



MODUL PRAKTIKUM

PRAKTIKUM SISTEM ADMINISTRASI DAN INFORMASI TERDISTRIBUSI

SVPL214405

Oleh:

**Dr. Eng. Ir. Ganjar Alfian, S.T., M. Eng.,
dan Tim Mahasiswa**

NIP. 111198701202201101

TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
2026

HALAMAN PENGESAHAN

MODUL PRAKTIKUM

Nama mata kuliah : Praktikum Sistem Administrasi dan Informasi Terdistribusi
Kode Mata Kuliah : SVPL214405
Nama Penulis : Dr.Eng. Ir. Ganjar Alfian, S.T., M.Eng.
NIP : 111198701202201101
Departemen / Prodi : Teknik Elektro dan Informatika/ Teknologi Rekayasa Perangkat Lunak

Yogyakarta, 25 – 01 – 2026

Menyetujui,
Ketua Program Studi

Penulis

Dr. Umar Taufiq, S.Kom., M.Cs.
NIP. 111198212201610101


Dr.Eng. Ir. Ganjar Alfian, S.T., M.Eng.
NIP. 111198701202201101

Mengetahui,
Ketua Departemen Teknik Elektro dan Informatika,

Ir. Yuris Mulya Saputra, S.T., M.Sc., Ph.D., IPM.
NIP. 111198806201605101

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga Modul Praktikum Sistem Administrasi dan Informasi Terdistribusi ini dapat diselesaikan dengan baik. Penyusunan modul ini bertujuan untuk memberikan panduan pembelajaran yang sistematis bagi mahasiswa dalam memahami konsep, teknik, dan implementasi penambangan data, khususnya pada Program Studi Teknologi Rekayasa Perangkat Lunak, Departemen Teknik Elektro dan Informatika, Sekolah Vokasi Universitas Gadjah Mada.

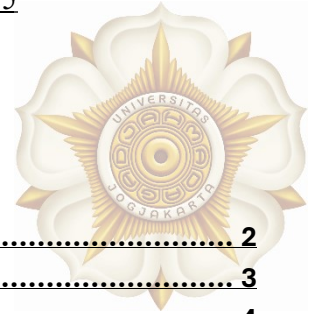
Modul ini diharapkan dapat menjadi sumber pembelajaran yang efektif dalam pelaksanaan praktikum Sistem Administrasi Informasi Terdistribusi, serta membantu mahasiswa meningkatkan kemampuan dalam memahami, mengelola, dan mengimplementasikan sistem administrasi berbasis arsitektur terdistribusi dengan dukungan berbagai metode dan perangkat lunak terkait. Selain itu, modul ini juga diharapkan dapat mendorong mahasiswa untuk lebih aktif mengeksplorasi pengembangan, penelitian, dan inovasi di bidang sistem informasi terdistribusi.

Ucapan terima kasih kami sampaikan kepada semua pihak yang telah berkontribusi dalam penyusunan modul ini, termasuk dosen, asisten laboratorium, dan rekan-rekan yang telah memberikan masukan serta dukungan selama proses penyusunan berlangsung.

Kami menyadari bahwa modul ini masih jauh dari sempurna. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari para pembaca demi penyempurnaan modul ini pada edisi berikutnya.

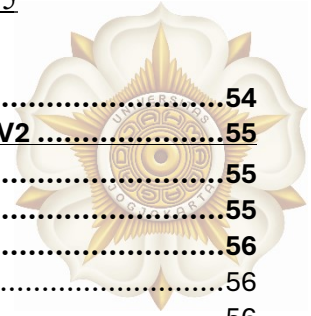
Yogyakarta, 25-01-2026

Penyusun

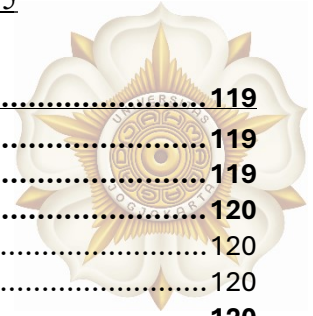


DAFTAR ISI

HALAMAN PENGESAHAN	2
KATA PENGANTAR	3
DAFTAR ISI	4
DAFTAR GAMBAR	7
DAFTAR TABEL	12
PERTEMUAN I INSTALASI VIRTUAL MACHINE DI HOST KOMPUTER	13
1.1. TUJUAN PEMBELAJARAN	13
1.2. DASAR TEORI.....	13
1.3. ALAT DAN BAHAN	14
1.3.1. PERANGKAT KERAS	14
1.3.2. PERANGKAT LUNAK	14
1.4. LANGKAH PERCOBAAN	14
1.5. TUGAS & ANALISIS	21
1.6. REFERENSI.....	21
PERTEMUAN II INSTALASI FTP CLIENT, TERMINAL EMULATOR, PHP MYSQL DI UBUNTU SERVER (VIRUTAL MACHINE)	22
2.1. TUJUAN PEMBELAJARAN	22
2.2. DASAR TEORI.....	22
2.3. ALAT DAN BAHAN	23
2.3.1. PERANGKAT KERAS	23
2.3.2. PERANGKAT LUNAK	23
2.4. LANGKAH PERCOBAAN	23
2.5. TUGAS & ANALISIS	36
2.6. REFERENSI.....	36
PERTEMUAN III INSTALASI WEB SERVER DAN DATABASE DALAM MESIN (KOMPUTER) YANG BERBEDA.....	37
3.1. TUJUAN PEMBELAJARAN	37
3.2. DASAR TEORI.....	37
3.3. ALAT DAN BAHAN	38
3.3.1. PERANGKAT KERAS	38
3.3.2. PERANGKAT LUNAK	38
3.4. LANGKAH PERCOBAAN	38
3.5. TUGAS & ANALISIS	42
3.6. REFERENSI.....	43
PERTEMUAN IV PENGGUNAAN REST API DI UBUNTU SERVER	44
4.1. TUJUAN PEMBELAJARAN	44
4.2. DASAR TEORI.....	44
4.3. ALAT DAN BAHAN	45
4.3.1. PERANGKAT KERAS	45
4.3.2. PERANGKAT LUNAK	45
4.4. LANGKAH PERCOBAAN	45
4.5. TUGAS & ANALISIS	54



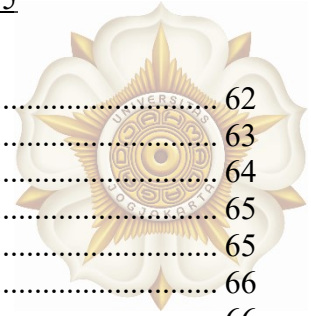
4.6. REFERENSI.....	54
PERTEMUAN V MEMBANGUN & MENGAKSES REST API DI SERVER UBUNTU V2	55
5.1. TUJUAN PEMBELAJARAN	55
5.2. DASAR TEORI.....	55
5.3. ALAT DAN BAHAN	56
5.3.1. PERANGKAT KERAS	56
5.3.2. PERANGKAT LUNAK	56
5.4. LANGKAH PERCOBAAN	56
5.5. TUGAS & ANALISIS	58
5.6. REFERENSI.....	58
PERTEMUAN VI REPLIKASI DATABASE MYSQL DI UBUNTU SERVER	60
6.1. TUJUAN PEMBELAJARAN	60
6.2. DASAR TEORI.....	60
6.2.1. REPLIKASI DATABASE	60
6.3. ALAT DAN BAHAN	62
6.3.1. PERANGKAT KERAS	62
6.3.2. PERANGKAT LUNAK	62
6.4. LANGKAH PERCOBAAN	62
6.5. TUGAS & ANALISIS	74
6.6. REFERENSI.....	74
PERTEMUAN VII SINKRONISASI DATABASE MENGGUNAKAN REST API	75
7.1. TUJUAN PEMBELAJARAN	75
7.2. DASAR TEORI.....	75
7.3. ALAT DAN BAHAN	77
7.3.1. PERANGKAT KERAS	77
7.3.2. PERANGKAT LUNAK	77
7.4. LANGKAH PERCOBAAN	77
7.5. TUGAS & ANALISIS	84
7.6. REFERENSI.....	85
PERTEMUAN VIII API AUTHENTICATION	86
8.1. TUJUAN PEMBELAJARAN	86
8.2. DASAR TEORI.....	86
8.3. ALAT DAN BAHAN	87
8.3.1. PERANGKAT KERAS	87
8.3.2. PERANGKAT LUNAK	87
8.4. LANGKAH PERCOBAAN	87
8.5. TUGAS & ANALISIS	98
8.6. REFERENSI.....	98
PERTEMUAN IX API PHP DENGAN DOCKER	99
9.1. TUJUAN PEMBELAJARAN	99
9.2. DASAR TEORI.....	99
9.3. ALAT DAN BAHAN	100
9.3.1. PERANGKAT KERAS	100
9.3.2. PERANGKAT LUNAK	100
9.4. LANGKAH PERCOBAAN	100
9.5. TUGAS & ANALISIS	118
9.6. REFERENSI.....	118



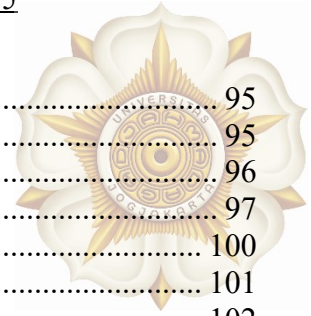
PERTEMUAN X ORKESTRASI DOCKER DENGAN DOCKER COMPOSE	119
10.1. TUJUAN PEMBELAJARAN.....	119
10.2. DASAR TEORI	119
10.3. ALAT DAN BAHAN.....	120
10.3.1. PERANGKAT KERAS.....	120
10.3.2. PERANGKAT LUNAK	120
10.4. LANGKAH PERCOBAAN	120
10.5. TUGAS & ANALISIS	131
10.6. REFERENSI	131
PERTEMUAN XI CI/CD PIPELINE LOKAL DENGAN GITEA ACTIONS	132
11.1. TUJUAN PEMBELAJARAN.....	132
11.2. DASAR TEORI	132
11.3. ALAT DAN BAHAN.....	133
11.3.1. PERANGKAT KERAS.....	133
11.3.2. PERANGKAT LUNAK	133
11.4. LANGKAH PERCOBAAN	133
11.5. TUGAS & ANALISIS	161
11.6. REFERENSI	162
PERTEMUAN XII IMPLEMENTASI DEPLOYMENT DAN LOAD BALANCING PADA VPS	163
12.1. TUJUAN PEMBELAJARAN.....	163
12.2. DASAR TEORI	163
12.3. ALAT DAN BAHAN.....	165
12.3.1. PERANGKAT KERAS.....	166
12.3.2. PERANGKAT LUNAK	166
12.4. LANGKAH PERCOBAAN	166
12.5. TUGAS & ANALISIS	183
12.6. REFERENSI	183

DAFTAR GAMBAR

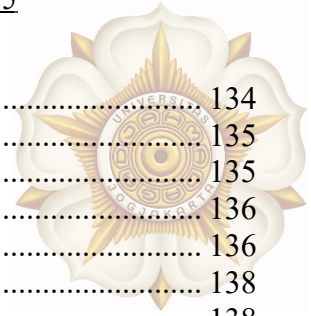
Gambar 1.4.1 Tampilan awal aplikasi VirtualBox	15
Gambar 1.4.2 Konfigurasi nama dan ISO (Pastikan Skip Unattended Installation dicentang)	16
Gambar 1.4.3 Rangkuman spesifikasi mesin virtual	16
Gambar 1.4.4 Tampilan awal proses booting instalasi Ubuntu Server.....	17
Gambar 1.4.5 Tampilan terminal Ubuntu Server setelah berhasil login.....	18
Gambar 1.4.6 Perbedaan segmen IP antara Ubuntu (NAT) (kiri) dan Komputer Host (kanan)...	19
Gambar 1.4.7 Uji coba ping dari Host ke Ubuntu yang gagal.....	19
Gambar 1.4.8 IP Ubuntu telah berubah dan satu segmen dengan Host.....	20
Gambar 1.4.9 Ping berhasil dari Host ke Ubuntu	20
Gambar 2.4.1 Instalasi Ekstensi Remote - SSH pada VS Code.....	24
Gambar 2.4.2 Icon Remote SSH.....	24
Gambar 2.4.3 Memasukkan perintah koneksi SSH di VS Code	25
Gambar 2.4.4 Status Apache2.....	25
Gambar 2.4.5 Tampilan default page di browser.....	26
Gambar 2.4.6 Tampilan pengecekan versi PHP di terminal.....	26
Gambar 2.4.7 Tampilan berhasil masuk ke konsol MySQL.....	27
Gambar 2.4.8 Screenshot terminal saat mengetik perintah SQL pembuatan user dan database ..	28
Gambar 2.4.9 Konfirmasi grup www-data di user saat ini	29
Gambar 2.4.10 Tampilan File Explorer VS Code yang membuka folder /var/www/htm	29
Gambar 2.4.11 Tampilan koding file tes_koneksi.php di VS Code	29
Gambar 2.4.12 Tampilan browser yang menampilkan pesan sukses koneksi	30
Gambar 3.4.1 Tampilan awal Laragon dengan status Apache berjalan.....	39
Gambar 3.4.2 Mengubah konfigurasi bind-address menjadi 0.0.0.0 via terminal VS Code	40
Gambar 3.4.3 Mengecek IP Address Host melalui CMD Windows	41
Gambar 3.4.4 Perintah SQL pembuat user baru di terminal VS Code	41
Gambar 3.4.5 Kode program tes_koneksi.php (dibuat di Windows)	42
Gambar 3.4.6 Tampilan browser yang menyatakan koneksi sukses	42
Gambar 4.4.1 Client side to side.....	46
Gambar 4.4.2 Proses drag-and-drop folder API ke sidebar VS Code	46
Gambar 4.4.3 Tampilan terminal VS Code saat verifikasi database	47
Gambar 4.4.4 Mengedit file config.php langsung di editor VS Code	48
Gambar 4.4.5 Tampilan data JSON saat diakses melalui browser	49
Gambar 4.4.6 Menampilkan daftar mahasiswa (GET) di Postman	50
Gambar 4.4.7 Menambahkan data baru (POST) di Postman.....	51
Gambar 4.4.8 Memperbarui data (Update) di Postman	51
Gambar 4.4.9 Menghapus data (DELETE) di Postman.....	52
Gambar 4.4.10 Modifikasi variabel API URL pada kode program Frontend	53
Gambar 4.4.11 Tampilan akhir website Frontend yang menampilkan data dari server	53
Gambar 5.4.1 Tampilan konfigurasi parameter dan URL API pada website Open-Meteo	57
Gambar 5.4.2 Tampilan akhir dashboard yang menampilkan Data Mahasiswa dan Data Cuaca sekaligus.....	58
Gambar 6.2.1 Ilustrasi master-slave dalam replikasi DB	61



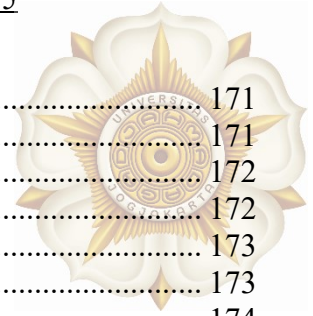
Gambar 6.4.1 Cek versi MySQL di Ubuntu Server (Master)	62
Gambar 6.4.2 Halaman MySQL Community Server.....	63
Gambar 6.4.3 Konfigurasi my.ini	64
Gambar 6.4.4 Inisiasi MySQL	65
Gambar 6.4.5 Menjalankan MySQL Server	65
Gambar 6.4.6 CLI MySQL	66
Gambar 6.4.7 Mengecek versi MySQL	66
Gambar 6.4.8 Merubah bind-address file /etc/mysql.conf.d/mysqld.cnf.....	67
Gambar 6.4.9 Mengaktifkan server-id dan log_bin di mysqld.cnf.....	67
Gambar 6.4.10 Restart Server MySQL Ubuntu Server	68
Gambar 6.4.11 Melakukan cek user yang telah ada di database	68
Gambar 6.4.12 Menambah User MySQL	68
Gambar 6.4.13 Memberikan izin Replication Slave pada replica_user.....	69
Gambar 6.4.14 Refresh Tabel Perizinan	69
Gambar 6.4.15 Menambahkan server-id di konfigurasi my.ini Slave	70
Gambar 6.4.16 Menjalankan ulang MySQL Server	70
Gambar 6.4.17 Mengecek position di MySQL Master.....	71
Gambar 6.4.18 Konfigurasi MySQL slave bagian 2.....	71
Gambar 6.4.19 Menjalankan replikasi	72
Gambar 6.4.20 Status Replikasi MySQL.....	72
Gambar 6.4.21 Perbandingan awal database (Master MySQL, Slave MySQL)	73
Gambar 6.4.22 Uji coba replikasi (kiri Master MySQL, kanan Slave MySQL)	73
Gambar 7.2.1 Konsep Kasus 3.....	76
Gambar 7.2.2 Konsep sinkronisasi database MySQL kasus 6.....	77
Gambar 7.4.1 Ekspor database mahasiswa	78
Gambar 7.4.2 Impor mahasiswa2 ke sait_db	78
Gambar 7.4.3 Mengecek tabel mahasiswa di MySQL Lokal (kiri) dan MySQL VPS (kanan) ...	79
Gambar 7.4.4 Menambahkan database lokal	80
Gambar 7.4.5 Ubah kode pemanggilan API dengan SQL ke database lokal	81
Gambar 7.4.6 Hasil implementasi penambahan tampilan database lokal.....	82
Gambar 7.4.7 Menambahkan kode insert database lokal	83
Gambar 7.4.8 Form Submit di insert.php secara full. Insert ke database lokal (merah) dan insert via API (kuning)	84
Gambar 8.2.1 Autentikasi dan Otorisasi	86
Gambar 8.4.1 CLI Git clone	88
Gambar 8.4.2 Source Code JWT	88
Gambar 8.4.3 Pembuatan JWT di Login	89
Gambar 8.4.4 Decode JWT.....	90
Gambar 8.4.5 Konfigurasi file config/database.php	91
Gambar 8.4.6 SQL Tabel User untuk contoh aplikasi JWT	92
Gambar 8.4.7 Injeksi SQL manual	93
Gambar 8.4.8 Pembuatan Variable Base di Postman	93
Gambar 8.4.9 Request BASE.....	94
Gambar 8.4.10 Request Register	94
Gambar 8.4.11 Request Login	94
Gambar 8.4.12 Request Get All Users.....	94



Gambar 8.4.13 Uji Register pada Postman	95
Gambar 8.4.14 Login	95
Gambar 8.4.15 Debug JWT	96
Gambar 8.4.16 Melihat data user	97
Gambar 9.4.1 docs.docker.com	100
Gambar 9.4.2 Set up Docker's apt repository	101
Gambar 9.4.3 Hasil sukses menambah repository docker	102
Gambar 9.4.4 Install the Docker packages	102
Gambar 9.4.5 Melakukan pengecekan status Docker	103
Gambar 9.4.6 Instalasi Docker Success	104
Gambar 9.4.7 Menjalankan docker tanpa sudo	105
Gambar 9.4.8 Membuat folder di Home VPS	105
Gambar 9.4.9 Mengubah konfigurasi Database	106
Gambar 9.4.10 Tambahkan file .env	107
Gambar 9.4.11 Development Server PHP	107
Gambar 9.4.12 Mengubah Variable Postman	107
Gambar 9.4.13 Mengecek API Development di Postman	108
Gambar 9.4.14 File .htaccess	109
Gambar 9.4.15 Router baru	111
Gambar 9.4.16 Perubahan impor	111
Gambar 9.4.17 Dockerfile	113
Gambar 9.4.18 Docker Build	115
Gambar 9.4.19 Image php-jwt-example berhasil dibuat	115
Gambar 9.4.20 Docker Run php-jwt-example	116
Gambar 9.4.21 Docker ps	116
Gambar 9.4.22 Aplikasi Docker berhasil jalan	116
Gambar 9.4.23 Menambahkan user MySQL	117
Gambar 9.4.24 Postman Docker Login	117
Gambar 9.4.25 Postman Docker Get All Users (Admin)	118
Gambar 10.4.1 Struktur folder Pertemuan Sebelumnya	121
Gambar 10.4.2 Menghentikan dan menghapus kontainer php-jwt lama	121
Gambar 10.4.3 Docker Compose	123
Gambar 10.4.4 Menjalankan Docker Compose Up	123
Gambar 10.4.5 Rebuild dan Menjalankan Ulang	124
Gambar 10.4.6 Service Database MySQL	124
Gambar 10.4.7 Penambahan Persistent Volume	125
Gambar 10.4.8 Menambah ketergantungan	126
Gambar 10.4.9 Mengubah env DB_HOST ke service MySQL Docker	126
Gambar 10.4.10 Docker Compose Dua Service	127
Gambar 10.4.11 Jalankan docker compose up untuk membuat service database	128
Gambar 10.4.12 Register dengan database di docker	128
Gambar 10.4.13 Login dengan database di docker	128
Gambar 10.4.14 Get All Users dari database di Docker	129
Gambar 10.4.15 Menambahkan route docker	130
Gambar 10.4.16 Hot reload berhasil	130
Gambar 11.4.1 Mengecek sisa penyimpanan	133



Gambar 11.4.2 Menghapus Image Docker tidak terpakai	134
Gambar 11.4.3 Mematikan service database docker	135
Gambar 11.4.4 Penyesuaian file .env.....	135
Gambar 11.4.5 Menghidupkan kembali MySQL	136
Gambar 11.4.6 Membuat folder gitea	136
Gambar 11.4.7 Instalasi docker-compose.yml gitea	138
Gambar 11.4.8 Menjalankan service gitea.....	138
Gambar 11.4.9 Tahap konfigurasi awal gitea	139
Gambar 11.4.10 Register akun di gitea.....	140
Gambar 11.4.11 Halaman utama gitea.....	140
Gambar 11.4.12 Site Administration Button	141
Gambar 11.4.13 Halaman Action Runners (Langkah 1)	142
Gambar 11.4.14 Docker Compose Gitea Lengkap	143
Gambar 11.4.15 Docker compose up.....	144
Gambar 11.4.16 Runner Gitea tersedia.....	144
Gambar 11.4.17 Repository baru php-jwt-example.....	145
Gambar 11.4.18 Pentunjuk awal konfigurasi repository	145
Gambar 11.4.19 Menghapus folder .git	146
Gambar 11.4.20 Menambahkan .gitignore	147
Gambar 11.4.21 Staging kode ke gitea	148
Gambar 11.4.22 Git push	148
Gambar 11.4.23 Copy URL Repository	149
Gambar 11.4.24 Git clone	150
Gambar 11.4.25 Membuka kode di VS code.....	150
Gambar 11.4.26 Deploys YAML Workflows.....	151
Gambar 11.4.27 Tombol setting di kanan baris 3	152
Gambar 11.4.28 Halaman Secrets.....	153
Gambar 11.4.29 Tambah Server IP VPS	153
Gambar 11.4.30 Tambah Username VPS	154
Gambar 11.4.31 Tambah Password VPS.....	154
Gambar 11.4.32 Tiga secrets	155
Gambar 11.4.33 Route baru cicd	156
Gambar 11.4.34 Tampilan url CICD awal.....	156
Gambar 11.4.35 Git Push dan Kredensial.....	157
Gambar 11.4.36 Lanjutan terminal ketika push di izinkan.....	157
Gambar 11.4.37 Actions Berjalan di Gitea	158
Gambar 11.4.38 Detail Actions	159
Gambar 11.4.39 Action berhasil dijalankan	160
Gambar 11.4.40 Aplikasi berhasil diperbarui	160
Gambar 12.2.1 Load Balancing (Sumber: geeksforgeeks.org).....	165
Gambar 12.4.1 Tampilan terminal cek versi PHP	166
Gambar 12.4.2 Status apache2 running	168
Gambar 12.4.3 Menjalankan apache2.....	169
Gambar 12.4.4 Cek version mysql.....	169
Gambar 12.4.5 Output sukses instalasi Composer.....	170
Gambar 12.4.6 Output perintah composer	170



Gambar 12.4.7 Cek npm	171
Gambar 12.4.8 Proses instalasi node versi lts	171
Gambar 12.4.9 Proses cloning repositori via terminal	172
Gambar 12.4.10 Mengubah kepemilikan folder	172
Gambar 12.4.11 Mengubah perizinan folder	173
Gambar 12.4.12 Proses instalasi vendor Laravel	173
Gambar 12.4.13 Proses npm install dan npm run build	174
Gambar 12.4.14 Mengedit konfigurasi database di file .env	174
Gambar 12.4.15 Migrasi dan import database	175
Gambar 12.4.16 Generate app key	175
Gambar 12.4.17 Jalankan aplikasi laravel	176
Gambar 12.4.18 Buka aplikasi laravel dengan website	176
Gambar 12.4.19 Membuat folder menjadi www-data	177
Gambar 12.4.20 Menjalankan npm install dan npm run build	178
Gambar 12.4.21 Duplikasi folder aplikasi	179
Gambar 12.4.22 Mengatur kepemilikan folder duplikat	179
Gambar 12.4.23 Mengubah welcome.blade.php	180
Gambar 12.4.24 Mengaktifkan kedua situs	181
Gambar 12.4.25 Mengaktifkan Modul Proxy	181
Gambar 12.4.26 Mengakses server 1	183
Gambar 12.4.27 Mengakses server 2	183

DAFTAR TABEL

No table of figures entries found.



PERTEMUAN I

INSTALASI VIRTUAL MACHINE DI HOST KOMPYUTER

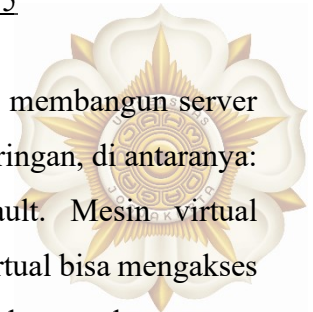
1.1. TUJUAN PEMBELAJARAN

- A. Mahasiswa mampu memahami prinsip kerja virtualisasi, perbedaan antara Host OS dan Guest OS, serta peran Hypervisor dalam manajemen sumber daya.
- B. Mahasiswa mampu melakukan instalasi dan konfigurasi awal Oracle VM VirtualBox pada komputer host.
- C. Mahasiswa mampu mengonfigurasi Virtual Machine (VM) dan menginstal sistem operasi Linux Server (Ubuntu) di dalamnya.
- D. Mahasiswa memahami konfigurasi jaringan pada VirtualBox untuk menghubungkan Host Computer dengan Virtual Machine.

1.2. DASAR TEORI

Virtualisasi dan Virtual Machine Virtualisasi adalah teknologi yang memungkinkan kita untuk membuat versi maya (virtual) dari suatu sumber daya teknologi, seperti perangkat keras, sistem operasi, atau perangkat penyimpanan. Dalam praktikum ini, kita menggunakan VirtualBox sebagai hypervisor tipe 2 yang berjalan di atas sistem operasi utama (Host OS) untuk menjalankan sistem operasi tamu (Guest OS). Hal ini memungkinkan kita menjalankan Ubuntu Server secara terisolasi tanpa mengganggu sistem operasi utama (Windows).

Ubuntu Server Ubuntu Server adalah varian dari distro Linux Ubuntu yang didesain khusus untuk kebutuhan server. Berbeda dengan versi Desktop, versi Server secara default tidak menggunakan antarmuka grafis (GUI) melainkan berbasis teks (Command Line Interface). Hal ini membuat Ubuntu Server lebih ringan dan efisien dalam penggunaan sumber daya (RAM dan CPU).



Konfigurasi Jaringan pada VirtualBox Salah satu aspek krusial dalam membangun server virtual adalah konektivitas. VirtualBox menyediakan beberapa mode jaringan, di antaranya:

1. NAT (Network Address Translation): Merupakan mode default. Mesin virtual mendapatkan IP privat yang terisolasi (biasanya 10.0.2.x). Mesin virtual bisa mengakses internet melalui host, namun host tidak bisa mengakses mesin virtual secara langsung.
2. Bridged Adapter: Mode ini menghubungkan kartu jaringan mesin virtual langsung ke jaringan fisik yang digunakan oleh host. Mesin virtual akan dianggap sebagai perangkat fisik independen dalam jaringan yang sama dengan host, sehingga keduanya akan memiliki segmen IP yang sama (misalnya 192.168.x.x) dan dapat saling berkomunikasi dua arah.

Pada praktikum ini, perubahan dari NAT ke Bridged Adapter diperlukan agar server yang dibangun dapat diakses (di-ping atau di-remote) oleh komputer host atau komputer lain dalam jaringan yang sama.

1.3. ALAT DAN BAHAN

1.3.1. Perangkat Keras

- Komputer / Laptop

1.3.2. Perangkat Lunak

- Oracle VM VirtualBox,
- ISO Image Ubuntu Server (Versi 22.04 LTS atau 24.04 LTS),
- Web Browser

1.4. LANGKAH PERCOBAAN

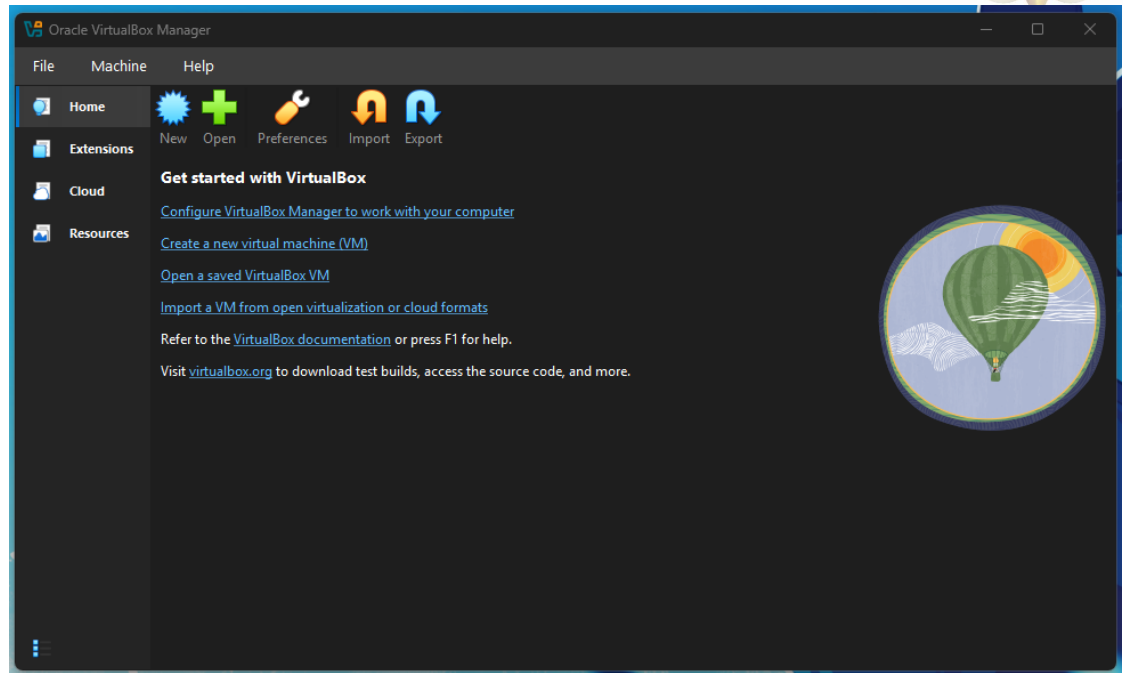
1. Membuat Mesin Virtual pada Komputer Host dengan VirtualBox

Pada praktikum ini, kita memerlukan perangkat lunak VirtualBox. Aplikasi ini berfungsi untuk membuat dan menjalankan mesin virtual (virtual machine) pada komputer utama (Komputer Host). Dengan virtualisasi, kita dapat menjalankan sistem operasi lain, seperti Ubuntu Server, secara terisolasi tanpa memerlukan perangkat fisik tambahan.

Langkah pertama, unduh dan install aplikasi VirtualBox melalui tautan berikut:

<https://www.virtualbox.org/wiki/Downloads>

Setelah instalasi selesai, buka aplikasi tersebut hingga muncul tampilan awal seperti Gambar 1.4.1.



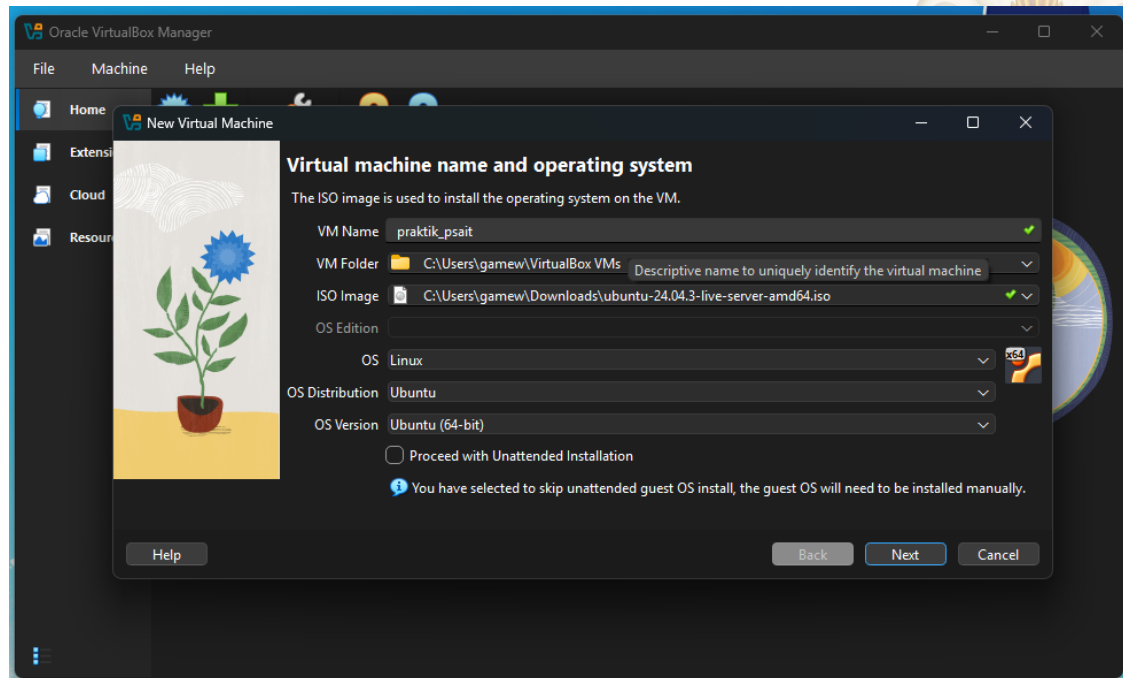
Gambar 1.4.1 Tampilan awal aplikasi VirtualBox

Selanjutnya, kita akan membuat mesin virtual baru. Pastikan telah memiliki file .iso Ubuntu Server. Jika belum, unduh terlebih dahulu melalui link dibawah ini.

<https://ubuntu.com/download/server>

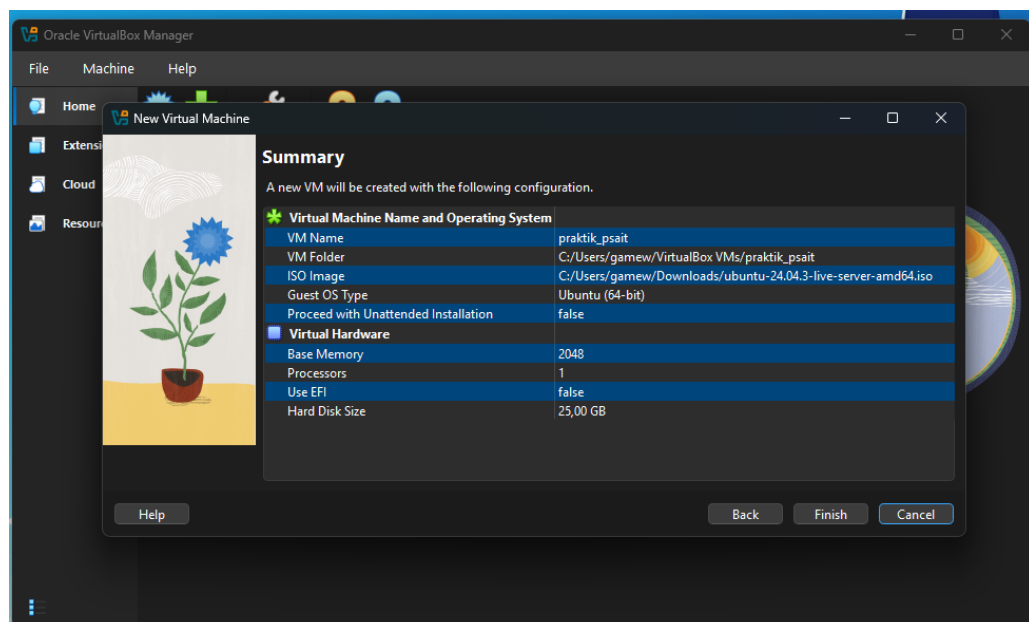
Klik tombol New/Open pada menu utama VirtualBox. Akan muncul jendela konfigurasi awal:

- Name: Isi nama mesin virtual (contoh: UbuntuServer_NIM).
- ISO Image: Pilih file .iso Ubuntu Server yang telah diunduh.
- Penting: Jangan centang opsi "Proceed with Unattended Installation". Hal ini diperlukan agar kita tidak perlu mengatur konfigurasi hostname dan domain.



Gambar 1.4.2 Konfigurasi nama dan ISO (Pastikan Skip Unattended Installation dicentang)

Klik Next. Pada bagian Virtual Hardware, atur Base Memory (RAM) minimal 2048 MB (2 GB), Processors minimal 1 CPU agar kinerja server optimal dan Virtual Hard disk dengan kapasitas rekomendasi minimal 20 GB. Terakhir, tinjau rangkuman spesifikasi pada halaman Summary. Jika sudah sesuai, klik Finish.



Gambar 1.4.3 Rangkuman spesifikasi mesin virtual



Mesin virtual telah berhasil dibuat. Klik tombol Start untuk menyalakan mesin dan memulai proses instalasi.

2. Instalasi Ubuntu Server di Mesin Virtual

Setelah mesin virtual berhasil booting, pilih opsi "Try or Install Ubuntu Server". Ikuti instruksi instalasi yang muncul (disarankan memilih Bahasa Inggris untuk memudahkan pemahaman istilah teknis).

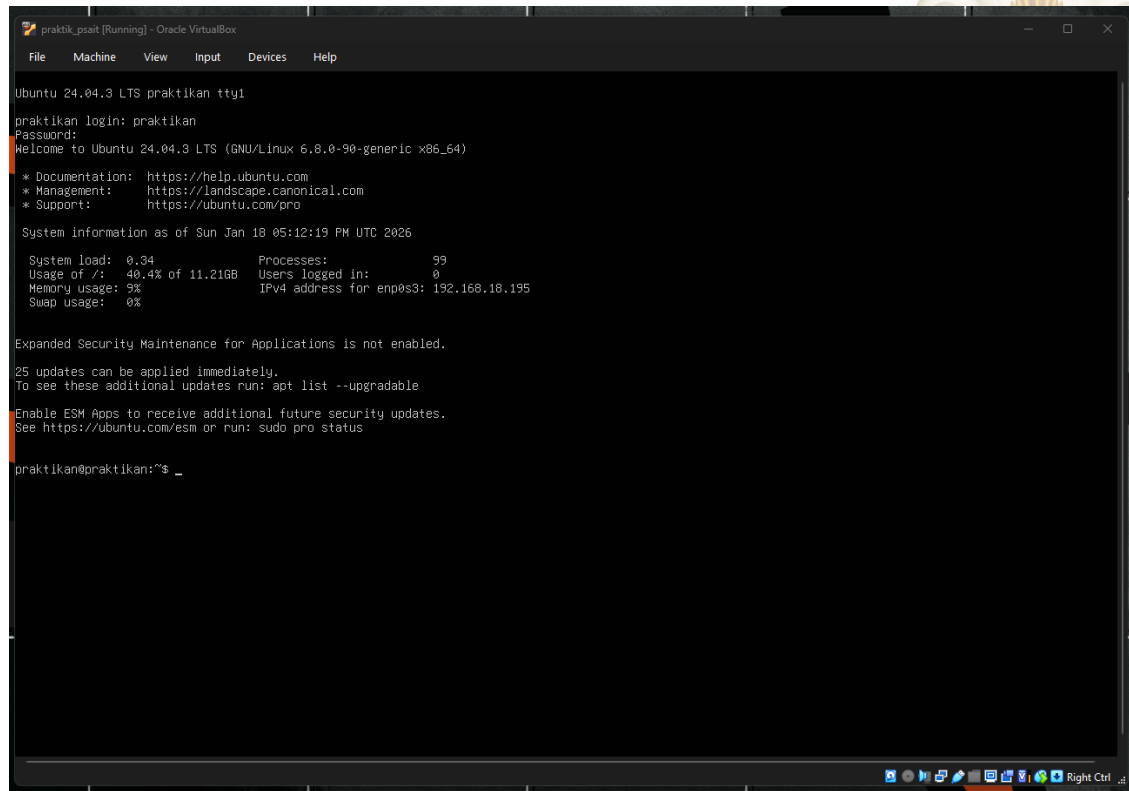


Gambar 1.4.4 Tampilan awal proses booting instalasi Ubuntu Server

Lakukan konfigurasi standar hingga mencapai tahap Profile Setup. Buatlah username dan password untuk akun administrator server. Pastikan untuk mengingatkannya karena akan digunakan untuk login selanjutnya.

Penting: Pada halaman SSH Setup, pastikan untuk mencentang opsi [x] Install OpenSSH server. Fitur ini wajib diaktifkan agar server dapat diakses dari jarak jauh (remote) yang akan dipraktikkan pada bab selanjutnya.

Lanjutkan proses hingga instalasi selesai, lalu pilih Reboot Now. Setelah restart, sistem akan meminta login. Masukkan username dan password yang telah dibuat. Jika berhasil, akan muncul tampilan terminal (command line) seperti Gambar 1.4.5.



Gambar 1.4.5 Tampilan terminal Ubuntu Server setelah berhasil login

3. Setting Network Mesin Virtual

Secara default, VirtualBox menggunakan mode jaringan NAT. Pada mode ini, mesin virtual dapat mengakses internet, namun Komputer Host tidak dapat mengakses mesin virtual secara langsung karena berada di segmen jaringan yang berbeda.

Untuk memverifikasinya, lakukan pengecekan alamat IP:

- Di Ubuntu Server VBOX, ketik perintah berikut.

```
ip a
```

Biasanya IP NAT adalah 10.0.2.15.

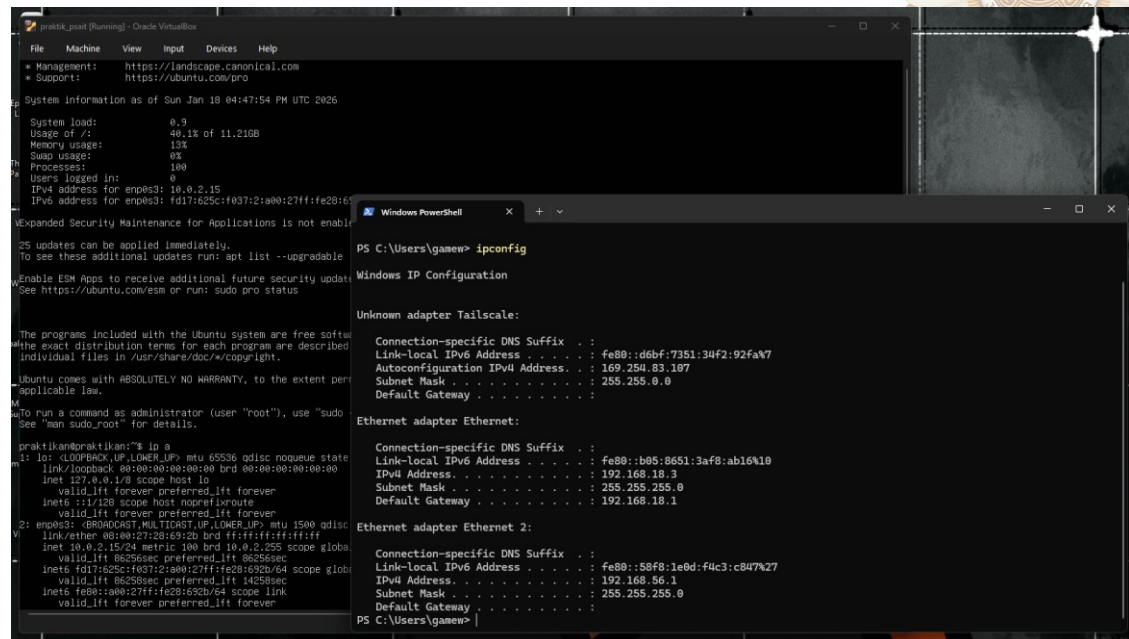
- Di Windows (Host), buka CMD dan ketik:

```
ipconfig
```

Biasanya IP Host adalah 192.168.x.x.

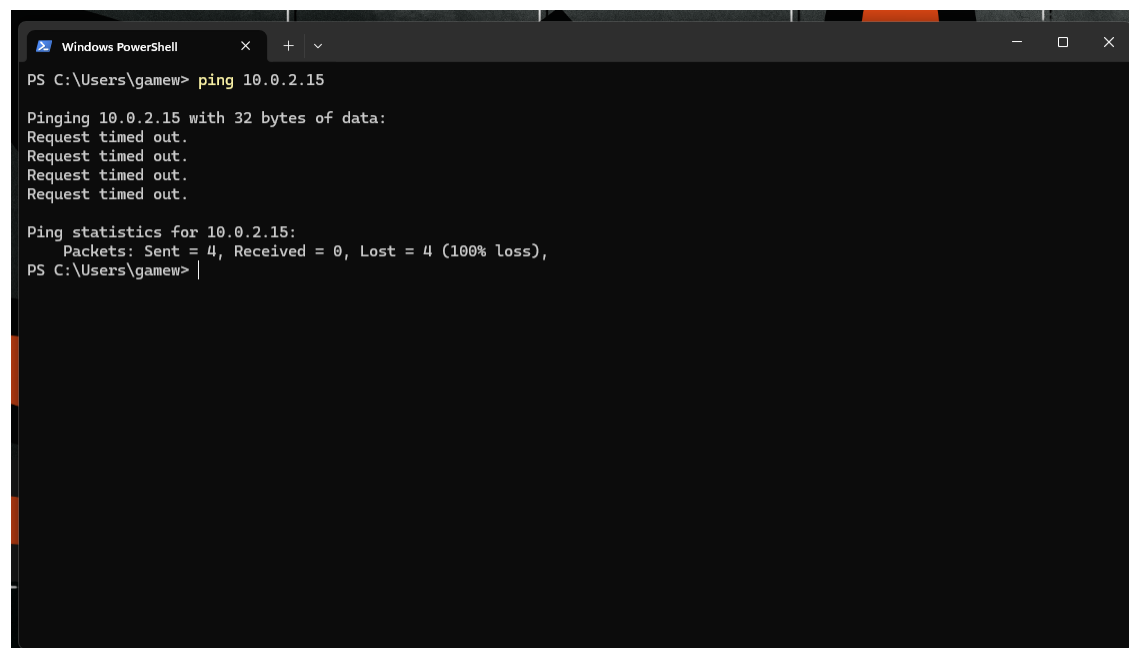
Oleh karena perbedaan segmen IP tersebut (10.x.x.x vs 192.x.x.x), koneksi antar kedua mesin tidak akan terjalin. Di kasus pembuat modul Gambar 1.4.6, koneksi di Ubuntu

Server yang ada di VBox menggunakan enp0s3 yang merupakan ethernet virtual vbox dengan ip 10.0.2.15, sedangkan untuk Windows 192.168.18.1.

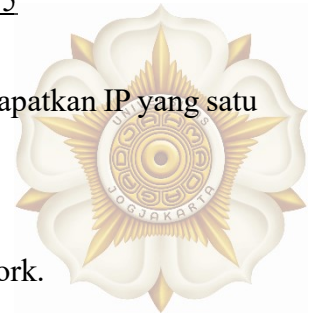


Gambar 1.4.6 Perbedaan segmen IP antara Ubuntu (NAT) (kiri) dan Komputer Host (kanan)

Jika dilakukan uji koneksi (ping) dari Host ke IP Ubuntu, hasilnya akan Request Timed Out.



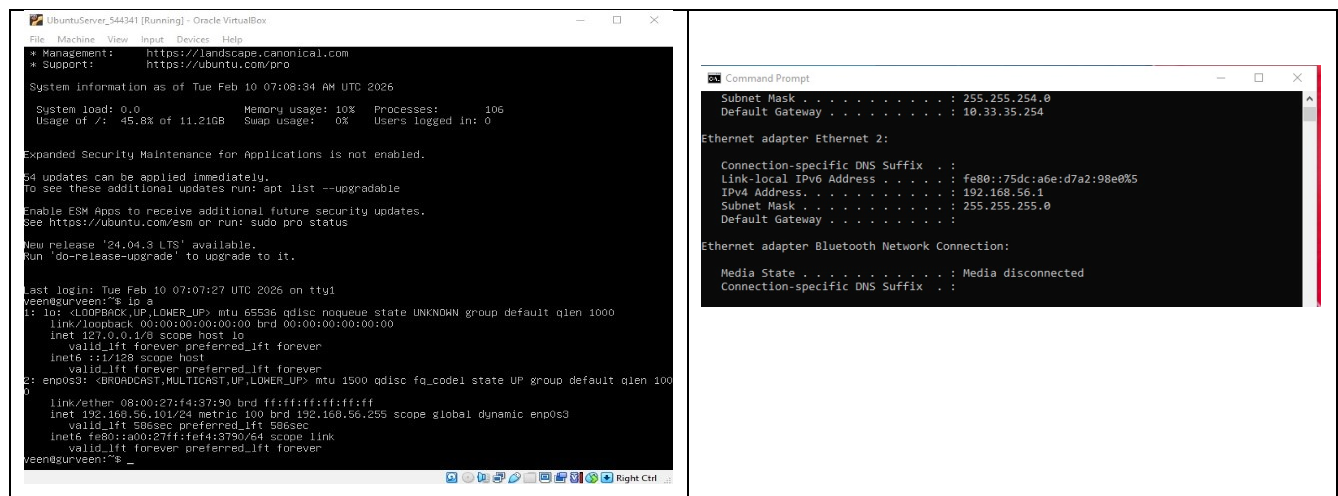
Gambar 1.4.7 Uji coba ping dari Host ke Ubuntu yang gagal



Solusi: Menggunakan Host Only Adapter Agar Ubuntu Server mendapatkan IP yang satu segmen dengan Komputer Host, lakukan langkah berikut:

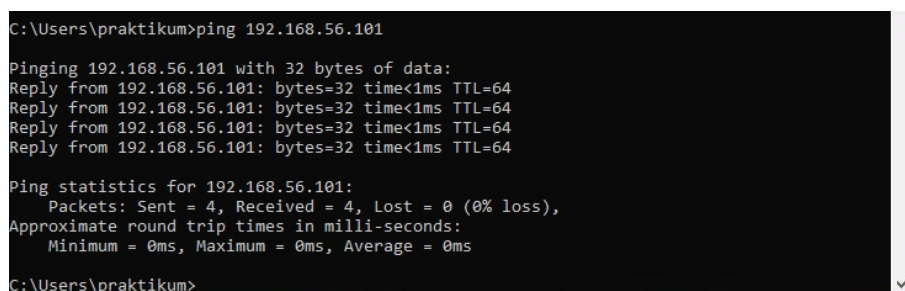
- Matikan Ubuntu Server dengan perintah: `sudo poweroff`.
- Buka menu Settings pada VirtualBox, lalu pilih menu Network.
- Ubah "Attached to" dari NAT menjadi Host Only Adapter.

Nyalakan kembali Ubuntu Server dan login. Cek kembali alamat IP dengan perintah “`ip a`”. Pastikan IP Address Ubuntu kini sudah berubah mengikuti segmen jaringan Host (contoh: oleh karena subnet mask adalah 255.255.255.0 dan IP Gateway adalah 192.168.xx.1 maka IP dari Ubuntu Server yang ada di VBox harus berada di jaringan 192.168.xx.xxx).

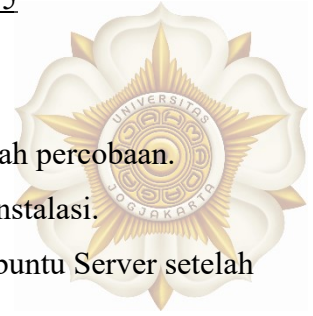


Gambar 1.4.8 IP Ubuntu telah berubah dan satu segmen dengan Host

Lakukan pengujian terakhir dengan melakukan ping dari CMD Windows ke alamat IP baru Ubuntu Server. Jika muncul respon "Reply from...", maka konfigurasi jaringan telah berhasil dan server siap digunakan.



Gambar 1.4.9 Ping berhasil dari Host ke Ubuntu



1.5. TUGAS & ANALISIS

1. Lakukan instalasi VirtualBox dan Ubuntu Server sesuai dengan langkah percobaan.
2. Pastikan Anda mencentang opsi Install OpenSSH Server saat proses instalasi.
3. Lakukan uji koneksi (Ping) dari Command Prompt Windows ke IP Ubuntu Server setelah mode jaringan diubah menjadi Bridged Adapter.

1.6. REFERENSI

- [1]. Oracle. (n.d.). Oracle VM VirtualBox User Manual: Chapter 6. Virtual Networking. Retrieved from <https://www.virtualbox.org/manual/ch06.html>
- [2]. Canonical. (n.d.). Install Ubuntu Server. Ubuntu Documentation. Retrieved from <https://ubuntu.com/server/docs/installation>
- [3]. Tanenbaum, A. S., & Wetherall, D. J. (2011). Computer Networks (5th ed.). Pearson Education.



PERTEMUAN II

INSTALASI FTP CLIENT, TERMINAL EMULATOR, PHP MYSQL DI UBUNTU SERVER (VIRTUAL MACHINE)

2.1. TUJUAN PEMBELAJARAN

- A. Mahasiswa mampu melakukan konfigurasi akses jarak jauh (remote access) ke server Linux menggunakan protokol SSH pada Visual Studio Code.
- B. Mahasiswa mampu membangun infrastruktur web server dengan melakukan instalasi paket Apache, PHP, dan MySQL (LAMP Stack) pada sistem operasi Ubuntu Server.
- C. Mahasiswa mampu memahami dan menerapkan manajemen hak akses (file permissions) pada direktori server untuk keperluan pengembangan web.
- D. Mahasiswa mampu melakukan manajemen database dasar melalui CLI (Command Line Interface), termasuk membuat database dan user baru.
- E. Mahasiswa mampu membuat skrip PHP sederhana untuk memverifikasi konektivitas antara Web Server dan Database Server.

2.2. DASAR TEORI

Secure Shell (SSH) dan Remote Development SSH adalah protokol jaringan kriptografi yang digunakan untuk mengoperasikan layanan jaringan secara aman di atas jaringan yang tidak aman. Dalam pengembangan web modern, developer jarang bekerja langsung di depan mesin fisik server. Sebaliknya, mereka menggunakan SSH untuk mengakses terminal server dari komputer pribadi. Ekstensi Remote - SSH pada VS Code memungkinkan editor teks tersebut berfungsi seolah-olah file yang diedit berada di komputer lokal, padahal file tersebut berada di server, sekaligus menyediakan terminal terintegrasi.

LAMP Stack adalah akronim dari paket perangkat lunak populer yang digunakan untuk membangun situs web dinamis dan aplikasi web. Komponennya terdiri dari:

- Linux: Sistem operasi (Ubuntu Server).



- Apache: Aplikasi web server yang bertugas menerima permintaan HTTP dari browser dan mengirimkan kembali halaman web.
- MySQL: Sistem manajemen basis data (RDBMS) untuk menyimpan dan mengelola data aplikasi.
- PHP: Bahasa pemrograman server-side yang memproses logika aplikasi dan berkomunikasi dengan database sebelum hasilnya dikirimkan oleh Apache ke browser.

Manajemen Hak Akses (Permissions) di Linux Secara default, direktori `/var/www/html` (tempat file web disimpan) dimiliki oleh root. Agar user biasa (praktikan) dapat membuat dan mengedit file di dalamnya tanpa menggunakan sudo terus-menerus, kepemilikan direktori perlu diubah menggunakan perintah `chown` (change owner) dan `chmod` (change mode), serta memasukkan user ke dalam grup `www-data` (grup standar web server).

2.3. ALAT DAN BAHAN

2.3.1. Perangkat Keras

- Laptop/PC (Host) yang terhubung ke jaringan yang sama dengan Server.
- Mesin Virtual Ubuntu Server yang telah berjalan (hasil dari Bab 1).

2.3.2. Perangkat Lunak

- Visual Studio Code (VS Code).
- Ekstensi VS Code: Remote - SSH (oleh Microsoft).
- Web Browser (Chrome/Edge/Firefox).
- Paket instalasi: Apache2, PHP, MySQL-Server.

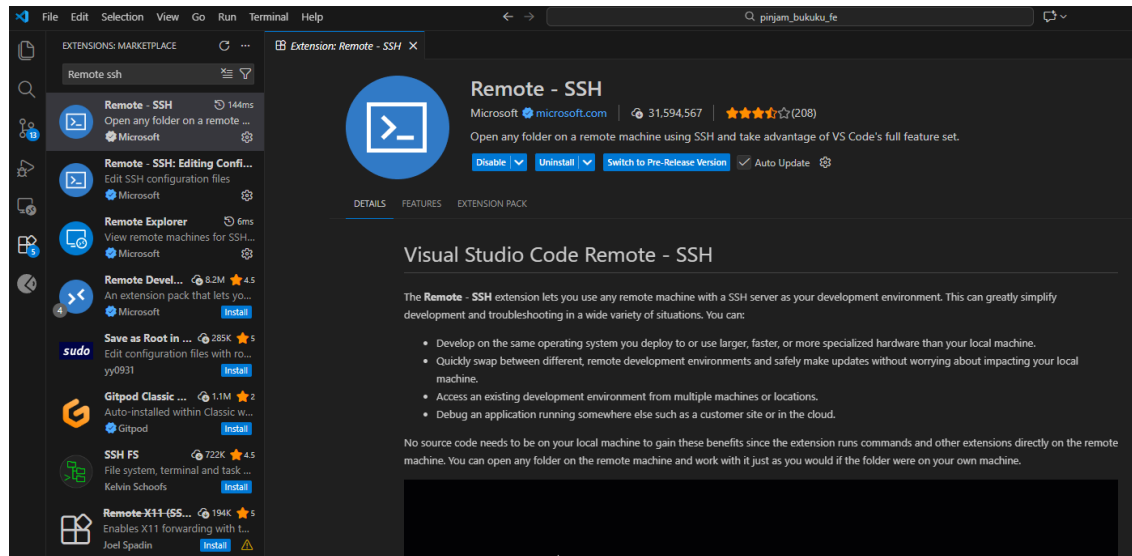
2.4. LANGKAH PERCOBAAN

1. Instalasi VS Code dan Ekstensi Remote SSH

Pada bab ini, kita akan mencoba membuat agar server kita dapat diakses dari jauh / remote melalui komputer host. Untuk itu kita memerlukan satu aplikasi utama yaitu VS Code dengan Ekstensi Remote SSH yang akan berfungsi sebagai sebagai Terminal Emulator (untuk mengetik perintah) dan FTP Client (untuk transfer file). Aplikasi ini

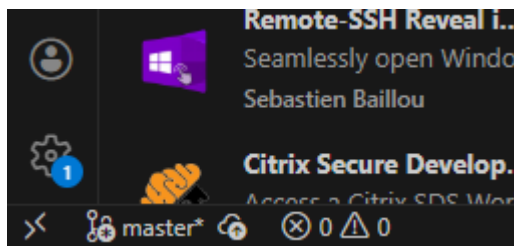
memudahkan kita mengelola server tanpa harus terpaku pada jendela VirtualBox, serta memungkinkan fitur copy-paste perintah.

Pertama, instal aplikasi VS Code tersebut di komputer host. Jika sudah buka VS Code, klik menu Extensions dan cari "Remote - SSH" buatan Microsoft, lalu klik Install.



Gambar 2.4.1 Instalasi Ekstensi Remote - SSH pada VS Code

Setelah terinstal, klik ikon di pojok kiri bawah tampilan VS Code dan pilih “Connect to Host” (atau tekan F1 lalu ketik "Remote-SSH: Connect to Host").

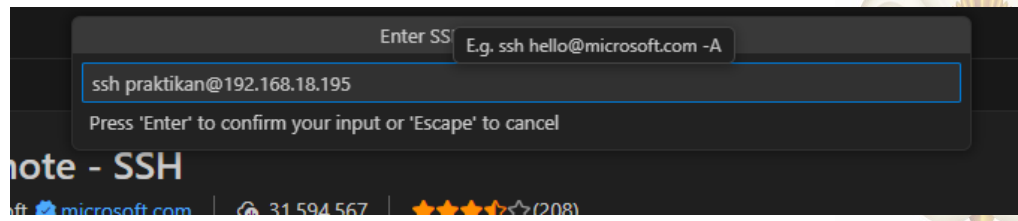


Gambar 2.4.2 Icon Remote SSH

Masukkan perintah koneksi dengan format:

```
ssh username@ip_address_ubuntu
```

Sebagai contoh: “ssh [praktikan@192.168.18.195](#)”, kemudian tekan enter. Jika diminta untuk memilih OS pilih OS SSH yang digunakan, di kasus ini OS yang digunakan adalah Linux (Ubuntu) dan masukkan password yang sudah dibuat sebelumnya.



Gambar 2.4.3 Memasukkan perintah koneksi SSH di VS Code

2. Instalasi Apache, PHP dan MySQL di Ubuntu

Sebelum menginstal package baru, jalankan perintah berikut melalui terminal VS Code.

```
sudo apt-get update
```

Langkah ini penting untuk memperbarui daftar paket agar kita mendapatkan versi perangkat lunak terbaru.

A. Instalasi Web Server (Apache)

Setelah itu jalankan perintah berikut untuk menginstal Apache.

```
sudo apt install apache2 -y
```

Setelah proses instalasi selesai, verifikasi status instalasi dengan perintah dibawah ini.

```
sudo systemctl status apache2
```

Pastikan statusnya adalah active (running) seperti Gambar 2.4.4.

```
praktikan@praktikan:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-01-19 09:14:32 UTC; 1min 9s ago
     Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 2425 (apache2)
      Tasks: 55 (limit: 2268)
    Memory: 5.1M (peak: 5.2M)
       CPU: 41ms
    CGroup: /system.slice/apache2.service
            └─2425 /usr/sbin/apache2 -k start
              2428 /usr/sbin/apache2 -k start
              2429 /usr/sbin/apache2 -k start

Jan 19 09:14:32 praktikan systemd[1]: Starting apache2.service - The Apache HTTP Server...
Jan 19 09:14:32 praktikan apachectl[2424]: AH00558: apache2: Could not reliably determine the server's
Jan 19 09:14:32 praktikan systemd[1]: Started apache2.service - The Apache HTTP Server.
praktikan@praktikan:~$
```

Gambar 2.4.4 Status Apache2



Untuk pengujian visual, buka browser di komputer host dan ketikkan IP Address Ubuntu. Jika muncul halaman "Apache2 Ubuntu Default Page", maka web server Apache2 sudah berjalan normal.



Gambar 2.4.5 Tampilan default page di browser

B. Instalasi PHP

Selanjutnya, install bahasa pemrograman PHP beserta modul penghubungnya ke Apache dan MySQL dengan perintah dibawah ini.

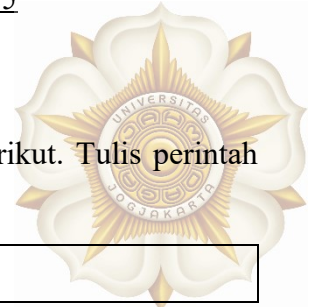
```
sudo apt install php libapache2-mod-php php-mysql -y
```

Tunggu hingga proses instalasi selesai. Lalu jalankan perintah dibawah ini untuk melihat versi php yang telah terinstal. Hasil akan terlihat seperti Gambar 2.4.6, versi PHP yang telah terunduh adalah versi 8.3.6.

```
php -v
```

```
praktikan@praktikan:~$ php -v
PHP 8.3.6 (cli) (built: Jan 7 2026 08:40:32) (NTS)
Copyright (c) The PHP Group
Zend Engine v4.3.6, Copyright (c) Zend Technologies
with Zend OPcache v8.3.6, Copyright (c), by Zend Technologies
```

Gambar 2.4.6 Tampilan pengecekan versi PHP di terminal



C. Instalasi Database (MySQL)

Terakhir, install sistem manajemen basis data dengan perintah berikut. Tulis perintah “Y” ketika meminta izin pengunduhan.

```
sudo apt install mysql-server
```

Tunggu hingga instalasi selesai, setelah itu uji coba dengan masuk ke konsol database menggunakan perintah dibawah ini.

```
sudo mysql
```

Jika terminal berubah menjadi prompt MySQL seperti Gambar 2.4.7, instalasi berhasil dilakukan. Ketik “exit” untuk keluar dari MySQL.

```
praktikan@praktikan:~$ sudo mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.44-0ubuntu0.24.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> exit
Bye
praktikan@praktikan:~$
```

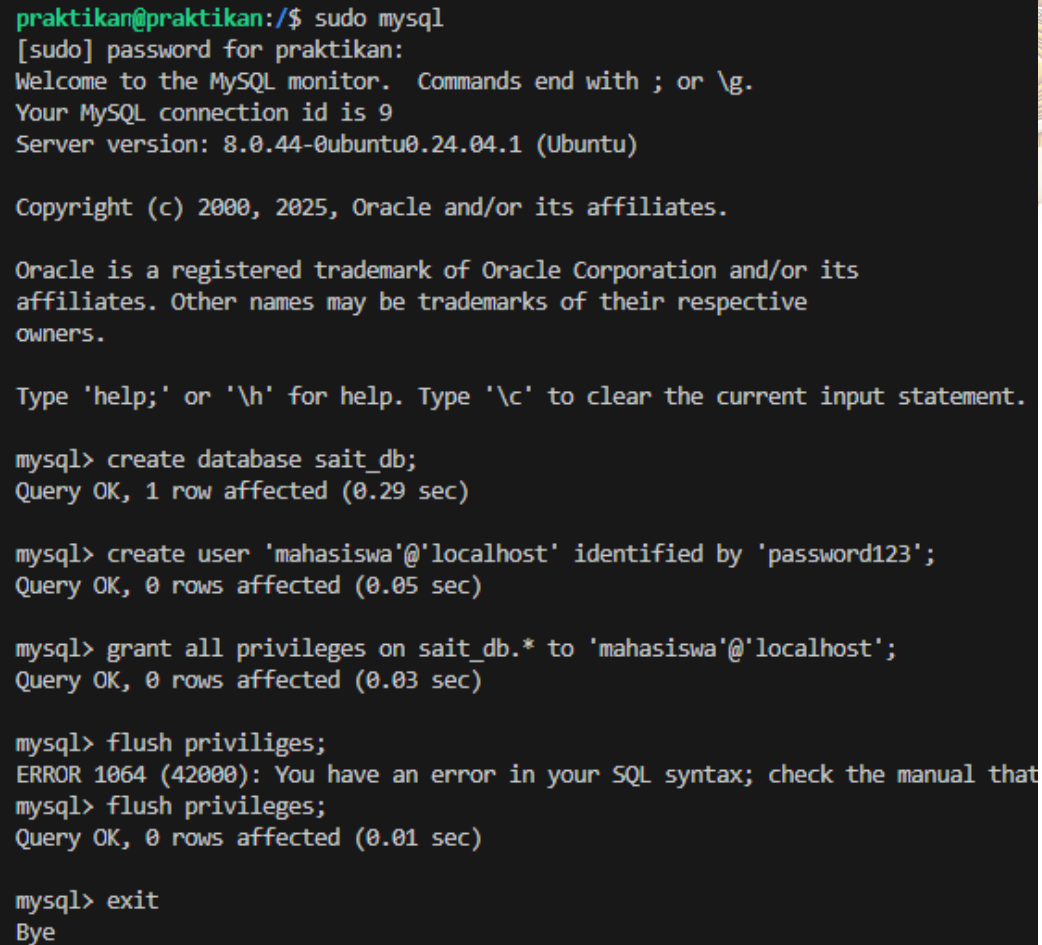
Gambar 2.4.7 Tampilan berhasil masuk ke konsol MySQL

3. Persiapan Database dan Uji Coba Koneksi

Di sinilah bagian pentingnya. Sebelum kita melakukan uji coba koneksi, kita harus membuat tujuan koneksinya terlebih dahulu.

A. Membuat Database dan User Baru di MySQL

Masuk kembali ke MySQL dengan perintah `sudo mysql` yang telah kita pelajari sebelumnya. Kita harus membuat user khusus untuk aplikasi web yang akan kita buat, karena menggunakan user root di script PHP seringkali gagal karena alasan keamanan. Jalankan perintah seperti Gambar 2.4.8 untuk membuat user dan database baru.



```
praktikan@praktikan:/$ sudo mysql
[sudo] password for praktikan:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.44-0ubuntu0.24.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database sait_db;
Query OK, 1 row affected (0.29 sec)

mysql> create user 'mahasiswa'@'localhost' identified by 'password123';
Query OK, 0 rows affected (0.05 sec)

mysql> grant all privileges on sait_db.* to 'mahasiswa'@'localhost';
Query OK, 0 rows affected (0.03 sec)

mysql> flush priviliges;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that
mysql> flush privileges;
Query OK, 0 rows affected (0.01 sec)

mysql> exit
Bye
```

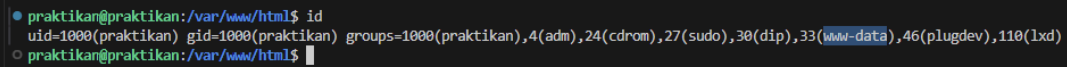
Gambar 2.4.8 Screenshot terminal saat mengetik perintah SQL pembuatan user dan database

B. Membuka Folder Server di VS Code

Selanjutnya Agar kita bisa membuat file PHP dengan mudah, kita perlu membuka folder web server di VS Code. Pertama, oleh karena secara default `/var/www/html` dimiliki oleh group `www-data`, hal ini membuat user biasa tidak memiliki izin akses untuk mengubah-ubah file tersebut kecuali tanpa `sudo`. Untuk mengatasi hal tersebut kita dapat menambahkan user ke dalam grup `www-data` dengan cara berikut.

```
sudo usermod -aG www-data $USER
sudo chown -R www-data:www-data /var/www/html
sudo chmod -R 775 /var/www/html
```

Untuk memastikan bahwa izin telah dijalankan oleh sistem, jalankan perintah “id” seperti berikut. Jika terdapat, www-data maka perizinan sistem berhasil diterapkan.

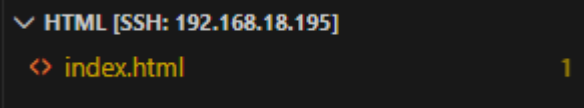


```
praktikan@praktikan:/var/www/html$ id
uid=1000(praktikan) gid=1000(praktikan) groups=1000(praktikan),4(adm),24(cdrom),27(sudo),30(dip),33(www-data),46(plugdev),110(lxd)
praktikan@praktikan:/var/www/html$
```

Gambar 2.4.9 Konfirmasi grup www-data di user saat ini

Jika belum ada grup www-data di user dalam kurun waktu 1 menit, paksa perubahan dengan cara berikut. Jalankan perintah “sudo reboot” untuk merestart ulang VPS dan tutup VS Code. Setelah 1 menit lebih, buka ulang VS Code dan coba perintah “id” kembali, pastikan terdapat grup www-data.

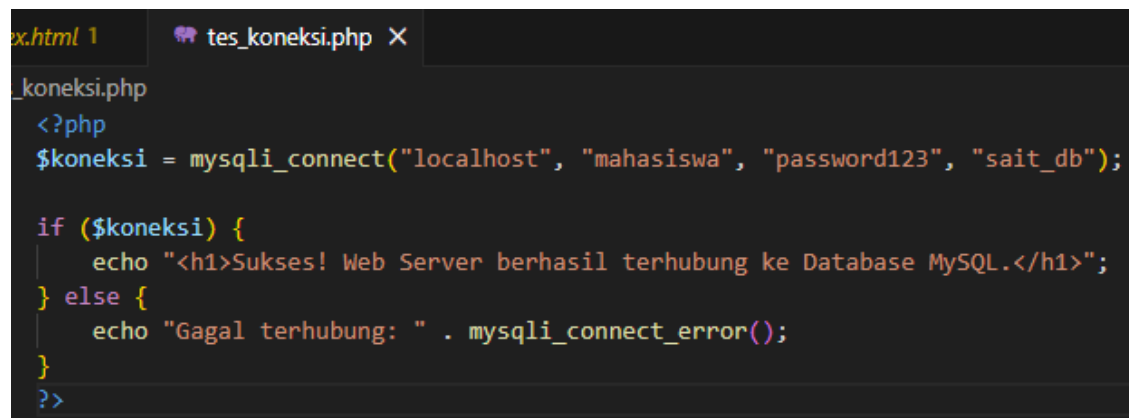
Kedua, pada menu VS Code, klik File > Open Folder. Di kolom yang muncul, ketik /var/www/html/ lalu klik OK. Masukkan password ubuntu jika diminta. Sekarang sidebar kiri VS Code menampilkan isi folder server.



Gambar 2.4.10 Tampilan File Explorer VS Code yang membuka folder /var/www/htm

C. Membuat File Tes Koneksi

Selanjutnya kita akan membuat file PHP untuk melakukan tes koneksi dengan klik kanan di sidebar kiri, pilih New File. Beri nama tes_koneksi.php. Ketikkan kode sesuai dengan Gambar 2.4.11. Perhatikan bahwa kita menggunakan localhost karena skrip PHP ini berada di dalam mesin yang sama dengan MySQL.



```
tes_koneksi.php
<?php
$koneksi = mysqli_connect("localhost", "mahasiswa", "password123", "sakit_db");

if ($koneksi) {
    echo "<h1>Sukses! Web Server berhasil terhubung ke Database MySQL.</h1>";
} else {
    echo "Gagal terhubung: " . mysqli_connect_error();
}
?>
```

Gambar 2.4.11 Tampilan koding file tes_koneksi.php di VS Code



D. Melakukan Pengujian di Browser

Selanjutnya kita akan melakukan pengujian di browser komputer host kita dengan mengakses alamat `http://[IP_UBUNTU_ANDA]/tes_koneksi.php`. Jika muncul tulisan "Sukses! Web Server berhasil terhubung...", maka web server dan database server sudah saling terhubung.



Gambar 2.4.12 Tampilan browser yang menampilkan pesan sukses koneksi

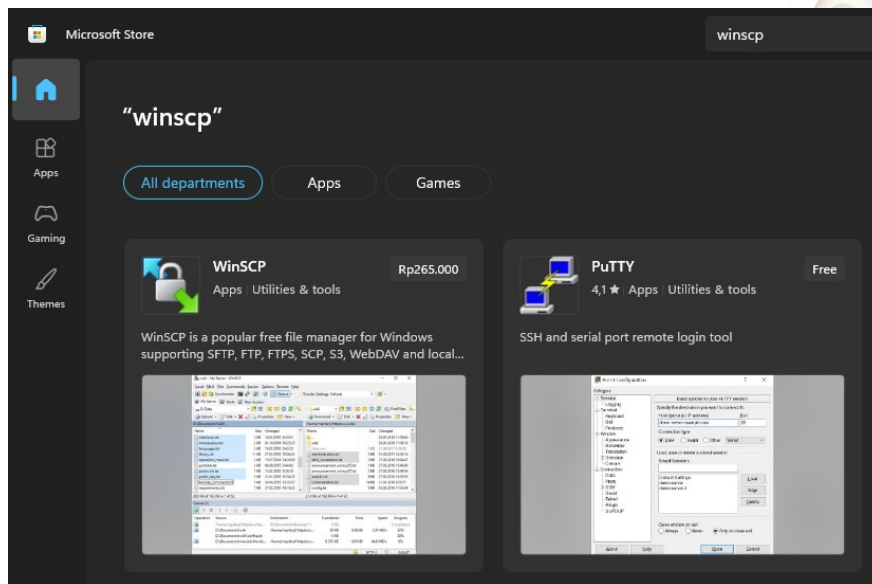
Alternatif SSH dan FTP

Selain menggunakan VS Code untuk SSH dan FTP, terdapat berbagai solusi aplikasi untuk melakukan hal tersebut, salah satunya untuk aplikasi Windows adalah PuTTY dan WinSCP.

- PuTTY merupakan aplikasi open-source yang digunakan sebagai terminal emulator untuk melakukan koneksi jarak jauh melalui protokol SSH. Hal ini memungkinkan untuk melakukan kontrol server via perintah (command line)
- WinSCP adalah aplikasi klien SFTP dan FTP berbasis grafis yang memudahkan proses transfer file secara aman antara komputer lokal dan server jarak jauh.

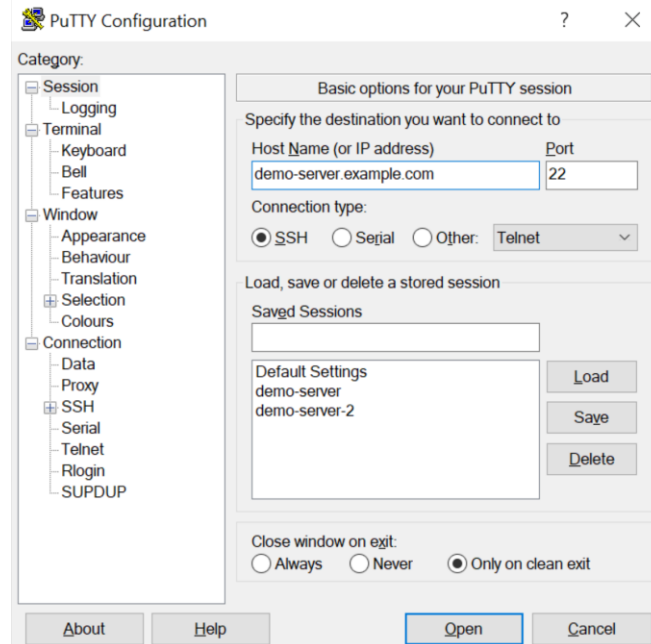
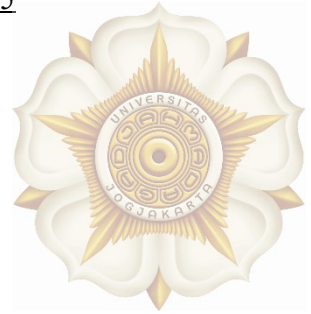
Kedua program ini dapat diunduh secara praktis melalui Microsoft Store untuk memastikan aplikasi yang diunduh adalah aplikasi yang aman. Kedua aplikasi di Microsoft Store akan terlihat seperti Gambar 2.4.13 WinSCP dan PuTTY di Microsoft Store. Jika sistem operasi Windows yang Anda gunakan adalah sistem operasi Windows lama yang belum support Microsoft Store, pengunduhan WinSCP dan PuTTY dapat dilakukan di website. Sebagai contoh dengan link berikut:

- PuTTY: <https://www.chiark.greenend.org.uk/~sgtatham/putty/latest.html>
- WinSCP: <https://winscp.net/eng/download.php>



Gambar 2.4.13 WinSCP dan PuTTY di Microsoft Store

Untuk melakukan koneksi SSH dengan menggunakan PuTTY, lakukan langkah-langkah berikut. Disetiap versi PuTTY akan terdapat sedikit perbedaan, namun memiliki konsep dasar yang sama. Ketika membuka PuTTY, kita akan diperlihatkan dengan dialog konfigurasi. Di konfigurasi ini, PuTTY meminta informasi IP address (dapat berupa DNS/hostname), Port, dan di versi terbaru ada yang meminta username dan password (jika tidak ada username dan password akan diminta ketika melakukan koneksi/open). Secara default, port dari SSH adalah 22, ketika kita menggunakan VS Code kita tidak menuliskan port karena perintah tersebut juga akan menerima port 22 sebagai parameter default. Isi dialog konfigurasi seperti dibawah ini, jangan lupa ubah hostname sesuai dengan IP yang digunakan!



Gambar 2.4.14 Konfigurasi Koneksi PuTTY

Jika ada permintaan fingerprint, tekan dialog ACC untuk melanjutkan. Setelah berhasil, kita bisa menjalankan perintah dari jarak jauh seperti ketika menggunakan VS Code Terminal.

```
praktikan@praktikan: ~
login as: praktikan
praktikan@192.168.18.195's password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-90-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Feb 18 03:21:46 PM UTC 2026

System load: 0.55          Processes:            109
Usage of /:  61.2% of 11.21GB Users logged in:          0
Memory usage: 28%          IPv4 address for enp0s3: 192.168.18.195
Swap usage:  0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

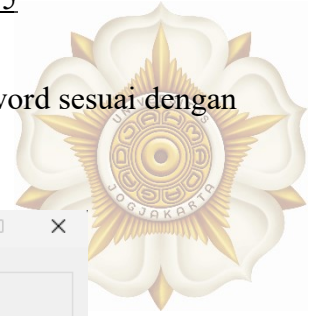
Expanded Security Maintenance for Applications is not enabled.

32 updates can be applied immediately.
1 of these updates is a standard security update.
```

Gambar 2.4.15 PuTTY berhasil terhubung

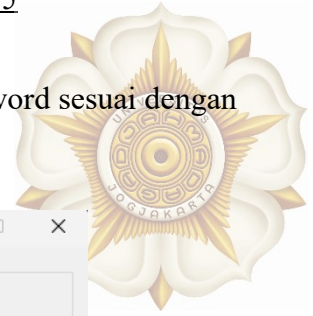
Untuk melakukan FTP dengan WinSCP, buka aplikasi WinSCP yang telah terinstall. Perlu diketahui juga, untuk beberapa versi terbaru memungkinkan untuk memiliki tampilan yang

word sesuai dengan



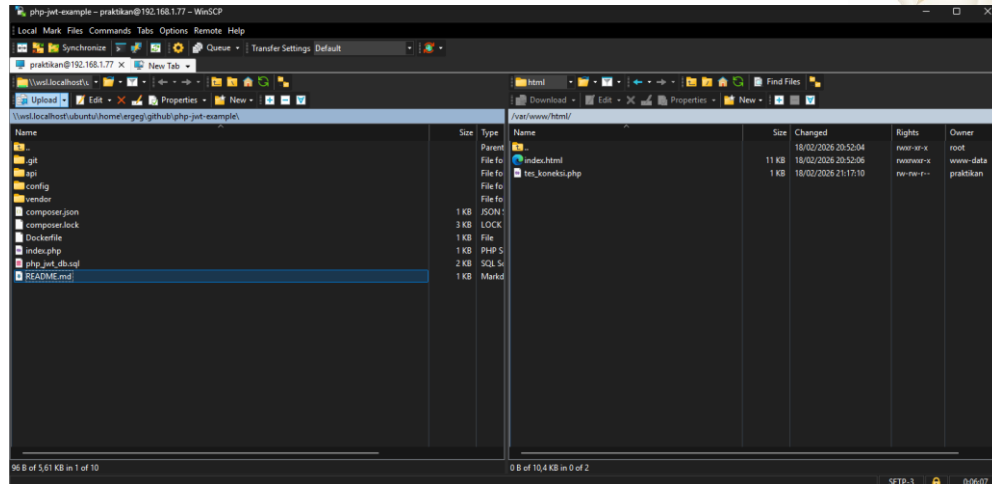
word sesuai dengan

word sesuai dengan

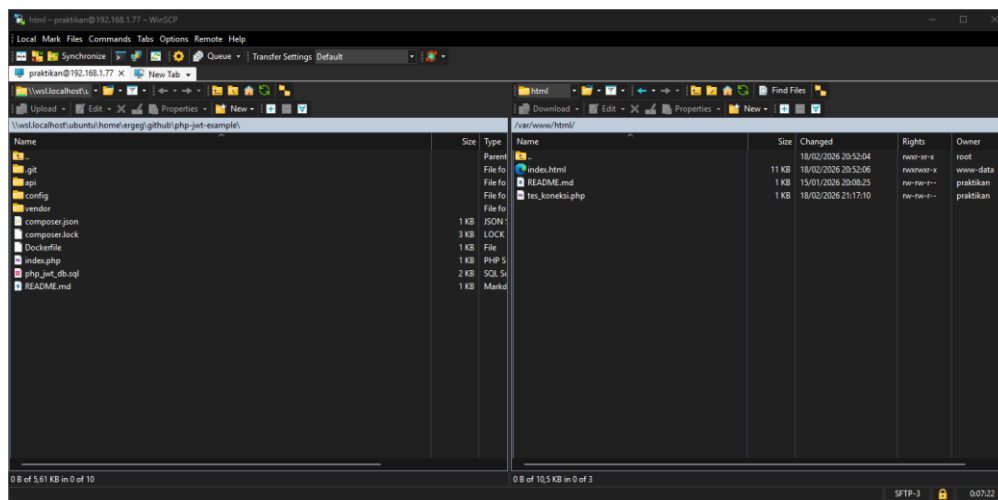


word sesuai dengan

Saat ini, kita akan mencoba menambahkan file dengan menggunakan WinSCP. Klik file atau folder yang ingin di upload. Klik tombol upload untuk mengupload file atau folder ke direktori target (Ubuntu) yang sedang dibuka (di kolom bagian kanan).



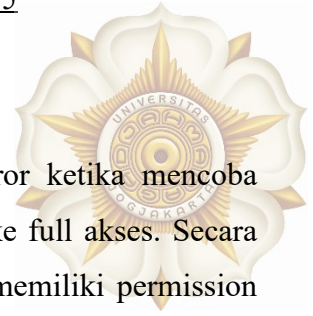
Gambar 2.4.18 Upload File via WinSCP



Gambar 2.4.19 File berhasil terupload via WinSCP

Alternatif Permission Error

Menghadapi permission error tentu akan cukup mengesalkan. Sebagai contoh, jika penambahan user grup www-data untuk kasus apache2 tidak berhasil ke user yang sedang kita gunakan, kita masih memiliki alternatif cara cadangan. Cara ini harus digunakan dengan penuh kehati-hatian, penggunaan cara ini akan membuka risiko keamanan ke ranah publik.



Jika ketika kita menggunakan SSH atau SFTP dan mengalami error ketika mencoba melakukan penulisan, kita dapat melakukan pengubahan permission ke full akses. Secara default, akses untuk folder apache /var/www/html adalah 775 yang memiliki permission seperti Gambar 2.4.20. Permission ini meniadakan akses write untuk pengguna publik. Akses secara eksternal seperti SSH atau SFTP dengan VS Code atau PuTTY dan WinSCP dianggap sebagai akses publik. Salah satu cara untuk mengatasi hal tersebut adalah membuka folder/file tersebut ke seluruh akses (777 – yang memiliki arti seperti Gambar 2.4.21). Anda juga dapat melihat kode akses di link berikut <https://chmod-calculator.com/>

Chmod Calculator

An awesome Chmod Calculator to convert Linux file permissions between different formats.

Owner	Group	Public
Read ☒	Read ☒	Read ☒
Write ☒	Write ☒	Write ☐
Execute ☒	Execute ☒	Execute ☒

Linux Permissions:

Gambar 2.4.20 Permission 775

Chmod Calculator

An awesome Chmod Calculator to convert Linux file permissions between different formats.

Owner	Group	Public
Read ☒	Read ☒	Read ☒
Write ☒	Write ☒	Write ☒
Execute ☒	Execute ☒	Execute ☒

Linux Permissions:

Gambar 2.4.21 Full Access Permission 777

Untuk melakukan perubahan akses folder, lakukan dengan perintah berikut.

```
sudo chmod -R 777 target_file_nya
```

Sebagai contoh untuk target file folder apache2 (/var/www/html) adalah perintah

```
sudo chmod -R 777 /var/www/html
```



2.5. TUGAS & ANALISIS

1. Lakukan instalasi ekstensi Remote SSH pada VS Code dan hubungkan ke Ubuntu Server Anda.
2. Lakukan instalasi paket LAMP (Apache, MySQL, PHP) sesuai langkah percobaan.
3. Buat user database baru dan ubah hak akses folder /var/www/html.
4. Buat file tes_koneksi.php dan buktikan bahwa koneksi berhasil dengan membukanya di browser.

2.6. REFERENSI

- [1].Microsoft. (n.d.). Remote Development using SSH. Visual Studio Code Documentation. Retrieved from <https://code.visualstudio.com/docs/remote/ssh>
- [2].DigitalOcean. (2023). How To Install Linux, Apache, MySQL, PHP (LAMP) stack on Ubuntu. Retrieved from <https://www.digitalocean.com/community/tutorials/how-to-install-linux-apache-mysql-php-lamp-stack-on-ubuntu>
- [3].PHP.net. (n.d.). MySQLi Extension. Retrieved from <https://www.php.net/manual/en/book.mysqli.php>
- [4].Ubuntu Documentation. (n.d.). File Permissions and ACLs. Retrieved from <https://ubuntu.com/server/docs/security-users-and-groups>



PERTEMUAN III

INSTALASI WEB SERVER DAN DATABASE

DALAM MESIN (KOMPUTER) YANG BERBEDA

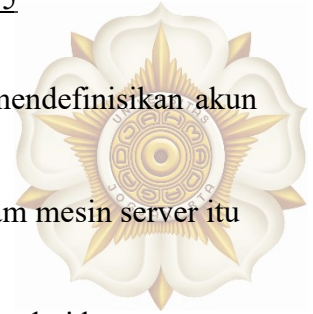
3.1. TUJUAN PEMBELAJARAN

- A. Mahasiswa mampu memahami konsep arsitektur sistem terdistribusi sederhana dengan memisahkan Web Server dan Database Server pada mesin yang berbeda.
- B. Mahasiswa mampu melakukan konfigurasi layanan MySQL pada server Linux agar dapat menerima koneksi dari jaringan luar (modifikasi bind-address).
- C. Mahasiswa mampu melakukan manajemen user dan hak akses (privileges) MySQL berbasis alamat IP (host-specific) untuk keamanan koneksi.
- D. Mahasiswa mampu mengimplementasikan koneksi basis data pada skrip PHP yang berjalan di lingkungan Windows (Host) menuju server Linux (Guest).

3.2. DASAR TEORI

Arsitektur Client-Server Terdistribusi Pada bab sebelumnya, aplikasi web dan database berjalan di satu mesin yang sama (localhost). Namun, dalam lingkungan produksi skala besar, Web Server dan Database Server seringkali dipisahkan ke dalam mesin fisik atau virtual yang berbeda. Hal ini bertujuan untuk membagi beban kerja (load balancing), meningkatkan keamanan, dan memudahkan skalabilitas. Dalam skenario ini, komputer Host (Windows) bertindak sebagai klien yang mengirimkan query, dan Ubuntu Server bertindak sebagai penyedia layanan data.

MySQL Bind-Address Secara default, instalasi MySQL memiliki konfigurasi keamanan yang ketat. Parameter bind-address pada file konfigurasi biasanya diset ke 127.0.0.1. Ini berarti MySQL hanya akan "mendengarkan" permintaan yang berasal dari dalam komputer itu sendiri (localhost). Agar MySQL dapat diakses oleh komputer lain dalam jaringan, nilai ini harus diubah menjadi 0.0.0.0 (menerima koneksi dari semua antarmuka jaringan) atau alamat IP spesifik dari server tersebut.



Manajemen User MySQL Berbasis Host Sistem keamanan MySQL mendefinisikan akun pengguna berdasarkan nama user DAN host asal koneksi.

- 'praktikan'@'localhost': User ini hanya bisa login jika berada di dalam mesin server itu sendiri.
- 'praktikan'@'192.168.1.10': User ini hanya bisa login jika mengakses dari komputer dengan IP 192.168.1.10.
- 'praktikan'@'%': User ini bisa login dari IP mana saja (kurang aman). Pada praktikum ini, kita menggunakan pendekatan kedua untuk memastikan hanya komputer Host kita yang diizinkan mengakses database.

3.3. ALAT DAN BAHAN

3.3.1. Perangkat Keras

- Komputer / Laptop
- Virtual Private Server

3.3.2. Perangkat Lunak

- VS Code (dengan ekstensi Remote SSH).
- Laragon atau XAMPP (terinstal di Windows sebagai Web Server).
- Browser (Chrome/Edge/Firefox).
- MySQL Server (di Ubuntu).

3.4. LANGKAH PERCOBAAN

1. Modifikasi MySQL Agar Bisa Diakses dari Luar

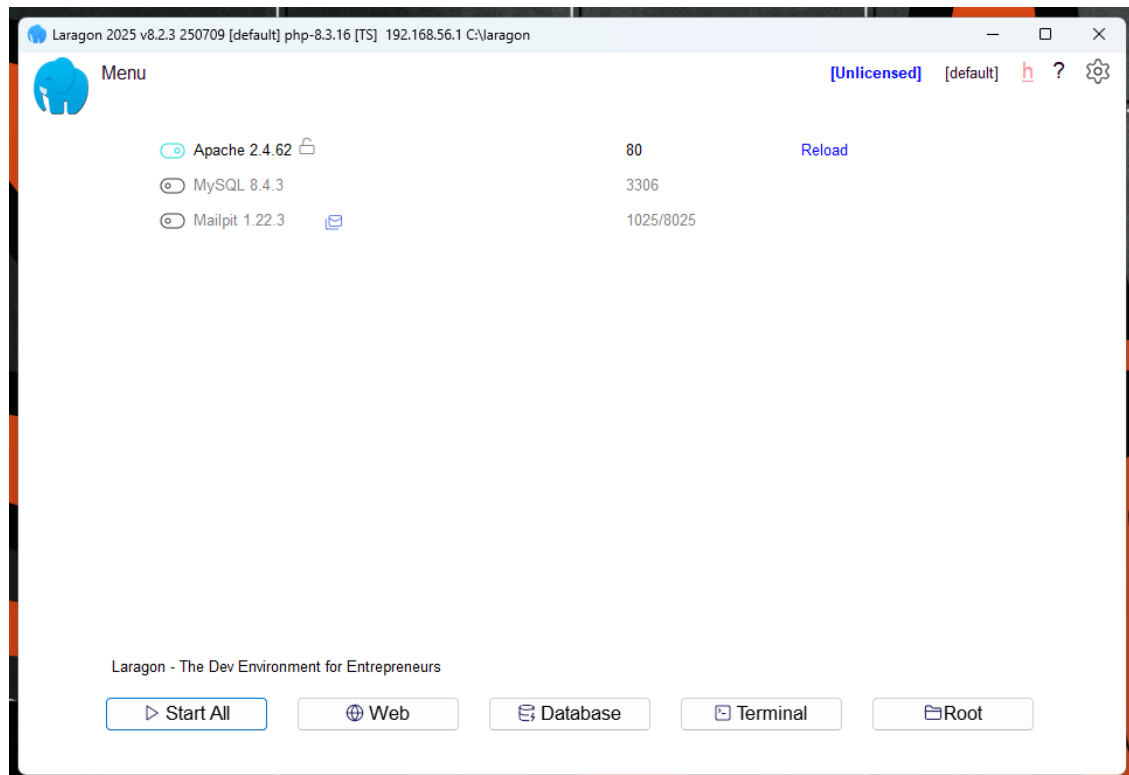
Pada bab sebelumnya, Web Server dan Database Server berjalan di satu mesin yang sama (Ubuntu). Sekarang, kita akan memisahkannya untuk mensimulasikan sistem terdistribusi yang sesungguhnya. Kita akan menggunakan komputer host (Windows) sebagai Web Server (menggunakan aplikasi Laragon atau XAMPP), sedangkan Ubuntu Server hanya akan bertindak sebagai Database Server.

Pertama, pastikan aplikasi Laragon (atau XAMPP) sudah terinstal di komputer host. Jika belum, unduh dan instal terlebih dahulu melalui link berikut.

<https://laragon.org/download>



Setelah aplikasi selesai diinstall, buka dan klik kanan pada aplikasi laragon lalu pilih apache dan klik start.



Gambar 3.4.1 Tampilan awal Laragon dengan status Apache berjalan

Selanjutnya, kita perlu mengatur MySQL di Ubuntu agar mau menerima koneksi dari komputer luar. Buka kembali VS Code yang sudah terhubung ke Ubuntu Server (seperti di halaman 22). Buka terminal di dalam VS Code, lalu edit file konfigurasi MySQL dengan perintah berikut.

```
sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf
```

Cari baris bertuliskan bind-address. Secara default, nilainya adalah 127.0.0.1. Ubah nilai tersebut menjadi 0.0.0.0. Perubahan ini penting agar MySQL "mendengarkan" permintaan akses dari IP manapun, bukan hanya dari dirinya sendiri.



```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL
GNU nano 7.2
# If MySQL is running as a replication slave, this should be
# changed. Ref https://dev.mysql.com/doc/refman/8.0/en/replication.html
# tmpdir                    = /tmp
#
# Instead of skip-networking the default is now to listen on
# localhost which is more compatible and is not less secure
bind-address                = 0.0.0.0
mysqlx-bind-address         = 127.0.0.1
#
# * Fine Tuning
#
key_buffer_size             = 16M
^G Help      ^O Write Out  ^W Where Is
^X Exit      ^R Read File  ^_ Replace
    
```

Gambar 3.4.2 Mengubah konfigurasi bind-address menjadi 0.0.0.0 via terminal VS Code

Simpan file dengan menekan Ctrl + X, lalu Y, dan Enter. Terakhir, restart layanan MySQL agar perubahan diterapkan dengan perintah berikut.

```
sudo service mysql restart
```

2. Membuat User Baru pada MySQL

Agar komputer host diizinkan mengakses database, kita perlu mendaftarkan alamat IP komputer host ke dalam MySQL Ubuntu. Cek alamat IP komputer host terlebih dahulu. Buka CMD (Command Prompt) di Windows lalu ketik ipconfig. Cari alamat IPv4 pada adapter yang terhubung dengan VirtualBox (biasanya adapter WiFi atau VirtualBox Host-Only Network).

Ethernet adapter Ethernet:

```
Connection-specific DNS Suffix . :  
Link-local IPv6 Address . . . . . : fe80::b05:8651:3af8:ab16%11  
IPv4 Address. . . . . : 192.168.18.3  
Subnet Mask . . . . . : 255.255.255.0  
Default Gateway . . . . . : 192.168.18.1
```

Gambar 3.4.3 Mengecek IP Address Host melalui CMD Windows

Setelah mendapatkan IP Host, kembali ke terminal VS Code dan masuk ke konsol MySQL dengan perintah `sudo mysql`. Kita akan membuat user khusus yang hanya bisa diakses dari IP Host tersebut. Jalankan perintah SQL sesuai dengan Gambar 3.4.4 dan pastikan ip yang dimasukkan sesuai dengan ip address dari komputer host.

```
● praktikan@praktikan:/var/www/html$ sudo mysql  
Welcome to the MySQL monitor.  Commands end with ; or \g.  
Your MySQL connection id is 8  
Server version: 8.0.44-0ubuntu0.24.04.1 (Ubuntu)  
  
Copyright (c) 2000, 2025, Oracle and/or its affiliates.  
  
Oracle is a registered trademark of Oracle Corporation and/or its  
affiliates. Other names may be trademarks of their respective  
owners.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
mysql> CREATE USER 'praktikan_host'@'192.168.18.3' IDENTIFIED BY '123123123';  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> GRANT ALL PRIVILEGES ON *.* TO 'praktikan_host'@'192.168.18.3';  
Query OK, 0 rows affected (0.02 sec)  
  
mysql> FLUSH PRIVILEGES;  
Query OK, 0 rows affected (0.02 sec)  
  
mysql> EXIT;  
Bye
```

Gambar 3.4.4 Perintah SQL pembuat user baru di terminal VS Code

Perintah di atas berfungsi membuat user baru dan memberikan hak akses penuh (privileges) agar user tersebut bisa membaca dan mengubah data dari komputer host.



3. Menghubungkan Website PHP pada Host ke MySQL

Langkah terakhir adalah pengujian koneksi. Buka File Explorer di Windows.

Masuk ke folder penyimpanan web Laragon (secara default ada di C:\laragon\www\)\ atau XAMPP (secara default ada di C:\xampp\htdocs\). Buat folder baru, misalnya praktikum_psait dan di dalam folder tersebut, buat file baru bernama tes_koneksi.php. Isi file tersebut dengan skrip koneksi PHP seperti Gambar 3.4.5. **PENTING:** Pada bagian \$host atau \$servername, isikan IP Address Ubuntu Server, bukan localhost. Gunakan username dan password yang baru saja dibuat di langkah ke-2 tadi.

```

tes_koneksi.php X
praktikum_psait > tes_koneksi.php > ...
1  <?php
2  $koneksi = mysqli_connect(["192.168.18.195", "praktikan_host", "123123123", "sait_db"]);
3
4  if ($koneksi) {
5      echo "<h1>Sukses! Web Server berhasil terhubung ke Database MySQL.</h1>";
6  } else {
7      echo "Gagal terhubung: " . mysqli_connect_error();
8  }
9  ?>

```

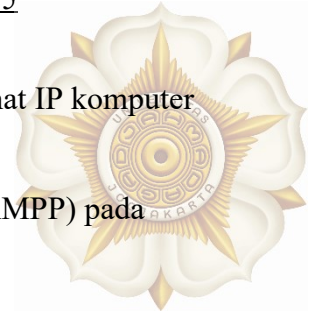
Gambar 3.4.5 Kode program tes_koneksi.php (dibuat di Windows)

Simpan file tersebut, lalu buka browser di komputer host. Akses alamat http://localhost/praktikum_psait/tes_koneksi.php. Jika layar menampilkan pesan "Koneksi Berhasil", berarti Web Server di Windows telah berhasil terhubung dengan Database Server di Ubuntu.

Gambar 3.4.6 Tampilan browser yang menyatakan koneksi sukses

3.5. TUGAS & ANALISIS

1. Pastikan layanan Apache pada Laragon/XAMPP di Windows sudah berjalan.
2. Lakukan konfigurasi mysqld.cnf di Ubuntu Server untuk mengubah bind-address.



3. Buat user baru di MySQL Ubuntu dengan spesifikasi host sesuai alamat IP komputer Windows Anda.
4. Buat skrip tes_koneksi.php di folder www (Laragon) atau htdocs (XAMPP) pada Windows, lalu uji koneksinya melalui browser.

3.6. REFERENSI

- [1].MySQL Documentation. (n.d.). Using System Variables: bind_address. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/server-system-variables.html#sysvar_bind_address
- [2].MySQL Documentation. (n.d.). Account Names and Password Management. Retrieved from <https://dev.mysql.com/doc/refman/8.0/en/account-names.html>
- [3].Laragon. (n.d.). Getting Started. Retrieved from <https://laragon.org/docs/>
- [4].W3Schools. (n.d.). PHP Connect to MySQL. Retrieved from https://www.w3schools.com/php/php_mysql_connect.asp



PERTEMUAN IV

PENGUNAAN REST API DI UBUNTU SERVER

4.1. TUJUAN PEMBELAJARAN

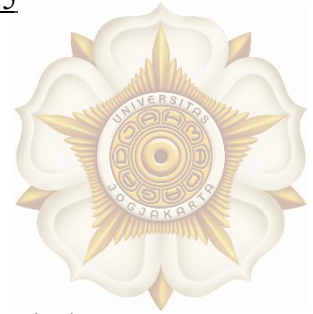
1. Mahasiswa mampu memahami arsitektur Client-Server dengan memisahkan sisi Backend (API di Ubuntu Server) dan Frontend (Aplikasi Web di Komputer Host).
2. Mahasiswa mampu melakukan deployment kode sumber REST API ke server Linux menggunakan fitur transfer file pada VS Code Remote SSH.
3. Mahasiswa mampu melakukan impor database dari file SQL ke dalam server MySQL melalui terminal.
4. Mahasiswa mampu melakukan pengujian fungsionalitas REST API (GET, POST, DELETE) menggunakan aplikasi Postman.
5. Mahasiswa mampu mengintegrasikan aplikasi Frontend berbasis PHP di sisi klien agar dapat berkomunikasi dan bertukar data dengan REST API yang berada di server.

4.2. DASAR TEORI

REST API (Representational State Transfer) adalah gaya arsitektur perangkat lunak yang menyediakan standar komunikasi antara sistem komputer yang berbeda melalui protokol HTTP. Dalam praktikum ini, kita membangun REST API yang bertugas melayani permintaan data. API ini tidak memiliki tampilan antarmuka (UI), melainkan hanya memberikan respon berupa data mentah, biasanya dalam format JSON (JavaScript Object Notation).

JSON (JavaScript Object Notation) JSON adalah format pertukaran data yang ringan, mudah dibaca oleh manusia, dan mudah diuraikan oleh mesin. Ketika aplikasi Frontend meminta daftar mahasiswa, Server akan mengirimkan data teks berformat JSON, yang kemudian oleh Frontend akan diubah menjadi tampilan tabel yang rapi.

Postman Postman adalah aplikasi populer yang digunakan oleh pengembang untuk menguji API. Sebelum API dihubungkan ke aplikasi Frontend yang sesungguhnya, pengembang wajib memastikan bahwa setiap fungsi (Endpoint) berjalan benar.



- GET: Meminta data dari server (Menampilkan mahasiswa).
- POST: Mengirim data baru ke server (Menambah mahasiswa).
- DELETE: Menghapus data di server.

Arsitektur Frontend-Backend Terpisah Pada bab ini, kita menerapkan pemisahan tugas yang jelas:

- Backend (Ubuntu Server): Fokus pada logika bisnis dan akses database.
- Frontend (Windows Host): Fokus pada antarmuka pengguna (UI/UX) dan memanggil API. Keuntungan arsitektur ini adalah satu Backend bisa melayani berbagai jenis Frontend (Web, Mobile Android, iOS) secara bersamaan.

4.3. ALAT DAN BAHAN

4.3.1. Perangkat Keras

- Komputer / Laptop
- Virtual Private Server

4.3.2. Perangkat Lunak

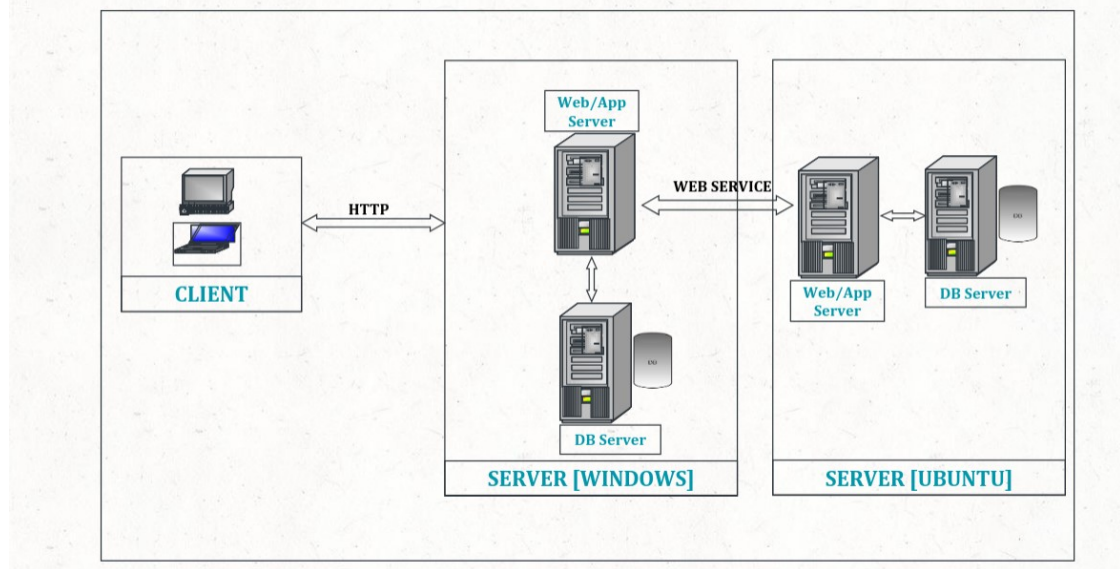
- Postman (Diinstal di Windows/Host).
- VS Code + Ekstensi Remote SSH.
- Web Server Lokal (Laragon/XAMPP).
- Browser (Chrome/Edge).
- Source Code: Folder `sait_api` (Backend) dan `sait_frontend` (Frontend).

4.4. LANGKAH PERCOBAAN

1. Deploy REST API ke Ubuntu Server

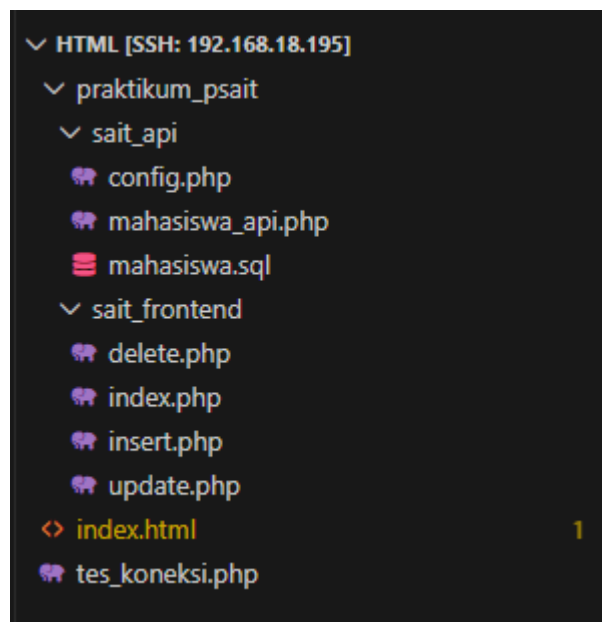
Pada bab ini, kita akan menerapkan arsitektur Client-Server. Sisi Server (Backend) akan diletakkan di Ubuntu menggunakan REST API, sedangkan sisi Client (Frontend) akan berjalan di Komputer Host (Windows).

KASUS - 3

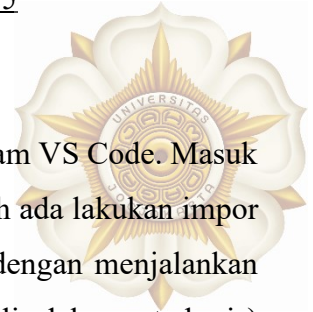


Gambar 4.4.1 Client side to side

Langkah pertama, pastikan Anda sudah mengunduh folder kode sumber (source code) API yang disediakan. Buka VS Code yang sudah terhubung ke Ubuntu Server (Remote SSH). Pada sidebar kiri (File Explorer) di VS Code, buka direktori `/var/www/html/`. Cukup copy dan paste folder `sait_api` dari Windows Explorer Anda langsung ke dalam sidebar VS Code tersebut.



Gambar 4.4.2 Proses drag-and-drop folder API ke sidebar VS Code



Selanjutnya, kita perlu menyiapkan database. Buka Terminal di dalam VS Code. Masuk ke MySQL dan buat database baru dengan nama `sait_db`. Jika sudah ada lakukan impor tabel menggunakan file SQL yang ada di dalam folder API tadi dengan menjalankan perintah berikut di terminal di luar konsol MySQL (perintah ini ditulis dalam satu baris)

```
sudo mysql -u root -p sait_db <
/var/www/html/praktikum_psait/sait_api/mahasiswa.sql
```

Verifikasi hasil impor dengan masuk ke konsol MySQL (`sudo mysql`) dan mengecek isi tabelnya. Jika muncul tabel mahasiswa beserta isinya, berarti database berhasil disiapkan.

```
● praktikan@praktikan:/var/www/html$ sudo mysql -u root -p sait_db < /var/www/html/praktikum_psait/sait_api/mahasiswa.sql
Enter password:
○ praktikan@praktikan:/var/www/html$ sudo mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 14
Server version: 8.0.44-0ubuntu0.24.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use sait_db
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

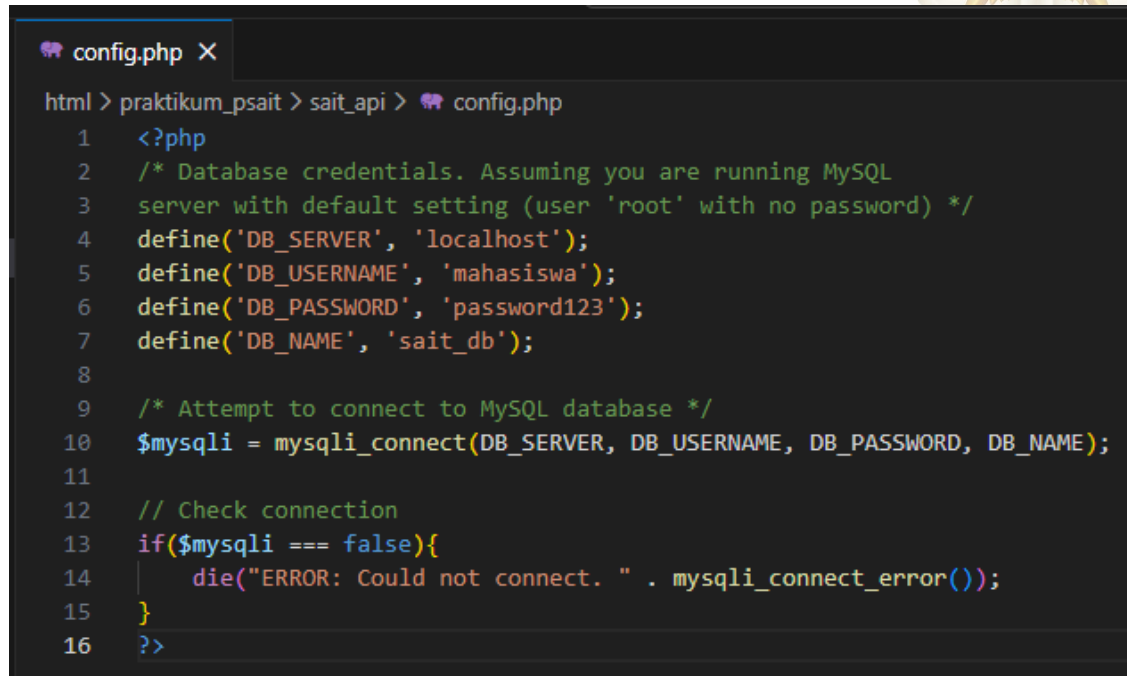
Database changed
mysql> show tables;
+-----+
| Tables_in_sait_db |
+-----+
| mahasiswa         |
+-----+
1 row in set (0.00 sec)

mysql> select * from mahasiswa
-> ;
+-----+-----+-----+
| id_mhs | nama  | alamat |
+-----+-----+-----+
| 1      | Andi  | Surabaya |
| 2      | Dwi   | Semarang |
| 3      | David | Sleman   |
| 4      | Paul  | Bantul   |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> 
```

Gambar 4.4.3 Tampilan terminal VS Code saat verifikasi database

Terakhir, kita perlu mengatur konfigurasi koneksi. Cari file config.php di sidebar VS Code, lalu klik dua kali untuk mengeditnya. Sesuaikan username dan password dengan konfigurasi MySQL di Ubuntu Server Anda, lalu simpan dengan Ctrl + S.



```
config.php X
html > praktikum_psait > sait_api > config.php
1  <?php
2  /* Database credentials. Assuming you are running MySQL
3  server with default setting (user 'root' with no password) */
4  define('DB_SERVER', 'localhost');
5  define('DB_USERNAME', 'mahasiswa');
6  define('DB_PASSWORD', 'password123');
7  define('DB_NAME', 'sait_db');
8
9  /* Attempt to connect to MySQL database */
10 $mysqli = mysqli_connect(DB_SERVER, DB_USERNAME, DB_PASSWORD, DB_NAME);
11
12 // Check connection
13 if($mysqli === false){
14     die("ERROR: Could not connect. " . mysqli_connect_error());
15 }
16 ?>
```

Gambar 4.4.4 Mengedit file config.php langsung di editor VS Code

2. Menjalankan REST API dengan Postman

Sebelum dihubungkan ke aplikasi Frontend, kita wajib menguji apakah API berjalan dengan baik secara logika. Uji coba awal dapat dilakukan melalui browser di komputer host dengan mengakses alamat berikut.

[http://\[IP_UBUNTU\]/praktikum_psait/sait_api/mahasiswa_api.php](http://[IP_UBUNTU]/praktikum_psait/sait_api/mahasiswa_api.php).

Jika berhasil, browser akan menampilkan data dalam format JSON.



```
{
  "status": 1,
  "message": "Get List Mahasiswa Successfully.",
  "data": [
    {
      "id_mhs": "1",
      "nama": "Andi",
      "alamat": "Surabaya"
    },
    {
      "id_mhs": "2",
      "nama": "Dwi",
      "alamat": "Semarang"
    },
    {
      "id_mhs": "3",
      "nama": "David",
      "alamat": "Sleman"
    },
    {
      "id_mhs": "4",
      "nama": "Paul",
      "alamat": "Bantul"
    }
  ]
}
```

Gambar 4.4.5 Tampilan data JSON saat diakses melalui browser

Untuk pengujian fungsi yang lebih kompleks (seperti menambah atau menghapus data), kita menggunakan aplikasi Postman di komputer host:

- GET: Digunakan untuk mengambil daftar mahasiswa.
- POST: Digunakan untuk menambah data mahasiswa baru.
- POST (Update): Digunakan untuk memperbarui data (sesuai logika kode API yang menggunakan method POST dengan parameter ID).

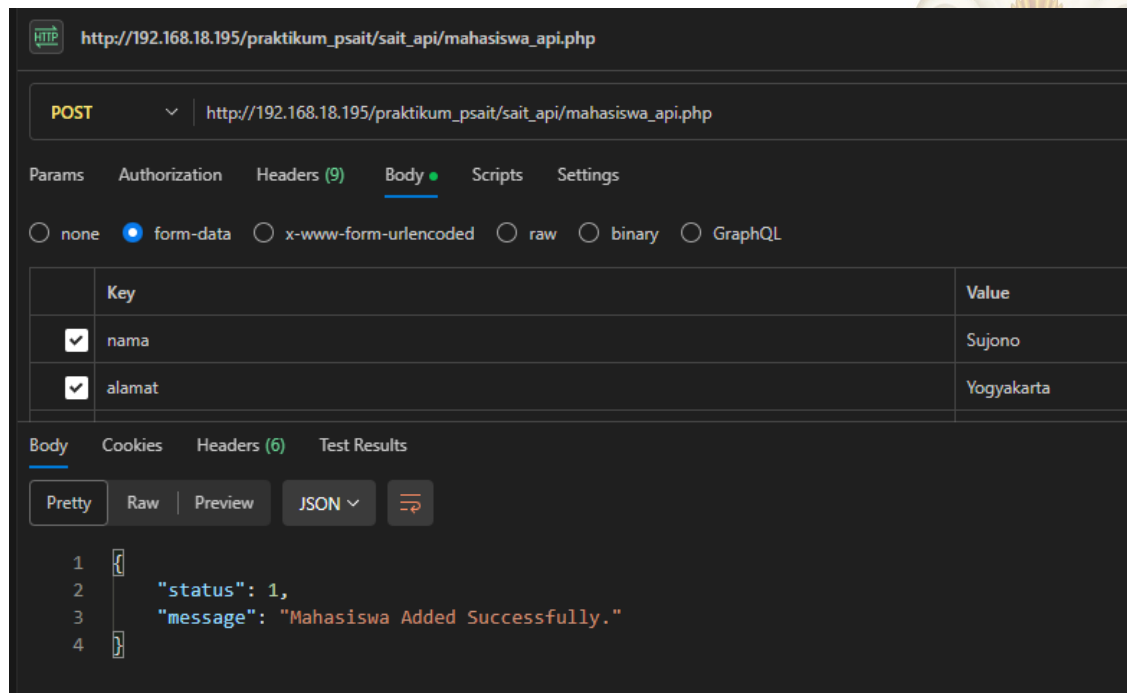
- DELETE: Digunakan untuk menghapus data.

Lakukan pengujian setiap fungsi tersebut di Postman dan pastikan responnya sukses (Status 1 atau 200 OK).

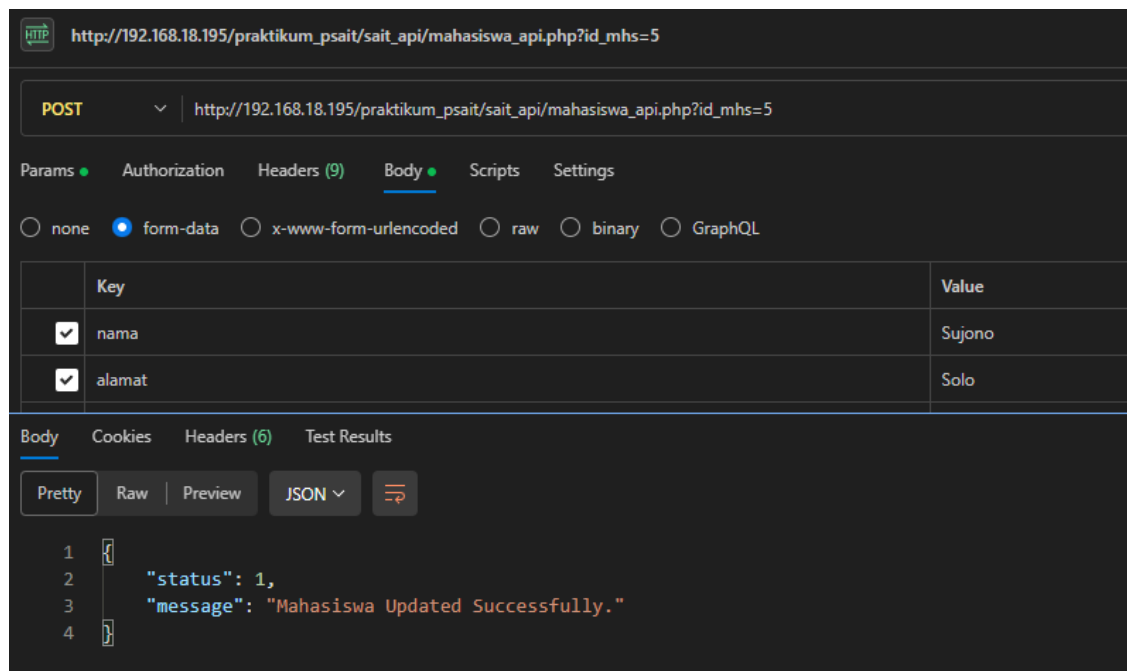


```
1 {
2   "status": 1,
3   "message": "Get List Mahasiswa Successfully.",
4   "data": [
5     {
6       "id_mhs": "1",
7       "nama": "Andi",
8       "alamat": "Surabaya"
9     },
10    {
11      "id_mhs": "2",
12      "nama": "Dwi",
13      "alamat": "Semarang"
14    },
15    {
16      "id_mhs": "3",
17      "nama": "David",
18      "alamat": "Sleman"
19    },
20    {
21      "id_mhs": "4",
22      "nama": "Paul",
23      "alamat": "Bantul"
24    }
25  ]
26 }
```

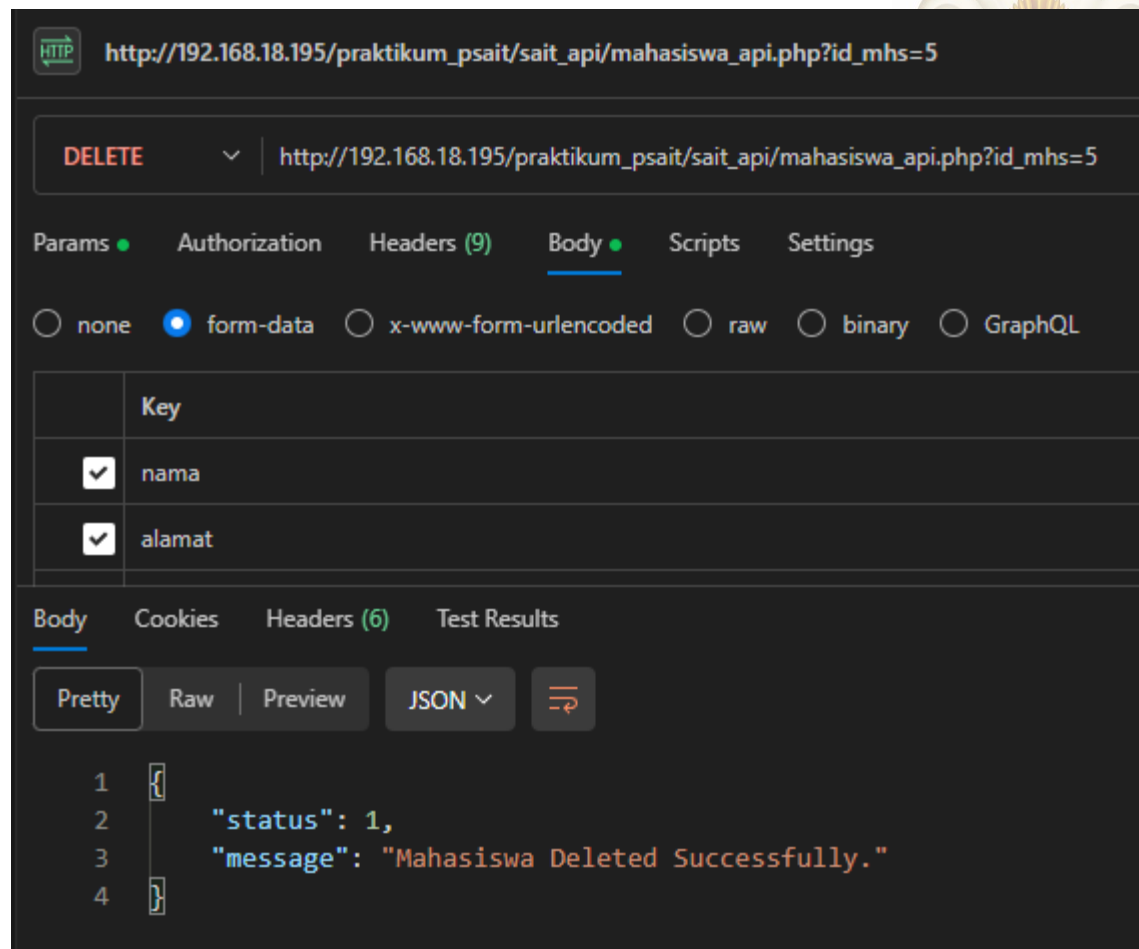
Gambar 4.4.6 Menampilkan daftar mahasiswa (GET) di Postman



Gambar 4.4.7 Menambahkan data baru (POST) di Postman



Gambar 4.4.8 Memperbarui data (Update) di Postman



Gambar 4.4.9 Menghapus data (DELETE) di Postman

3. Menjalankan REST API dengan Frontend di Komputer Host

Setelah API dipastikan berjalan lancar, langkah terakhir adalah menghubungkannya dengan antarmuka pengguna (Frontend). Siapkan folder `sait_frontend` dan letakkan di dalam folder web server komputer host (misalnya di `C:\laragon\www\` untuk Laragon atau `htdocs` untuk XAMPP). Buka folder tersebut menggunakan VS Code (jendela baru/lokal) atau text editor lainnya. Buka file-file PHP seperti `index.php`, `insert.php`, dll. Cari variabel `$api_url` dan ubah nilainya agar mengarah ke IP Address Ubuntu Server Anda. Contoh: `$api_url`, sebagai berikut:

`http://[IP_UBUNTU]/praktikum_psait/sait_api/mahasiswa_api.php`



```

<div class="overflow-x-auto">
  <table class="min-w-full bg-white border border-gray-300">
    <thead>
      <tr class="bg-gray-200 text-gray-600 uppercase text-sm leading-normal">
        <th class="py-3 px-6 text-left">ID</th>
        <th class="py-3 px-6 text-left">Nama</th>
        <th class="py-3 px-6 text-left">Alamat</th>
        <th class="py-3 px-6 text-center">Aksi</th>
      </tr>
    </thead>
    <tbody class="text-gray-600 text-sm font-light">
      <?php
        // URL API LOKAL (Pastikan API Ubuntu sudah jalan)
        $api_url = 'http://192.168.18.195/praktikum_psait/sait_api/mahasiswa_api.php';

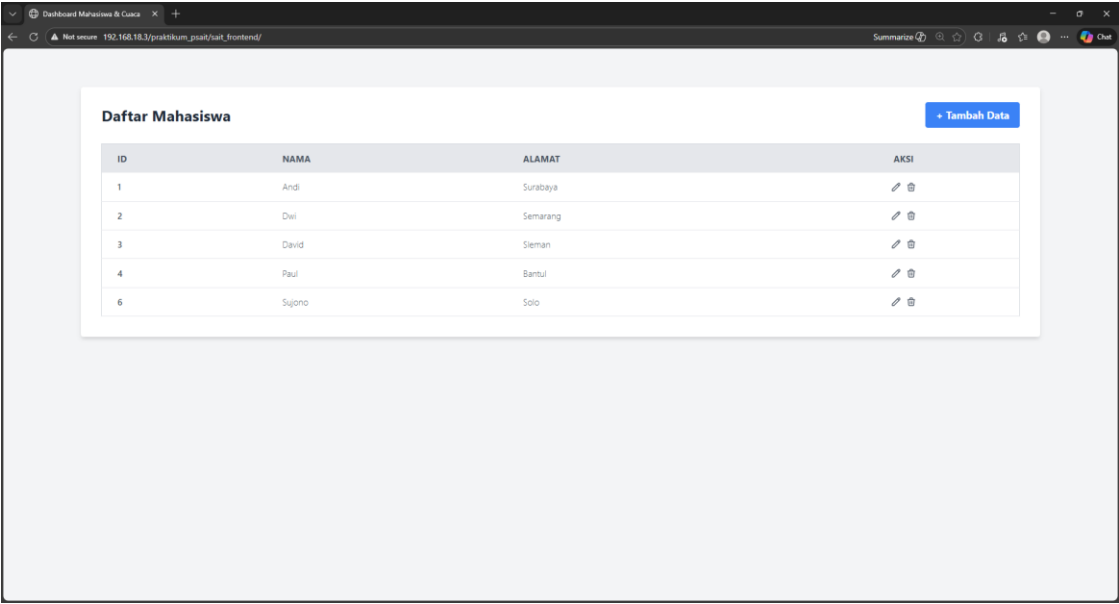
        $json_data = @file_get_contents($api_url);
        $mahasiswa = [];

        if($json_data) {
          $response = json_decode($json_data, true);
          if(isset($response['status']) && $response['status'] == 1 && isset($response['data'])) {
            $mahasiswa = $response['data'];
          }
        }
      </?php>
    </tbody>
  </table>
</div>

```

Gambar 4.4.10 Modifikasi variabel API URL pada kode program Frontend

Simpan semua perubahan. Buka browser di komputer host dan akses alamat lokal frontend Anda (misalnya: http://localhost/sait_frontend). Jika tabel data mahasiswa muncul, berarti aplikasi Frontend di Windows berhasil mengambil data dari Backend di Ubuntu Server. Coba lakukan fitur tambah, edit, dan hapus untuk memastikan integrasi berjalan sempurna.



ID	NAMA	ALAMAT	AKSI
1	Andi	Surabaya	 
2	Dwi	Semarang	 
3	David	Sieman	 
4	Paul	Bantul	 
6	Sujono	Solo	 

Gambar 4.4.11 Tampilan akhir website Frontend yang menampilkan data dari server



4.5. TUGAS & ANALISIS

1. Lakukan deployment folder `sait_api` ke direktori `/var/www/html` di Ubuntu Server dan impor database mahasiswa.sql.
2. Lakukan pengujian API menggunakan Postman untuk fungsi: Menampilkan semua data, Menambah data baru, Update data, dan Hapus data. Pastikan semua respon berstatus 200 OK atau Success.
3. Konfigurasi file pada folder `sait_frontend` di komputer Host agar variabel `$api_url` mengarah ke IP Ubuntu Server Anda.
4. Jalankan aplikasi Frontend di browser dan buktikan bahwa data yang tampil adalah data yang berasal dari Ubuntu Server.

4.6. REFERENSI

- [1]. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine. (Disertasi asli pencetus REST).
- [2]. Postman. (n.d.). Postman Learning Center. Retrieved from <https://learning.postman.com/docs/getting-started/introduction/>
- [3]. Mozilla Developer Network (MDN). (n.d.). HTTP request methods. Retrieved from <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>
- [4]. W3Schools. (n.d.). JSON - Introduction. Retrieved from https://www.w3schools.com/js/js_json_intro.asp



PERTEMUAN V

MEMBANGUN & MENGAKSES REST API DI SERVER UBUNTU V2

5.1. TUJUAN PEMBELAJARAN

1. Mahasiswa mampu memahami konsep External API (Public API) dan perbedaannya dengan Local API yang telah dibangun sebelumnya.
2. Mahasiswa mampu membaca dokumentasi teknis dari penyedia layanan API pihak ketiga (Open-Meteo) untuk menentukan parameter request yang dibutuhkan.
3. Mahasiswa mampu mengimplementasikan fungsi HTTP Request pada PHP (menggunakan `file_get_contents` atau `cURL`) untuk mengambil data dari server eksternal.
4. Mahasiswa mampu melakukan parsing (penguraian) data JSON yang kompleks dari pihak ketiga menjadi struktur data (Array/Object) yang dapat dibaca oleh aplikasi.
5. Mahasiswa mampu menyajikan data dari dua sumber berbeda (Database Lokal dan API Internet) dalam satu tampilan dashboard yang terintegrasi.

5.2. DASAR TEORI

Public / External API Berbeda dengan API yang kita bangun di Bab 4 (yang merupakan Private/Internal API), Public API adalah antarmuka pemrograman yang disediakan oleh pihak ketiga (perusahaan atau organisasi lain) untuk digunakan oleh pengembang umum. Contohnya adalah Google Maps API, Twitter API, atau dalam kasus ini, Open-Meteo. Penggunaan API eksternal memungkinkan aplikasi kita memiliki fitur canggih (seperti prakiraan cuaca, peta, atau pembayaran) tanpa kita harus membangun infrastruktur datanya dari nol.

HTTP Client pada PHP Untuk mengambil data dari API eksternal, aplikasi PHP kita bertindak sebagai Client. PHP menyediakan beberapa metode untuk melakukan HTTP Request:

- `file_get_contents()`: Cara paling sederhana untuk membaca konten dari sebuah URL (metode GET).



- cURL (Client URL Library): Pustaka yang lebih kompleks dan powerful, mendukung berbagai protokol, autentikasi, dan metode HTTP (POST, PUT, DELETE).

JSON Parsing (Decoding) Data yang dikirim oleh Open-Meteo berbentuk teks JSON. Agar bisa ditampilkan ke dalam tabel HTML menggunakan PHP, data tersebut harus diubah (decode) terlebih dahulu menjadi tipe data PHP (biasanya Array Asosiatif). Fungsi `json_decode($json_data, true)` digunakan untuk mengubah string JSON menjadi Array PHP agar elemen-elemennya bisa diakses menggunakan indeks atau key.

5.3. ALAT DAN BAHAN

5.3.1. Perangkat Keras

- Komputer / Laptop
- Virtual Private Server

5.3.2. Perangkat Lunak

- VS Code.
- Web Server Lokal (Laragon/XAMPP).
- Browser.
- Layanan: Open-Meteo API

5.4. LANGKAH PERCOBAAN

1. Mengakses REST API External

Pada bab sebelumnya, kita telah berhasil membangun dan mengakses REST API milik sendiri (Local API) yang tersimpan di Ubuntu Server. Pada bab ini, kita akan memperluas kemampuan aplikasi dengan mengambil data dari server pihak ketiga (External API). Kita akan menggunakan layanan gratis dari Open-Meteo untuk menampilkan informasi cuaca secara real-time.

Langkah pertama, buka situs dokumentasi Open-Meteo melalui browser di komputer host <https://open-meteo.com/en/docs>.

Pada halaman tersebut, gulir ke bawah hingga menemukan bagian API URL lalu gunakan API tersebut.



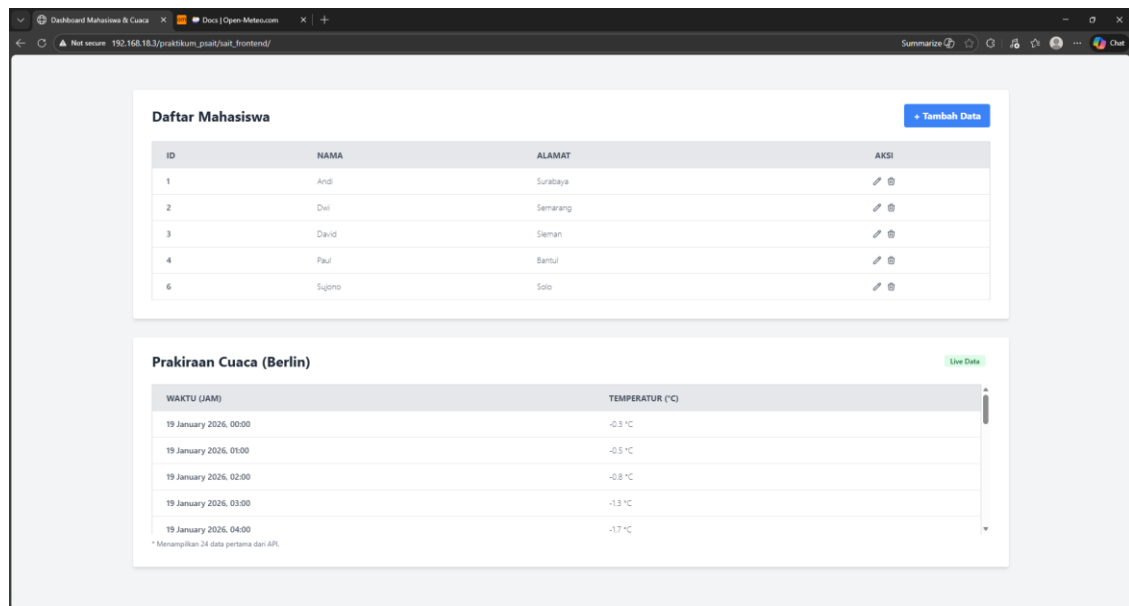
Gambar 5.4.1 Tampilan konfigurasi parameter dan URL API pada website Open-Meteo

2. Implementasi Kode pada Frontend

Langkah selanjutnya adalah memodifikasi kode program pada sisi frontend yang berada di komputer host. Silakan buka kembali folder `sait_frontend` yang tersimpan di dalam direktori web server lokal, baik itu di Laragon maupun XAMPP, menggunakan aplikasi VS Code. Buka file `index.php` dan tambahkan logika skrip PHP baru tepat di bawah tabel data mahasiswa yang sudah ada sebelumnya. Skrip ini berfungsi untuk memanggil data cuaca dengan mekanisme inisialisasi variabel URL yang berisi alamat dari Open-Meteo, kemudian mengambil data mentah berformat JSON menggunakan fungsi bawaan PHP seperti `file_get_contents()` atau pustaka `curl`. Data JSON yang diterima selanjutnya harus diubah atau di-decode menjadi Array PHP agar dapat diolah. Terakhir, lakukan perulangan atau looping untuk menampilkan data waktu dan suhu tersebut ke dalam baris-baris tabel HTML, lalu simpan perubahan file tersebut dengan menekan tombol simpan.

3. Pengujian Hasil Integrasi

Tahap terakhir dari praktikum ini adalah melakukan pengujian integrasi pada aplikasi. Buka browser di komputer host dan lakukan refresh pada halaman aplikasi frontend yang sedang berjalan. Jika integrasi berjalan dengan sukses, halaman dashboard kini akan menampilkan dua bagian data yang berasal dari sumber yang berbeda secara bersamaan. Bagian pertama adalah daftar mahasiswa yang datanya diambil dari database MySQL di Ubuntu Server melalui jaringan lokal, sedangkan bagian kedua adalah prakiraan cuaca yang datanya diambil langsung dari server Open-Meteo melalui jaringan internet.



The screenshot shows a web browser displaying a dashboard with two main sections. The top section, titled 'Daftar Mahasiswa', contains a table with student data. The bottom section, titled 'Prakiraan Cuaca (Berlin)', contains a table with weather forecast data. A 'Tambah Data' button is visible in the top right of the student list section, and a 'Live Data' button is visible in the top right of the weather forecast section.

ID	NAMA	ALAMAT	AKSI
1	Andi	Surabaya	 
2	Dwi	Semarang	 
3	David	Sleman	 
4	Paul	Bantul	 
5	Sugono	Solo	 

WAKTU (JAM)	TEMPERATUR (°C)
19 January 2026, 00:00	-0.3 °C
19 January 2026, 01:00	-0.5 °C
19 January 2026, 02:00	-0.8 °C
19 January 2026, 03:00	-1.3 °C
19 January 2026, 04:00	-1.7 °C

* Menampilkan 24 data pertama dari API.

Gambar 5.4.2 Tampilan akhir dashboard yang menampilkan Data Mahasiswa dan Data Cuaca sekaligus

5.5. TUGAS & ANALISIS

1. Buka dokumentasi Open-Meteo, atur lokasi (Latitude/Longitude) sesuai kota Anda, dan pilih parameter cuaca (misalnya: temperature dan time). Salin URL API yang dihasilkan.
2. Modifikasi file index.php pada folder `sait_frontend`. Tambahkan kode PHP untuk mengambil data dari URL Open-Meteo tersebut.
3. Lakukan decoding JSON dan tampilkan data cuaca (Waktu dan Suhu) ke dalam tabel HTML di bawah tabel Mahasiswa.
4. Pastikan halaman web menampilkan dua jenis data: Data Mahasiswa (dari Ubuntu Server) dan Data Cuaca (dari Open-Meteo) secara bersamaan.

5.6. REFERENSI



- [1]. Open-Meteo. (n.d.). Weather Forecast API Documentation. Retrieved from <https://open-meteo.com/en/docs>
- [2]. PHP Manual. (n.d.). function.json-decode. Retrieved from <https://www.php.net/manual/en/function.json-decode.php>
- [3]. PHP Manual. (n.d.). function.file-get-contents. Retrieved from <https://www.php.net/manual/en/function.file-get-contents.php>
- [4]. IBM. (n.d.). What is an API?. Retrieved from <https://www.ibm.com/topics/api>



PERTEMUAN VI

REPLIKASI DATABASE MYSQL DI UBUNTU SERVER

6.1. TUJUAN PEMBELAJARAN

A. Mahasiswa mampu melakukan replikasi database menggunakan MySQL

6.2. DASAR TEORI

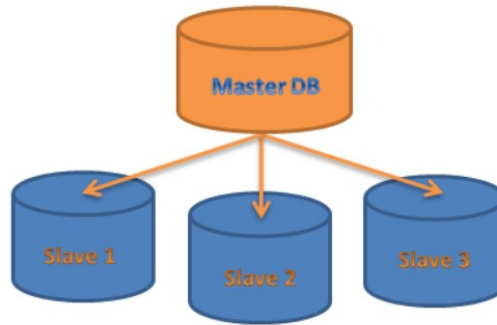
Replikasi database adalah teknik yang penting dalam manajemen basis data yang memungkinkan kita untuk membuat salinan atau replika data dari sebuah basis data utama ke basis data lainnya secara real-time. Dengan replikasi, kita dapat menghadirkan sistem yang toleran terhadap kegagalan dan meningkatkan ketersediaan data.

Dalam praktikum ini, kita akan menggunakan PuTTY atau alat SSH lainnya untuk melakukan remote Ubuntu Server untuk mengimplementasikan replikasi database MySQL. Kita akan belajar tentang konsep master-slave dalam replikasi, di mana basis data master berperan sebagai sumber utama data, sedangkan basis data slave berperan sebagai salinan data yang terus-menerus diperbarui.

Dengan pemahaman yang baik tentang replikasi database MySQL di Ubuntu Server, diharapkan dapat menambah pengetahuan dan keterampilan yang diperlukan untuk mengimplementasikan replikasi database dalam lingkungan server.

6.2.1. Replikasi Database

Replikasi master-slave memungkinkan dari satu server database (master) untuk di replikasi ke satu atau lebih server database lain (slave).



Gambar 6.2.1 Ilustrasi master-slave dalam replikasi DB

Replikasi MySQL adalah proses yang memungkinkan data dari satu server database MySQL (master) untuk disalin secara otomatis ke satu atau lebih server database MySQL (slave). Biasanya digunakan untuk menyebarkan akses baca di beberapa server untuk skalabilitas, meskipun dapat juga digunakan untuk keperluan lain seperti untuk failover atau menganalisis data pada slave agar tidak membebani master.

Karena replikasi master-slave adalah replikasi satu arah (dari master ke slave), hanya database master yang digunakan untuk operasi tulis, sedangkan operasi baca dapat tersebar di banyak database slave. Artinya, jika replikasi master-slave digunakan sebagai solusi penskalaan, Anda harus memiliki setidaknya dua sumber data yang ditentukan, satu untuk operasi tulis dan yang kedua untuk operasi baca.

- **Scale-out solutions**

Menyebarkan beban di antara banyak slave untuk meningkatkan kinerja. Di lingkungan ini, semua penulisan dan pembaruan harus dilakukan di server master. Membaca, bagaimanapun, dapat terjadi pada satu atau lebih slave. Model ini dapat meningkatkan kinerja penulisan (karena master didedikasikan untuk pembaruan), sekaligus meningkatkan kecepatan baca secara dramatis di sejumlah budak yang terus meningkat.

- **Data security**

Karena data di replikasi ke slave, dan slave dapat menjeda proses replikasi, dimungkinkan untuk menjalankan layanan pencadangan pada slave tanpa merusak data master yang sesuai.

- **Analytics**



Data langsung dapat dibuat di master, sedangkan analisis informasi dapat dilakukan di slave tanpa memengaruhi kinerja master.

- **Long-distance data distribution**

Jika kantor cabang ingin bekerja dengan salinan data utama, kami dapat menggunakan replikasi untuk membuat salinan data lokal untuk digunakan tanpa memerlukan akses permanen ke master.

6.3. ALAT DAN BAHAN

6.3.1. Perangkat Keras

- VPS
- Komputer/Laptop

6.3.2. Perangkat Lunak

- PuTTY atau SSH lainnya
- MySQL (1) sebagai Master (di VPS)
- MySQL (2) sebagai Slave (di Komputer/Laptop)

6.4. LANGKAH PERCOBAAN

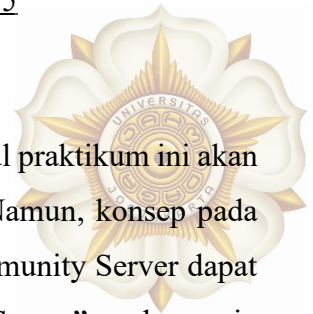
Percobaan latihan ini akan menggunakan MySQL di Ubuntu Server sebagai master dan MySQL di Windows Sebagai slave. Tahapan percobaan dapat dilakukan sebagai berikut:

1. Mengecek versi MySQL di VPS Master

Sebelum membuat MySQL slave, terlebih dulu kita mengecek versi MySQL yang akan digunakan. Anda perlu masuk ke dalam Ubuntu Server untuk mengecek versi MySQL. Pengecekan dapat dilakukan seperti Gambar 6.4.1. Versi MySQL yang digunakan di Ubuntu Server adalah versi 8.0.44. Versi ini akan digunakan untuk mengunduh versi MySQL Slave. Penggunaan versi yang sama akan meminimalisasikan terjadinya error.

```
praktikan@praktikan: ~  
praktikan@praktikan:~$ sudo mysql --version  
[sudo] password for praktikan:  
mysql Ver 8.0.44-0ubuntu0.22.04.1 for Linux on x86_64 ((Ubuntu))  
praktikan@praktikan:~$ |
```

Gambar 6.4.1 Cek versi MySQL di Ubuntu Server (Master)



2. Download MySQL di komputer Slave

Silakan Lompati tahap ini jika Anda sudah memiliki MySQL. Modul praktikum ini akan menggunakan MySQL Community Server sebagai percontohan. Namun, konsep pada dunia kerja kurang lebih sama. Untuk mendownload MySQL Community Server dapat dilakukan dengan melakukan pencarian “MySQL Community Server” pada mesin pencarian atau dengan link berikut:

<https://dev.mysql.com/downloads/mysql/>

Halaman download akan terlihat sebagai berikut. Pilih versi yang sesuai dengan yang ada di Ubuntu Server, yaitu versi 8.0.44 (Sesuaikan dengan versi yang Anda gunakan). Pada modul ini akan mencontohkan dengan menggunakan Windows (x86, 64-bit), ZIP Archive. Pemilihan download ini dimaksudkan sebagai percontohan saja, gunakan installer yang sesuai dengan kebutuhan produksi jika diperlukan.

MySQL Community Server 8.0.44

Select Version:

- 8.0.44 (Selected)
- 9.5.0 Innovation
- 8.4.7 LTS
- 8.0.44

Recommended Download:

MySQL Installer for Windows

All MySQL Products. For All Windows Platforms. In One Package.

Starting with MySQL 5.6 the MySQL Installer package replaces the standalone MSI packages.

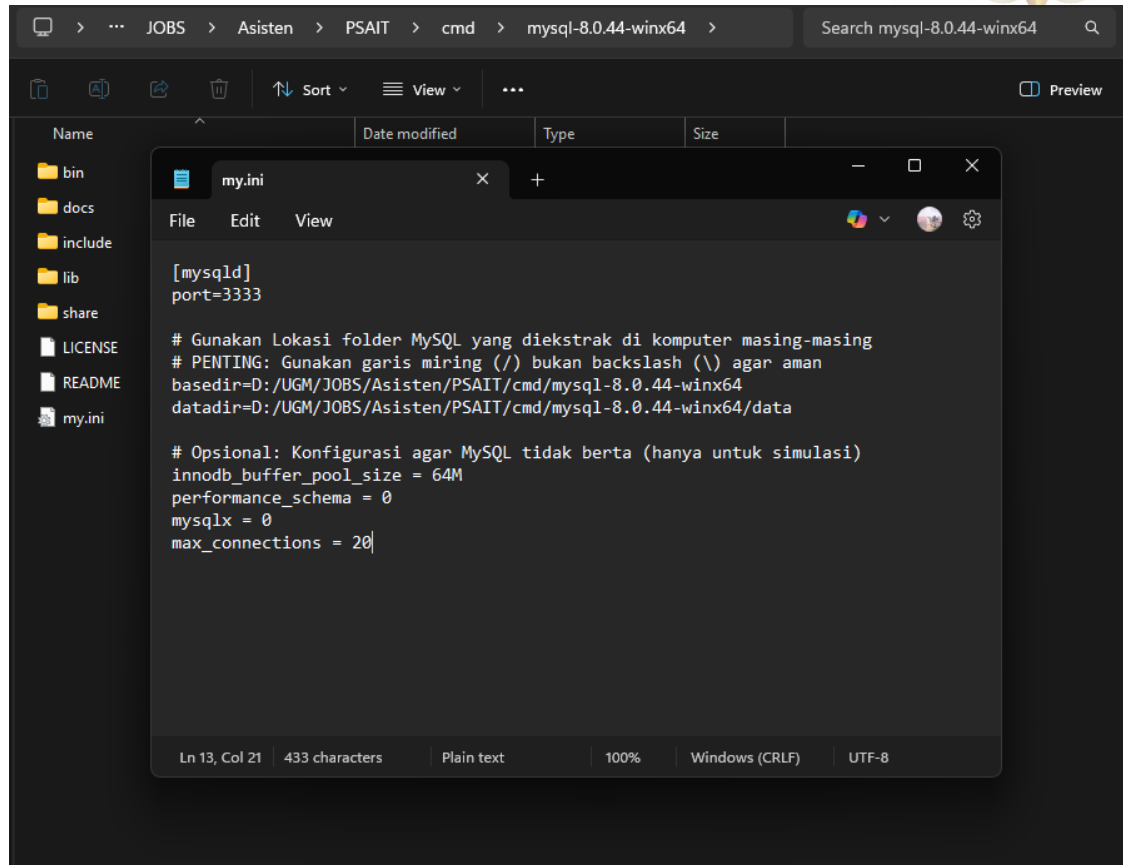
Windows (x86, 32 & 64-bit), MySQL Installer MSI [Go to Download Page >](#)

Other Downloads:

Platform	Version	Size	Action
Windows (x86, 64-bit), MSI Installer	8.0.44	123.6M	Download
(mysql-8.0.44-winx64.msi) MD5: d426fe79888c88665abfd95928c8f477 Signature			
Windows (x86, 64-bit), ZIP Archive	8.0.44	233.2M	Download
(mysql-8.0.44-winx64.zip) MD5: f5e62718c0e77122aee3476386f48051 Signature			
Windows (x86, 64-bit), ZIP Archive Debug Binaries & Test Suite	8.0.44	715.3M	Download
(mysql-8.0.44-winx64-debug-test.zip) MD5: ed4200d4826392550abc20e920c05a69 Signature			

Gambar 6.4.2 Halaman MySQL Community Server

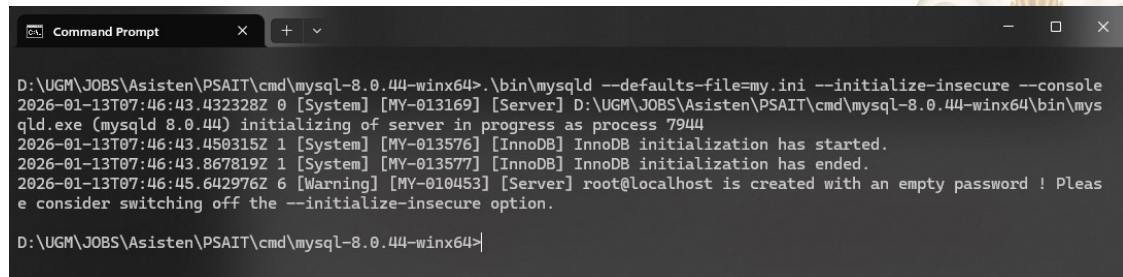
Setelah selesai mendownload, ekstrak file tersebut dan tambahkan file my.ini di dalam folder ekstraksi tersebut. Isi konfigurasi my.ini seperti Gambar 6.4.3. Pada konfigurasi ini MySQL akan dijalankan di port 3333, gunakan port lain jika port tersebut sedang digunakan.



Gambar 6.4.3 Konfigurasi my.ini

Inisiasi MySQL dengan perintah berikut di dalam folder ekstraksi. Hasil menjalankan perintah akan terlihat seperti Gambar 6.4.4. Perintah ini akan membuat data yang terletak sesuai dengan apa yang telah didefinisikan di my.ini.

```
.\bin\mysqld --defaults-file=my.ini --initialize-insecure --console
```



```

D:\UGM\JOBS\Asisten\PSAIT\cmd\mysql-8.0.44-win64>.bin\mysqld --defaults-file=my.ini --initialize-insecure --console
2026-01-13T07:46:43.432328Z 0 [System] [MY-013169] [Server] D:\UGM\JOBS\Asisten\PSAIT\cmd\mysql-8.0.44-win64\bin\mys
qld.exe (mysqld 8.0.44) initializing of server in progress as process 7944
2026-01-13T07:46:43.450315Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2026-01-13T07:46:43.867819Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2026-01-13T07:46:45.642976Z 6 [Warning] [MY-010453] [Server] root@localhost is created with an empty password ! Pleas
e consider switching off the --initialize-insecure option.

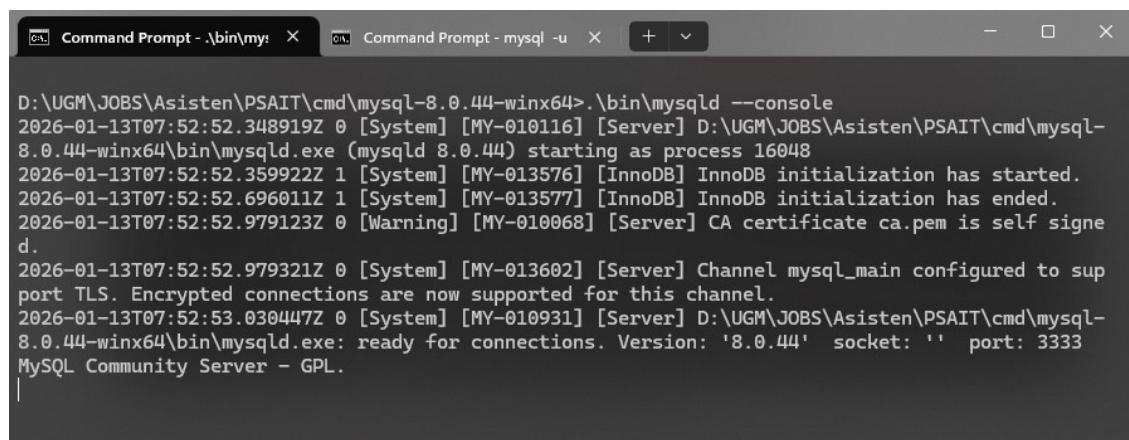
D:\UGM\JOBS\Asisten\PSAIT\cmd\mysql-8.0.44-win64>

```

Gambar 6.4.4 Inisiasi MySQL

Untuk menjalankan MySQL dapat dilakukan dengan perintah berikut. Perintah ini akan menjalankan MySQL Server dan menjadikan terminal yang digunakan untuk menjalankan perintah tersebut sebagai Log sehingga Anda tidak dapat mengetik lagi di terminal tersebut. MySQL Server ini akan tertutup jika terminal yang digunakan untuk menjalankannya tertutup, sehingga pastikan terminal tersebut tidak tertutup. Tampilan menjalankan MySQL Server akan terlihat seperti Gambar 6.4.5. MySQL ini akan berjalan di port 3333 sesuai dengan yang terdefiniskan di my.ini.

```
.\bin\mysqld --console
```



```

D:\UGM\JOBS\Asisten\PSAIT\cmd\mysql-8.0.44-win64>.bin\mysqld --console
2026-01-13T07:52:52.348919Z 0 [System] [MY-010116] [Server] D:\UGM\JOBS\Asisten\PSAIT\cmd\mysql-
8.0.44-win64\bin\mysqld.exe (mysqld 8.0.44) starting as process 16048
2026-01-13T07:52:52.359922Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
2026-01-13T07:52:52.696011Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
2026-01-13T07:52:52.979123Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signe
d.
2026-01-13T07:52:52.979321Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to sup
port TLS. Encrypted connections are now supported for this channel.
2026-01-13T07:52:53.030447Z 0 [System] [MY-010931] [Server] D:\UGM\JOBS\Asisten\PSAIT\cmd\mysql-
8.0.44-win64\bin\mysqld.exe: ready for connections. Version: '8.0.44' socket: '' port: 3333
MySQL Community Server - GPL.

```

Gambar 6.4.5 Menjalankan MySQL Server

Untuk masuk ke dalam MySQL, Anda dapat membuka terminal baru dan mengetikkan perintah berikut. Ganti PORT dengan port yang Anda gunakan. Pada modul ini menggunakan port 3333 seperti konfigurasi yang telah dilakukan sebelumnya. Perintah tersebut akan menjalankan aplikasi CLI seperti Gambar 6.4.6.



mysql -u root -P PORT

```

Microsoft Windows [Version 10.0.26200.7462]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ASUS>mysql -u root -P 3333
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.44 MySQL Community Server - GPL

Copyright (c) 2000, 2022, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> |
    
```

Gambar 6.4.6 CLI MySQL

Pada saat menjalankan aplikasi MySQL, Anda dapat melihat versi MySQL di bagian pembukaan seperti Gambar 6.4.6. Jika ingin mengecek dengan menggunakan SQL, Anda dapat menjalankan perintah seperti Gambar 6.4.7 di dalam MySQL CLI.

```

mysql> select @@version;
+-----+
| @@version |
+-----+
| 8.0.44    |
+-----+
1 row in set (0.00 sec)

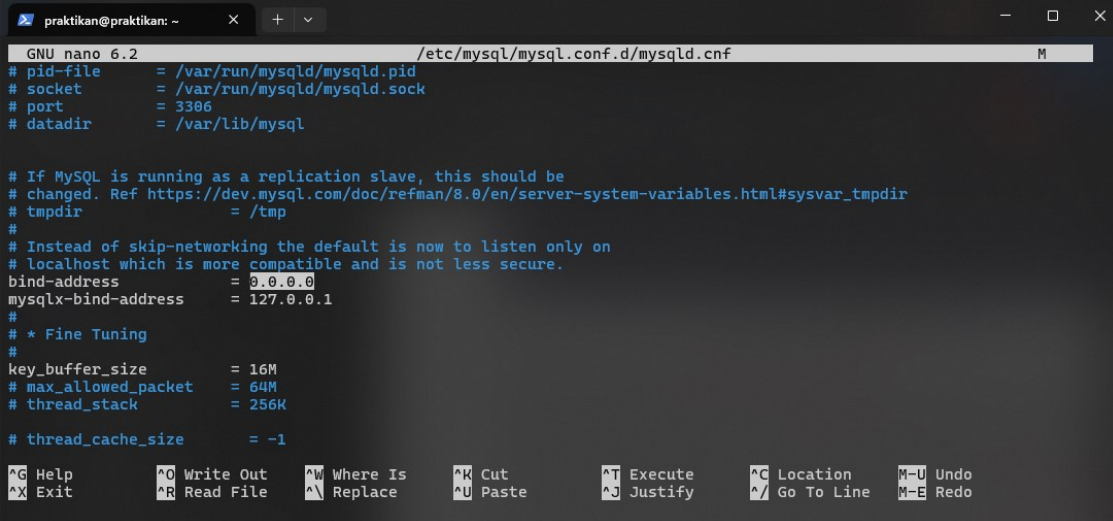
mysql> |
    
```

Gambar 6.4.7 Mengecek versi MySQL

3. Mengatur konfigurasi MySQL di Master

Agar bisa melakukan replikasi database yang berada di master dan slave, harus melakukan beberapa perubahan pengaturan baik itu di master ataupun di slave. Untuk master, yang harus dipersiapkan terlebih dahulu adalah file config dengan mengubah bind-address menjadi 0.0.0.0 seperti Gambar 6.4.8. Mengakses file tersebut dapat dilakukan dengan memasukkan perintah sebagai berikut.

```
sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf
```



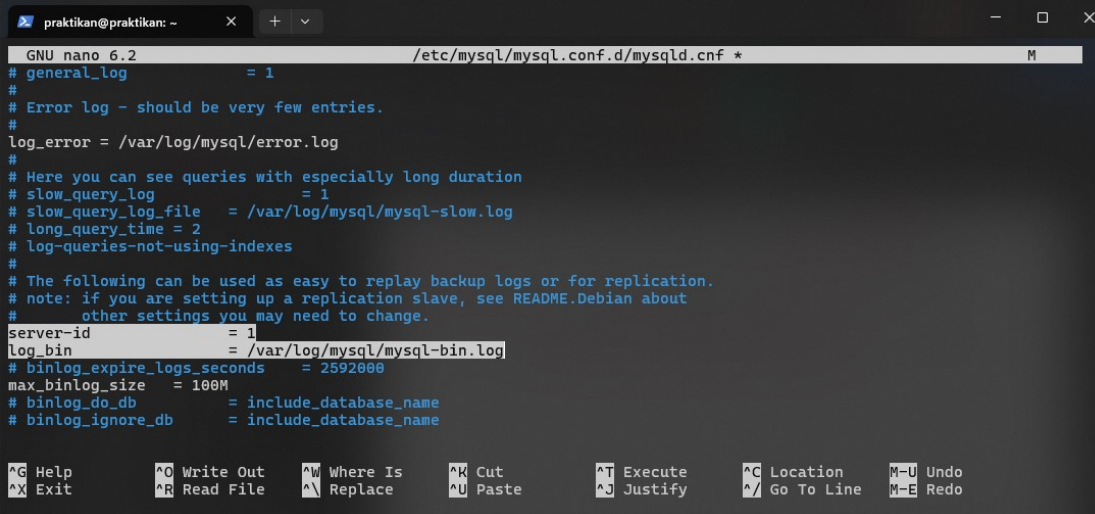
```
GNU nano 6.2 /etc/mysql/mysql.conf.d/mysqld.cnf
# pid-file             = /var/run/mysqld/mysqld.pid
# socket               = /var/run/mysqld/mysqld.sock
# port                 = 3306
# datadir              = /var/lib/mysql

# If MySQL is running as a replication slave, this should be
# changed. Ref https://dev.mysql.com/doc/refman/8.0/en/server-system-variables.html#sysvar_tmpdir
# tmpdir               = /tmp
#
# Instead of skip-networking the default is now to listen only on
# localhost which is more compatible and is not less secure.
bind-address           = 0.0.0.0
mysqlx-bind-address    = 127.0.0.1
#
# * Fine Tuning
#
key_buffer_size        = 16M
# max_allowed_packet   = 64M
# thread_stack         = 256K
#
# thread_cache_size    = -1

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^N Replace   ^U Paste     ^J Justify   ^_/ Go To Line M-E Redo
```

Gambar 6.4.8 Merubah bind-address file /etc/mysql/mysql.conf.d/mysqld.cnf

Selain bind-address, server-id dan log_bin perlu diaktifkan atau didefinisikan. Hapus tanda “#” untuk mengaktifkan definisi server-id dan log_bin seperti Gambar 6.4.9.

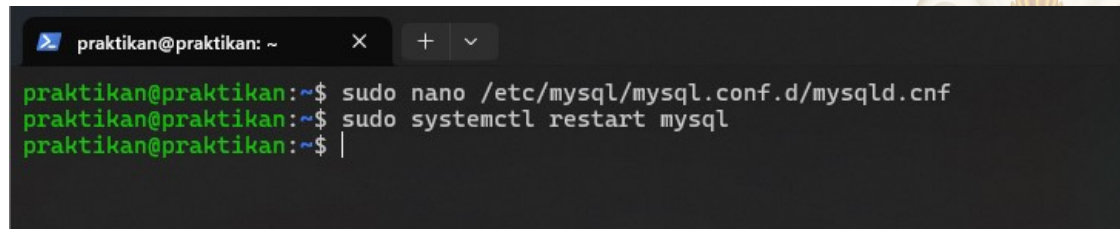


```
GNU nano 6.2 /etc/mysql/mysql.conf.d/mysqld.cnf *
# general_log          = 1
#
# Error log - should be very few entries.
#
log_error = /var/log/mysql/error.log
#
# Here you can see queries with especially long duration
# slow_query_log        = 1
# slow_query_log_file   = /var/log/mysql/mysql-slow.log
# long_query_time = 2
# log-queries-not-using-indexes
#
# The following can be used as easy to replay backup logs or for replication.
# note: if you are setting up a replication slave, see README.Debian about
# other settings you may need to change.
server-id              = 1
log_bin                = /var/log/mysql/mysql-bin.log
# binlog_expire_logs_seconds = 2592000
# max_binlog_size        = 100M
# binlog_do_db            = include_database_name
# binlog_ignore_db        = include_database_name

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^N Replace   ^U Paste     ^J Justify   ^_/ Go To Line M-E Redo
```

Gambar 6.4.9 Mengaktifkan server-id dan log_bin di mysqld.cnf

Setelah selesai mengedit file konfigurasi mysqld.cnf, tekan CTRL+S untuk menyimpan dan CTRL+X untuk keluar dari editor. Restart server MySQL untuk menjalankan perubahan konfigurasi seperti Gambar 6.4.10.

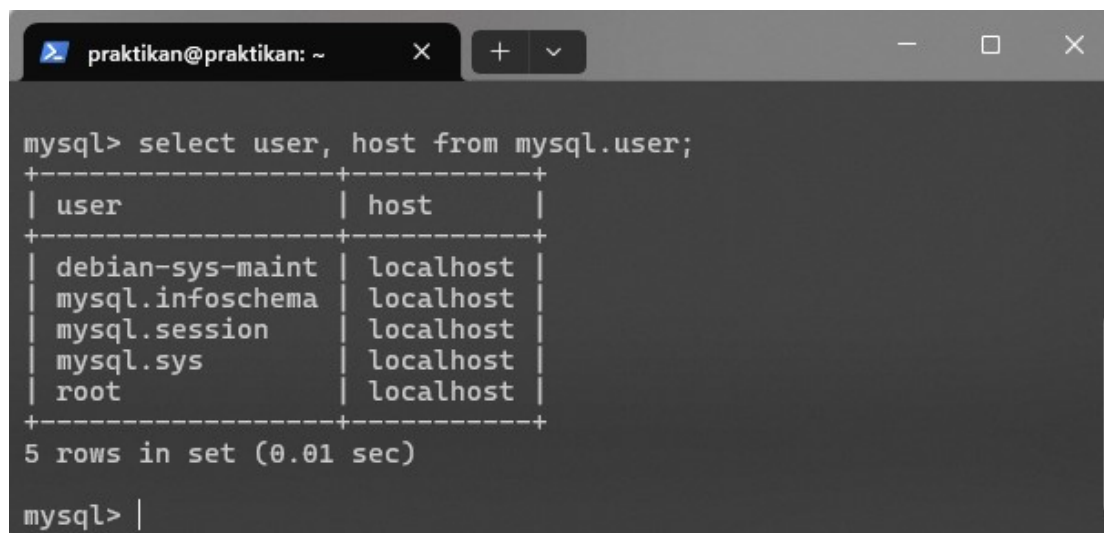


```
praktikan@praktikan: ~
praktikan@praktikan:~$ sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf
praktikan@praktikan:~$ sudo systemctl restart mysql
praktikan@praktikan:~$
```

Gambar 6.4.10 Restart Server MySQL Ubuntu Server

4. Membuat MySQL user baru di Master (Ubuntu Server)

Sebelum membuat user baru di Master (Ubuntu Server), cek user yang telah didefinisikan terlebih dahulu. Masuk ke CLI MySQL di Ubuntu Server dan jalankan perintah SQL seperti Gambar 6.4.11.

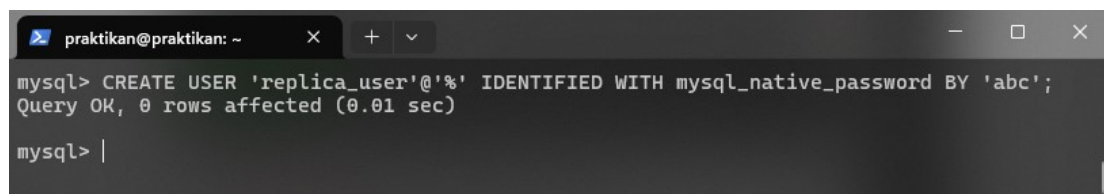


```
mysql> select user, host from mysql.user;
+-----+-----+
| user          | host      |
+-----+-----+
| debian-sys-maint | localhost |
| mysql.infoschema | localhost |
| mysql.session   | localhost |
| mysql.sys       | localhost |
| root           | localhost |
+-----+-----+
5 rows in set (0.01 sec)

mysql>
```

Gambar 6.4.11 Melakukan cek user yang telah ada di database

Modul praktikum ini akan membuat user baru dengan nama “replica user” dengan password “abc”. Pembuatan user baru dapat dilakukan dengan perintah SQL seperti Gambar 6.4.12.

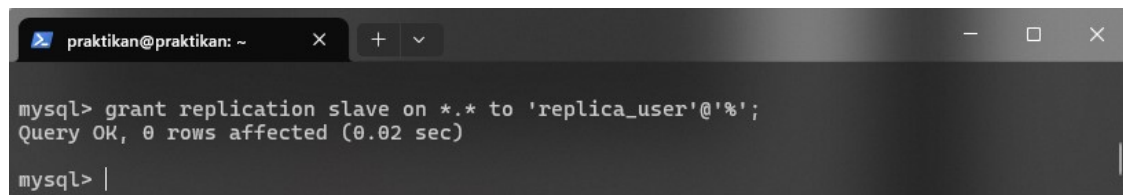


```
mysql> CREATE USER 'replica_user'@'%' IDENTIFIED WITH mysql_native_password BY 'abc';
Query OK, 0 rows affected (0.01 sec)

mysql>
```

Gambar 6.4.12 Menambah User MySQL

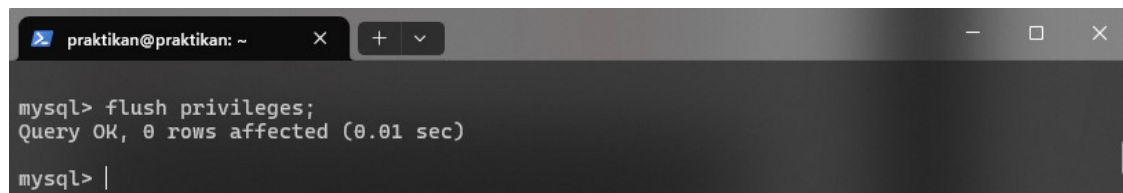
Berikan izin akses replication slave ke “replica_user” agar MySQL Slave dapat membaca data dari MySQL Master sebagai slave seperti Gambar 6.4.13. Pada SQL ini, izin “replication slave” untuk akses seluruh database “*” dan seluruh tabel “*” diberikan kepada user “replica_user” dengan ip “%”. Perlu diketahui bahwa pembuatan perizinan akses seperti ini tidak cocok untuk produksi karena minim keamanan, mungkin Anda dapat menspesifikan target replikasi. Namun, untuk latihan hal ini digunakan sebagai upaya mempermudah latihan replikasi.



```
praktikan@praktikan: ~  
mysql> grant replication slave on *.* to 'replica_user'@'%';  
Query OK, 0 rows affected (0.02 sec)  
mysql> |
```

Gambar 6.4.13 Memberikan izin Replication Slave pada replica_user

Setelah mendefinisikan perisinan akses, lakukan perintah seperti Gambar 6.4.14 untuk melakukan muat ulang tabel perizinan di MySQL Server Master.

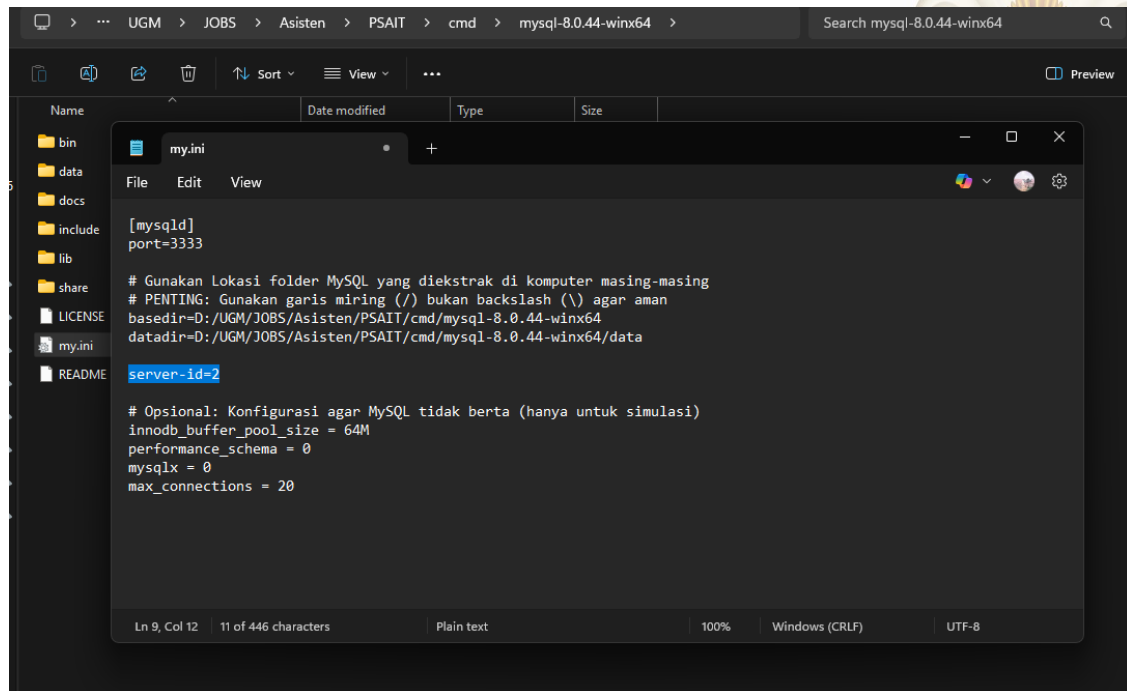


```
praktikan@praktikan: ~  
mysql> flush privileges;  
Query OK, 0 rows affected (0.01 sec)  
mysql> |
```

Gambar 6.4.14 Refresh Tabel Perizinan

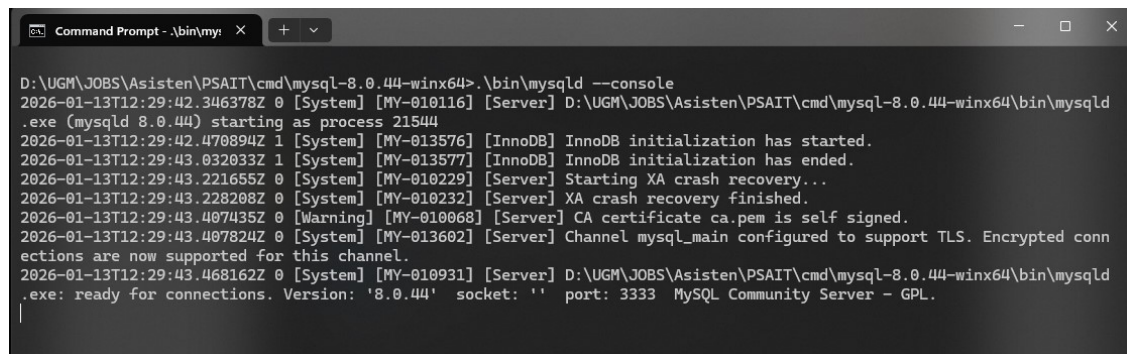
5. Mengatur konfigurasi di MySQL Slave

Tutup terminal MySQL server yang ada di komputer slave dan buka konfigurasi my.ini kembali. Tambahkan definisi server-id di konfigurasi MySQL server slave seperti Gambar 6.4.15. Server-ide yang digunakan di MySQL slave harus menggunakan id selain id milik MySQL master. Oleh karena itu, server-id di MySQL slave di definisikan sebagai 2.



Gambar 6.4.15 Menambahkan server-id di konfigurasi my.ini Slave

Jalankan ulang MySQL server untuk menerapkan perubahan. Pada modul ini karena menggunakan MySQL versi portabel, menjalankan ulang dilakukan dengan perintah yang sama seperti tahapan nomor 2.

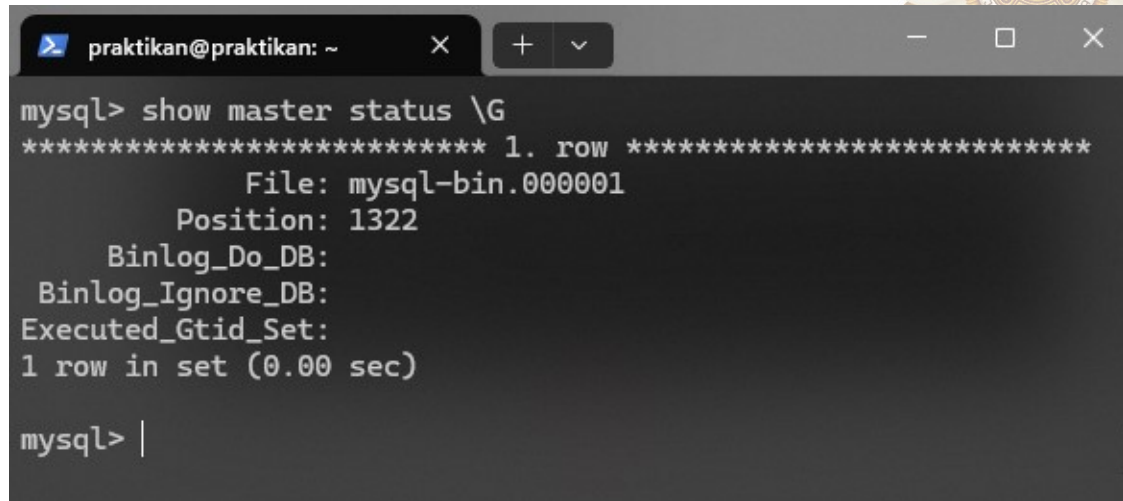


Gambar 6.4.16 Menjalankan ulang MySQL Server

6. Mengecek position MySQL Master

Masuk kembali ke Ubuntu Server dan buka MySQL. Masukkan perintah SQL seperti Gambar 6.4.17. Dari perintah tersebut kita dapat mengetahui bahwa posisi master berada di 1322 dengan file mysql-bin.000001. Perlu menjadi catatan bahwa nilai ini dapat berbeda-beda tergantung dengan kasus MySQL yang praktikan gunakan. Catat file dan

posisi dari MySQL Master, data ini akan digunakan untuk melakukan konfigurasi MySQL Slave.



```
praktikan@praktikan: ~
mysql> show master status \G
***** 1. row *****
      File: mysql-bin.000001
     Position: 1322
    Binlog_Do_DB:
   Binlog_Ignore_DB:
 Executed_Gtid_Set:
1 row in set (0.00 sec)

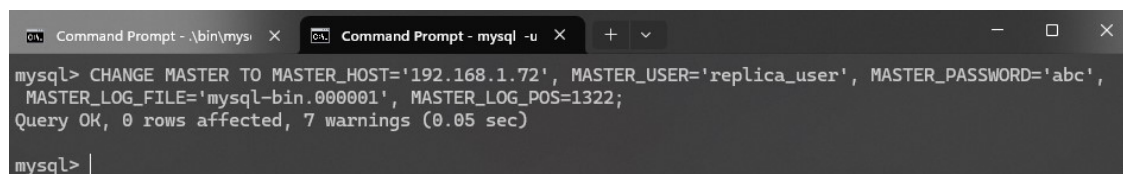
mysql> |
```

Gambar 6.4.17 Mengecek position di MySQL Master

7. Mengatur konfigurasi di MySQL slave bagian 2

Buka kembali komputer yang digunakan untuk menjalankan MySQL slave. Masuklah ke dalam MySQL CLI dan tambahkan perintah SQL berikut. Perintah ini akan mendefinisikan konfigurasi master di MySQL slave. Gunakan IP_VPS yang dimiliki, gunakan REPLICA_USER yang telah dibuat di tahap sebelumnya, masukkan PASSWORD, FILE, dan POS yang sesuai dari data-data sebelumnya di konfigurasi master. Hasil menjalankan perintah akan terlihat seperti Gambar 6.4.18.

```
CHANGE MASTER TO MASTER_HOST='IP_VPS',
MASTER_USER='REPLICA_USER', MASTER_PASSWORD='PASSWORD',
MASTER_LOG_FILE='FILE', MASTER_LOG_POS=POS;
```



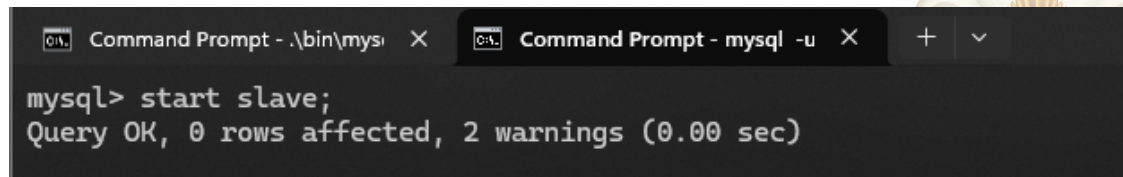
```
Command Prompt - \bin\mys: x Command Prompt - mysql -u x
mysql> CHANGE MASTER TO MASTER_HOST='192.168.1.72', MASTER_USER='replica_user', MASTER_PASSWORD='abc',
MASTER_LOG_FILE='mysql-bin.000001', MASTER_LOG_POS=1322;
Query OK, 0 rows affected, 7 warnings (0.05 sec)

mysql> |
```

Gambar 6.4.18 Konfigurasi MySQL slave bagian 2

8. Jalankan MySQL Slave

Menjalankan slave dapat dengan perintah SQL berikut.



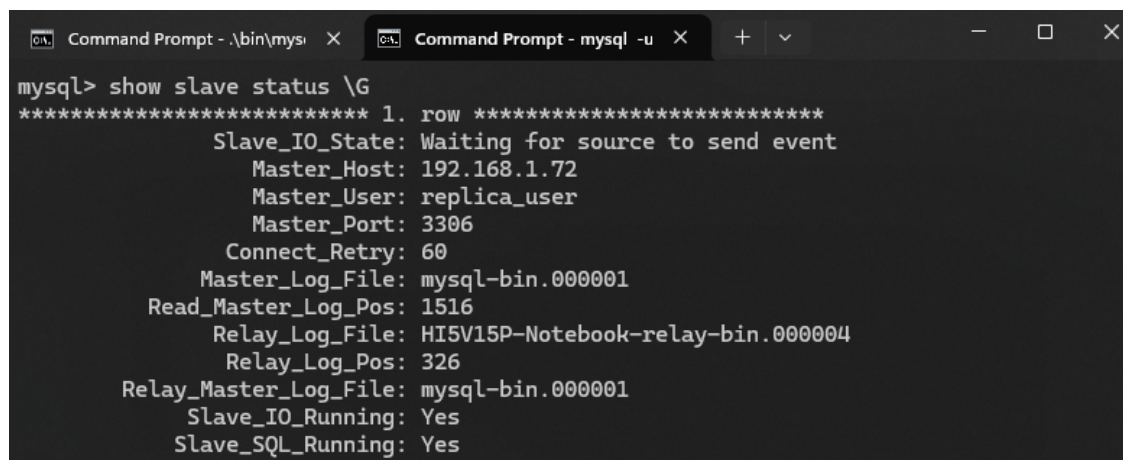
```

Command Prompt - .\bin\mysi X Command Prompt - mysql -u X + v
mysql> start slave;
Query OK, 0 rows affected, 2 warnings (0.00 sec)

```

Gambar 6.4.19 Menjalankan replikasi

Untuk memastikan bahwa replikasi sudah terhubung di MySQL slave, cek status dengan perintah SQL seperti Gambar 6.4.20. Status ini menunjukkan bahwa koneksi replikasi terhubung, ditandai dengan penjelasan `Slave_IO_State` yang menunggu event dari Master, `Slave_IO_Running` dan `Slave_SQL_Running` yang memberitahukan bahwa replikasi berjalan.



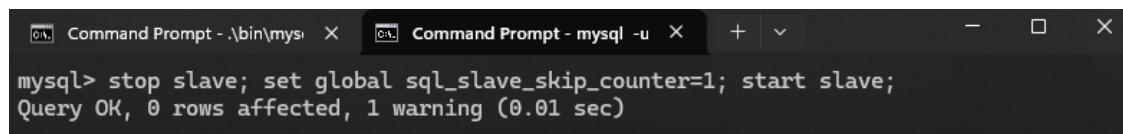
```

Command Prompt - .\bin\mysi X Command Prompt - mysql -u X + v - □ X
mysql> show slave status \G
***** 1. row *****
Slave_IO_State: Waiting for source to send event
Master_Host: 192.168.1.72
Master_User: replica_user
Master_Port: 3306
Connect_Retry: 60
Master_Log_File: mysql-bin.000001
Read_Master_Log_Pos: 1516
Relay_Log_File: HI5V15P-Notebook-relay-bin.000004
Relay_Log_Pos: 326
Relay_Master_Log_File: mysql-bin.000001
Slave_IO_Running: Yes
Slave_SQL_Running: Yes

```

Gambar 6.4.20 Status Replikasi MySQL

Jika terdapat error, jalankan perintah SQL berikut dan tunggu beberapa saat. Anda dapat mengecek status kembali untuk memastikan.



```

Command Prompt - .\bin\mysi X Command Prompt - mysql -u X + v - □ X
mysql> stop slave; set global sql_slave_skip_counter=1; start slave;
Query OK, 0 rows affected, 1 warning (0.01 sec)

```

9. Menguji replikasi

Untuk menguji apakah replikasi berhasil, kita dapat mencoba menambahkan database baru pada MySQL Master. Jika database yang baru saja tertambahkan juga terlihat di MySQL Slave, berarti konfigurasi replikasi tersebut berhasil dilakukan.



Sebelum menguji, pastikan kita mengecek terlebih dahulu kedua database yang tersedia. Di Gambar 6.4.21, sebelah kiri merupakan database yang ada di Master, sedangkan bagian kanan merupakan daftar database yang ada di Slave.

```

praktikan@praktikan: ~
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sait_db |
| sys |
+-----+
5 rows in set (0.00 sec)

Command Prompt - .\bin\mysq
mysql> show databases;
1 row in set, 1 warning (0.00 sec)

mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
4 rows in set (0.00 sec)
    
```

Gambar 6.4.21 Perbandingan awal database (Master MySQL, Slave MySQL)

Buatlah database baru dengan nama “uji_coba” pada MySQL Master. Di Gambar 6.4.22, bagian kiri merupakan tampilan MySQL Master yang melakukan penambahan database baru, ketika melakukan pengecekan di slave dengan menampilkan database, database “uji_coba” tersebut juga ikut muncul di MySQL slave. Hal ini menunjukkan replikasi berhasil dilakukan, database baru yang dibuat di Master, ter-copy secara otomatis ke database di slave.

```

praktikan@praktikan: ~
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sait_db |
| sys |
+-----+
5 rows in set (0.00 sec)

mysql> create database uji_coba;
Query OK, 1 row affected (0.02 sec)

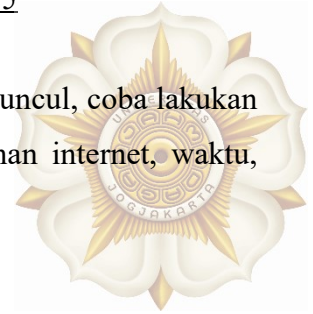
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sait_db |
| sys |
| uji_coba |
+-----+
6 rows in set (0.00 sec)

Command Prompt - .\bin\mysq
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
4 rows in set (0.00 sec)

mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| uji_coba |
+-----+
5 rows in set (0.00 sec)

mysql>
mysql>
mysql>
mysql>
    
```

Gambar 6.4.22 Uji coba replikasi (kiri Master MySQL, kanan Slave MySQL)



Jika replikasi gagal, coba tunggu beberapa saat. Jika masih tidak muncul, coba lakukan ulang dari tahap 8. Kegagalan ini bisa terjadi karena permasalahan internet, waktu, ataupun tidak konsisten isi data.

6.5. TUGAS & ANALISIS

Dari latihan uji coba diatas terdapat perbedaan jumlah data, di database MySQL Master terdapat 6 database, sedangkan di database MySQL Slave hanya terdapat 5 database. Mengapa database `sait_db` tidak tereplikasi ke MySQL Slave? Bagaimana solusi yang harus dilakukan agar database `sait_db` juga ter-copy ke MySQL Slave dan ikut tereplikasi ketika terdapat penambahan data? Tunjukkan langkah-langkah solusinya!

6.6. REFERENSI

- [1]. Hitesh Jethva, “How to Setup MariaDB Replication Tutorial (Step by Step),” Cloud Infrastructure Services, Jan. 21, 2022. <https://cloudinfrastructureservices.co.uk/setup-mariadb-replication/> (accessed Jan. 24, 2026).
- [2]. <https://www.facebook.com/MariaDB.dbms>, “Database Master-Slave Replication in the Cloud | MariaDB,” MariaDB, May 2013. <https://mariadb.com/resources/blog/database-master-slave-replication-in-the-cloud> (accessed Jan. 24, 2026).
- [3]. V. Grippa, “Replicating from MySQL 8.0 to MySQL 5.7,” Percona Database Performance Blog, Aug. 07, 2018. <https://www.percona.com/blog/replicating-mysql-8-0-mysql-5-7/> (accessed Jan. 24, 2026). <https://www.percona.com/blog/replicating-mysql-8-0-mysql-5-7/>
- [4]. M. W, “Basic setup of master-slave GTID replication on MySQL 8,” Medium, Nov. 15, 2019. https://medium.com/@michael_w_s/basic-setup-of-master-slave-gtid-replication-on-mysql-8-8f39ea29765c (accessed Jan. 24, 2026).



PERTEMUAN VII

SINKRONISASI DATABASE MENGGUNAKAN REST API

7.1. TUJUAN PEMBELAJARAN

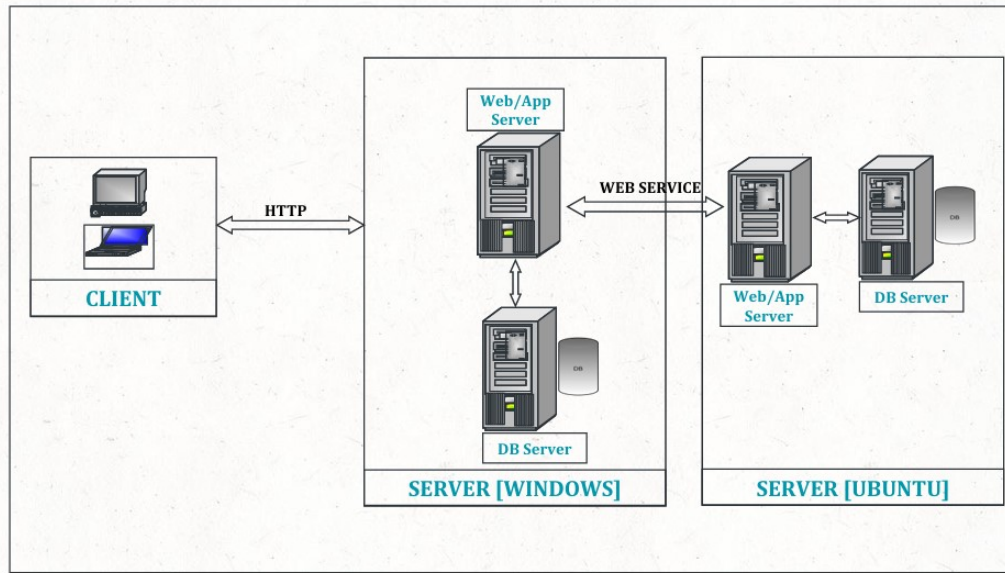
- A. Mahasiswa mampu menggunakan REST API dari Server Ubuntu yang telah dibuatnya di aplikasi front-end yang berada di komputer/laptop yang berbeda dari Server Ubuntu.
- B. Mahasiswa mampu memodifikasi aplikasi front-end guna menambahkan fungsionalitas insert, update, delete data di database lokal.
- C. Mahasiswa mampu menerapkan sinkronisasi database lokal dengan database yang ada di Server Ubuntu menggunakan REST API.

7.2. DASAR TEORI

Dalam praktikum ini, kita akan mempelajari konsep dan langkah-langkah untuk melakukan sinkronisasi antara dua database MySQL, sehingga data tetap konsisten di server yang terlibat. Dengan sinkronisasi, kita dapat memastikan bahwa perubahan yang terjadi pada satu server akan secara otomatis diterapkan pada server lainnya, sehingga data di semua server tetap selaras.

Kita akan menggunakan REST API dari Ubuntu Server yang telah digunakan sebelumnya dari kasus 3. Hanya saja kita juga akan menggunakan MySQL di komputer/laptop secara bersamaan dan memastikan setiap ada perubahan data bisa tetap sinkron seperti pada replikasi database di BAB sebelumnya. Dengan pemahaman yang baik tentang sinkronisasi database menggunakan MySQL, Anda akan memiliki kemampuan untuk mengelola lingkungan yang kompleks dengan beberapa server yang terhubung secara efisien.

KASUS - 3

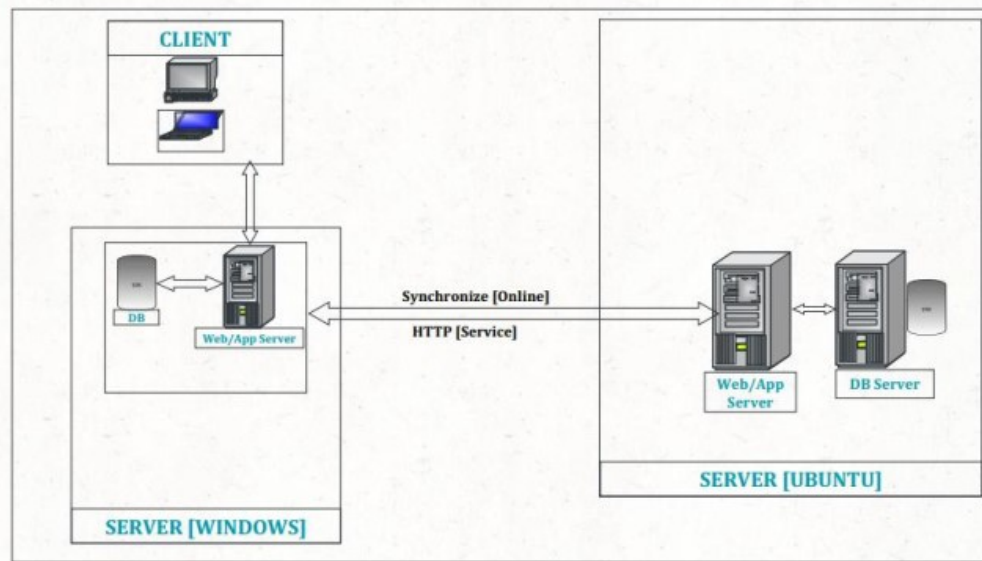


Gambar 7.2.1 Konsep Kasus 3

Dalam praktikum kali ini, konsep arsitektur yang akan digunakan adalah konsep kasus 6 Gambar 7.2.2. Perbedaan dari kasus 3 sebelumnya adalah, di kasus 6 kita akan melakukan sinkronisasi database Server dengan menggunakan REST API, yang mana aplikasi di Server Ubuntu akan menjalankan aksi ke database ketika mendapatkan permintaan dari aplikasi yang kita buat di komputer/laptop (windows).



KASUS - 6



Gambar 7.2.2 Konsep sinkronisasi database MySQL kasus 6

7.3. ALAT DAN BAHAN

7.3.1. Perangkat Keras

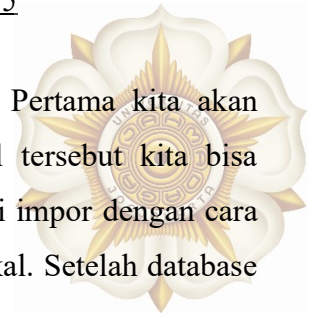
- VPS
- Komputer/Laptop

7.3.2. Perangkat Lunak

- PuTTY atau SSH lainnya
- Aplikasi Webservice REST API (di VPS)
- MySQL (1) (di VPS)
- MySQL (2) (di Komputer/Laptop)

7.4. LANGKAH PERCOBAAN

Pada kasus ini kita akan melanjutkan menggunakan REST API yang sebelumnya sudah pernah dibuat di pertemuan sebelumnya. Oleh karena yang akan kita lakukan merupakan sinkronisasi menggunakan RESTful API maka kita harus mematikan terlebih dahulu sinkronisasi pada pertemuan sebelumnya. Cek kembali materi sebelumnya untuk mematikan sinkronisasi MySQL!



Setelah mematikan sinkronisasi ini tahap yang akan kita lakukan. Pertama kita akan memastikan bahwa isi database adalah sama. Untuk mencapai hal tersebut kita bisa melakukan ekspor database di VPS. Kemudian hasil ekspor tersebut di impor dengan cara menimpaa (hapus yang lama, ganti dengan yang baru) di komputer lokal. Setelah database sama, kita dapat melakukan tahap sinkronisasi dengan RESTful API.

1. Ekspor database di server.

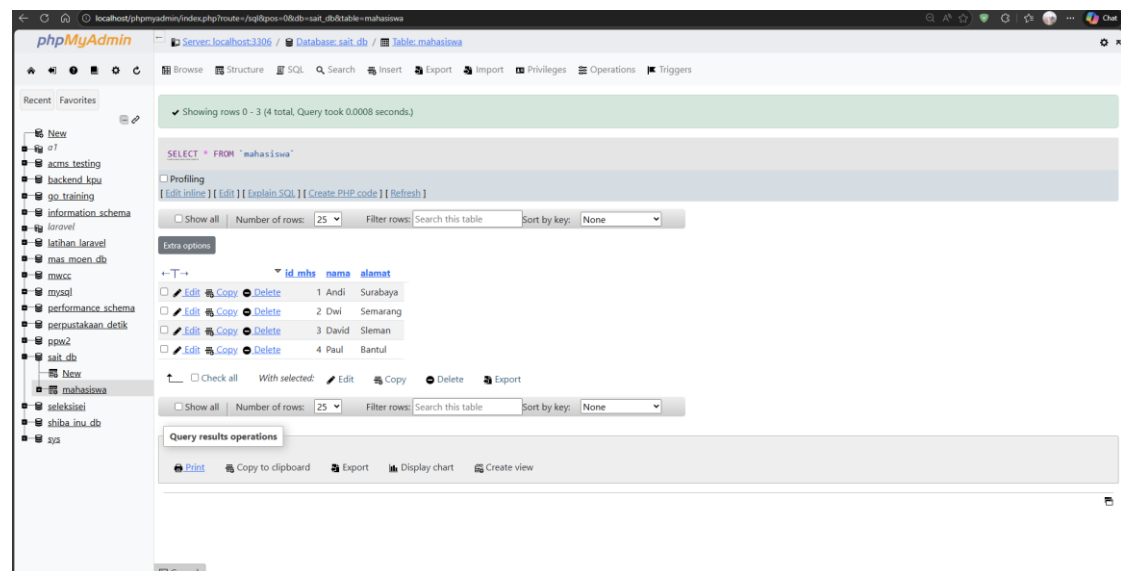
Buka VPS dan masuk ke `sait_project_api` atau `sait_api` atau aplikasi `php mahasiswa_api` dan jalankan perintah berikut.

```
praktikan@praktikan:/var/www/html/sait_project_api$ sudo mysqldump -u root -p sait_db > mahasiswa2.sql
[sudo] password for praktikan:
praktikan@praktikan:/var/www/html/sait_project_api$
```

Gambar 7.4.1 Ekspor database mahasiswa

2. Impor database di komputer lokal

Sebelum melakukan impor, hapus seluruh data yang ada di `sait_db`. Masukkan file `mahasiswa2.sql` ke dalam database `sait_db` dengan fitur import yang telah disediakan `phpMyAdmin` atau aplikasi lain yang kalian gunakan.



Gambar 7.4.2 Impor mahasiswa2 ke sait_db

3. Pastikan kedua tabel di dua MySQL yang berbeda memiliki data yang sama



Kita dapat mengecek kedua tabel dengan masuk ke MySQL dan menggunakan database `sait_db` serta mengecek tabel mahasiswa. Di Gambar 7.4.3, kita dapat yakin bahwa tabel yang tersimpan memiliki data yang sama.

The image shows two terminal windows. The left window is a local MySQL terminal on a Windows machine (PS D:\laragon\www\sait_frontend>). The right window is a MySQL terminal on a VPS (praktikan@praktikan: /var/www/html/sait_project_api\$).

Left Terminal (Local MySQL):

```
mysql -u root -p
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 126
Server version: 8.0.30 MySQL Community Server - GPL

Copyright (c) 2000, 2022, Oracle and/or its affiliates.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use sait_db;
Database changed
mysql> select * from mahasiswa;
+----+-----+-----+
| id_mhs | nama | alamat |
+----+-----+-----+
| 1 | Andi | Surabaya |
| 2 | Dedi | Semarang |
| 3 | David | Sleman |
| 4 | Paul | Bantul |
+----+-----+-----+
4 rows in set (0.00 sec)

mysql>
```

Right Terminal (MySQL VPS):

```
mysql -u root -p
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9346
Server version: 8.0.44-0ubuntu0.22.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

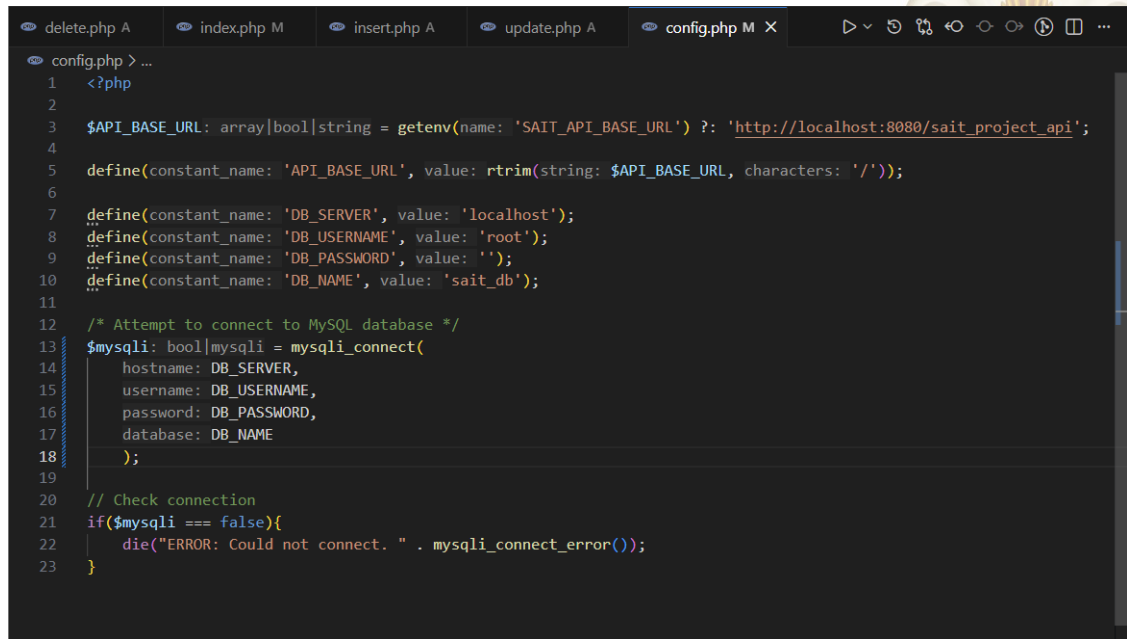
mysql> use sait_db;
Database changed
mysql> select * from mahasiswa;
+----+-----+-----+
| id_mhs | nama | alamat |
+----+-----+-----+
| 1 | Andi | Surabaya |
| 2 | Dedi | Semarang |
| 3 | David | Sleman |
| 4 | Paul | Bantul |
+----+-----+-----+
4 rows in set (0.00 sec)

mysql>
```

Gambar 7.4.3 Mengecek tabel mahasiswa di MySQL Lokal (kiri) dan MySQL VPS (kanan)

4. Tambahkan konfigurasi database lokal

Buat file `config.php` jika belum tersedia dan tambahkan kode berikut. Konfigurasi ini akan menyimpan URL BASE API dan juga konfigurasi database lokal. `DB_SERVER` pada konfigurasi ini tetaplah `localhost` karena berada di dalam satu mesin.



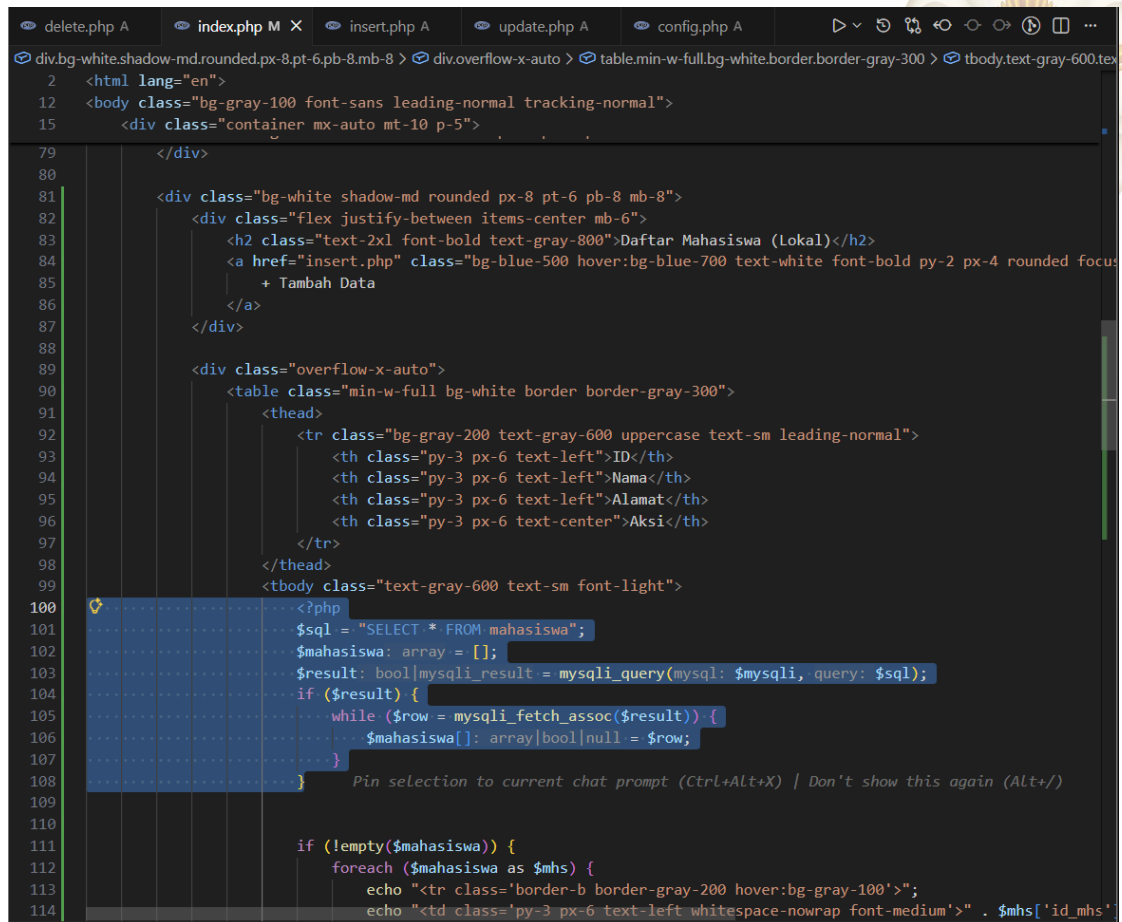
Gambar 7.4.4 Menambahkan database lokal

Jangan lupa untuk impor config.php di file yang membutuhkan. Letakkan kode impor berikut di bagian paling atas setiap file yang membutuhkan koneksi ke database.

```
<?php require_once 'config.php'; ?>
```

5. Tambahkan tampilan data dari database lokal di frontend

Salin kode penampilan database via API dan tempelkan di bawahnya sebagai template untuk penampilan database lokal. Ubah kode pemanggilan data via API dengan pemanggilan database secara lokal seperti Gambar 7.4.5. Jangan lupa ubah heading 2 pada tiap kartu agar dapat lebih jelas bagian mana yang merupakan kartu untuk database via API dan mana yang lokal.



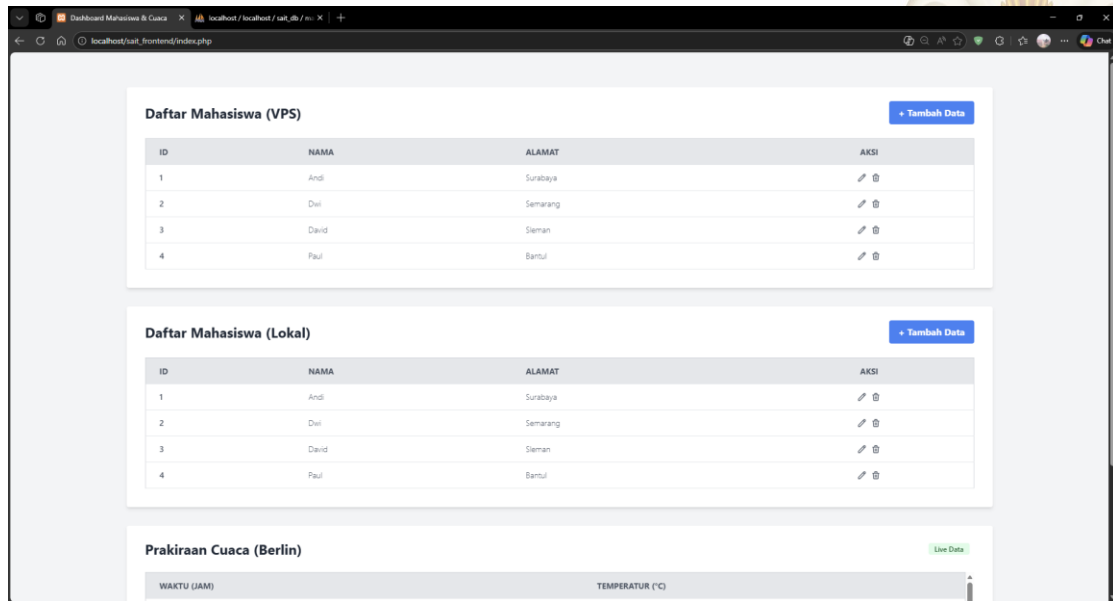
```

delete.php A  index.php M X  insert.php A  update.php A  config.php A
div.bg-white.shadow-md.rounded.px-8.pt-6.pb-8.mb-8 > div.overflow-x-auto > table.min-w-full.bg-white.border.border-gray-300 > tbody.text-gray-600.tex
2  <html lang="en">
12 <body class="bg-gray-100 font-sans leading-normal tracking-normal">
15 <div class="container mx-auto mt-10 p-5">
79 </div>
80
81 <div class="bg-white shadow-md rounded px-8 pt-6 pb-8 mb-8">
82 <div class="flex justify-between items-center mb-6">
83 <h2 class="text-2xl font-bold text-gray-800">Daftar Mahasiswa (Lokal)</h2>
84 <a href="insert.php" class="bg-blue-500 hover:bg-blue-700 text-white font-bold py-2 px-4 rounded focus:
85 + Tambah Data
86 </a>
87 </div>
88
89 <div class="overflow-x-auto">
90 <table class="min-w-full bg-white border border-gray-300">
91 <thead>
92 <tr class="bg-gray-200 text-gray-600 uppercase text-sm leading-normal">
93 <th class="py-3 px-6 text-left">ID</th>
94 <th class="py-3 px-6 text-left">Nama</th>
95 <th class="py-3 px-6 text-left">Alamat</th>
96 <th class="py-3 px-6 text-center">Aksi</th>
97 </tr>
98 </thead>
99 <tbody class="text-gray-600 text-sm font-light">
100 <?php
101 ..... $sql = "SELECT * FROM mahasiswa";
102 ..... $mahasiswa = array = [];
103 ..... $result: bool|mysqli_result = mysqli_query(mysqli: $mysqli, query: $sql);
104 ..... if ($result) {
105 ..... while ($row = mysqli_fetch_assoc($result)) {
106 ..... $mahasiswa[]: array|bool|null = $row;
107 ..... }
108 ..... }
109 .....
110 .....
111 if (!empty($mahasiswa)) {
112 foreach ($mahasiswa as $mhs) {
113 echo "<tr class='border-b border-gray-200 hover:bg-gray-100'>";
114 echo "<td class='py-3 px-6 text-left whitespace-nowrap font-medium'> . $mhs['id_mhs'

```

Gambar 7.4.5 Ubah kode pemanggilan API dengan SQL ke database lokal

Hasil implementasi kode tersebut akan terlihat seperti Gambar 7.4.6. Di program frontend yang ada di komputer lokal menampilkan dua data yang diambil dari API dan juga database lokal komputer. Di tahap ini kita telah menerapkan sinkronisasi baca.



Gambar 7.4.6 Hasil implementasi penambahan tampilan database lokal

6. Penambahan intervensi pengiriman data via RESTful API di frontend

Buka kode insert.php, di bagian submit, tambahkan kode berikut seperti Gambar 7.4.7. Kode ini melakukan penambahan data ke database lokal dengan menggunakan query SQL. Selanjutnya kode akan memasukkan data ke dalam database di VPS Ubuntu melalui REST API. Secara utuh kode dapat dilihat di Gambar 7.4.8, bagian merah merupakan yang kita tambahkan yaitu insert ke lokal DB, dan yang kuning merupakan insert ke DB yang ada di VPS, yang merupakan kode lama.

```
$sql = "INSERT INTO mahasiswa (nama, alamat)
VALUES ('" . $data['nama'] . "', '" . $data['alamat'] . "')";
if (mysqli_query($mysqli, $sql)) {
    echo "<script>
        const Toast = Swal.mixin({
            toast: true,
            position: 'top-end',
            showConfirmButton: false,
            timer: 2000,
            timerProgressBar: true,
            didOpen: (toast) => {
                toast.addEventListener('mouseenter', Swal.stopTimer);
                toast.addEventListener('mouseleave', Swal.resumeTimer);
            }
        });
    Toast.fire({
        icon: 'success',
        title: 'Data berhasil ditambahkan ke database lokal'
    });
    </script>";
}
```



```

    });
    Toast.fire({
        icon: 'success',
        title: 'Data mahasiswa berhasil ditambahkan di lokal
database.'
    });
    setTimeout(() => {}, 2000);
</script>";
} else {
    echo "ERROR: Could not able to execute $sql. " .
mysqli_error($mysqli);
}

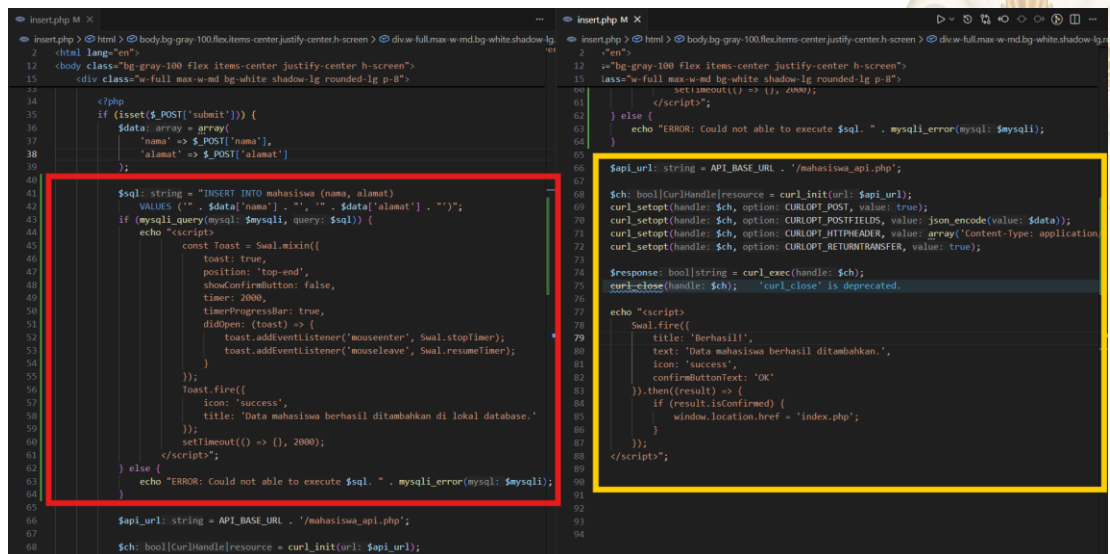
```

```

34 <?php
35 if (isset($_POST['submit'])) {
36     $data: array = array(
37         'nama' => $_POST['nama'],
38         'alamat' => $_POST['alamat']
39     );
40
41     $sql: string = "INSERT INTO mahasiswa (nama, alamat)
42     VALUES ('" . $data['nama'] . "', '" . $data['alamat'] . "')";
43     if (mysqli_query($mysqli, $sql)) {
44         echo "<script>
45             const Toast = Swal.mixin({
46                 toast: true,
47                 position: 'top-end',
48                 showConfirmButton: false,
49                 timer: 2000,
50                 timerProgressBar: true,
51                 didOpen: (toast) => {
52                     toast.addEventListener('mouseenter', Swal.stopTimer);
53                     toast.addEventListener('mouseleave', Swal.resumeTimer);
54                 }
55             });
56             Toast.fire({
57                 icon: 'success',
58                 title: 'Data mahasiswa berhasil ditambahkan di lokal database.'
59             });
60             setTimeout(() => {}, 2000);
61         </script>";
62     } else {
63         echo "ERROR: Could not able to execute $sql. " . mysqli_error($mysqli);
64     }
65
66     $api_url: string = API_BASE_URL . '/mahasiswa_api.php';
67
68     $ch: bool|CurlHandle|resource = curl_init($api_url);
69     curl_setopt($ch, CURLOPT_POST, true);

```

Gambar 7.4.7 Menambahkan kode insert database lokal



Gambar 7.4.8 Form Submit di insert.php secara full. Insert ke database lokal (merah) dan insert via API (kuning)

7. Informasi penting

Pada implementasi saat ini, sinkronisasi database lokal dengan database VPS melalui REST API memiliki kelemahan. Jika internet mati sesaat ketika melakukan menambahkan data, data tidak akan tersalin. Hal tersebut berbeda dengan sinkronisasi menggunakan master slave dari MySQL. Namun, hal tersebut tidak membuat bahwa cara sinkronisasi menggunakan API tidak bagus, melainkan kita harus meningkatkan algoritma sinkronisasi API seperti membuat algoritma Queue dan sejenisnya untuk mengatasi permasalahan offline.

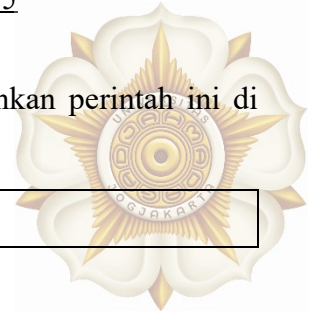
7.5. TUGAS & ANALISIS

Lanjutkan program yang telah Anda buat sebelumnya (sesuai studi kasus masing-masing):

- Silakan modifikasi program front-end Anda sehingga data akan tersimpan di dua dataabse, lokal, dan server ubuntu! Pastikan fungsi insert, update, delete bekerja dengan benar

Catatan Troubleshooting:

Agar proses update dan delete bekerja dengan baik, maka nilai dari “ID_MGS” dari data yang adda di database lokal dengan ubuntu harus sama. Gunakan solusi berikut untuk mengatasi permasalahan tersebut:



- Hapus semua data terdahulu di database lokal dan ubuntu. Jalankan perintah ini di MySQL

```
delete from mahasiswa
```

- Set auto increment menjadi 1. Jalankan perintah ini di MySQL

```
alter table mahasiswa set auto_increment=1;
```

- Atau silakan menggunakan cara lain, sehingga ID_MHS yang ada di database lokal dengan database server ubuntu menjadi sama.

7.6. REFERENSI

- [1]. IBM, “Data synchronization,” Ibm.com, Dec. 12, 2024.
<https://www.ibm.com/think/topics/data-synchronization>



PERTEMUAN VIII

API AUTHENTICATION

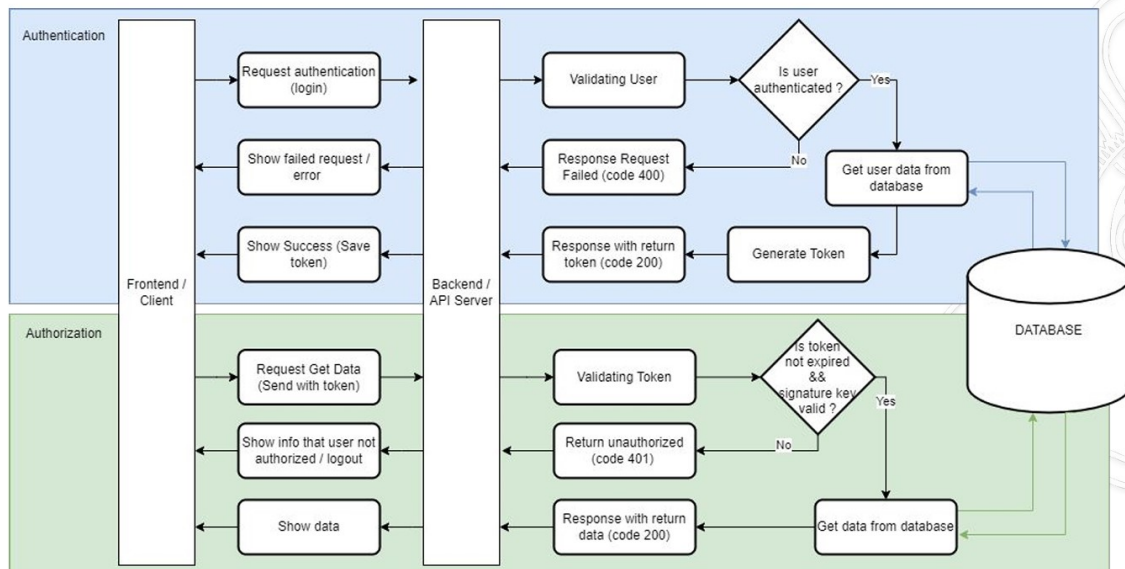
8.1. TUJUAN PEMBELAJARAN

- A. Dapat melakukan autentikasi pada API
- B. Menjalankan sistem register yang dapat membuat token
- C. Melakukan login menggunakan token yang ada berdasarkan role yang tersedia

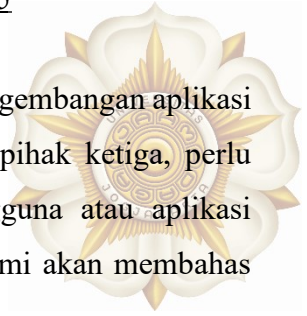
8.2. DASAR TEORI

Dalam praktikum ini, kita akan mempelajari pentingnya dan langkah-langkah untuk mengimplementasikan sistem autentikasi (Authentication) pada API. Namun, sebelum mengimplementasikan sistem autentikasi, kita perlu mengetahui terlebih dahulu perbedaan autentikasi dan otorisasi. Untuk memahaminya dapat dengan cepat, silakan pahami alur diagram Gambar 8.2.1. Secara singkat autentikasi kurang lebih seperti pertanyaan “siapa kamu?”, sedangkan otorisasi lebih ke pertanyaan “apakah kamu punya akses?”

Which comes first, authentication or authorization ?



Gambar 8.2.1 Autentikasi dan Otorisasi



API Authentication adalah salah satu aspek yang sangat kritis dalam pengembangan aplikasi modern. Saat API digunakan oleh pengguna eksternal atau aplikasi pihak ketiga, perlu adanya mekanisme yang aman untuk memverifikasi identitas pengguna atau aplikasi sebelum memberikan akses ke data atau layanan yang terlindungi. Kami akan membahas metode autentikasi JSON Web Tokens (JWT).

Dengan pemahaman yang baik tentang API Authentication, Anda akan dapat memberikan tingkat keamanan yang tinggi pada API Anda dan memberikan kontrol akses yang tepat kepada pengguna yang berwenang.

8.3. ALAT DAN BAHAN

8.3.1. Perangkat Keras

- Komputer/Laptop

8.3.2. Perangkat Lunak

- Postman
- Aplikasi contoh JWT
- Visual Studio Code
- Git

8.4. LANGKAH PERCOBAAN

1. Mempersiapkan source code yang akan digunakan

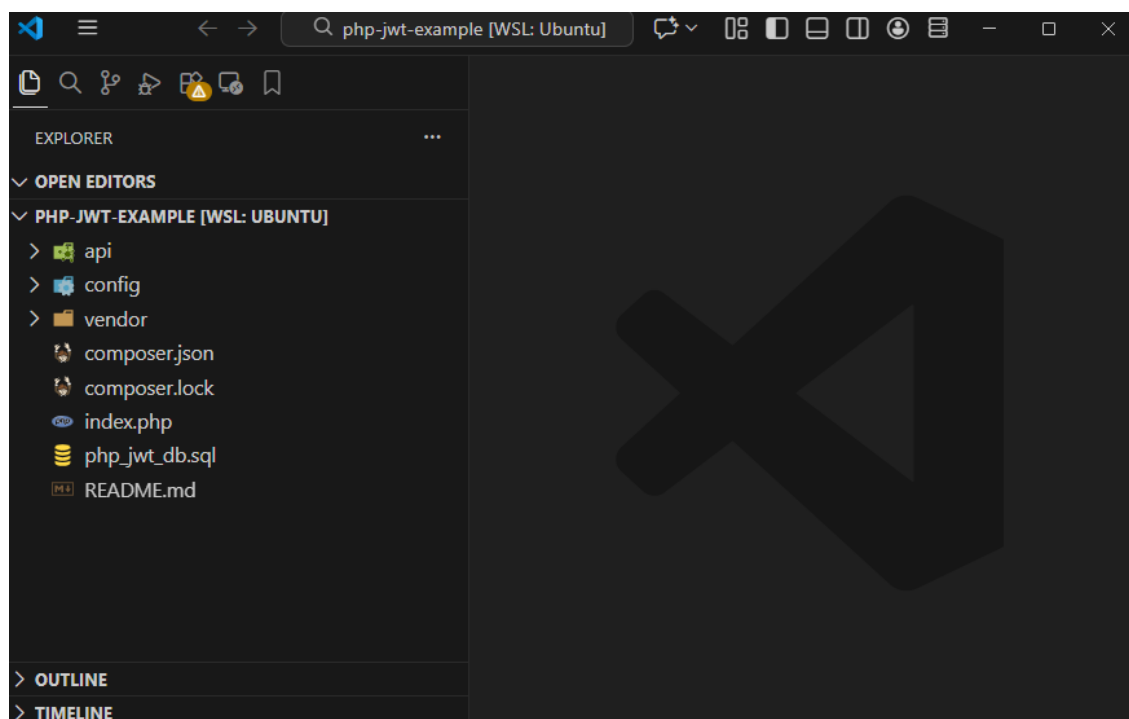
Atau juga bisa melalui CLI seperti Gambar 8.4.1. Buka source code dengan Visual Studio Code, tampilan akan terlihat seperti Gambar 8.4.2.

```
git clone https://github.com/hananhanafi/php-jwt-example.git
```



```
[20:05:04] ~/github
$ git clone https://github.com/hananhanafi/php-jwt-example.git
Cloning into 'php-jwt-example'...
remote: Enumerating objects: 57, done.
remote: Counting objects: 100% (57/57), done.
remote: Compressing objects: 100% (45/45), done.
remote: Total 57 (delta 7), reused 51 (delta 4), pack-reused 0 (from 0)
Receiving objects: 100% (57/57), 37.12 KiB | 48.00 KiB/s, done.
Resolving deltas: 100% (7/7), done.
[20:08:25] ~/github
$ cd php-jwt-example
[20:08:43] ~/github/php-jwt-example
$
```

Gambar 8.4.1 CLI Git clone



Gambar 8.4.2 Source Code JWT

Di file index.php di folder api/login, Anda dapat melihat contoh proses pembuatan JWT di PHP menggunakan library JWT dari Firebase seperti Gambar 8.4.3. Oleh karena JWT merupakan enkripsi dua arah yang memerlukan kata kunci, secret key dari sebuah aplikasi yang kalian buat tidak boleh tersebar keluar. Anda dapat menambahkan secret ketika aplikasi sudah berada di dalam komputer produksi. Secret Key ini akan digunakan untuk decode dan encode. Perlu menjadi perhatian bahwa enkripsi yang digunakan di JWT tidak sepenuhnya aman, sehingga dapat dipertimbangkan untuk tidak mengirimkan



informasi yang sangat sensitif. Di Gambar 8.4.3, informasi yang dikirim meliputi id, firstname, email, dan role, yang mana informasi ini cukup untuk proses otorisasi yang akan dilakukan.

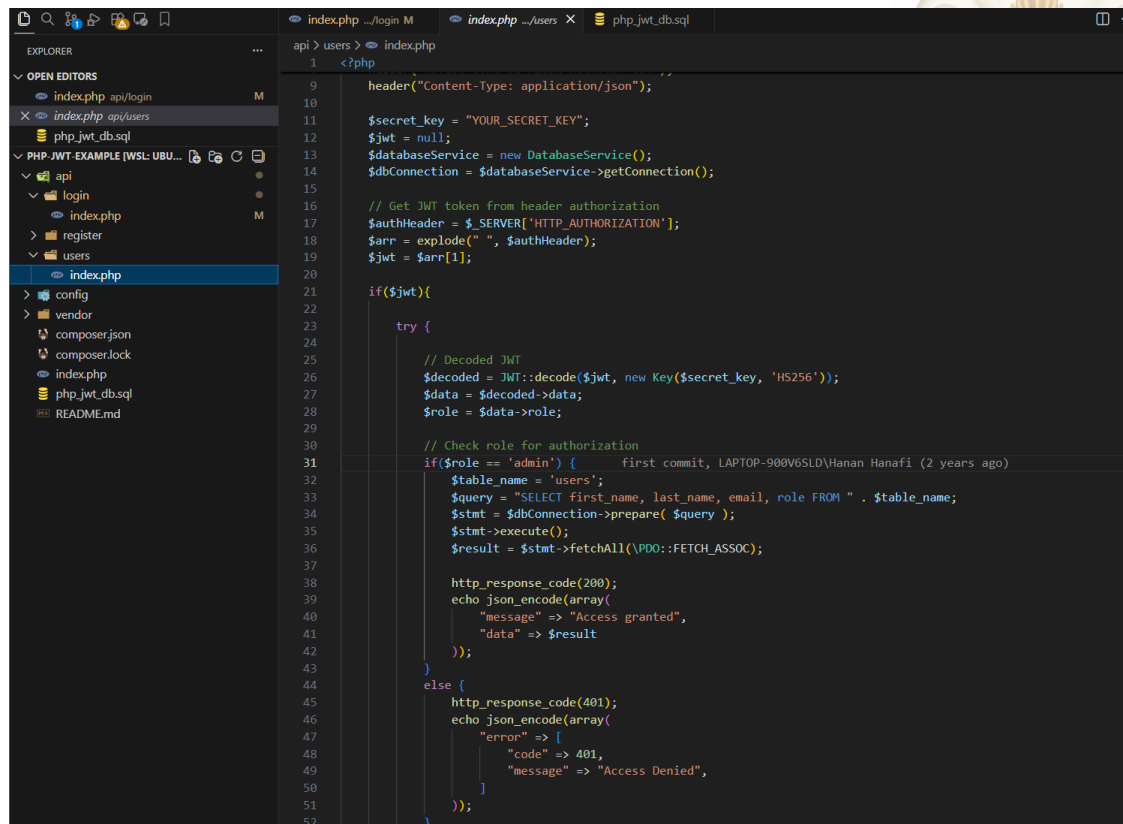
```

1  <?php
28 if($num > 0){
34     $hashedPassword = $row['password'];
35
36     if(password_verify($password, $hashedPassword)){
37         $secret_key = "YOUR_SECRET_KEY";
38         $issuer_claim = "THE_ISSUER"; // this can be the servername
39         $audience_claim = "THE_AUDIENCE"; // this can be the client name
40         $issuedat_claim = time(); // issued at
41         // $expire_claim = $issuedat_claim + 600; // expire time in seconds
42         $expire_claim = $issuedat_claim + 10; // expire time in seconds
43         $token = array(
44             "iss" => $issuer_claim,
45             "aud" => $audience_claim,
46             "iat" => $issuedat_claim,
47             "exp" => $expire_claim,
48             "data" => array(
49                 "id" => $id,
50                 "firstname" => $firstname,
51                 "lastname" => $lastname,
52                 "email" => $email,
53                 "role" => $role
54             )
55         );
56
57         // jwt encoded
58         $jwt = JWT::encode($token, $secret_key, 'HS256');
59
60         http_response_code(200);
61         echo json_encode(
62             array(
63                 "message" => "Successful login.",
64                 "jwt" => $jwt,
65                 "email" => $email,
66                 "expireAt" => $expire_claim
67             )
68         );
69     }

```

Gambar 8.4.3 Pembuatan JWT di Login

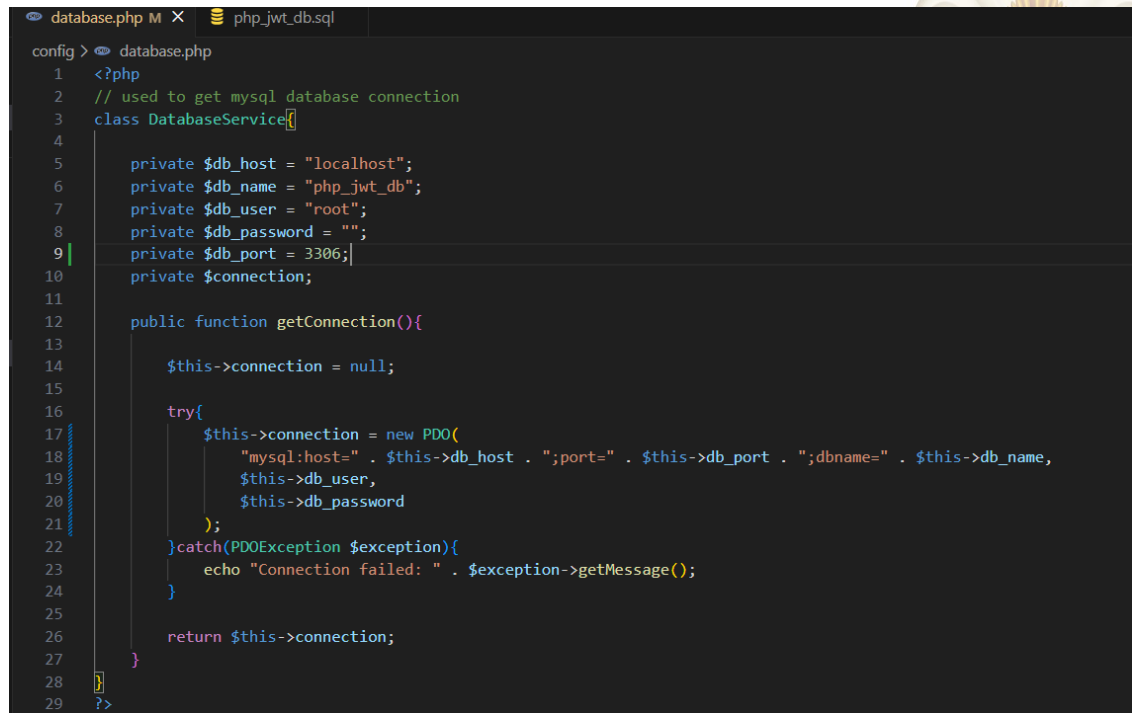
Untuk melakukan pembacaan JWT yang dibuat, Anda memerlukan secret key yang sama ketika pembuatan. Secret key ini digunakan untuk memvalidasi bahwa JWT ini tidak dilakukan tampering. Di Gambar 8.4.4, JWT didecode dengan menggunakan library yang sama ketika pembuatan dan dengan konfigurasi secret key dan algoritma yang sama. Di library yang digunakan, jika JWT dilakukan tampering oleh user, error akan terjadi sehingga pembacaan JWT tidak dilanjutkan. Setelah JWT berhasil didecode, payload yang sebelumnya dilampirkan di JWT dapat dibaca dan digunakan untuk otorisasi, contohnya mengecek role user. Manfaat adanya JWT seperti hal tersebut adalah mengurangi beban server karena jika role harus diambil melalui database, maka kebutuhan komputasi menjadi lebih besar, dengan adanya payload role di JWT yang dibuat, pengecekan role dapat dilakukan dengan stateless atau tanpa membaca database.



Gambar 8.4.4 Decode JWT

2. Mempersiapkan konfigurasi konektivitas database

Setelah memahami konsep pembuatan JWT dan decode JWT, untuk mencoba implementasi JWT di source code ini, Anda perlu melakukan konfigurasi database terlebih dahulu. Ubah file database.php sesuai dengan konfigurasi yang kalian gunakan. Sesuaikan host MySQL, nama database yang telah dibuat (jika belum, Anda dapat membuatnya terlebih dahulu), user database dan passwordnya, dan juga port database (secara default port yang digunakan untuk MySQL adalah 3306, namun jika Anda membuat custom seperti pada BAB sebelumnya, maka ganti port tersebut sesuai dengan port yang kalian gunakan). Tambahan penulisan port dilakukan seperti Gambar 8.4.5.



```

config > database.php
1 <?php
2 // used to get mysql database connection
3 class DatabaseService{
4
5     private $db_host = "localhost";
6     private $db_name = "php_jwt_db";
7     private $db_user = "root";
8     private $db_password = "";
9     private $db_port = 3306;
10    private $connection;
11
12    public function getConnection(){
13
14        $this->connection = null;
15
16        try{
17            $this->connection = new PDO(
18                "mysql:host=" . $this->db_host . ";port=" . $this->db_port . ";dbname=" . $this->db_name,
19                $this->db_user,
20                $this->db_password
21            );
22        }catch(PDOException $exception){
23            echo "Connection failed: " . $exception->getMessage();
24        }
25
26        return $this->connection;
27    }
28 }
29 ?>

```

Gambar 8.4.5 Konfigurasi file config/database.php

3. Tambahkan tabel ke database

Selanjutnya, masuk ke dalam kode php_jwt_db.sql yang ada di source code Gambar 8.4.6. Jika Anda perhatikan, hasil export SQL ini dibuat oleh phpMyAdmin versi 5 dengan PHP versi 8.1. Perlu diketahui bahwa melakukan impor database langsung mungkin dapat terjadi kegagalan oleh karena bedanya versi COLLATE. Jika Anda mengalami kegagalan, maka ada beberapa opsi yang dapat Anda lakukan (pilih salah satu saja):

- Mengubah COLLATE menjadi utf8mb4_general_ci atau sesuai konfigurasi COLLATE yang digunakan di database Anda.
- Melakukan impor secara manual seperti Gambar 8.4.7.
- Pembuatan manual menggunakan phpMyAdmin:
 - Buka phpMyAdmin lalu click new dan masukkan nama database yang diinginkan. Jangan lupa untuk mengatur collation yang sesuai dengan file SQL. Lakukan import database SQL melalui bagian import, kemudian jalankan. Tampilan setelah import database akan memberikan informasi sukses dengan warna kuning, sedangkan warna merah untuk memberikan informasi bahwa database



gagal diimpor. Tabel users akan berisikan id, first_name, last_name, email, password dan role. ---

- Pembuatan manual dengan bantuan RDMS lainnya.

```

1  -- phpMyAdmin SQL Dump
2  -- version 5.2.0
3  -- https://www.phpmyadmin.net/
4  --
5  -- Host: localhost:3306
6  -- Waktu pembuatan: 22 Bulan Mei 2023 pada 10.33
7  -- Versi server: 8.0.30
8  -- Versi PHP: 8.1.10
9
10 SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO";
11 START TRANSACTION;
12 SET time_zone = "+00:00";
13
14
15 /*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
16 /*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
17 /*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
18 /*!40101 SET NAMES utf8mb4 */;
19
20 --
21 -- Database: `php_jwt_db`
22 --
23
24
25
26
27 -- Struktur dari tabel `users`
28 --
29
30 CREATE TABLE `users` (
31   `id` int NOT NULL,
32   `first_name` varchar(150) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci DEFAULT NULL,
33   `last_name` varchar(150) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci DEFAULT NULL,
34   `email` varchar(255) DEFAULT NULL,
35   `password` varchar(255) DEFAULT NULL,
36   `role` varchar(100) DEFAULT NULL
37 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
38
39

```

Gambar 8.4.6 SQL Tabel User untuk contoh aplikasi JWT



```
mysql> create database php_jwt_db;
Query OK, 1 row affected (0.01 sec)

mysql> use php_jwt_db;
Database changed
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE TABLE `users` (
  -> `id` int NOT NULL,
  -> `first_name` varchar(150) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci DEFAULT NULL,
  -> `last_name` varchar(150) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci DEFAULT NULL,
  -> `email` varchar(255) DEFAULT NULL,
  -> `password` varchar(255) DEFAULT NULL,
  -> `role` varchar(100) DEFAULT NULL
  -> );
Query OK, 0 rows affected (0.02 sec)

mysql> ALTER TABLE `users`
  -> ADD PRIMARY KEY (`id`);
Query OK, 0 rows affected (0.03 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> ALTER TABLE `users`
  -> MODIFY `id` int NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=3;
Query OK, 0 rows affected (0.03 sec)
Records: 0 Duplicates: 0 Warnings: 0

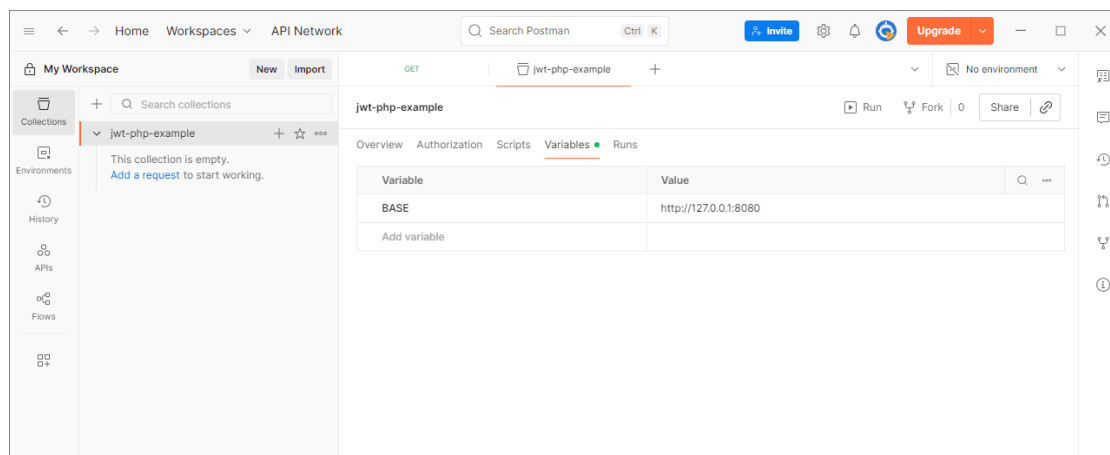
mysql> SHOW TABLES;
+-----+
| Tables_in_php_jwt_db |
+-----+
| users                 |
+-----+
1 row in set (0.01 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)
```

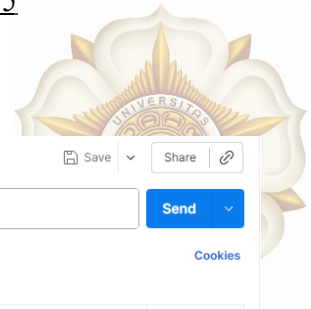
Gambar 8.4.7 Injeksi SQL manual

4. Membuat konfigurasi API di Postman

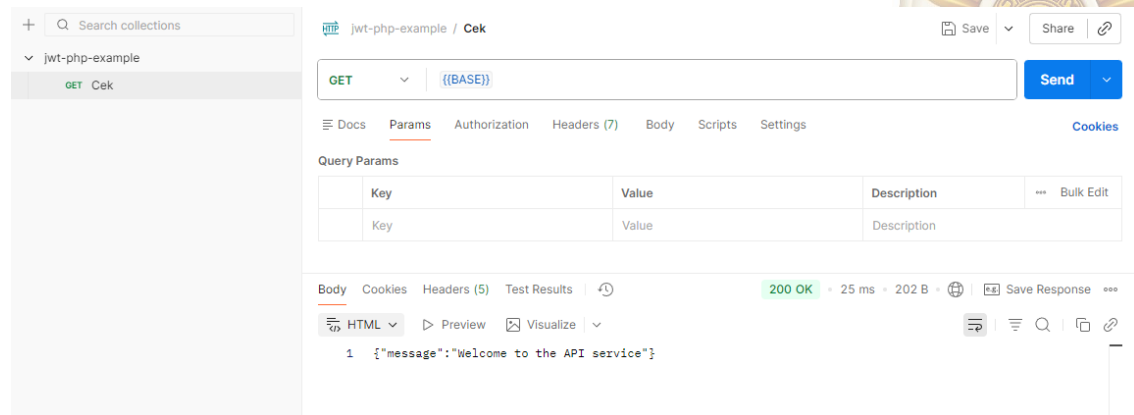
Buat collection baru, sebagai contoh jwt-php-example. Klik collection tersebut dan klik menu variable. Tambahkan variable BASE dan isikan dengan base url, sebagai contoh dapat dilihat di Gambar 8.4.8 dan simpan.



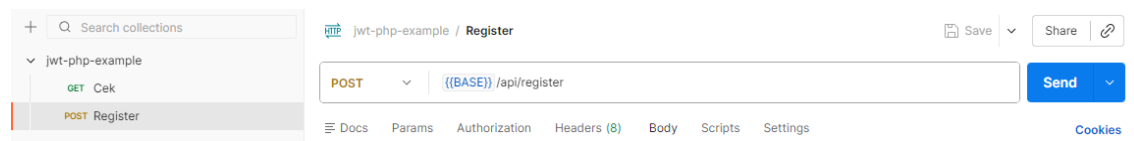
Gambar 8.4.8 Pembuatan Variable Base di Postman



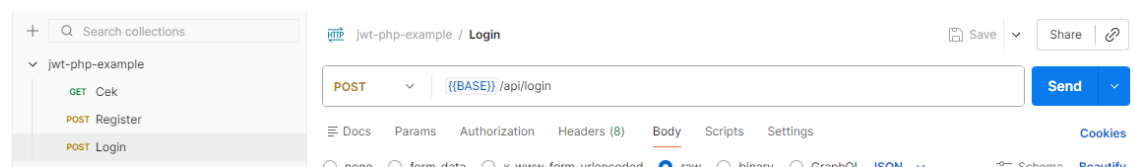
Tambahkan request-request berikut:



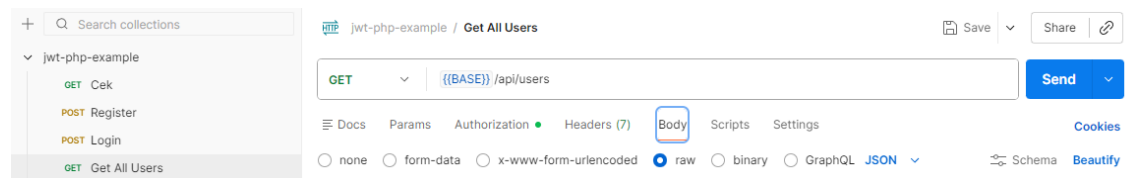
Gambar 8.4.9 Request BASE



Gambar 8.4.10 Request Register



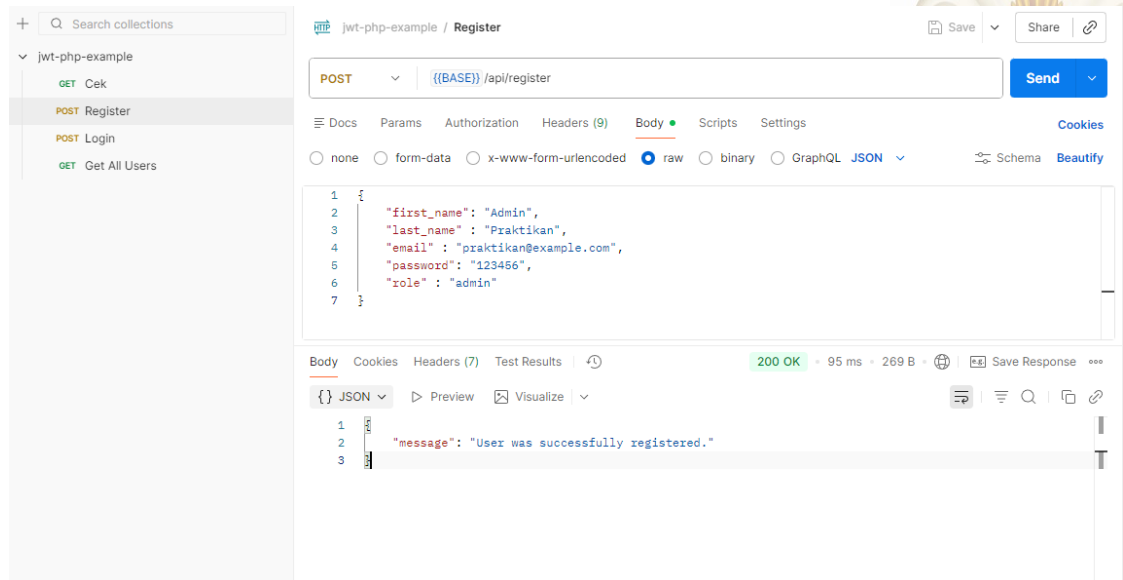
Gambar 8.4.11 Request Login



Gambar 8.4.12 Request Get All Users

5. Uji Register User pada Postman

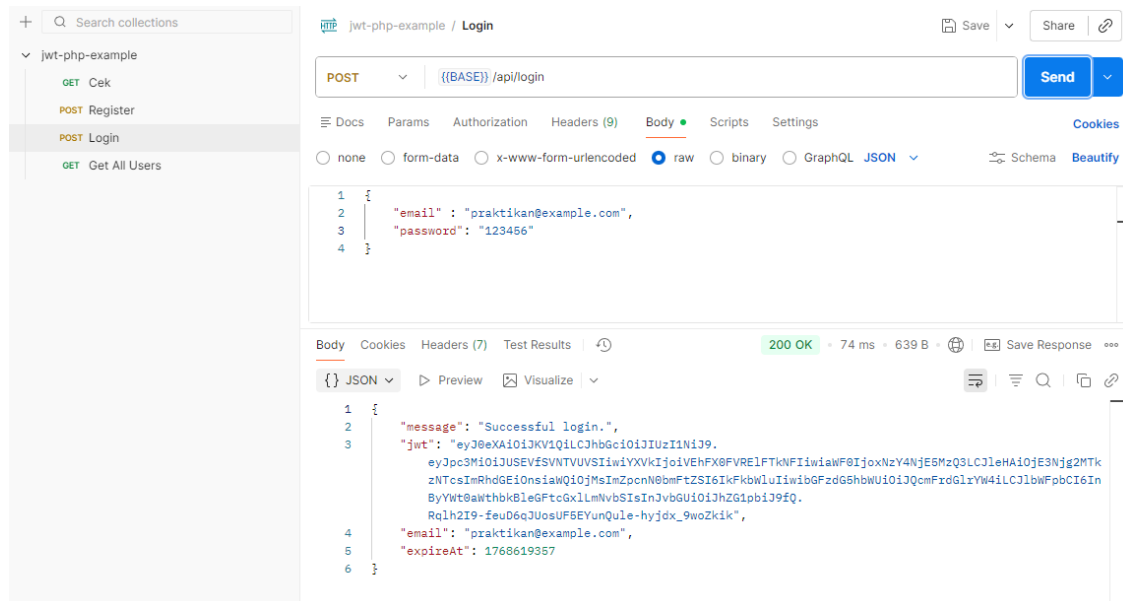
Masuk ke dalam request Register bagian Body Raw. Buat JSON First name, last name, email, password pada bagian tersebut dapat diubah bebas sesuai preferensi. Berbeda untuk role yang akan memiliki akses pada masing-masing rolenya seperti admin ataupun user biasa. Lakukan send untuk mengirimkan data tersebut ke dalam tabel users.



Gambar 8.4.13 Uji Register pada Postman

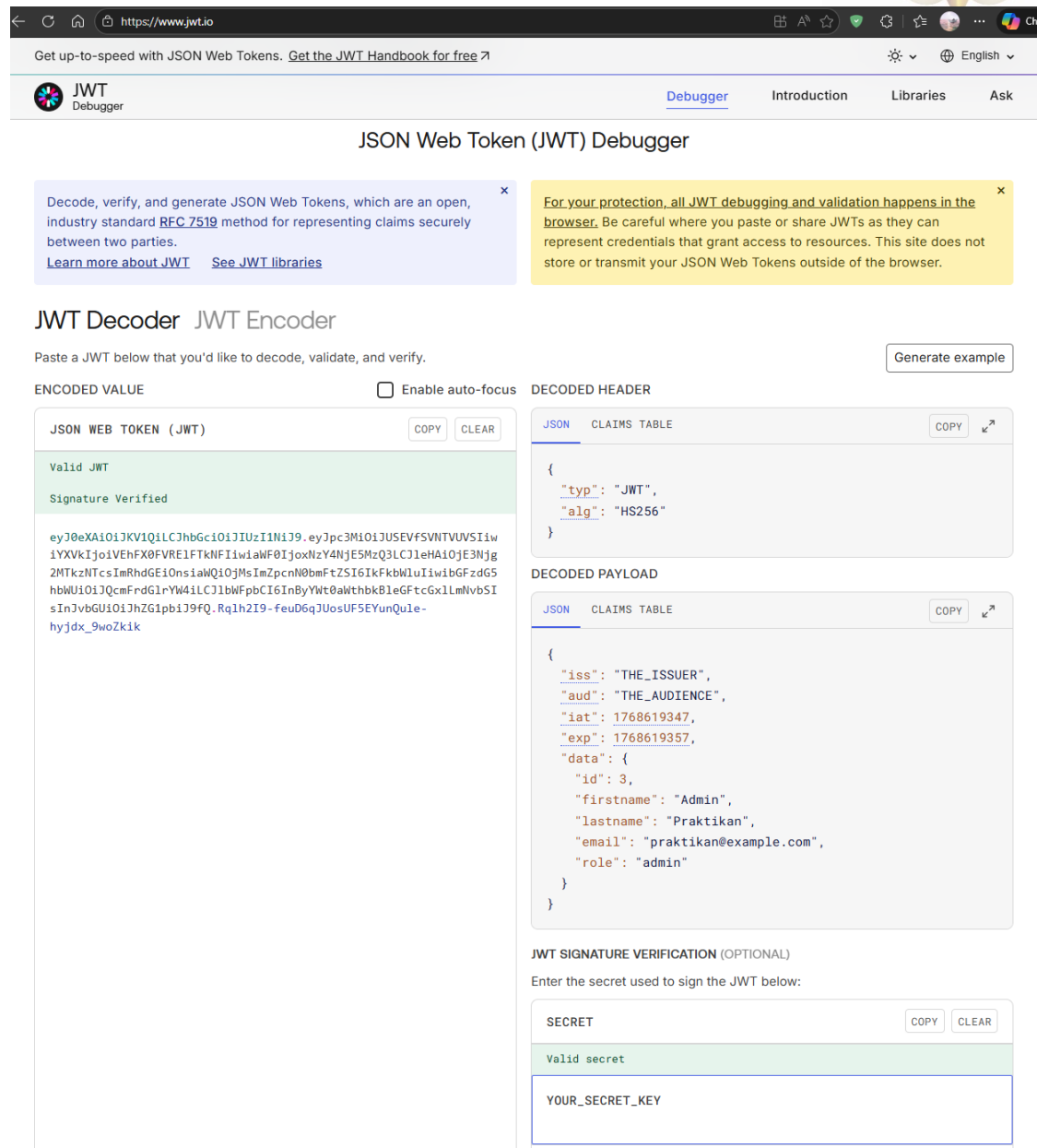
6. Uji Login pada Postman

Apabila sudah melakukan register, selanjutnya masuk ke dalam tahapan login. Tahapan ini nantinya akan memerlukan email dan password sesuai yang sudah dibuat sebelumnya pada register. Lakukan send untuk melakukan uji coba login pada postman. Ketika proses berhasil dijalankan, maka akan memunculkan pesan “Successful login” dan akan memberikan token/jwt kepada user berdasarkan role.



Gambar 8.4.14 Login

Anda juga bisa melakukan debugging isi JWT melalui website www.jwt.io seperti Gambar 8.4.15. Tambahkan token JWT dan SECRET_KEY pada kolom pengisian di website. Sebagai catatan penting untuk menghindari kebocoran data rahasia, pastikan website debugger JWT yang digunakan oleh Anda merupakan website yang terpercaya!



Get up-to-speed with JSON Web Tokens. [Get the JWT Handbook for free](#)

JWT Debugger Introduction Libraries Ask

JSON Web Token (JWT) Debugger

Decode, verify, and generate JSON Web Tokens, which are an open, industry standard [RFC 7519](#) method for representing claims securely between two parties. [Learn more about JWT](#) [See JWT libraries](#)

For your protection, all JWT debugging and validation happens in the browser. Be careful where you paste or share JWTs as they can represent credentials that grant access to resources. This site does not store or transmit your JSON Web Tokens outside of the browser.

JWT Decoder JWT Encoder

Paste a JWT below that you'd like to decode, validate, and verify. [Generate example](#)

ENCODED VALUE ☐ Enable auto-focus

JSON WEB TOKEN (JWT) [COPY](#) [CLEAR](#)

Valid JWT

Signature Verified

```
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJUSEVfSVNTVUUVSIiwiaXNkIjoieHh0FVRElFTkNFIFwiYW0IjoxNzY4NjE5MzQ3L3leHAiOiJlbnJgZMTkzNTcsImRhdGEiOi0nsiaWQ10jMsImZpcnN0bmFtZSI6IktkbW1uIiwibGFzdG5hbWUiOiJQcmFrdGlrYW4iLCJlbWVpbCI6InByYWt0aWthbkB1eGZtcGx1LmNvbSI9InJvbGU10iJhZG1pbjI9FQ.Rq1h2I9-feuD6qJUosUF5EYunQu1e-hyjdX_9woZkik
```

DECODED HEADER

JSON CLAIMS TABLE [COPY](#)

```
{
  "typ": "JWT",
  "alg": "HS256"
}
```

DECODED PAYLOAD

JSON CLAIMS TABLE [COPY](#)

```
{
  "iss": "THE_ISSUER",
  "aud": "THE_AUDIENCE",
  "iat": 1768619347,
  "exp": 1768619357,
  "data": {
    "id": 3,
    "firstname": "Admin",
    "lastname": "Praktikan",
    "email": "praktikan@example.com",
    "role": "admin"
  }
}
```

JWT SIGNATURE VERIFICATION (OPTIONAL)

Enter the secret used to sign the JWT below:

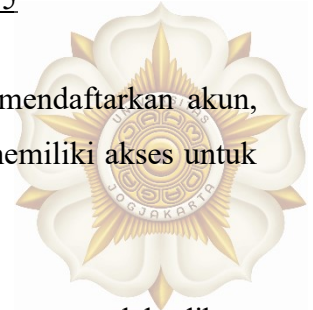
SECRET [COPY](#) [CLEAR](#)

Valid secret

YOUR_SECRET_KEY

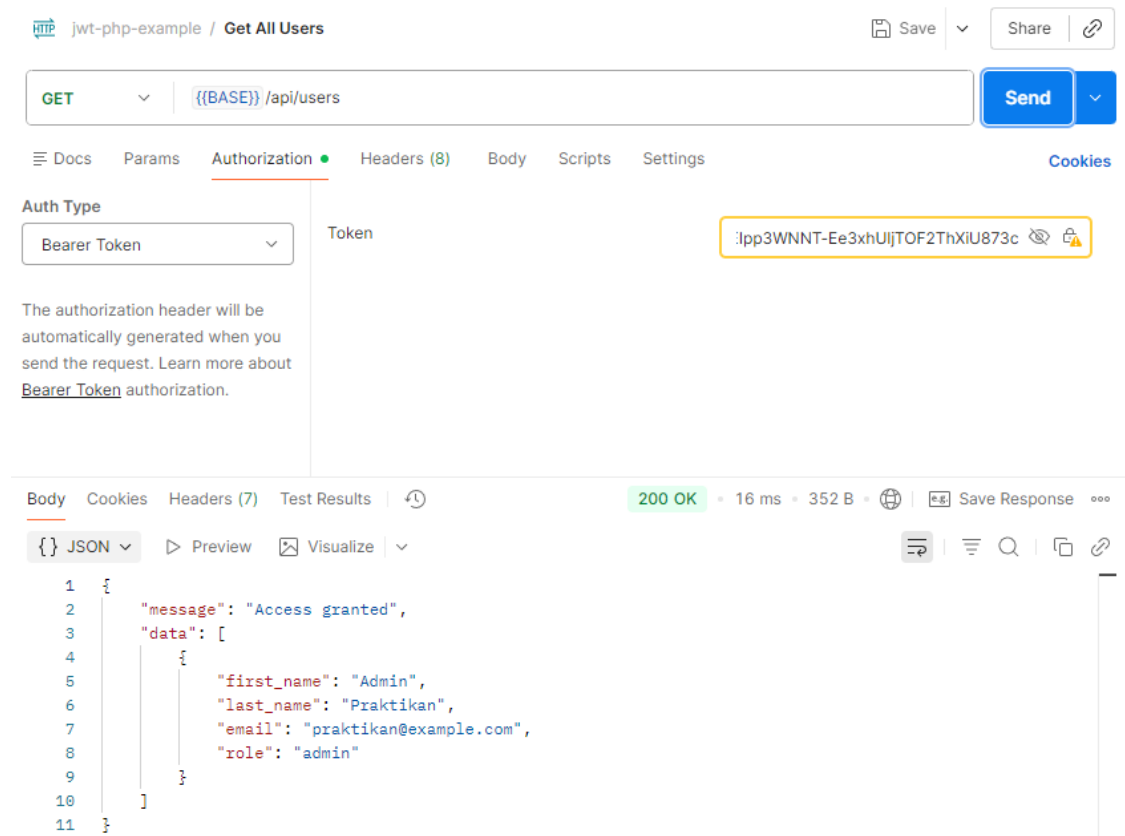
Gambar 8.4.15 Debug JWT

- Uji data seluruh User pada Postman menggunakan akun admin



Admin dapat melihat data tiap masing-masing user yang sudah mendaftarkan akun, mulai dari first name, last name, email dan role. Role user tidak memiliki akses untuk melihat data user karena hanya admin yang dapat melihatnya.

Untuk mengaksesnya, silakan gunakan method GET All Users yang telah dibuat sebelumnya, pada bagian authorization Gambar 8.4.16, ubah tipe Auth menjadi Bearer Token dan masukkan token yang dapat dilihat setelah melakukan login Gambar 8.4.14. Pastikan segera memasukkan token dan mengirimkan request karena di Gambar 8.4.3, kita telah mengatur validasi token selama 10 detik. Jika hal tersebut terlalu cepat, Anda dapat mengubahnya sesuai kebutuhan.



Gambar 8.4.16 Melihat data user

Ketika user biasa ingin melihat data yang ada dengan memasukkan tokennya, maka akses melihat data akan ditolak. Apabila admin yang memasukkan token, maka akses akan diterima dan admin dapat melihat data-data dari tiap masing-masing user.



8.5. TUGAS & ANALISIS

1. Apa yang terjadi ketika melakukan request akses data user menggunakan akun dengan role admin dan jwt token yang valid?
2. Apa yang terjadi ketika melakukan request akses data user menggunakan akun dengan role admin dan jwt token yang tidak valid?
3. Apa yang terjadi jika akun dengan role user biasa (bukan admin) mengakses data user?
4. Apa yang terjadi jika melakukan register dengan menginput role selain admin dan user biasa?
5. Apa yang terjadi jika akun dengan role selain admin dan user biasa mengakses data user?

8.6. REFERENSI

- [1]. GeeksforGeeks, “What is API Authentication? Definition and Working,” GeeksforGeeks, Mar. 11, 2025. <https://www.geeksforgeeks.org/ethical-hacking/what-is-api-authentication-definition-and-working/>



PERTEMUAN IX

API PHP DENGAN DOCKER

9.1. TUJUAN PEMBELAJARAN

- A. Mahasiswa diharapkan dapat memahami konsep penggunaan DOcker dan membuat PHP API menggunakan Docker
- B. Mahasiswa diharapkan dapat memahami dan melakukan implementasi verifikasi API menggunakan Docker

9.2. DASAR TEORI

Dalam praktikum ini, kita akan mempelajari cara menggunakan Docker untuk membuat dan mengelola lingkungan pengembangan yang terisolasi, sehingga dapat menjalankan API PHP dengan mudah dan efisien.

Docker adalah platform open-source yang memungkinkan kita untuk mengemas, mendistribusikan, dan menjalankan aplikasi dalam wadah (container) yang ringan dan portabel. Dengan menggunakan Docker, kita dapat memastikan bahwa lingkungan pengembangan di setiap mesin adalah seragam, sehingga menghindari masalah yang terjadi karena perbedaan konfigurasi.

Tetapi kali ini kita hanya akan fokus pada penggunaan Docker untuk menjalankan API PHP. Kita akan belajar bagaimana membuat sebuah wadah Docker yang berisi konfigurasi PHP dan server web (misalnya Apache) yang diperlukan untuk menjalankan API. Kita juga akan membahas tentang bagaimana memasukkan kode API PHP ke dalam wadah dan mengeksposnya ke luar melalui port yang sesuai.

Dengan pemahaman yang baik tentang penggunaan API PHP dengan Docker, kalian akan memiliki kemampuan untuk mengembangkan dan menyebarkan API PHP Anda dengan cepat, konsisten, dan portable di berbagai lingkungan pengembangan dan produksi.



9.3. ALAT DAN BAHAN

9.3.1. Perangkat Keras

- VPS

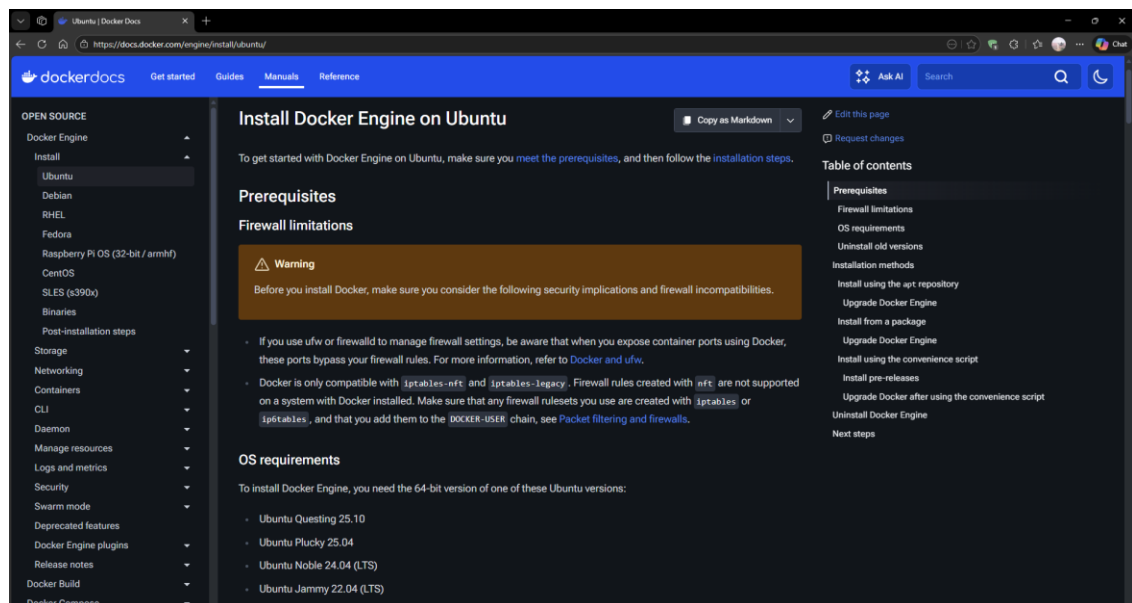
9.3.2. Perangkat Lunak

- Docker
- Visual Studio Code
- Postman

9.4. LANGKAH PERCOBAAN

1. Dokumentasi Docker

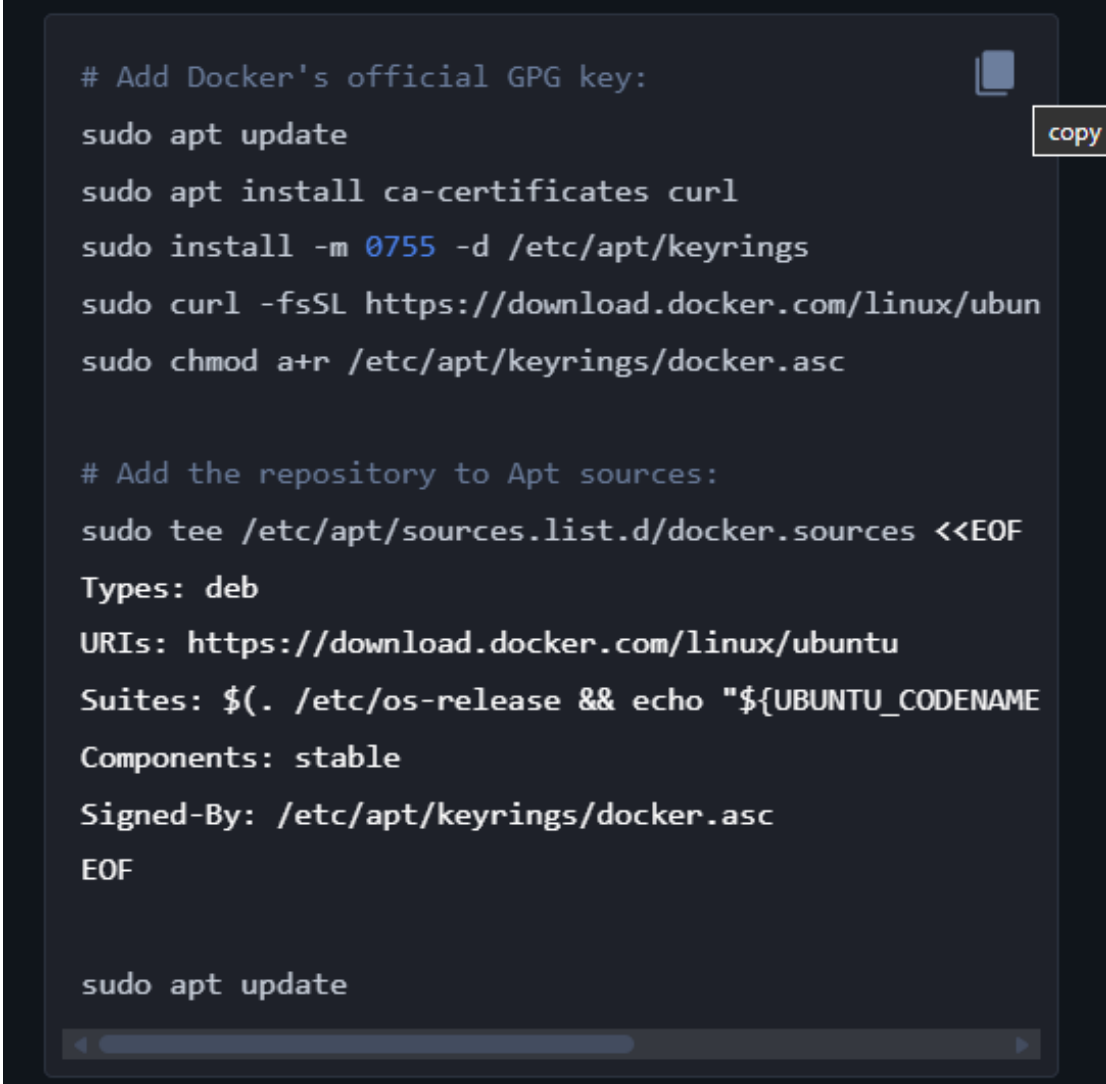
Sebelum melakukan instalasi, sebaiknya perlu mengecek terlebih dahulu dokumentasi resmi Docker di docs.docker.com. Di laman tersebut, terdapat syarat instalasi dan prosedur instalasi. Oleh karena instalasi akan dilakukan di Ubuntu 22, maka pilihlah menu install ubuntu seperti Gambar 9.4.1. Jika Anda mencoba mengunduh versi Windows, gunakan Docker Desktop. Docker Desktop untuk Windows juga memiliki syarat, yaitu komputer harus support WSL2 terbaru dan layanan Windows tidak dinonaktifkan oleh Microsoft.



Gambar 9.4.1 docs.docker.com

2. Menyiapkan repository docker di Ubuntu Server

Jalankan kode Gambar 9.4.2 di terminal Ubuntu Server. Jika semua tertulis done, maka tidak ada error seperti Gambar 9.4.3.

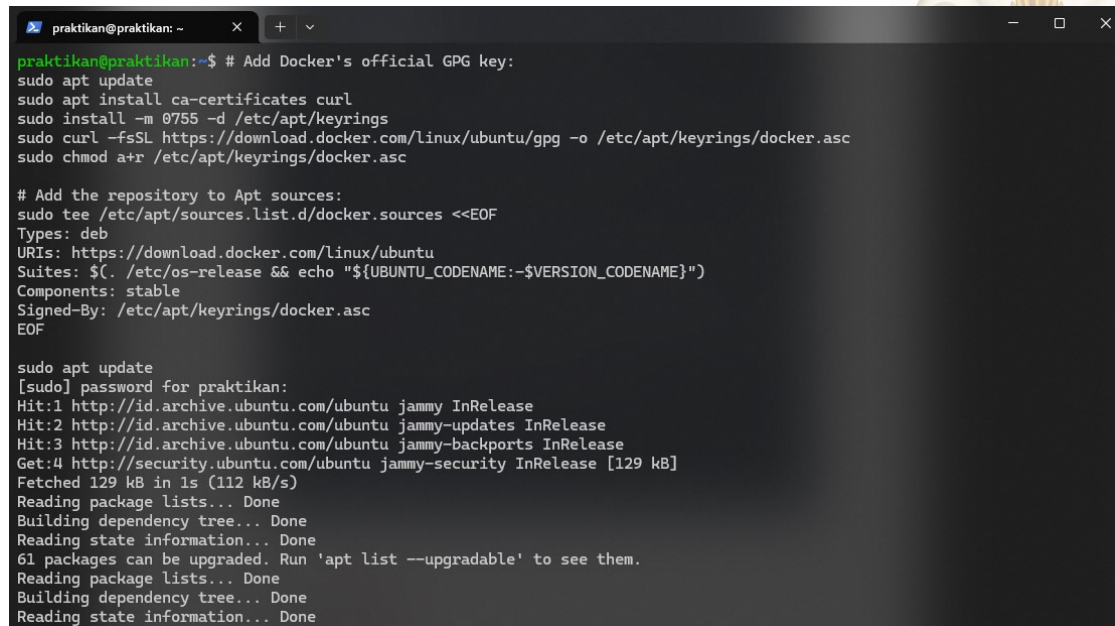


```
# Add Docker's official GPG key:
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME}
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

Gambar 9.4.2 Set up Docker's apt repository

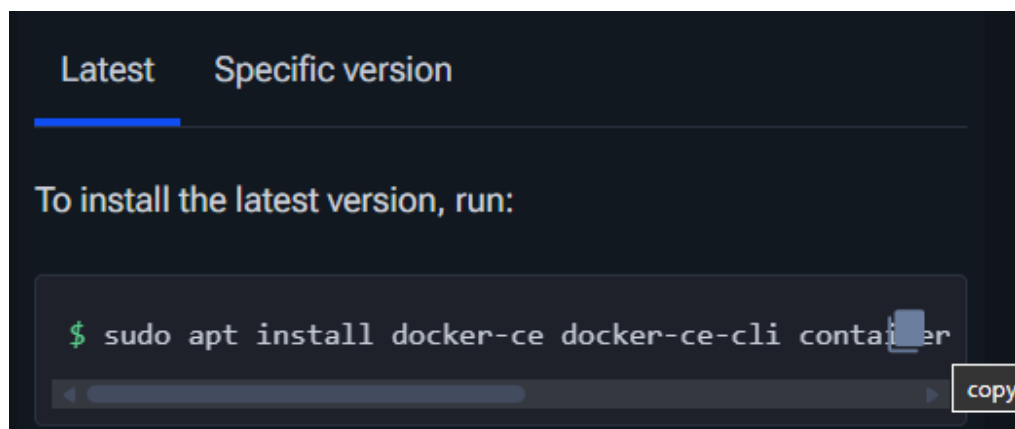


```
praktikan@praktikan: ~  
praktikan@praktikan:~$ # Add Docker's official GPG key:  
sudo apt update  
sudo apt install ca-certificates curl  
sudo install -m 0755 -d /etc/apt/keyrings  
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc  
sudo chmod a+r /etc/apt/keyrings/docker.asc  
  
# Add the repository to Apt sources:  
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF  
Types: deb  
URIs: https://download.docker.com/linux/ubuntu  
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")  
Components: stable  
Signed-By: /etc/apt/keyrings/docker.asc  
EOF  
  
sudo apt update  
[sudo] password for praktikan:  
Hit:1 http://id.archive.ubuntu.com/ubuntu jammy InRelease  
Hit:2 http://id.archive.ubuntu.com/ubuntu jammy-updates InRelease  
Hit:3 http://id.archive.ubuntu.com/ubuntu jammy-backports InRelease  
Get:4 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]  
Fetched 129 kB in 1s (112 kB/s)  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
61 packages can be upgraded. Run 'apt list --upgradable' to see them.  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done
```

Gambar 9.4.3 Hasil sukses menambah repository docker

3. Install Docker di Ubuntu Server

Jalankan kode dari bagian install docker Gambar 9.4.4 di terminal Ubuntu Server. Ketika tahap instalasi, akan ada pertanyaan tentang instalasi, ketik “y” di terminal.

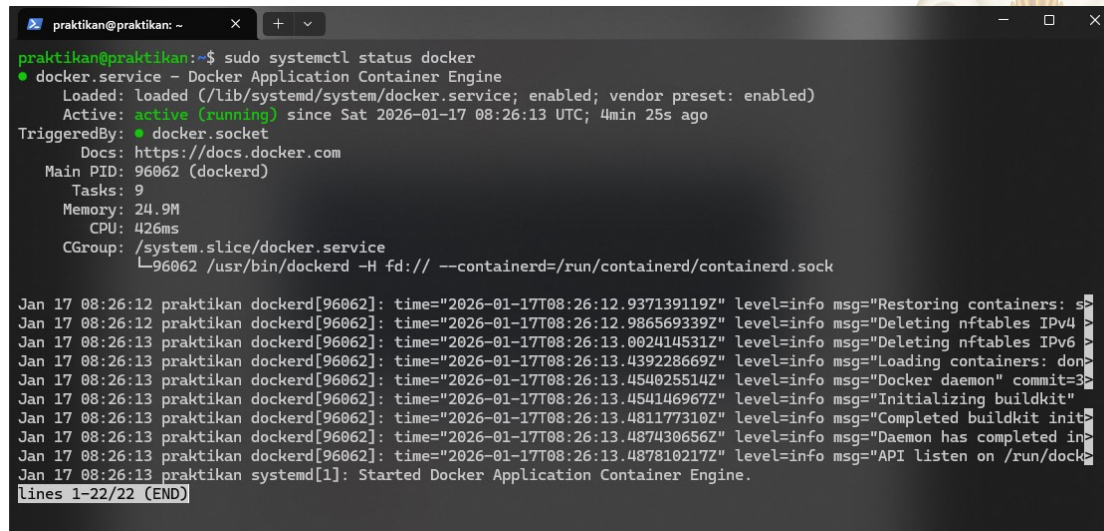


```
Latest   Specific version  
-----  
To install the latest version, run:  
  
$ sudo apt install docker-ce docker-ce-cli containerd.io
```

Gambar 9.4.4 Install the Docker packages

4. Verifikasi status Docker

Untuk melakukan verifikasi status Docker, dapat dilakukan dengan menggunakan `systemctl` seperti Gambar 9.4.5. Dapat dilihat bahwa status Docker tersebut adalah active (running) yang menandakan bahwa aplikasi service Docker berjalan. Jika belum berjalan, untuk menjalankannya dapat dilakukan dengan perintah “`sudo systemctl start docker`”.



```
praktikan@praktikan: ~  
praktikan@praktikan:~$ sudo systemctl status docker  
● docker.service - Docker Application Container Engine  
   Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor preset: enabled)  
   Active: active (running) since Sat 2026-01-17 08:26:13 UTC; 4min 25s ago  
 TriggeredBy: ● docker.socket  
    Docs: https://docs.docker.com  
   Main PID: 96062 (dockerd)  
     Tasks: 9  
    Memory: 24.9M  
       CPU: 426ms  
    CGroup: /system.slice/docker.service  
            └─96062 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock  
  
Jan 17 08:26:12 praktikan dockerd[96062]: time="2026-01-17T08:26:12.937139119Z" level=info msg="Restoring containers: s  
Jan 17 08:26:12 praktikan dockerd[96062]: time="2026-01-17T08:26:12.986569339Z" level=info msg="Deleting nftables IPv4  
Jan 17 08:26:13 praktikan dockerd[96062]: time="2026-01-17T08:26:13.002414531Z" level=info msg="Deleting nftables IPv6  
Jan 17 08:26:13 praktikan dockerd[96062]: time="2026-01-17T08:26:13.439228669Z" level=info msg="Loading containers: don  
Jan 17 08:26:13 praktikan dockerd[96062]: time="2026-01-17T08:26:13.454025514Z" level=info msg="Docker daemon" commit=3  
Jan 17 08:26:13 praktikan dockerd[96062]: time="2026-01-17T08:26:13.454146967Z" level=info msg="Initializing buildkit"  
Jan 17 08:26:13 praktikan dockerd[96062]: time="2026-01-17T08:26:13.481177310Z" level=info msg="Completed buildkit init  
Jan 17 08:26:13 praktikan dockerd[96062]: time="2026-01-17T08:26:13.487430656Z" level=info msg="Daemon has completed in  
Jan 17 08:26:13 praktikan dockerd[96062]: time="2026-01-17T08:26:13.487810217Z" level=info msg="API listen on /run/dock  
Jan 17 08:26:13 praktikan systemd[1]: Started Docker Application Container Engine.  
lines 1-22/22 (END)
```

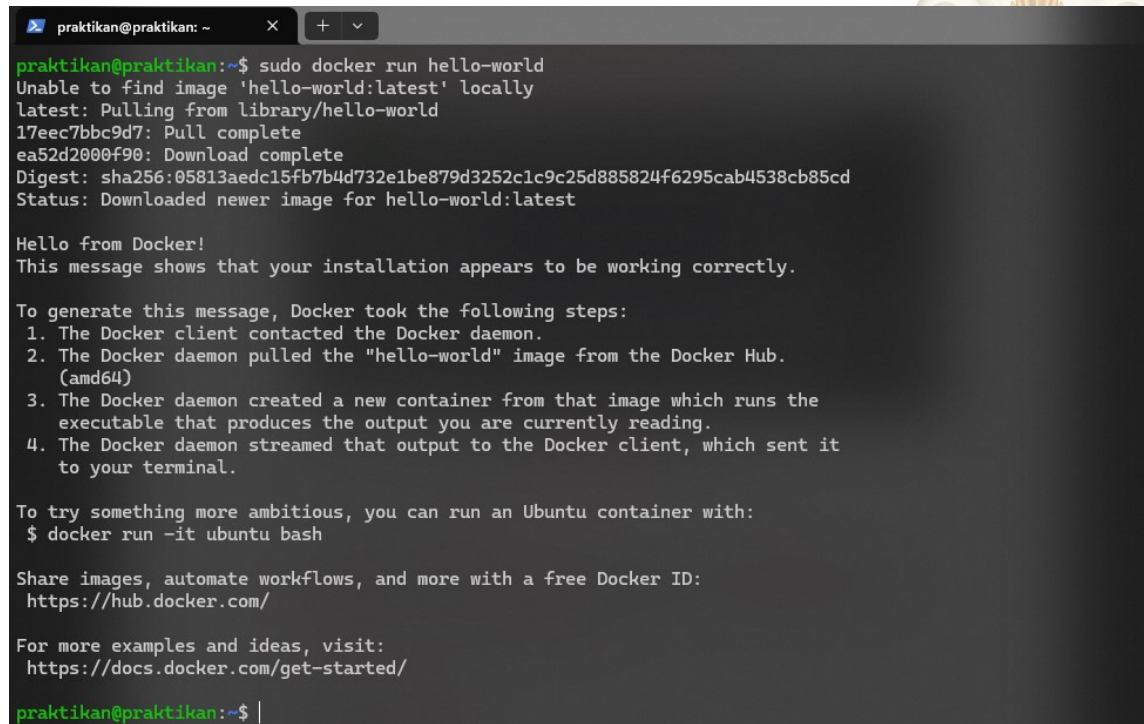
Gambar 9.4.5 Melakukan pengecekan status Docker

Untuk kembali ke terminal dari systemctl status, tekan CTRL + C.

5. Verifikasi instalasi Docker

Tahap terakhir dari instalasi Docker adalah mengecek sistem Docker itu sendiri. Docker telah menyediakan Image yang dapat digunakan untuk mengecek instalasi Docker berjalan dengan mulus atau tidak. Jalankan perintah dibawah ini untuk verifikasi instalasi Docker. Tunggu beberapa saat karena proses ini memerlukan pengunduhan data. Jika berhasil maka akan tertampil seperti Gambar 9.4.6.

```
sudo docker run hello-world
```



```
praktikan@praktikan: ~  
praktikan@praktikan:~$ sudo docker run hello-world  
Unable to find image 'hello-world:latest' locally  
latest: Pulling from library/hello-world  
17eec7bbc9d7: Pull complete  
ea52d2000f90: Download complete  
Digest: sha256:05813aedc15fb7b4d732e1be879d3252c1c9c25d885824f6295cab4538cb85cd  
Status: Downloaded newer image for hello-world:latest  
  
Hello from Docker!  
This message shows that your installation appears to be working correctly.  
  
To generate this message, Docker took the following steps:  
1. The Docker client contacted the Docker daemon.  
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.  
   (amd64)  
3. The Docker daemon created a new container from that image which runs the  
   executable that produces the output you are currently reading.  
4. The Docker daemon streamed that output to the Docker client, which sent it  
   to your terminal.  
  
To try something more ambitious, you can run an Ubuntu container with:  
$ docker run -it ubuntu bash  
  
Share images, automate workflows, and more with a free Docker ID:  
https://hub.docker.com/  
  
For more examples and ideas, visit:  
https://docs.docker.com/get-started/  
praktikan@praktikan:~$
```

Gambar 9.4.6 Instalasi Docker Success

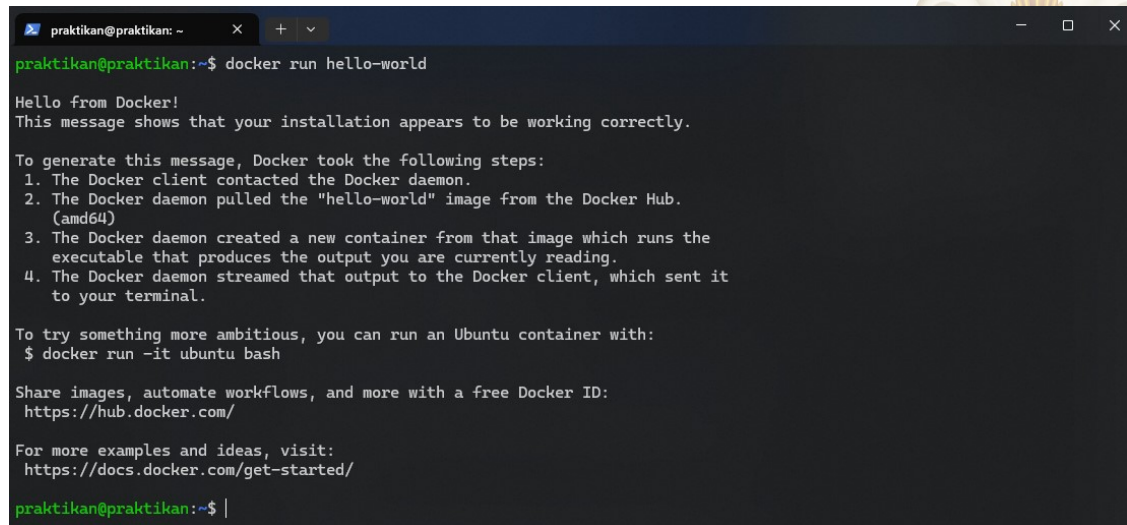
Jika terjadi error, Anda dapat melakukan troubleshooting sesuai dengan instruksi yang ada di website Docker bagian tahap instalasi yang telah kita buka.

6. Menambahkan non-root user ke Grup Docker (opsional)

Agar saat menggunakan docker tidak perlu menggunakan sudo, bisa menambahkan user yang sedang digunakan ke grup Docker. Gunakan perintah dibawah ini dan lakukan log out, dan kembali lagi.

```
sudo usermod -aG docker $USER
```

Ketika sudah login kembali, coba cek dengan menjalankan “docker run hello-world” seperti sebelumnya tanpa perintah sudo.



```
praktikan@praktikan: ~$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

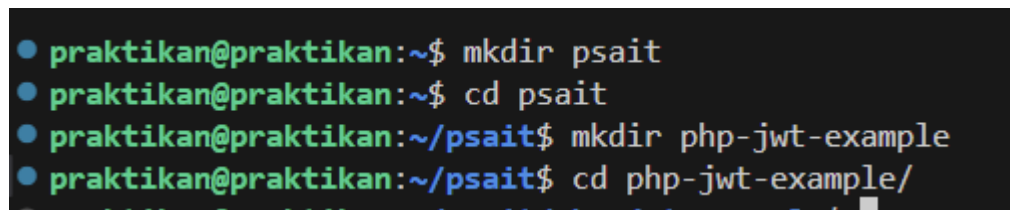
praktikan@praktikan:~$ |
```

Gambar 9.4.7 Menjalankan docker tanpa sudo

7. Buat folder baru di home direktori

Untuk mencoba menggunakan Docker, buat folder baru di home directory VPS bernama psait/php-jwt-example. Kita akan mencoba membuat ulang php-jwt-example yang sebelumnya berhasil dijalankan namun saat ini dijalankan menggunakan Docker.

```
cd ~
mkdir psait
cd psait
mkdir php-jwt-example
```

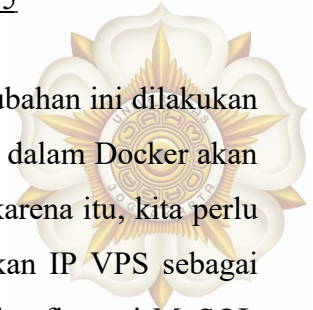


```
praktikan@praktikan:~$ mkdir psait
praktikan@praktikan:~$ cd psait
praktikan@praktikan:~/psait$ mkdir php-jwt-example
praktikan@praktikan:~/psait$ cd php-jwt-example/
```

Gambar 9.4.8 Membuat folder di Home VPS

8. Pindahkan aplikasi ke folder tersebut

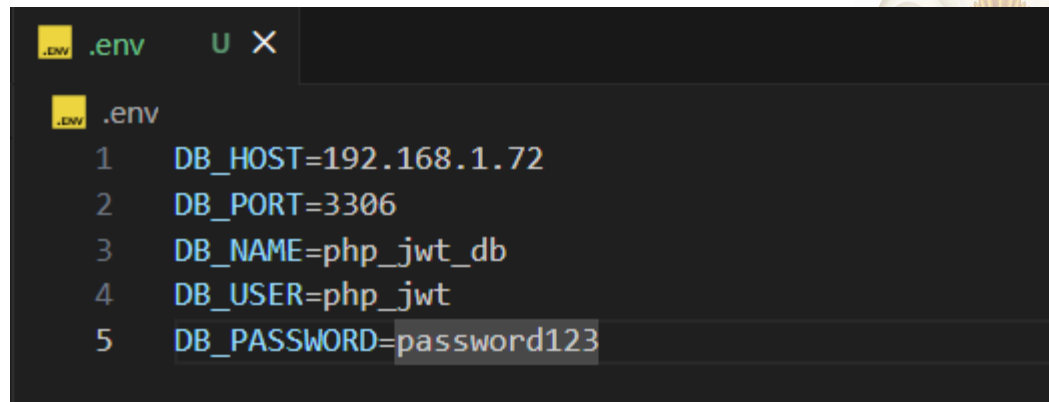
Gunakan WinSCP atau VS Code atau alat lain untuk memindahkan program php-jwt-example yang telah kita modifikasi ke dalam folder VPS psait/php-jwt-example. Setup ulang konfigurasi database, jangan lupa untuk membuat databasenya di VPS.



Ubah file config/database.php menjadi seperti Gambar 9.4.9. Pengubahan ini dilakukan karena Docker membuat virtual network sendiri sehingga localhost dalam Docker akan mengarah ke localhost docker sendiri bukan localhost VPS. Oleh karena itu, kita perlu melakukan konfigurasi database secara spesifik dengan memberikan IP VPS sebagai contoh Gambar 9.4.10, sesuaikan konfigurasi .env sesuai dengan konfigurasi MySQL server masing-masing.

```
database.php M x
config > database.php
1 <?php
2
3 class DatabaseService{
4     private $db_host;
5     private $db_name;
6     private $db_user;
7     private $db_password;
8     private $db_port;
9     private $connection;
10
11     public function __construct() {
12         // Ambil dari Environment Variable.
13         // Jika tidak ada, baru gunakan fallback (opsional)
14         $this->db_host = getenv('DB_HOST') ?: 'localhost';
15         $this->db_name = getenv('DB_NAME') ?: 'php_jwt_db';
16         $this->db_user = getenv('DB_USER') ?: 'root';
17         $this->db_password = getenv('DB_PASSWORD') ?: '';
18         $this->db_port = getenv('DB_PORT') ?: 3306;
19     }
20
21     public function getConnection(){
22
23         $this->connection = null;
24
25         try{
26             // Syntax connection string tetap sama, variabelnya yang dinamis
27             $this->connection = new PDO(
28                 "mysql:host=" . $this->db_host . ";port=" . $this->db_port . ";dbname=" . $this->db_name,
29                 $this->db_user,
30                 $this->db_password
31             );
32             $this->connection->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
33         }catch(PDOException $exception){
34             echo "Connection failed: " . $exception->getMessage();
35         }
36
37         return $this->connection;
38     }
39 }
40 ?>
```

Gambar 9.4.9 Mengubah konfigurasi Database

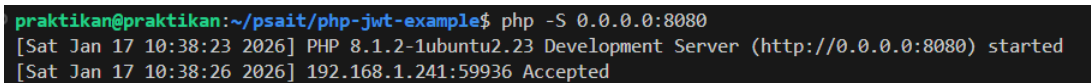


```
.env
1 DB_HOST=192.168.1.72
2 DB_PORT=3306
3 DB_NAME=php_jwt_db
4 DB_USER=php_jwt
5 DB_PASSWORD=password123
```

Gambar 9.4.10 Tambahkan file .env

9. Cek aplikasi dengan development mode sebelum membuat Docker Image

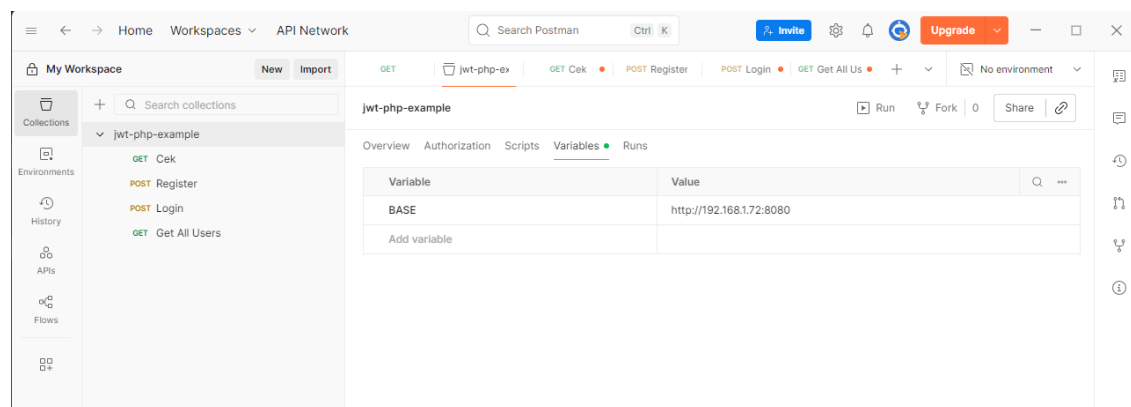
Sebelum membuat Docker Image ke aplikasi php-jwt-example ini, ada baiknya kita mengecek terlebih dahulu apakah aplikasi sudah ter-setup dengan benar dengan cara menjalankan developer server php seperti Gambar 9.4.11.



```
praktikan@praktikan:~/psait/php-jwt-example$ php -S 0.0.0.0:8080
[Sat Jan 17 10:38:23 2026] PHP 8.1.2-1ubuntu2.23 Development Server (http://0.0.0.0:8080) started
[Sat Jan 17 10:38:26 2026] 192.168.1.241:59936 Accepted
```

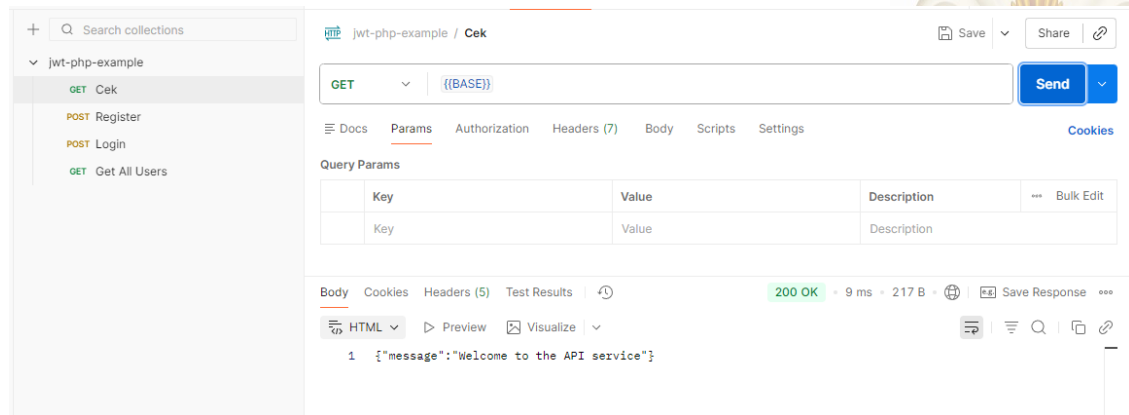
Gambar 9.4.11 Development Server PHP

Buka postman dan ganti variable base menjadi IP VPS, sebagai contoh dapat dilihat di Gambar 9.4.12.



Gambar 9.4.12 Mengubah Variable Postman

Jalankan request Cek, seharusnya program dapat berjalan lancar dan terlihat seperti Gambar 9.4.13.



Gambar 9.4.13 Mengecek API Development di Postman

Jika request tertolak, pastikan koneksi ke database telah disesuaikan dan database telah dibuat. Jika masih belum bisa, coba jalankan kode berikut: “sudo ufw allow 8080/tcp”. Kode tersebut akan membuka firewall Ubuntu Server untuk port 8080.

Jika sudah berhasil, tekan CTRL+C untuk menutup development server PHP.

10. Menjadikan aplikasi produksi

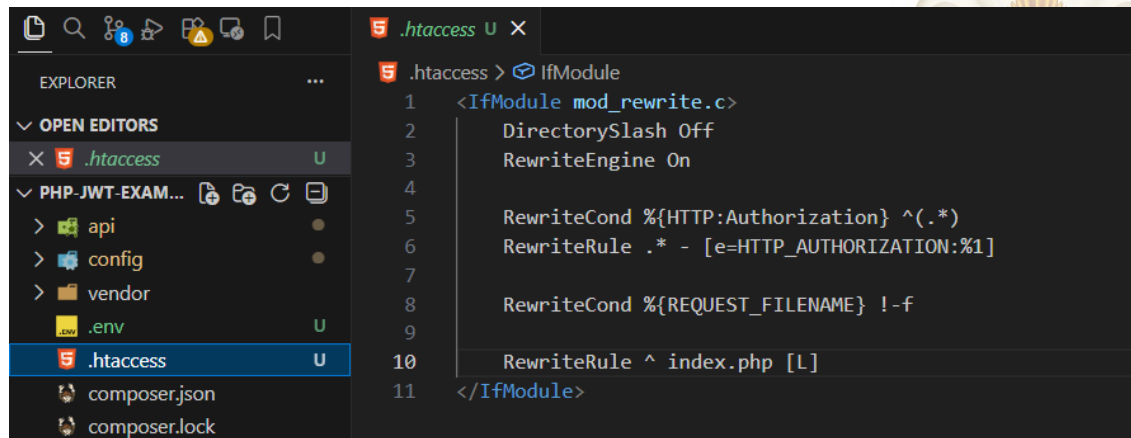
Tambahkan file .htaccess pada root project seperti Gambar 9.4.14. File ini bertujuan untuk membantu navigasi URL di apache dan aturan-aturannya.

```
<IfModule mod_rewrite.c>
    DirectorySlash Off
    RewriteEngine On

    RewriteCond %{HTTP:Authorization} ^(.*)
    RewriteRule .* - [e=HTTP_AUTHORIZATION:%1]

    RewriteCond %{REQUEST_FILENAME} !-f

    RewriteRule ^ index.php [L]
</IfModule>
```



Gambar 9.4.14 File .htaccess

Oleh karena folder tidak akan ditempatkan di folder di dalam html apache, maka kita perlu menambahkan router untuk mempermudah navigasi aplikasi. Ubah indeks.php di root project menjadi seperti Gambar 9.4.15.

```

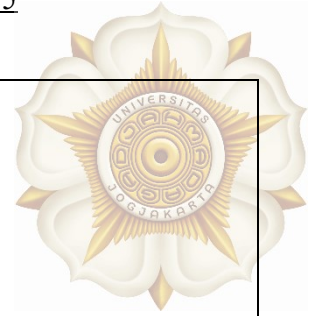
<?php
    header("Access-Control-Allow-Origin: *");
    header("Content-Type: application/json; charset=UTF-8");
    header("Access-Control-Allow-Methods: POST, GET, OPTIONS");
    header("Access-Control-Max-Age: 3600");
    header("Access-Control-Allow-Headers: Content-Type, Access-Control-Allow-Headers, Authorization, X-Requested-With");

    $requestUri = $_SERVER['REQUEST_URI'];
    $requestUri = strtok($requestUri, '?');
    if ($requestUri !== '/' && substr($requestUri, -1) === '/') {
        $requestUri = rtrim($requestUri, '/');
    }

    switch ($requestUri) {

        case '/api/login':
            if (file_exists('api/login/index.php')) {
                require 'api/login/index.php';
            } else {
                http_response_code(500);
                echo json_encode(["message" => "File api/login/index.php tidak ditemukan!"]);
            }
    }

```



```
        break;

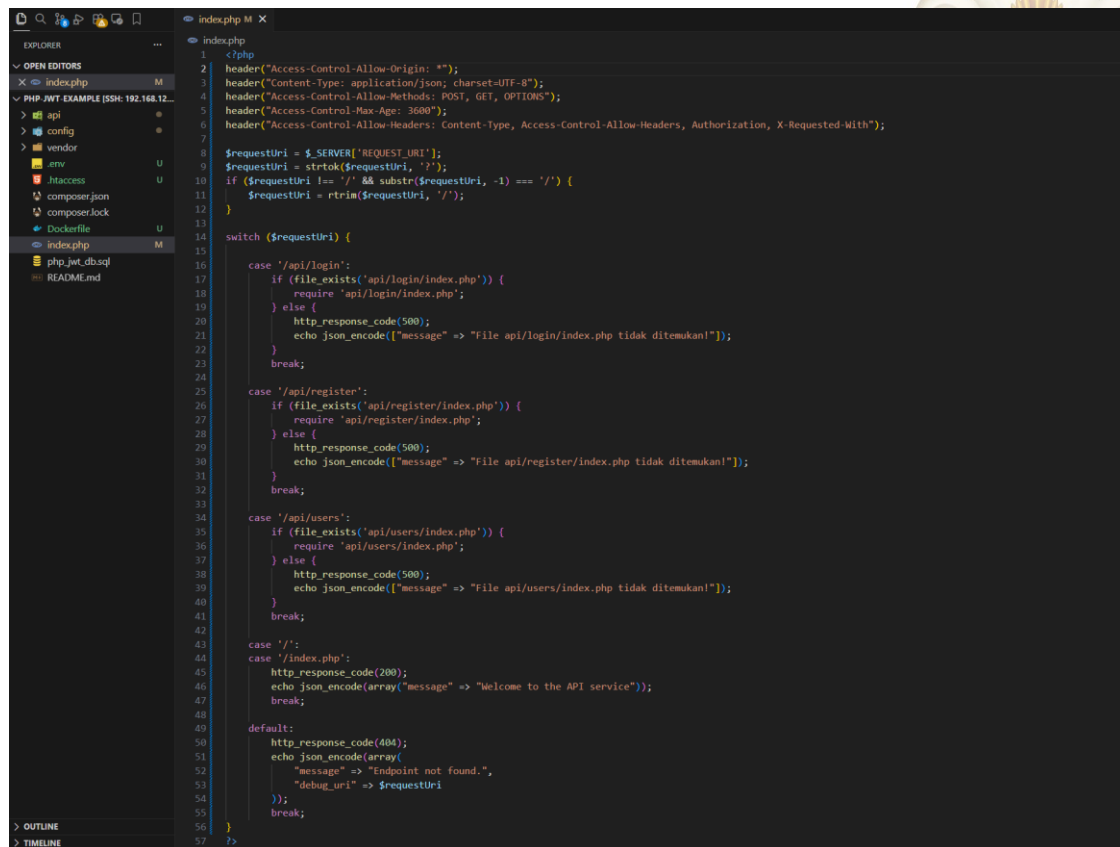
    case '/api/register':
        if (file_exists('api/register/index.php')) {
            require 'api/register/index.php';
        } else {
            http_response_code(500);
            echo json_encode(["message" => "File api/register/index.php tidak
ditemukan!"]);
        }
        break;

    case '/api/users':
        if (file_exists('api/users/index.php')) {
            require 'api/users/index.php';
        } else {
            http_response_code(500);
            echo json_encode(["message" => "File api/users/index.php tidak
ditemukan!"]);
        }
        break;

    case '/':
    case '/index.php':
        http_response_code(200);
        echo json_encode(array("message" => "Welcome to the API service"));
        break;

    default:
        http_response_code(404);
        echo json_encode(array(
            "message" => "Endpoint not found.",
            "debug_uri" => $requestUri
        ));
        break;
}

?>
```



Gambar 9.4.15 Router baru

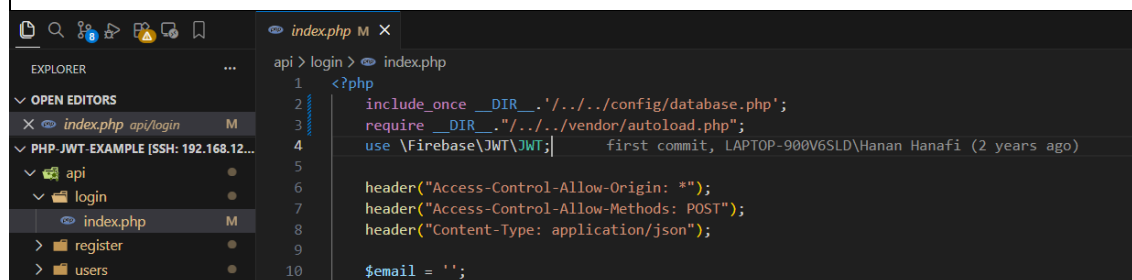
Kemudian metode impor akan sedikit berbeda dan akan menggunakan cara seperti yang dilakukan oleh Laravel.

Ubah bagian api/login/index.php impor menjadi seperti ini

```

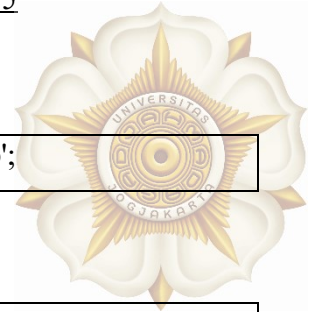
include_once __DIR__.'../../config/database.php';
require __DIR__.'../../vendor/autoload.php';

```



Gambar 9.4.16 Perubahan impor

Lakukan penambahan __DIR__.'/' yang sama seperti login ke register dan users.



Register:

```
include_once __DIR__.'../../config/database.php';
```

Users:

```
include_once __DIR__.'../../config/database.php';  
require __DIR__.'../../vendor/autoload.php';
```

Konstanta `__DIR__` yang kita tambahkan digunakan untuk memastikan bahwa impor tersebut dibaca dari lokasi file berada, jika tidak dilakukan seperti itu, lokasi impor dapat berbeda dan dapat menyebabkan navigasi impor yang salah.

11. Membuat script Dockerfile

Script Dockerfile ini akan dijalankan Docker untuk membuat sebuah image. Image ini kemudian akan dijalankan sebagai container. Pembuatan Dockerfile ini semacam sebuah ringkasan ketika kita menggunakan komputer asli. Tahapan yang dijalankan tidak jauh berbeda, dan perintah yang digunakan hampir serupa.

```
FROM php:8.2-apache  
RUN docker-php-ext-install pdo_mysql  
RUN a2enmod rewrite headers  
  
RUN sed -ri 's/AllowOverride None/AllowOverride All/g' /etc/apache2/apache2.conf  
  
RUN rm -f /var/www/html/index.html  
  
RUN sed -ri 's/Listen 80/Listen 8080/g' /etc/apache2/ports.conf \  
    && sed -ri 's/:80>/:8080>/g' /etc/apache2/sites-available/000-default.conf  
  
WORKDIR /var/www/html  
COPY . /var/www/html
```



EXPOSE 8080

CMD ["apache2-foreground"]

```

14
15 FROM php:8.2-apache
16
17 RUN docker-php-ext-install pdo_mysql
18 RUN a2enmod rewrite headers
19
20 RUN sed -ri 's/AllowOverride None/AllowOverride All/g' /etc/apache2/apache2.conf
21
22 RUN rm -f /var/www/html/index.html
23
24 RUN sed -ri 's/Listen 80/Listen 8080/g' /etc/apache2/ports.conf \
25 && sed -ri 's/::80>/:8080>/g' /etc/apache2/sites-available/000-default.conf
26
27 WORKDIR /var/www/html
28 COPY . /var/www/html
29
30 EXPOSE 8080
31 CMD ["apache2-foreground"]
  
```

Gambar 9.4.17 Dockerfile

Di baris satu “FROM php:8.2-apache” memerintahkan Docker untuk mengambil fondasi sistem operasi linux yang sudah terpasang Web Server Apache dan bahasa pemrograman PHP versi 8.2. Di tahap ini kita menganggap bahwa Docker telah mengunduh kebutuhan utama kita, yaitu PHP 8.2 dan Apache. Kita tidak perlu melakukan instalasi PHP seperti pada BAB sebelumnya karena Docker akan mengambil fondasi yang sudah memiliki PHP dan Apache yang dimaksud. Fondasi-fondasi Image yang didapat ini dapat Anda cari di website resmi Docker.

Di baris kedua, terdapat perintah RUN yang mana perintah tersebut mirip seperti perintah pada terminal. Perintah “docker-php-ext-install pdo_mysql” akan menginstall ekstensi php khusus docker.

Di baris ketiga, merupakan perintah yang dimiliki apache untuk mengaktifkan fitur URL Rewriting di Apache. Ini penting agar URL aplikasi web bisa terlihat rapi (contoh: domain.com/login) dan bukan mentah (contoh: domain.com/index.php?page=login).



Namun, dipraktikum ini tidak akan mencapai penamaan DNS sehingga mengakses tetap menggunakan IP_VPS:PORT.

Di baris selanjutnya, “RUN sed -ri 's/Listen 80/Listen 8080/g' /etc/apache2/ports.conf \&& sed -ri 's/:80>/:8080>/g' /etc/apache2/sites-available/000-default.conf” bertujuan untuk mengganti port Apache yang defaultnya 80 menjadi 8080. Tujuan mengganti ini agar tidak tabrakan dengan Apache yang ada di VPS.

Baris berikutnya, “WORKDIR /var/www/html” digunakan untuk menentukan folder kerja aktif ke lokasi file website apache.

Setelah menentukan folder kerja aktif, penyalinan dilakukan dengan “COPY . /var/www/html” yang memiliki arti salin semua file proyek (. – artinya semua) yang Anda miliki ke direktori /var/www/html kontainer aplikasi kontainer yang akan kita buat.

EXPOSE 8080, digunakan untuk menginformasikan bahwa container ini membuka jalur komunikasi di port 8080.

Kemudian yang terakhir, yaitu CMD [“apache2-foreground”] digunakan sebagai entryptpoint agar server Apache di kontainer akan menyala terus-meneurs.

12. Build Docker Image

Setelah membuat script Dockerfile, kita dapat membuat image. Jika diumpamakan, Dockerfile merupakan resep masakan, dan Docker Image merupakan makanan hangat. Untuk membuat Docker Image kita perlu memasak dengan Docker Build.

```
docker build -t nama-aplikasi-saya LokasiDockerFile
```

Penjelasan build dari perintah tersebut digunakan untuk membaca Dockerfile dan membuat Image. Tag (-t) digunakan untuk memberikan nama resep, di Gambar 9.4.18 nama resep tersebut adalah php-jwt-example. Lokasi Dockerfile perlu diinformasikan,



di Gambar 9.4.18, lokasi Dockerfile berada di root project (.) (~/.psait/php-jwt-example atau direktori aktif terminal saat ini).

```
praktikan@praktikan:~/psait/php-jwt-example$ docker build -t php-jwt-example .
[+] Building 3.4s (1/2)                                docker:default
=> [internal] load build definition from Dockerfile      0.1s
=> => transferring dockerfile: 342B                    0.0s
=> [internal] load metadata for docker.io/library/php:8.2-apache 3.3s
```

Gambar 9.4.18 Docker Build

Setelah berhasil, kita dapat mengecek resep atau Images tersebut dengan perintah seperti berikut. Jika berhasil, image yang telah dibuat akan muncul dalam daftar images seperti Gambar 9.4.19.

```
praktikan@praktikan:~/psait/php-jwt-example$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hello-world:latest	05813aedic15f	25.9kB	9.52kB	U
php-jwt-example:latest	7569ca9a0f9c	708MB	176MB	

```
praktikan@praktikan:~/psait/php-jwt-example$
```

Gambar 9.4.19 Image php-jwt-example berhasil dibuat

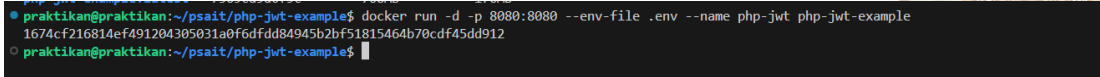
13. Menjalankan image sebagai kontainer

Setelah membuat images, jika kita buka aplikasi di halaman web IP_VPS:PORT, Anda tidak mendapat apa-apa. Hal tersebut karena Image atau masakan yang telah kita buat, belum dihidangkan. Untuk menghidangkan masakan Image yang telah kita buat, kita dapat menjalankan Docker run.

```
docker run -d -p 8080:8080 --env-file .env --name container-saya nama-image
```

Tag -d digunakan untuk menginformasikan bahwa docker berjalan di background sehingga kita dapat memanfaatkan terminal yang saat ini digunakan (analogi: jangan tampilkan log Docker di terminal ini karena saya masih ingin pakai!). Tag -p merupakan Port, port ini menerima dua argumen, port yang disebelah kiri merupakan Port user luar (VPS) mengakses aplikasi Docker, Port disebelah kanan merupakan port dari Docker Image yang telah didefinisikan di resepnya. Arti dari kedua port ini adalah port 8080 dari Docker dilakukan forwarding port ke port 8080 komputer VPS. Oleh karena kita menggunakan .env, maka tambahkan tag --env-file dan masukkan lokasi .env. Tag –

name merupakan nama container seperti pengunjung-1 sedangkan nama-image merupakan nama hidangan yang telah kita buat.

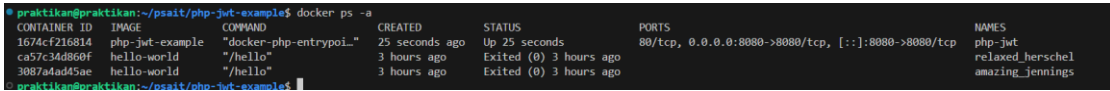


```
praktikan@praktikan:~/psait/php-jwt-example$ docker run -d -p 8080:8080 --env-file .env --name php-jwt php-jwt-example
1674cf216814ef491294305931a0f6dfdd84945b2bf51815464b70cdf45dd912
praktikan@praktikan:~/psait/php-jwt-example$
```

Gambar 9.4.20 Docker Run php-jwt-example

14. Verifikasi status Container Docker

Untuk memastikan container berjalan dan statusnya Up, masukkan perintah docker ps atau bisa juga dengan perintah docker ps -a seperti Gambar 9.4.21. Pastikan statusnya Up dan portnya adalah 8080.

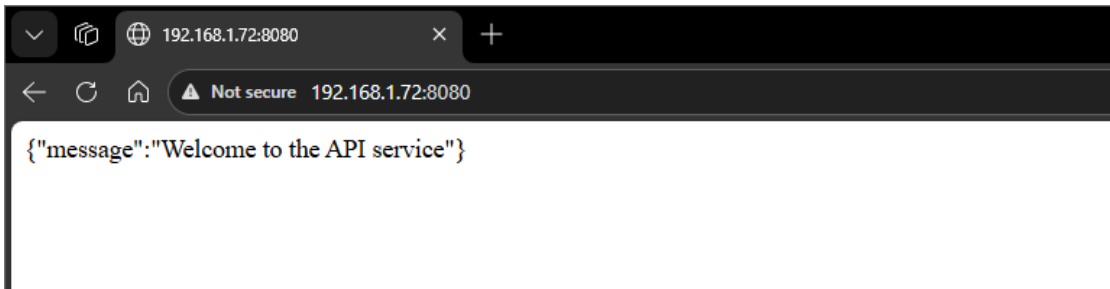


```
praktikan@praktikan:~/psait/php-jwt-example$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
1674cf216814   php-jwt-example "docker-php-entrypoi..." 25 seconds ago Up 25 seconds 80/tcp, 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp  php-jwt
ca57c34d860f   hello-world    "/hello"                 3 hours ago   Exited (0) 3 hours ago                                     relaxed_herschel
3087a4ad45ae   hello-world    "/hello"                 3 hours ago   Exited (0) 3 hours ago                                     amazing_jennings
```

Gambar 9.4.21 Docker ps

15. Mengecek aplikasi

Jika statusnya adalah Up, ketika kita membuka IP_VPS:Port, maka kita akan melihat aplikasi kita berjalan seperti ketika menjalankan di php developer server.



The browser address bar shows the URL `192.168.1.72:8080`. The page content displays a JSON response: `{"message": "Welcome to the API service"}`.

Gambar 9.4.22 Aplikasi Docker berhasil jalan

Jika terjadi error karena koneksi database, jangan lupa menambahkan user seperti Gambar 9.4.23. Sesuaikan user, password, dan database sesuai dengan file .env yang telah dibuat.

```
praktikan@praktikan:~/psait/php-jwt-example$ sudo mysql -u root -p
[sudo] password for praktikan:
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 8.0.44-0ubuntu0.22.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE USER 'php_jwt'@ '%' IDENTIFIED BY 'password123';
Query OK, 0 rows affected (0.02 sec)

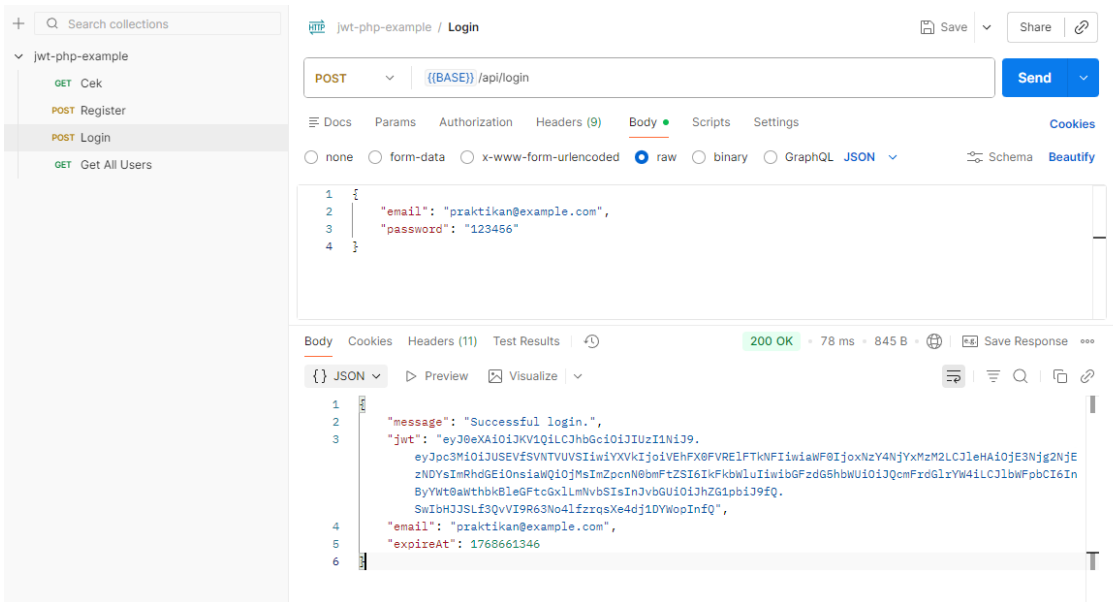
mysql> GRANT ALL PRIVILEGES ON php_jwt_db.* TO 'php_jwt'@ '%';
Query OK, 0 rows affected (0.02 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)
```

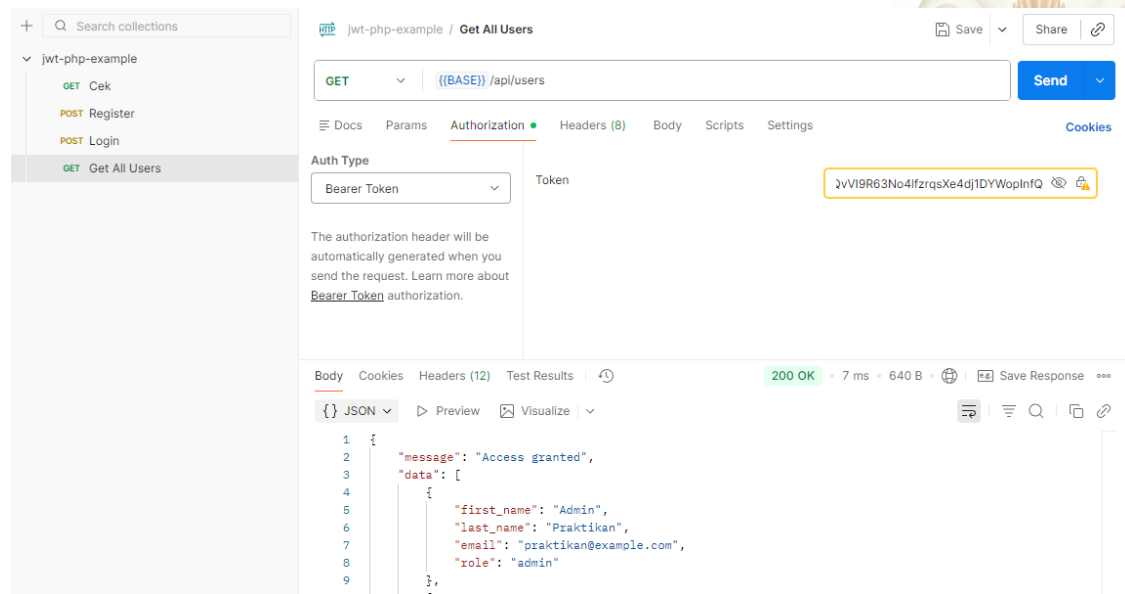
Gambar 9.4.23 Menambahkan user MySQL

16. Menguji di Postman

Jika aplikasi berjalan lancar, pengecekan API yang telah kita buat akan berjalan lancar di Postman.



Gambar 9.4.24 Postman Docker Login



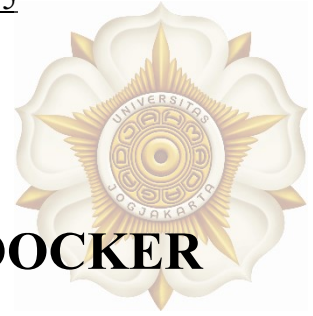
Gambar 9.4.25 Postman Docker Get All Users (Admin)

9.5. TUGAS & ANALISIS

1. Bagaimana cara menghapus Image Container Docker php-jwt yang telah dibuat?
2. Bagaimana jika source code terdapat perubahan dan Image dan Container ingin diperbarui?

9.6. REFERENSI

- [1]. Docker, “Docker Documentation,” Docker Documentation, Oct. 31, 2019.
<https://docs.docker.com/>



PERTEMUAN X

ORKESTRASI DOCKER DENGAN DOCKER COMPOSE

10.1. TUJUAN PEMBELAJARAN

- A. Memahami perbedaan antara Dockerfile (membangun image) dan Docker Compose (menjalankan container)
- B. Menstranslasikan perintah CLI docker run yang panjang menjadi file konfigurasi `docker-compose.yml`
- C. Mengimplementasikan fitur Volumes untuk Hot-Reloading (perubahan kode langsung terlihat tanpa rebuild).
- D. Mengelola environment variables menggunakan `.env` yang terintegrasi dengan Docker Compose

10.2. DASAR TEORI

Sebelumnya kita telah belajar mengenai Dockerfile, tentu mempelajari Dockerfile dalam waktu singkat sungguh melelahkan dan banyak tantangan. Di pertemuan kali ini, kita akan mempelajari Docker Compose, tentunya akan lebih mudah dari Dockerfile. Jika Dockerfile adalah “Resep Makanan” (cara membuat Image), maka Docker Compose adalah “Daftar Menu & Pelayan” yang mengatur bagaimana hidangan tersebut disajikan di meja makan (Container).

Docker Compose adalah alat untuk mendefinisikan dan menjalankan aplikasi Docker multi-kontainer (walaupun bisa juga untuk satu kontainer). Dengan Docker Compose, kita menggunakan file YAML untuk mengonfigurasi layanan aplikasi, jaringan, dan volume.

File YAML Docker Compose memiliki beberapa bagian, diantaranya Version, Services, Volume, dan Networks. Services merupakan daftar container yang akan dijalankan, seperti app-server, database, dan lain-lain sesuai dengan kebutuhan. Volumes merupakan mapping



folder host ke container agar file dapat tersimpan selamanya atau untuk mempercepat proses development seperti hot-reload. Networks merupakan jaringan virtual yang menghubungkan antar services.

Docker Compose memiliki perintah yang hampir mirip dengan Docker. Docker compose up digunakan untuk membangun/membuat/menjalankan container. Docker compose down digunakan untuk menghentikan dan menghapus container serta network. Docker compose logs -f digunakan untuk melihat log output dari semua service secara real-time. Untuk perintah lainnya dapat melihat dokumentasi Docker Compose yang telah disediakan di CLI ataupun Web.

10.3. ALAT DAN BAHAN

10.3.1. Perangkat Keras

- VPS

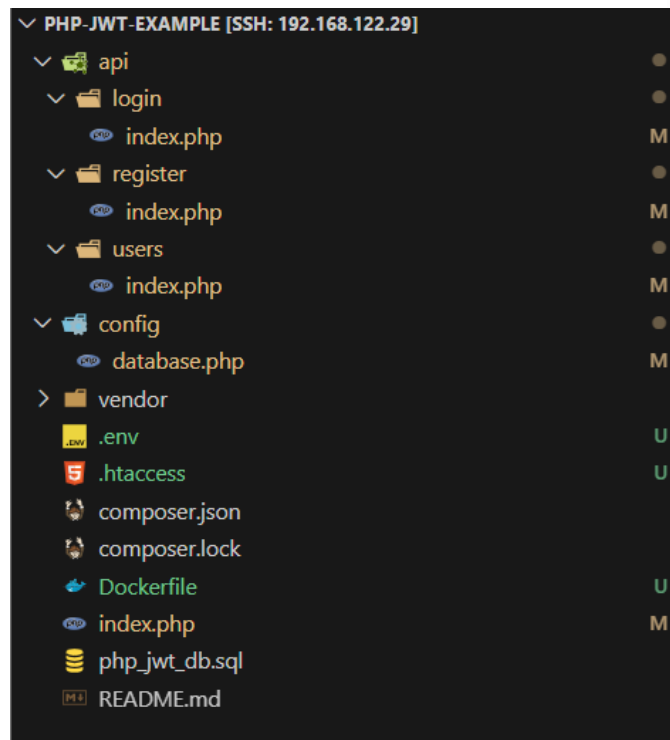
10.3.2. Perangkat Lunak

- Docker
- Visual Studio Code
- Postman

10.4. LANGKAH PERCOBAAN

1. Pastikan project praktikum sebelumnya telah selesai

Pada pertemuan sebelumnya, kita telah membuat Dockerfile dan aplikasi yang dapat dijalankan dengan apache2. Sebagai pengingat, kita telah router di index.php, mengubah konektivitas database menggunakan file .env, membenarkan import, dan pembuatan .htaccess. Aplikasi yang JWT yang dibuat pada pertemuan sebelumnya memiliki struktur folder seperti Gambar 10.4.1.



Gambar 10.4.1 Struktur folder Pertemuan Sebelumnya

Oleh karena kita akan menggunakan Docker Compose, sebaiknya kita matikan terlebih dahulu aplikasi Docker yang dijalankan di pertemuan sebelumnya (Jika masih menyala). Hentikan dan hapus kontainer php-jwt lama yang dibuat di pertemuan sebelumnya seperti Gambar 10.4.2.

```
praktikan@praktikan:~/psait/php-jwt-example$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
ac1b180e05bb   php-jwt-example  "docker-php-entrypoi..." 47 minutes ago Up 47 minutes 80/tcp, 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp  php-jwt
ca57c34d860f   hello-world     "/hello"                 7 hours ago   Exited (0) 7 hours ago                                relaxed_herschel
3087a4ad45ae   hello-world     "/hello"                 7 hours ago   Exited (0) 7 hours ago                                amazing_jennings

praktikan@praktikan:~/psait/php-jwt-example$ docker stop php-jwt
php-jwt

praktikan@praktikan:~/psait/php-jwt-example$ docker rm php-jwt
php-jwt

praktikan@praktikan:~/psait/php-jwt-example$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
ca57c34d860f   hello-world     "/hello"                 7 hours ago   Exited (0) 7 hours ago                                relaxed_herschel
3087a4ad45ae   hello-world     "/hello"                 7 hours ago   Exited (0) 7 hours ago                                amazing_jennings

praktikan@praktikan:~/psait/php-jwt-example$
```

Gambar 10.4.2 Menghentikan dan menghapus kontainer php-jwt lama

2. Membuat docker-compose.yml

Buat file baru bernama docker-compose.yml di root project. Tambahkan kode seperti Gambar 10.4.3. Dari kode tersebut Anda melihat bahwa terdapat dua bagian utama yaitu services dan networks. Di bagian services, terdapat sebuah service kontainer yang kita beri nama be-jwt dan container_name Docker akan diberi nama php-jwt-example. Service ini tentu memerlukan image Docker. Image docker yang kita gunakan adalah

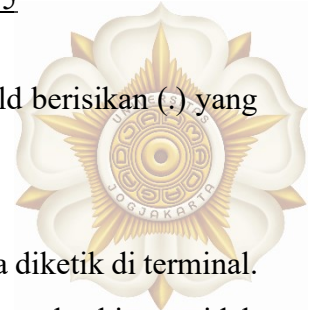
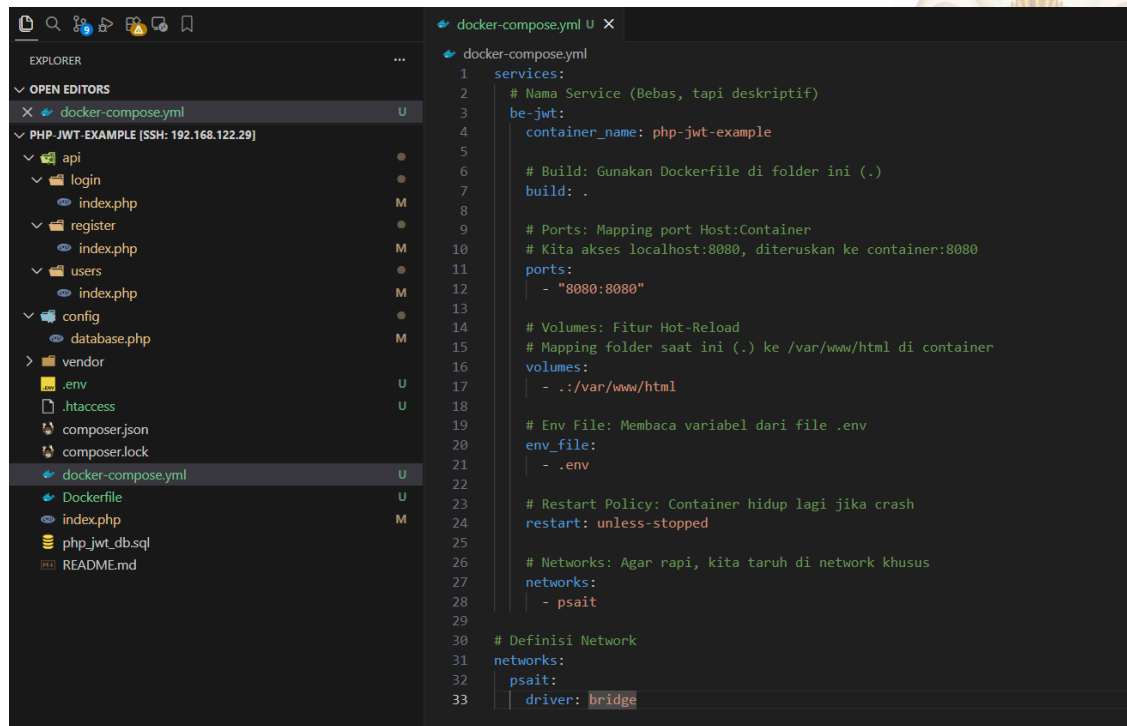


image docker yang telah kita buat sebelumnya. Oleh karena itu build berisikan (.) yang berarti Docker Compose akan mencari Dockerfile di root project.

Sebelumnya di Docker run, kita memasukkan port forwarding secara diketik di terminal. Di Docker Compose, hal tersebut dilakukan di file docker-compose.yml sehingga tidak perlu susah payah menulis terus menerus di terminal karena telah didefinisikan sekali di Docker Compose. Sama hal nya seperti file environment, sebelumnya di Docker run kita menambahkan argumen environmnet, di Docker Compose kita melakukan hal yang sama namun di file docker-compose.yml langsung. Konfigurasi Docker Compose ini tentu akan membantu mempersingkat waktu dan memberikan kemudahan dalam menjalankan kontainerisasi.

Volume di service be-jwt merupakan volume binding. Volume ini akan melakukan mounting file host ke kontainer. Hal ini dapat membuat fitur hot reload yang cocok untuk pengembangan (development). Di Gambar 10.4.3, root project di mount ke dalam kontainer /var/www/html sehingga ketika kita mengubah sesuatu di root project, secara otomatis perubahan tersebut akan terupdate di kontainer tanpa kita harus melakukan build ulang seperti pada kasus sebelumnya (Jika Anda mengalami error).

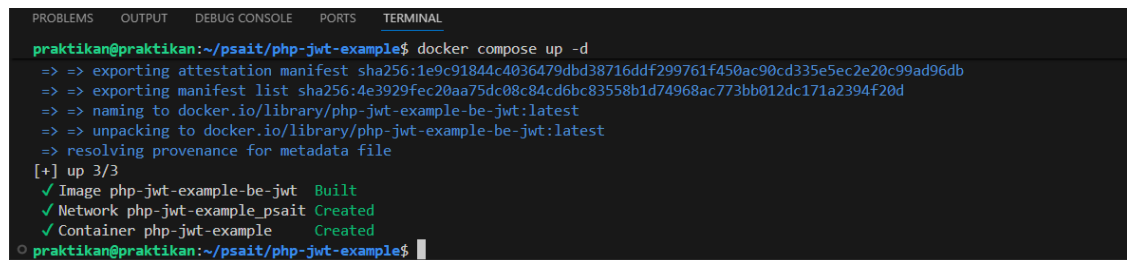
Terdapat juga konfigurasi seperti restart. Konfigurasi ini sangat membantu dalam proses pengembangan ataupun produksi. Images docker terkadang akan mati sendiri jika tidak ada aktivitas atau terlalu lama berjalan (jika tidak diberikan konfigurasi tertentu). Oleh karena itu, konfigurasi restart dengan unless-stopped membuat aplikasi terus berjalan kecuali jika di stop secara manual.



Gambar 10.4.3 Docker Compose

3. Menjalankan docker compose

Menjalankan docker compose cukup mudah, hanya dengan `docker compose up -d` seperti Gambar 10.4.4.



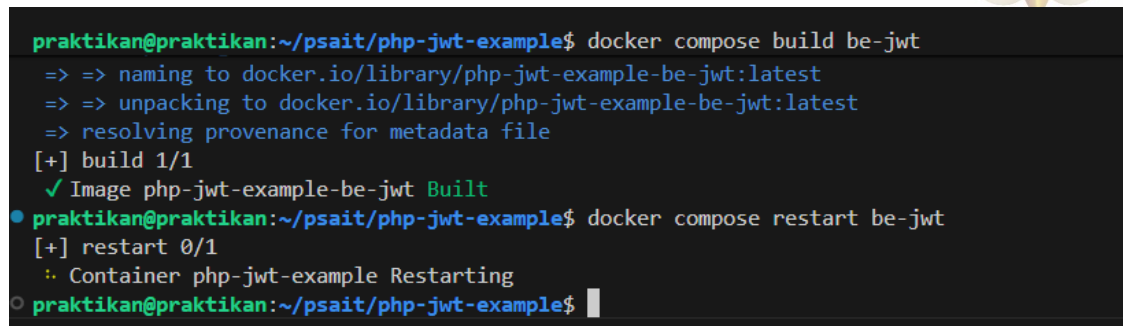
Gambar 10.4.4 Menjalankan Docker Compose Up

4. Mengecek status docker compose

Docker compose juga memiliki fungsi pengecekan status service yang sama seperti Docker. Gunakan perintah “`docker compose ps -a`” di root directory docker-compose.yml berada. Hasil `docker compose ps -a` hampir serupa dengan “`docker ps -a`”, hanya saja yang tertampil merupakan kontainer yang ada di docker-compose.

5. Melakukan build image ulang salah satu service docker compose

Sebagai contoh, jika kita ingin melakukan rebuild pada salah satu service, kita dapat mengetik `docker compose build service_name`. Kemudian dapat melakukan restart dengan `docker compose restart service_name`. Perintah-perintah ini lebih cepat dibanding menggunakan Docker seperti pertemuan sebelumnya.

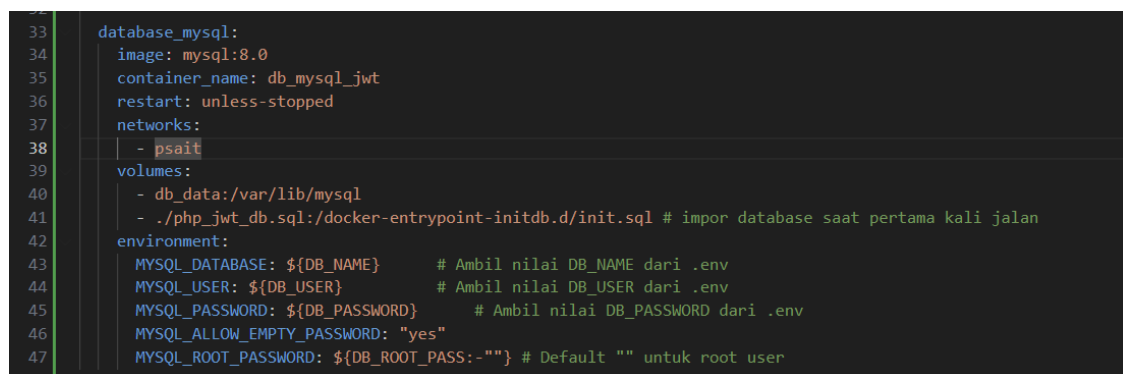


```
praktikan@praktikan:~/psait/php-jwt-example$ docker compose build be-jwt
=> => naming to docker.io/library/php-jwt-example-be-jwt:latest
=> => unpacking to docker.io/library/php-jwt-example-be-jwt:latest
=> resolving provenance for metadata file
[+] build 1/1
✓ Image php-jwt-example-be-jwt Built
praktikan@praktikan:~/psait/php-jwt-example$ docker compose restart be-jwt
[+] restart 0/1
  Container php-jwt-example Restarting
praktikan@praktikan:~/psait/php-jwt-example$
```

Gambar 10.4.5 Rebuild dan Menjalankan Ulang

6. Menambahkan service baru

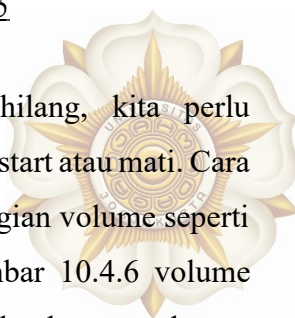
Menambahkan service serupa ketika melakukan pembuatan service `be-jwt`. Dengan indentasi yang sama dengan `be-jwt`, buatlah service seperti Gambar 10.4.6. Jika kalian perhatikan, tidak ada konfigurasi `build` pada service `database_mysql` ini. Konfigurasi image akan digunakan sebagai pengganti `build`. Tidak jauh berbeda dengan `Dockerfile`, image disini akan mengambil fondasi sistem operasi linux yang telah memiliki MySQL versi 8. Di service ini, terdapat konfigurasi `environment` yang sebelumnya tidak ada di `be-jwt`. Konfigurasi `environment` ini dibutuhkan oleh image MySQL dalam pembantuan pembuatan database, user dengan password tertentu. Ketika Anda mencoba membuat service, untuk mengetahui argumen `environment`, Anda dapat mencari di website Docker.



```
33 database_mysql:
34   image: mysql:8.0
35   container_name: db_mysql_jwt
36   restart: unless-stopped
37   networks:
38     - psait
39   volumes:
40     - db_data:/var/lib/mysql
41     - ./php_jwt_db.sql:/docker-entrypoint-initdb.d/init.sql # impor database saat pertama kali jalan
42   environment:
43     MYSQL_DATABASE: ${DB_NAME} # Ambil nilai DB_NAME dari .env
44     MYSQL_USER: ${DB_USER} # Ambil nilai DB_USER dari .env
45     MYSQL_PASSWORD: ${DB_PASSWORD} # Ambil nilai DB_PASSWORD dari .env
46     MYSQL_ALLOW_EMPTY_PASSWORD: "yes"
47     MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASS:-""} # Default "" untuk root user
```

Gambar 10.4.6 Service Database MySQL

Oleh karena sifat kontainer jika dimatikan data akan menghilang, kita perlu menambahkan persistent volume agar data-data tidak hilang ketika restart atau mati. Cara menambahkan persistent volume ini dengan cara mendefinisikan bagian volume seperti Gambar 10.4.6 dan gunakan volume di Gambar 10.4.6. Di Gambar 10.4.6 volume tersebut dilakukan data binding dari persistent volume di mounting ke data mysql yang berada di dalam kontainer. Kemudian untuk mempermudah impor SQL pertama kali, kita dapat melakukan mounting SQL ke lokasi init.sql. Mounting tersebut akan membantu pembuatan database pertama kali.



```
34 volumes:
35 | db_data:
36
37 # Definisi Network
38 networks:
39 | psait:
40 | driver: bridge
```

Gambar 10.4.7 Penambahan Persistent Volume

Sekarang, karena service be-jwt akan menggunakan database dari database_mysql, kita akan menambahkan konfigurasi ketergantungan (depends_on) seperti Gambar 10.4.8. Konfigurasi ini akan membuat service be-jwt akan berjalan jika service database_mysql berjalan.



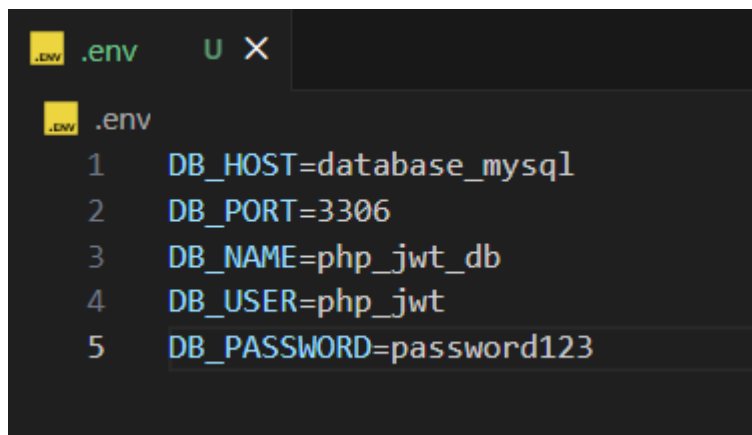
```

2  be-jwt:
3      container_name: php-jwt-example
4      build: .
5      ports:
6      | - "8080:8080"
7      volumes:
8      | - ./var/www/html
9      env_file:
10     | - .env
11     restart: unless-stopped
12     networks:
13     | - psait
14
15     depends_on:
16     | - database_mysql      Not Committed Yet
17

```

Gambar 10.4.8 Menambah ketergantungan

Service be-jwt tidak begitu saja akan menggunakan service database_mysql yang kita buat. Kita perlu mengubah file .env dan mengarahkan ke service database_mysql. Di file .env, ubah DB_HOST menjadi nama service database yang telah dibuat seperti Gambar 10.4.9. Konfigurasi DB_HOST database_mysql ini akan di translate oleh docker sendiri menurut virtual network environmentnya sehingga kita tidak perlu mengetahui IP Addressnya. Namun, kita tetap bisa mengetahui IP Address nya dengan masuk ke sistem Docker dan untuk saat ini kita tidak sedang mempelajari hal tersebut.

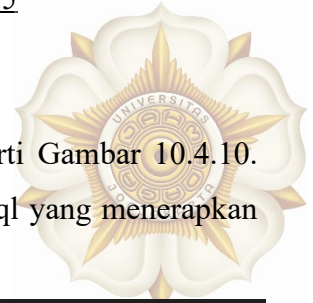


```

1  DB_HOST=database_mysql
2  DB_PORT=3306
3  DB_NAME=php_jwt_db
4  DB_USER=php_jwt
5  DB_PASSWORD=password123

```

Gambar 10.4.9 Mengubah env DB_HOST ke service MySQL Docker



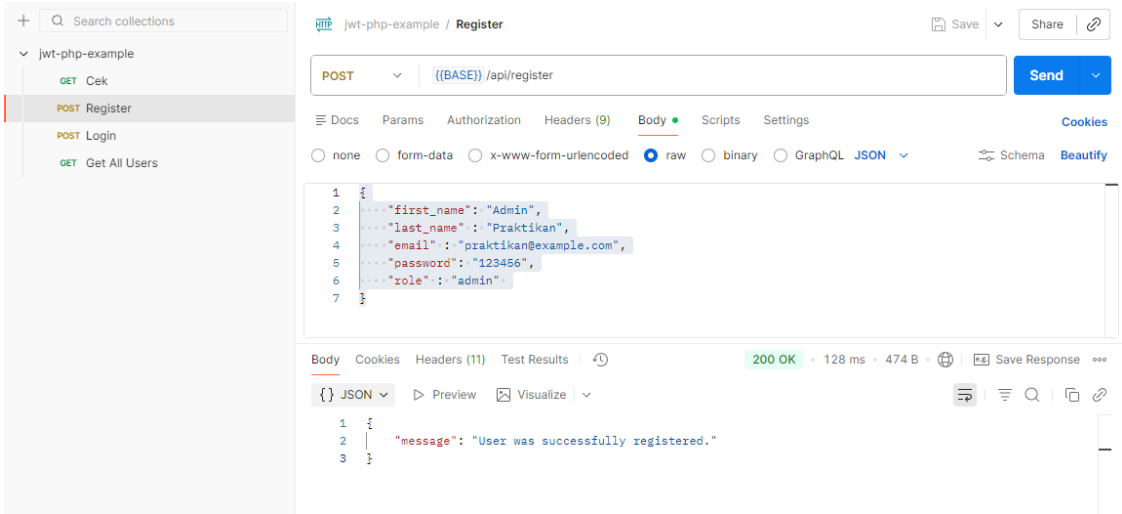
Secara keseluruhan pembuatan docker-compose.yml terlihat seperti Gambar 10.4.10. Kita telah membuat service be-jwt dan juga service database_mysql yang menerapkan persistent volume.

```
docker-compose.yml
1  services:
2    be-jwt:
3      container_name: php-jwt-example
4      build: .
5      ports:
6        - "8080:8080"
7      volumes:
8        - ./var/www/html
9      env_file:
10       - .env
11      restart: unless-stopped
12      networks:
13        - psait
14
15      depends_on:
16        - database_mysql
17
18    database_mysql:
19      image: mysql:8.0
20      container_name: db_mysql_jwt
21      restart: unless-stopped
22      networks:
23        - psait
24      volumes:
25        - db_data:/var/lib/mysql
26        - ./php_jwt_db.sql:/docker-entrypoint-initdb.d/init.sql # impor database saat pertama kali jalan
27      environment:
28        MYSQL_DATABASE: ${DB_NAME}      # Ambil nilai DB_NAME dari .env
29        MYSQL_USER: ${DB_USER}          # Ambil nilai DB_USER dari .env
30        MYSQL_PASSWORD: ${DB_PASSWORD}  # Ambil nilai DB_PASSWORD dari .env
31        MYSQL_ALLOW_EMPTY_PASSWORD: "yes"
32        MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASS:-""} # Default "" untuk root user
33
34      volumes:
35        db_data:
36
37      # Definisi Network
38      networks:
39        psait:
40          driver: bridge
```

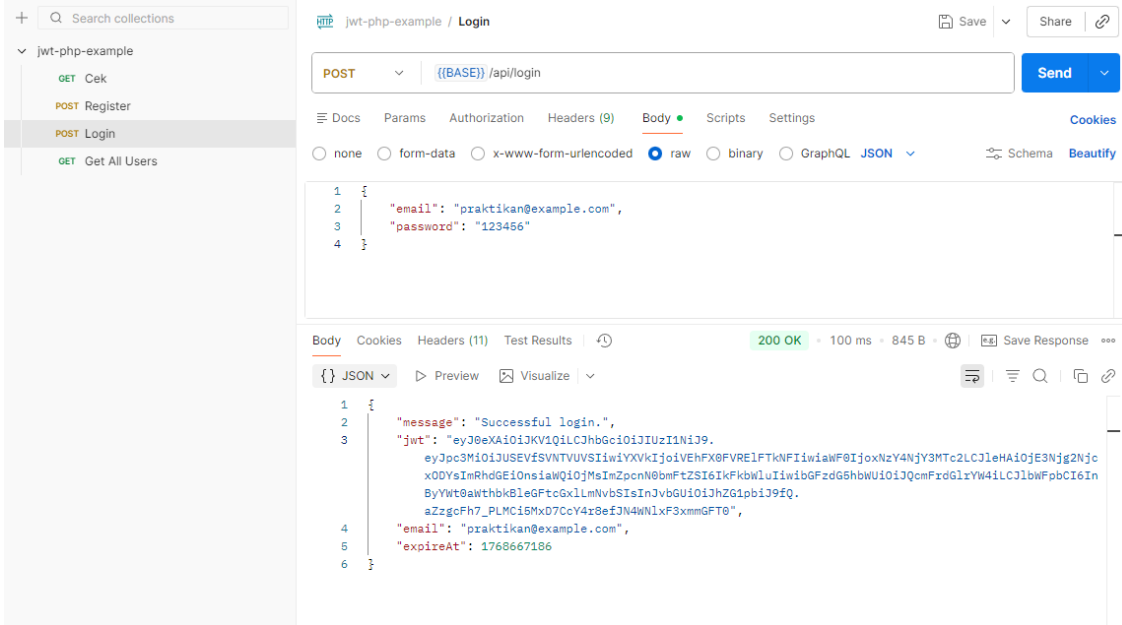
Gambar 10.4.10 Docker Compose Dua Service

Untuk menerapkan perubahan kita bisa mencoba melakukan docker compose up untuk menjalankan service yang kita buat. Ketika kita menguji ulang, seharusnya keseluruhan fitur dapat berjalan lancar.

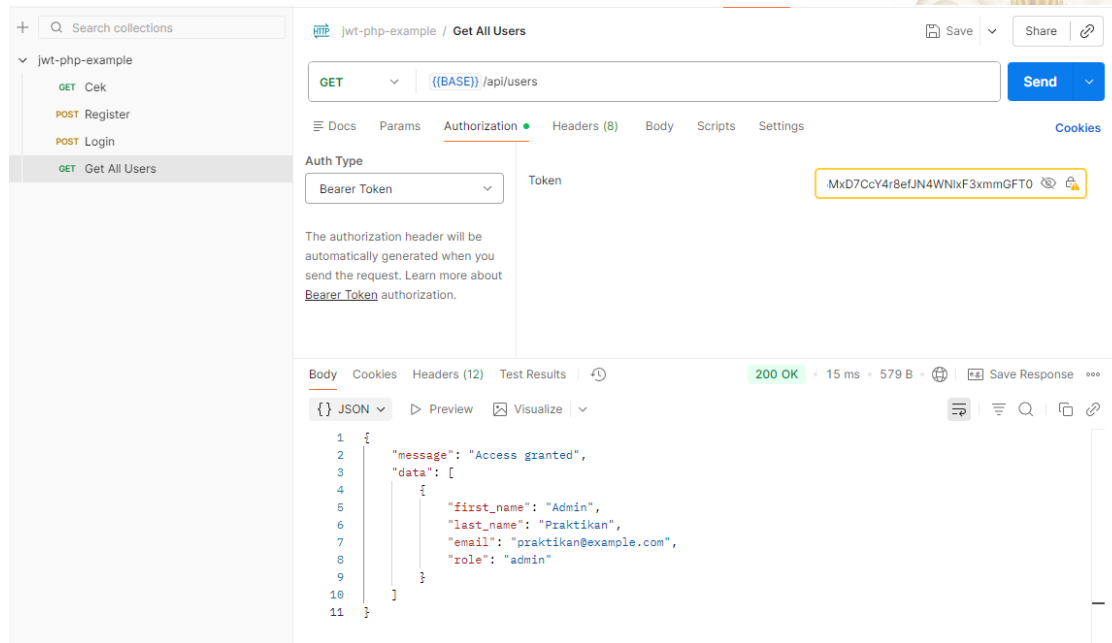
```
praktikan@praktikan:~/psait/php-jwt-example$ docker compose up -d
WARN[0000] No services to build
[+] up 2/2
  ✓ Container db_mysql_jwt      Running
  ✓ Container php-jwt-example   Running
```



Gambar 10.4.12 Register dengan database di docker



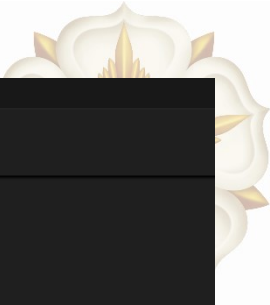
Gambar 10.4.13 Login dengan database di docker



Gambar 10.4.14 Get All Users dari database di Docker

7. Uji coba hot reload

Pada langkah percobaan sebelumnya, modul ini mention mengenai hot-reload. Hot reload ini sangat berguna untuk pengembangan aplikasi sehingga kita tidak perlu repot-repot build image ulang. Sebagai contoh, coba kita tambahkan route /docker di router index.php yang telah kita buat seperti Gambar 10.4.15. Kemudian kita **jangan** melakukan docker compose build be-jwt, langsung saja cek di browser dan lihat apa yang terjadi.



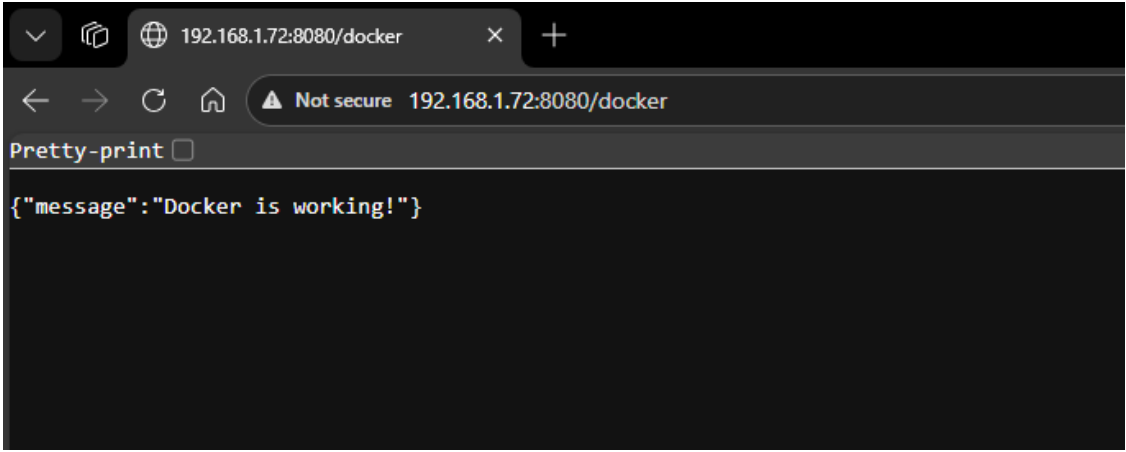
```

index.php M X
index.php
14  switch ($requestUri) {
25      case '/api/register':
32
33          break;
34
35      case '/api/users':
36          if (file_exists('api/users/index.php')) {
37              require 'api/users/index.php';
38          } else {
39              http_response_code(500);
40              echo json_encode(["message" => "File api/users/index.php tidak ditemukan!"]);
41          }
42          break;
43
44      case '/docker':
45          // Simple response to indicate Docker is working
46          http_response_code(200);
47          echo json_encode(array("message" => "Docker is working!"));
48          break;
49
50      case '/index.php':
51          http_response_code(200);
52          echo json_encode(array("message" => "Welcome to the API service"));
53          break;
54
55      default:
56          http_response_code(404);
57          echo json_encode(array(
58              "message" => "Endpoint not found.",
59              "debug_uri" => $requestUri
60          ));
61      break;
62  }
63  ?>

```

Gambar 10.4.15 Menambahkan route docker

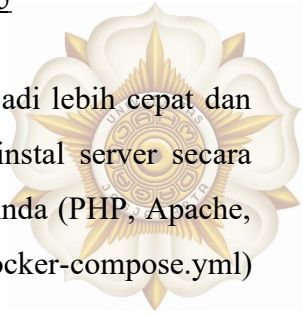
Ketika membuka route docker di url aplikasi dimana aplikasi be-jwt dijalankan, maka kita akan melihat perubahan yang telah kita implementasikan di Gambar 10.4.15. Di halaman web akan tertampil seperti Gambar 10.4.16.



Gambar 10.4.16 Hot reload berhasil

8. Tahap selanjutnya

Dengan adanya kontainerisasi seperti Docker Compose ini, kita dapat membuat alur Continuous Integration dan Continuous Delivery/Deployment (CI/CD) dengan mudah.



Selain itu ketika bekerja dalam tim, melakukan setup proyek menjadi lebih cepat dan meminimalisasikan terjadinya suatu error. Anda tidak lagi menginstal server secara manual (klik Next > Next > Finish). Seluruh infrastruktur server Anda (PHP, Apache, MySQL) kini didefinisikan dalam bentuk kode teks (Dockerfile & docker-compose.yml) yang mudah dibagikan dan diduplikasi.

10.5. TUGAS & ANALISIS

1. Tambahkan service phpMyAdmin ke dalam orkestrasi Docker Compose! Forward port phpMyAdmin tersebut ke port selain port 80 agar tidak tabrakan dengan Apache2 yang ada di VPS. Link image docker phpMyAdmin:

https://hub.docker.com/_/phpmyadmin/

2. Lakukan CRUD database aplikasi php_jwt-db dengan menggunakan phpMyAdmin yang telah dimuat di Docker Compose! Pastikan semua CRUD berjalan untuk memastikan keberhasilan penambahan service phpMyAdmin yang kalian lakukan!

10.6. REFERENSI

- [1]. Docker, “Docker Documentation,” Docker Documentation, Oct. 31, 2019.
<https://docs.docker.com/>



PERTEMUAN XI

CI/CD PIPELINE LOKAL DENGAN GITEA ACTIONS

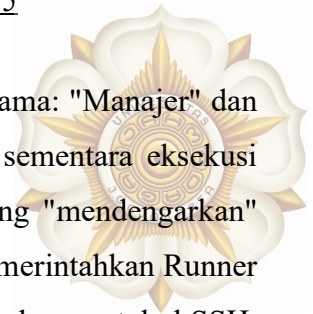
11.1. TUJUAN PEMBELAJARAN

- A. Memahami konsep dasar CI/CD (Continuous Integration & Continuous Deployment) dalam pengembangan perangkat lunak modern.
- B. Mengimplementasikan otomatisasi deployment menggunakan Gitea Actions
- C. Membuat script workflow .yaml untuk memerintahkan server melakukan update aplikasi secara otomatis via SSH
- D. Membangun infrastruktur CI/CD mandiri menggunakan Gitea dan Gitea Runner

11.2. DASAR TEORI

Pengembangan perangkat lunak modern telah meninggalkan cara lama di mana programmer harus menyalin file secara manual ke server setiap kali ada pembaruan fitur. Metode manual ini tidak hanya lambat, tetapi juga sangat rentan terhadap human error, seperti kesalahan menyalin file atau lupa merestart layanan. Sebagai solusinya, industri menerapkan konsep CI/CD (Continuous Integration & Continuous Deployment). CI berfokus pada otomatisasi pengujian dan penggabungan kode, sedangkan CD berfokus pada pengiriman kode tersebut ke server produksi secara otomatis agar dapat segera digunakan oleh pengguna akhir.

Dalam praktikum ini, kita menggunakan Gitea sebagai platform manajemen source code. Penting untuk dipahami bahwa penggunaan Gitea di sini hanyalah sebagai media pembelajaran yang memungkinkan simulasi CI/CD berjalan di jaringan lokal (intranet) kampus dan gratis. Namun, secara sintaks dan konsep, Gitea Actions mengadopsi standar yang hampir identik dengan GitHub Actions yang merupakan standar industri saat ini. Artinya, skrip YAML dan logika alur kerja yang Anda pelajari di modul ini dapat langsung diterapkan jika Anda menggunakan GitHub di dunia kerja nanti.



Arsitektur CI/CD yang akan kita bangun melibatkan dua komponen utama: "Manajer" dan "Pekerja". Gitea bertindak sebagai Manajer yang menyimpan kode, sementara eksekusi perintah dilakukan oleh Runner. Runner adalah program terpisah yang "mendengarkan" perintah dari Gitea. Ketika Anda melakukan push kode, Gitea akan memerintahkan Runner untuk melakukan tugas-tugas seperti login ke server produksi menggunakan protokol SSH, menarik kode terbaru, dan merestart container Docker. Mekanisme ini memastikan server selalu terupdate tanpa intervensi manual.

11.3. ALAT DAN BAHAN

11.3.1. Perangkat Keras

- VPS (Min RAM 2 GB & storage min 10GB)

11.3.2. Perangkat Lunak

- Project aplikasi sebelumnya php-jwt-example
- Visual Studio Code
- SSH

11.4. LANGKAH PERCOBAAN

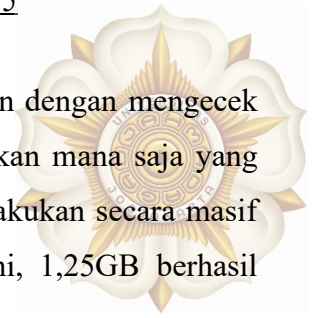
1. Persiapan (Opsional)

Dalam praktikum ini, kita akan mengunduh beberapa aplikasi sehingga perlu memastikan bahwa tempat penyimpanan dan ram dari VPS masih tersedia. Untuk mengecek sisa penyimpanan, jalankan perintah seperti berikut.

```
praktikan@praktikan:~$ df -h
Filesystem          Size  Used Avail Use% Mounted on
tmpfs                291M  1.5M  289M   1% /run
/dev/mapper/ubuntu--vg-ubuntu--lv 9.8G  9.3G   0 100% /
tmpfs                1.5G   0  1.5G   0% /dev/shm
tmpfs                5.0M   0  5.0M   0% /run/lock
/dev/vda2            1.8G  130M  1.5G   8% /boot
tmpfs                291M  4.0K  291M   1% /run/user/1000
```

Gambar 11.4.1 Mengecek sisa penyimpanan

Jika sisa penyimpanan yang dimiliki adalah 0 seperti Gambar 11.4.1, Anda perlu menghapus beberapa aplikasi yang tidak terpakai karena kita hanya akan menggunakan php-jwt-example. Sebagai contoh di Gambar 11.4.2, kita akan menghapus image docker pertemuan sebelumnya seperti phpMyAdmin dan MySQL. Jika aplikasi masih aktif,



Anda perlu menghentikan kontainer tersebut, hal ini bisa dilakukan dengan mengecek seluruh image dengan perintah docker ps -a, kemudian menargetkan mana saja yang ingin diberhentikan. Setelah diberhentikan, penghapusan dapat dilakukan secara masif dengan docker system prune -a. Pada kasus pembuat modul ini, 1,25GB berhasil didapatkan.

```
praktikan@praktikan:~/psait/php-jwt-example$ docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
c9d8e761ee74   phpmyadmin/phpmyadmin:latest        "/docker-entrypoint..." 11 hours ago   Up 24 minutes   80/tcp, 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   phpmyadmin_jwt
516dc1f1833e   php-jwt-example-be-jwt              "docker-php-entrypoi..." 20 hours ago   Up 24 minutes   3306/tcp, 33060/tcp                 db_mysql_jwt
738a253f27f5   mysql:8.0                           "docker-entrypoint.s..." 20 hours ago   Up 13 minutes   3306/tcp, 33060/tcp                 relaxed_herschel
ca57c34d866f   hello-world                          "/hello"                 20 hours ago   Exited (0) 28 hours ago                                     amazing_jennings
3887a4ad45ae   hello-world                          "/hello"                 20 hours ago   Exited (0) 28 hours ago                                     amazing_jennings

praktikan@praktikan:~/psait/php-jwt-example$ docker stop c9d8e761ee74
c9d8e761ee74
praktikan@praktikan:~/psait/php-jwt-example$ docker stop 738a253f27f5
738a253f27f5
praktikan@praktikan:~/psait/php-jwt-example$ docker images
REPOSITORY    TAG       IMAGE ID       SIZE
hello-world   latest    05813aedc15f   25.9kB
mysql:8.0     latest    9c338beac945   1.08GB
php-jwt-example-be-jwt:latest    b737e3938c21   768MB
php-jwt-example:latest           ad179d575d3e   768MB
phpmyadmin/phpmyadmin:latest     42a208db07b4   1.09GB

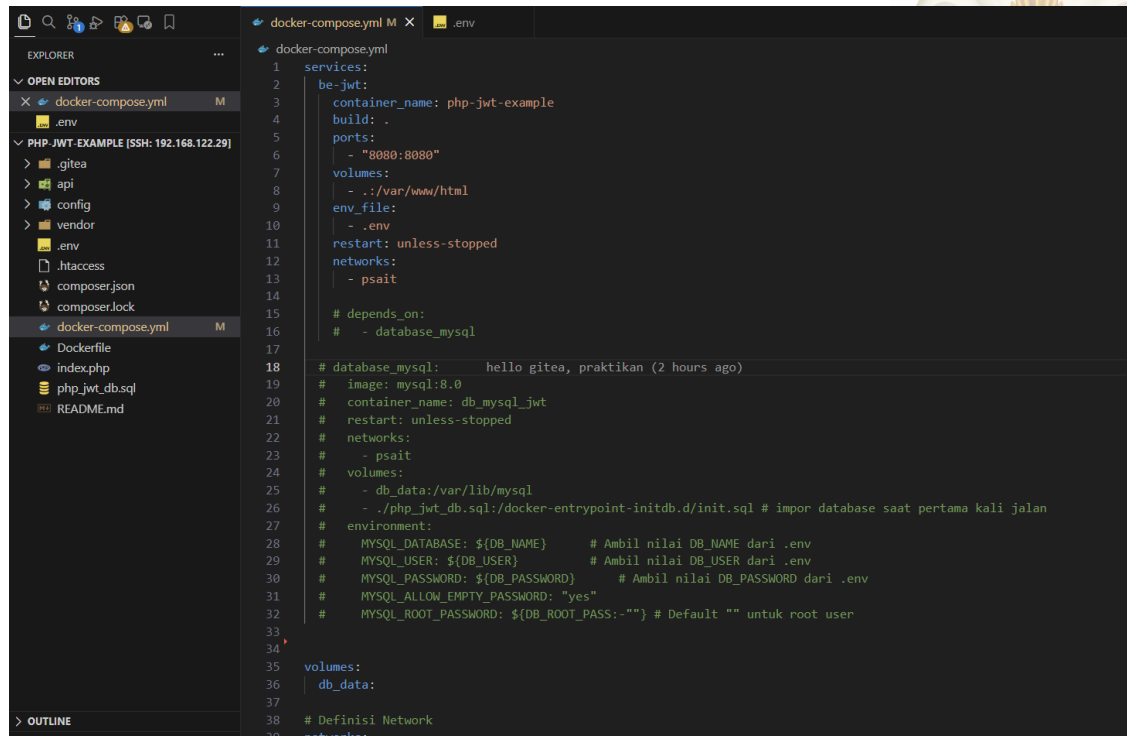
praktikan@praktikan:~/psait/php-jwt-example$ docker system prune -a
WARNING! This will remove:
- all stopped containers
- all networks not used by at least one container
- all images without at least one container associated to them
- all build cache

Are you sure you want to continue? [y/N] y
Deleted Containers:
c9d8e761ee74b37fc3234fe4013fe1520fb58d4a30f08b1b946bb66be271b842
738a253f27f51a5d1729bae9bac8d9e88727d89610811a90c33ec808f561e
ca57c34d866fb180baef1187857bb36c867e1be0d6fa15192e8079f5f1e71260
3887a4ad45aec1dcd89456dc5711e4974f3ca886878ef13414edfada242832e
```

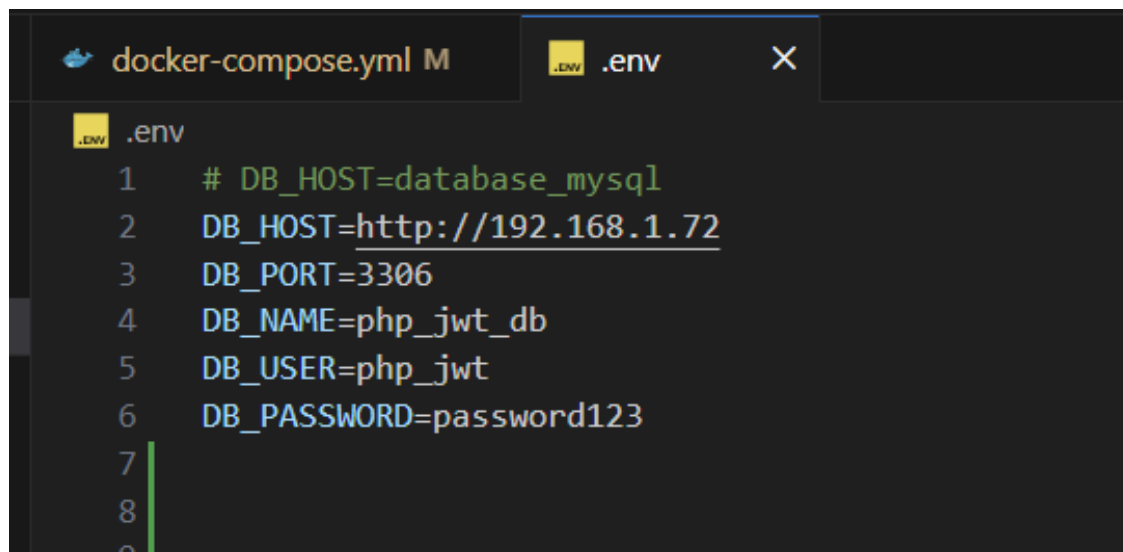
Gambar 11.4.2 Menghapus Image Docker tidak terpakai

2. Persiapan objek aplikasi

Dalam langkah percobaan ini, kita akan menggunakan hasil pekerjaan php-jwt-example yang telah dilakukan. Namun, kita perlu mengubah kode docker-compose.yml dan .env terlebih dahulu agar sistem tidak melebihi beban komputasi komputer VPS lab kampus (Jika Anda ada di posisi industri, tentu Anda perlu menyesuaikan sumber komputasi yang dimiliki. Pada kasus pembuatan modul, sumber daya sangat terbatas dan hanya mendapat RAM 2 GB dan storage 10GB). Di Gambar 11.4.3, kita akan mematikan service database_mysql yang telah dibuat sebelumnya dan ketergantungan ke database_mysql di be-jwt. Oleh karena kita tidak menggunakan MySQL yang dari docker, maka kita perlu menyesuaikan kembali ke database MySQL yang ada di VPS. Ubah file .env sesuai dengan koneksi ke MySQL yang ada di VPS seperti Gambar 11.4.4.

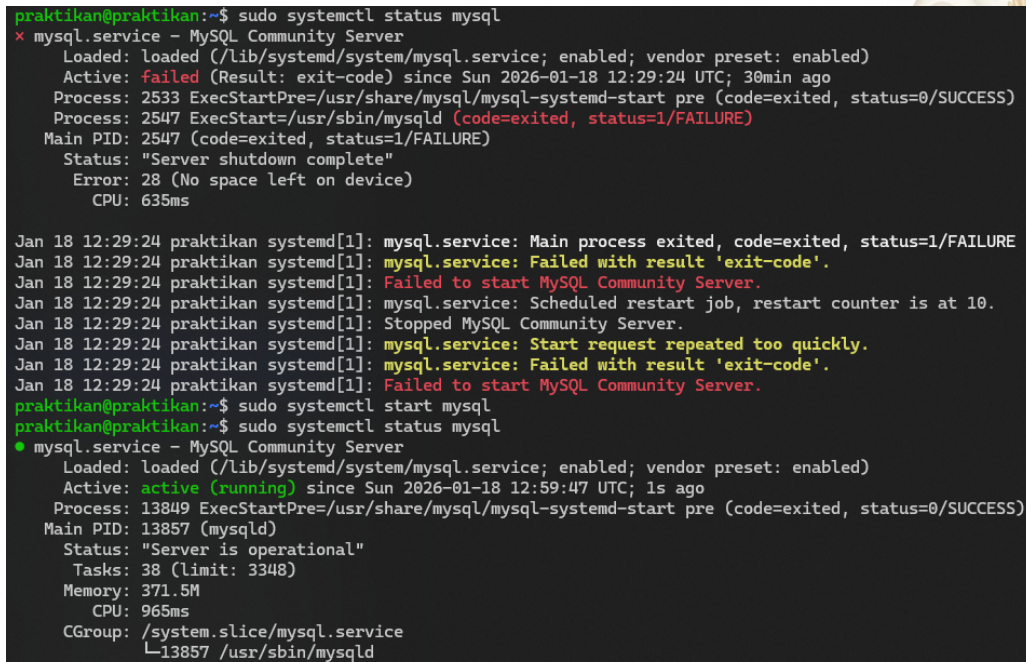


Gambar 11.4.3 Mematikan service database docker



Gambar 11.4.4 Penyesuaian file .env

Pastikan juga bahwa system service MySQL masih menyala di VPS. Jika system service MySQL mati seperti Gambar 11.4.5, hidupkan kembali dengan perintah start.



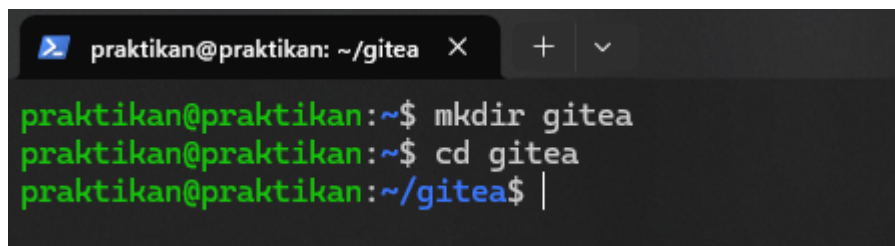
```
praktikan@praktikan:~$ sudo systemctl status mysql
* mysql.service - MySQL Community Server
   Loaded: loaded (/lib/systemd/system/mysql.service; enabled; vendor preset: enabled)
   Active: failed (Result: exit-code) since Sun 2026-01-18 12:29:24 UTC; 30min ago
   Process: 2533 ExecStartPre=/usr/share/mysql/mysql-systemd-start pre (code=exited, status=0/SUCCESS)
   Process: 2547 ExecStart=/usr/sbin/mysqld (code=exited, status=1/FAILURE)
   Main PID: 2547 (code=exited, status=1/FAILURE)
   Status: "Server shutdown complete"
   Error: 28 (No space left on device)
   CPU: 635ms

Jan 18 12:29:24 praktikan systemd[1]: mysql.service: Main process exited, code=exited, status=1/FAILURE
Jan 18 12:29:24 praktikan systemd[1]: mysql.service: Failed with result 'exit-code'.
Jan 18 12:29:24 praktikan systemd[1]: Failed to start MySQL Community Server.
Jan 18 12:29:24 praktikan systemd[1]: mysql.service: Scheduled restart job, restart counter is at 10.
Jan 18 12:29:24 praktikan systemd[1]: Stopped MySQL Community Server.
Jan 18 12:29:24 praktikan systemd[1]: mysql.service: Start request repeated too quickly.
Jan 18 12:29:24 praktikan systemd[1]: mysql.service: Failed with result 'exit-code'.
Jan 18 12:29:24 praktikan systemd[1]: Failed to start MySQL Community Server.
praktikan@praktikan:~$ sudo systemctl start mysql
praktikan@praktikan:~$ sudo systemctl status mysql
* mysql.service - MySQL Community Server
   Loaded: loaded (/lib/systemd/system/mysql.service; enabled; vendor preset: enabled)
   Active: active (running) since Sun 2026-01-18 12:59:47 UTC; 1s ago
   Process: 13849 ExecStartPre=/usr/share/mysql/mysql-systemd-start pre (code=exited, status=0/SUCCESS)
   Main PID: 13857 (mysqld)
   Status: "Server is operational"
   Tasks: 38 (Limit: 3348)
   Memory: 371.5M
   CPU: 965ms
   CGroup: /system.slice/mysql.service
           └─13857 /usr/sbin/mysqld
```

Gambar 11.4.5 Menghidupkan kembali MySQL

3. Instalasi Gitea menggunakan Docker Compose

Buat folder bernama gitea dari home directory seperti Gambar 11.4.6.

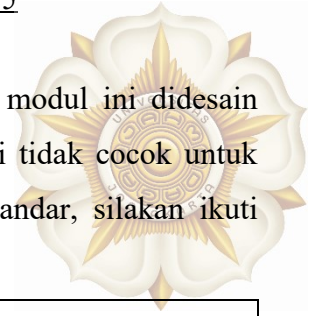


```
praktikan@praktikan: ~/gitea
praktikan@praktikan:~$ mkdir gitea
praktikan@praktikan:~$ cd gitea
praktikan@praktikan:~/gitea$ |
```

Gambar 11.4.6 Membuat folder gitea

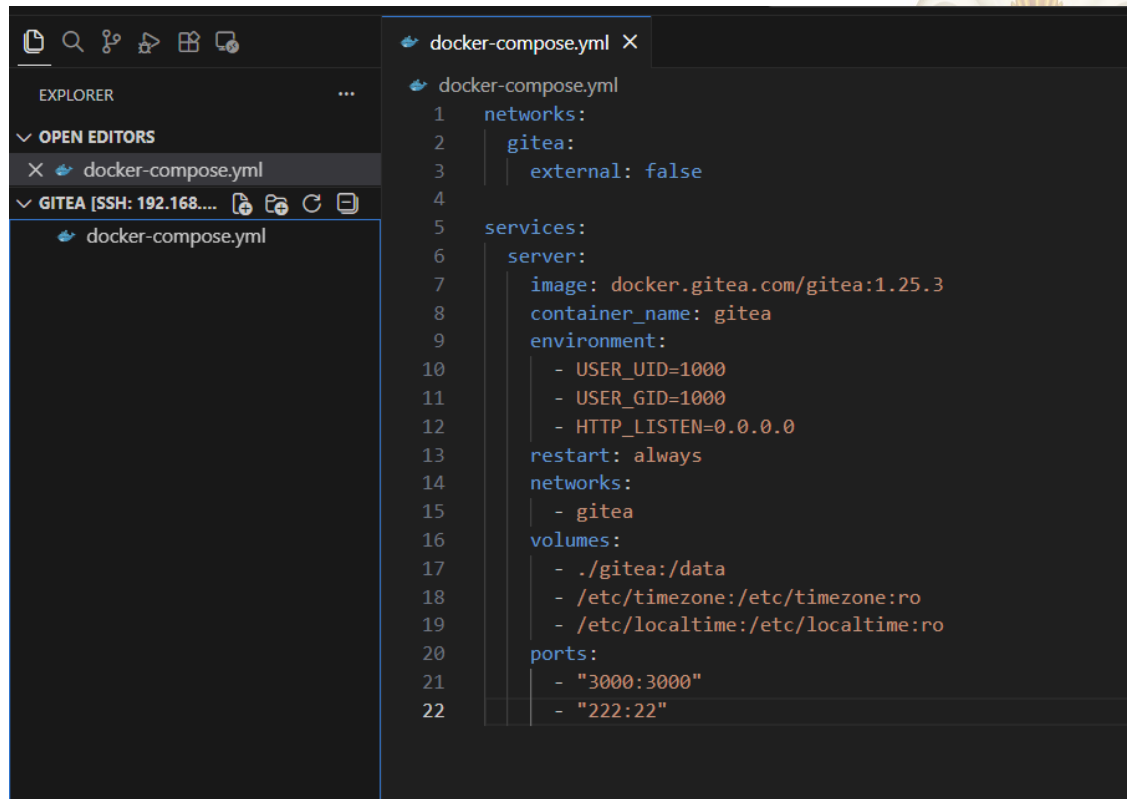
Di dalam folder tersebut kita akan melakukan instalasi dengan menggunakan Docker Compose untuk mempermudah instalasi aplikasi Gitea. Buat file bernama docker-compose.yml di dalam folder gitea yang telah dibuat seperti Gambar 11.4.7. Script docker-compose.yml ini didapat dari dokumentasi gitea, berbeda sedikit dengan contoh yang ada di website, di file ini kita tambahkan HTTP_LISTEN dari yang default adalah localhost, kita ubah menjadi 0.0.0.0 agar dapat digunakan di jaringan lab. Pada bagian port, Anda akan melihat port binding 3000:3000, port 3000 (bagian kanan) merupakan port asli aplikasi gitea. Ada juga Port gitea 22 yang berfungsi sebagai SSH, port ini dilakukan bind ke port VPS 222. Pastikan port 3000 dan 222 tidak sedang digunakan di VPS, jika sedang digunakan, Anda dapat menyesuaikan kode docker compose ini. Perlu

diketahui sebagai catatan, kode docker compose yang dibuat di modul ini didesain khusus untuk agar bisa jalan di komputer Lab, mungkin kode ini tidak cocok untuk spesifikasi industri, jika suatu saat ingin mencoba spesifikasi standar, silakan ikuti dokumentasi asli.



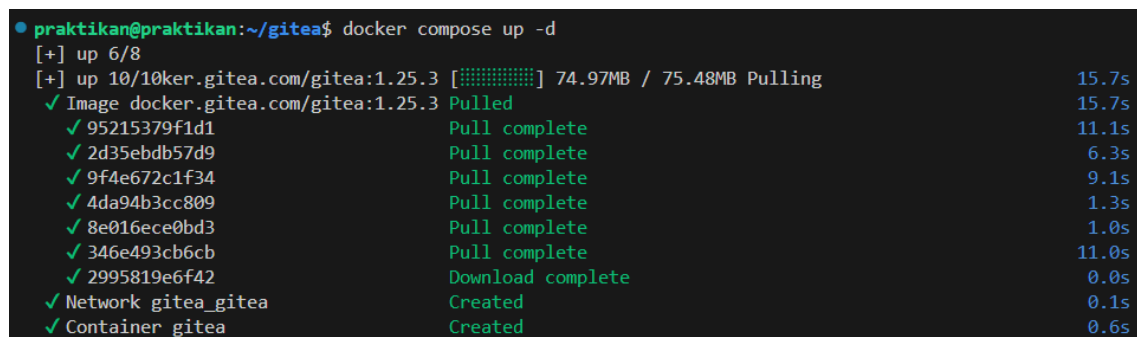
```
networks:
  gitea:
    external: false

services:
  server:
    image: docker.gitea.com/gitea:1.25.3
    container_name: gitea
    environment:
      - USER_UID=1000
      - USER_GID=1000
      - HTTP_LISTEN=0.0.0.0
    restart: always
    networks:
      - gitea
    volumes:
      - ./gitea:/data
      - /etc/timezone:/etc/timezone:ro
      - /etc/localtime:/etc/localtime:ro
    ports:
      - "3000:3000"
      - "222:22"
```



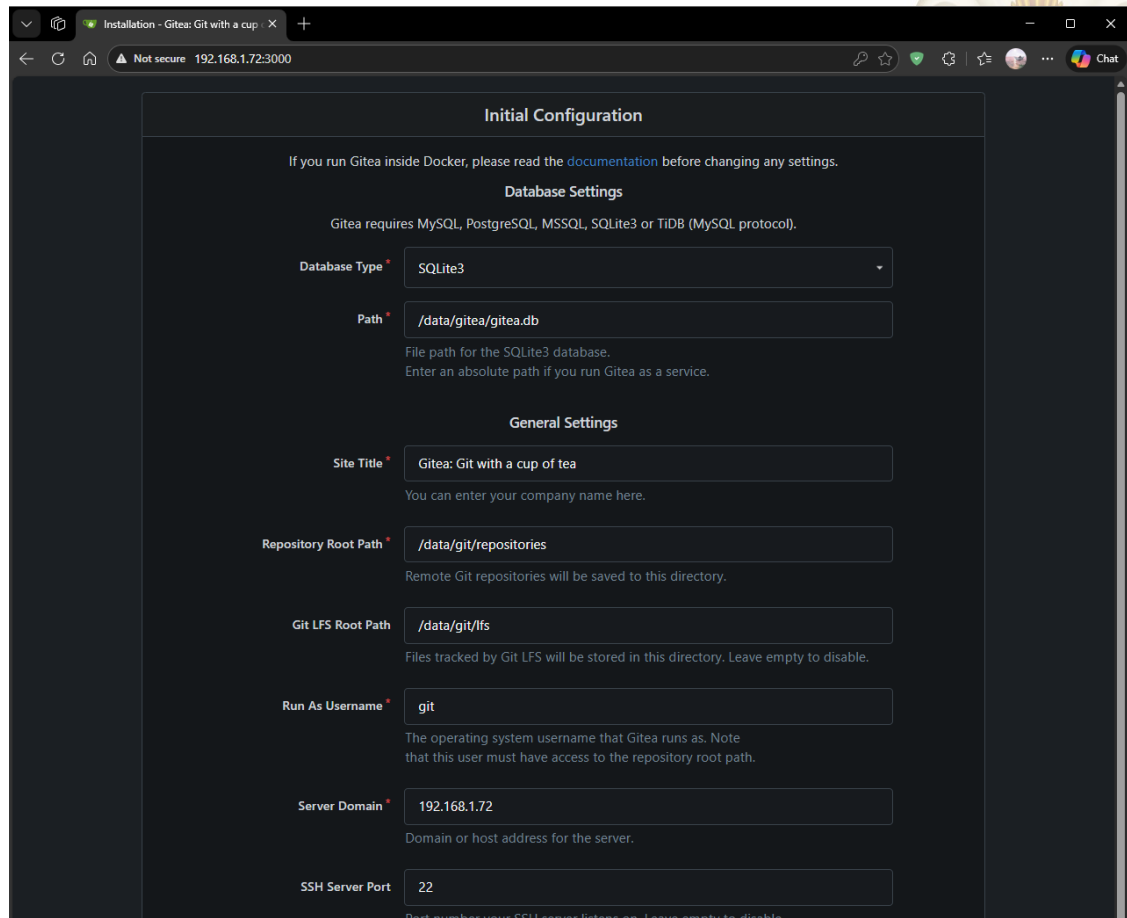
Gambar 11.4.7 Instalasi docker-compose.yml gitea

Seperti yang telah kita pelajari sebelumnya, kita dapat menjalankan service dari Docker Compose tersebut dengan `docker compose up` seperti Gambar 11.4.8.



Gambar 11.4.8 Menjalankan service gitea

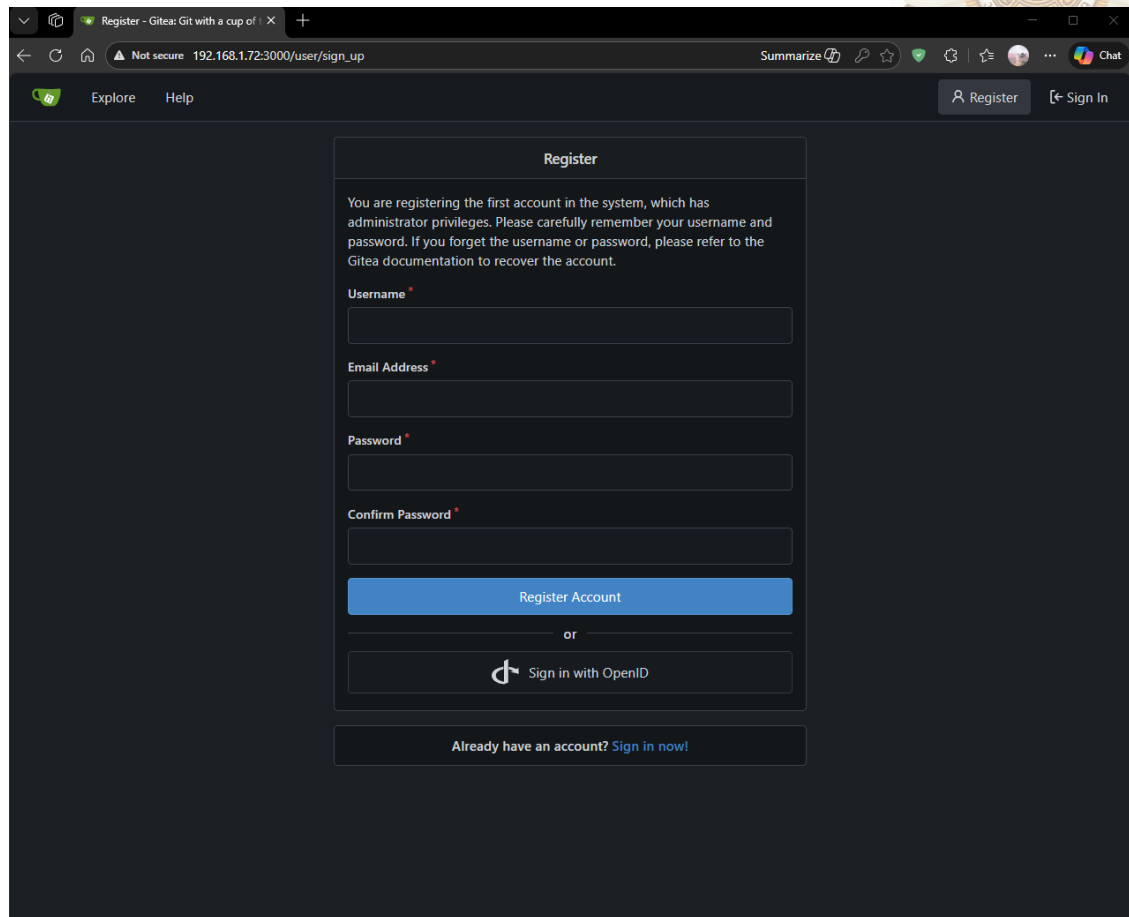
Setelah service docker tersebut berjalan, kita dapat membuka aplikasi dengan url `IP_VPS:port_gitea_di_vps` seperti Gambar 11.4.9. Untuk praktikum ini, kita abaikan perubahan konfigurasi dan klik install.



Gambar 11.4.9 Tahap konfigurasi awal gitea

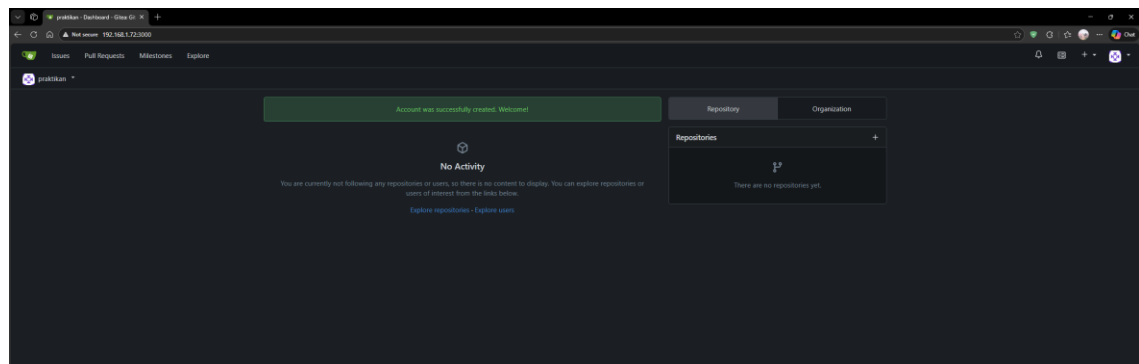
Tahap berikutnya adalah registrasi, di registrasi ini Anda akan diminta untuk mendaftarkan akun. Oleh karena gitea menggunakan teknologi git, aplikasi ini tidak jauh berbeda dengan gitlab, github, dan sejenisnya. Di github kita tahu bahwa kita tidak bisa sembarangan push staging ke repository orang lain, git pasti akan mendeteksi bahwa akun yang digunakan tidak benar atau tidak memiliki otorisasi. Untuk mempermudah tahap instalasi Gitea, Anda dapat melakukan registrasi menggunakan akun git yang kalian gunakan, data akan tersimpan secara lokal karena kita mendeploy aplikasi Gitea kita sendiri (self-hosted) sehingga tidak perlu khawatir (informasi berdasarkan waktu pembuatan modul, mungkin terdapat perubahan di masa mendatang, pastikan untuk mengecek informasi tersebut di dokumentasi resmi ataupun source code aslinya). Namun, jika Anda merasa tidak yakin dengan hal tersebut atau tidak yakin dengan lingkungan kampus (yang mana VPS ini dapat diakses orang lain juga dan kemungkinan

menggunakan password yang sama), Anda dapat membuat akun git baru dan melakukan konfigurasi git di laptop kalian juga untuk uji coba push staging ke gitea.



Gambar 11.4.10 Register akun di gitea

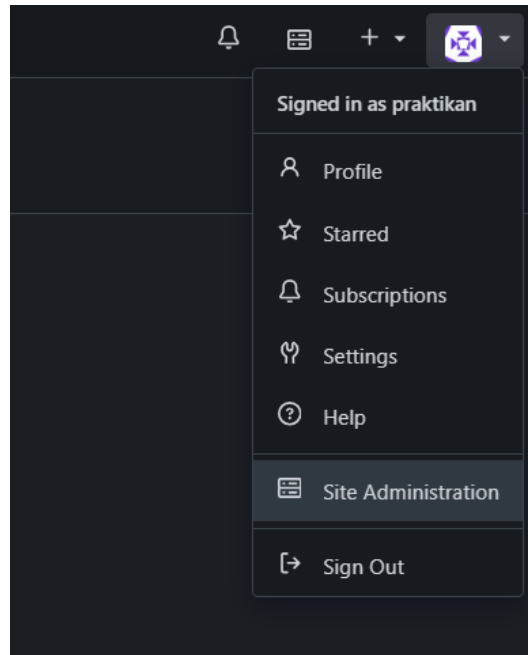
Setelah melakukan registrasi, Anda akan melihat halaman yang mungkin familiar bagi beberapa orang. Halaman tersebut akan terlihat seperti Gambar 11.4.11.



Gambar 11.4.11 Halaman utama gitea

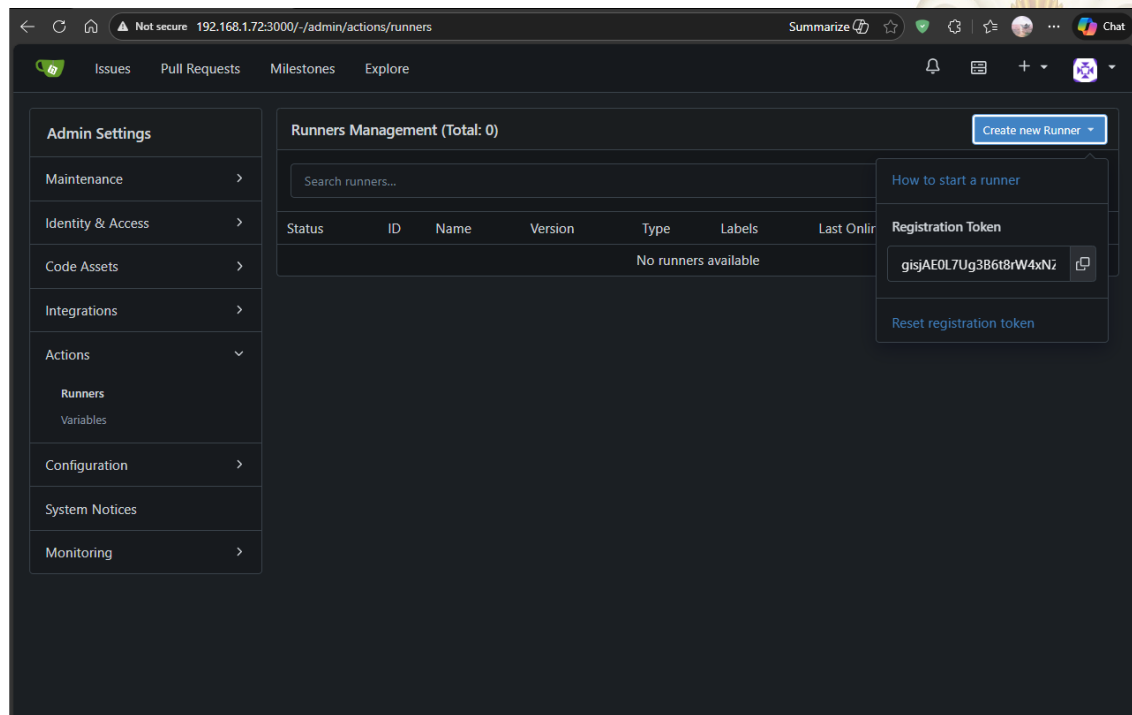


Pada tahap selanjutnya kita akan membuat runner agar dapat menjalankan CICD di gitea. Buka aplikasi gitea di browser, klik profile bagian kanan, klik menu Site Administration.



Gambar 11.4.12 Site Administration Button

Pilih menu Actions, dan pilih sub menu Runners, maka akan tertampil seperti Gambar 11.4.13. Klik tombol create new Runner dan copy kode token.



Gambar 11.4.13 Halaman Action Runners (Langkah 1)

Buka kembali file docker-compose.yml gitea yang ada di VPS. Tambahkan kode berikut.

```
runner:
  image: gitea/act_runner:latest
  container_name: gitea_runner
  restart: unless-stopped
  networks:
    - gitea
  depends_on:
    - server
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
    - ./runner-data:/data
  environment:
    # URL Gitea (Gunakan nama service 'gitea' karena satu jaringan)
    - GITEA_INSTANCE_URL=http://gitea:3000

    # PASTE TOKEN DARI LANGKAH 1 DI SINI 📌
    # -
    GITEA_RUNNER_REGISTRATION_TOKEN=PASTE_TOKEN_ANDA_DI_SINI
```



```
# Nama Runner (Bebas)
- GITEA_RUNNER_NAME=runner-1

# MAPPING LABEL (Sangat Penting untuk Storage Hemat!)
# Artinya: Jika ada job 'ubuntu-latest', pakai image 'node:16-alpine'
- GITEA_RUNNER_LABELS=ubuntu-latest:docker://node:16-alpine
```

```
1  networks:
2    gitea:
3      external: false
4
5  services:
6    server:
7      image: docker.gitea.com/gitea:1.25.3
8      container_name: gitea
9      environment:
10       - USER_UID=1000
11       - USER_GID=1000
12       - HTTP_LISTEN=0.0.0.0
13      restart: always
14      networks:
15       - gitea
16      volumes:
17       - ./gitea:/data
18       - /etc/timezone:/etc/timezone:ro
19       - /etc/localtime:/etc/localtime:ro
20      ports:
21       - "3000:3000"
22       - "222:22"
23
24    runner:
25      image: gitea/act_runner:latest
26      container_name: gitea_runner
27      restart: unless-stopped
28      networks:
29       - gitea
30      depends_on:
31       - server
32      volumes:
33       - /var/run/docker.sock:/var/run/docker.sock
34       - ./runner-data:/data
35      environment:
36       # URL Gitea (Gunakan nama service 'gitea' karena satu jaringan)
37       - GITEA_INSTANCE_URL=http://gitea:3000
38
39       # PASTE TOKEN DARI LANGKAH 1 DI SINI
40       # - GITEA_RUNNER_REGISTRATION_TOKEN=PASTE_TOKEN_ANDA_DISINI
41       - GITEA_RUNNER_REGISTRATION_TOKEN=gisjAE0L7Ug3B6t8rW4xNZR098CsnidyapuIZtvf
42
43       # Nama Runner (Bebas)
44       - GITEA_RUNNER_NAME=runner-1
45
46       # MAPPING LABEL (Sangat Penting untuk Storage Hemat!)
47       # Artinya: Jika ada job 'ubuntu-latest', pakai image 'node:16-alpine'
48       - GITEA_RUNNER_LABELS=ubuntu-latest:docker://node:16-alpine
```

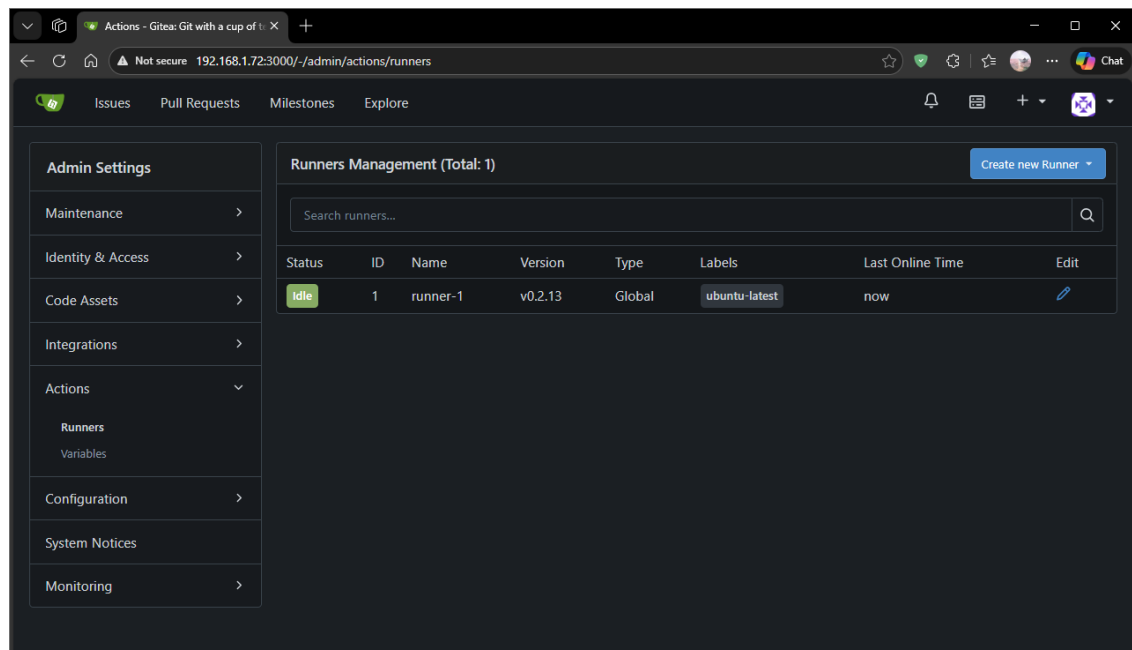
Gambar 11.4.14 Docker Compose Gitea Lengkap

Jalankan service tersebut dengan docker compose up seperti Gambar 11.4.15. Setelah menjalankan, buka aplikasi gitea di halaman Actions Runners, maka akan terlihat 1 runner yang tersedia seperti Gambar 11.4.16. Runner ini akan digunakan untuk CICD, jika Anda bertanya mengenai runner Github, Github telah menyediakan banyak runner

dan sudah siap pakai. Di Gitea yang kita self-hosted sendiri, kita sediakan 1 runner saja sebagai contoh.

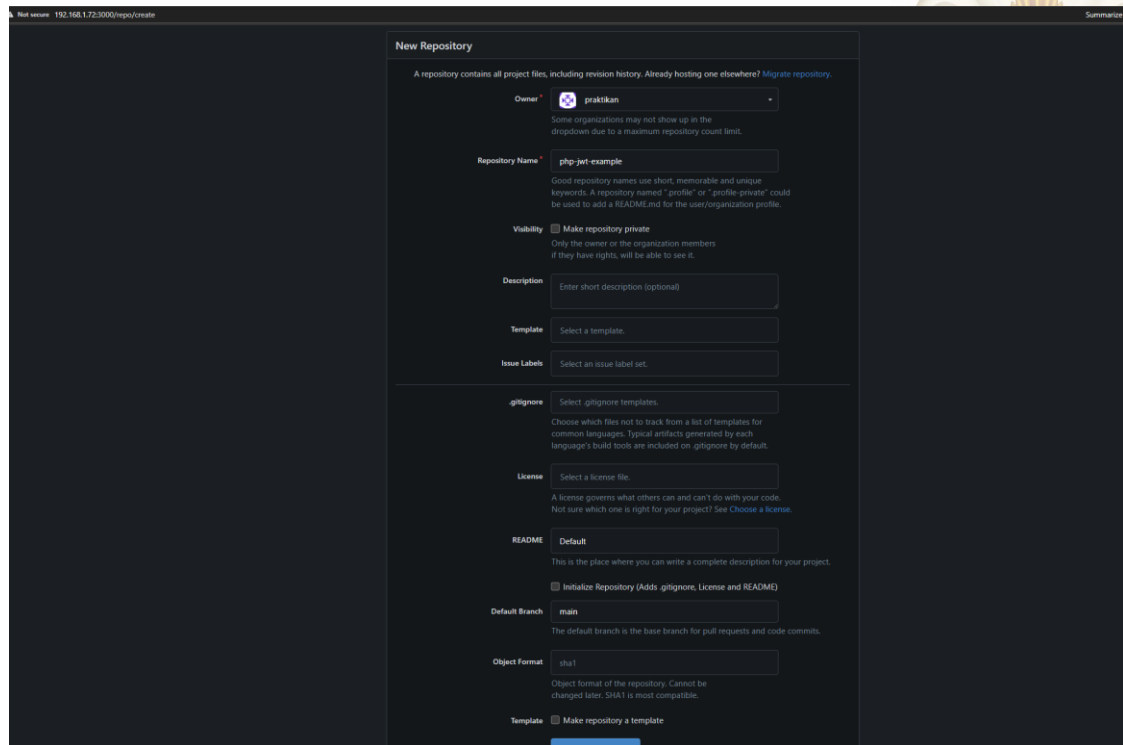
```
praktikan@praktikan:~/gitea$ docker compose up -d
[+] up 6/6
[+] up 8/8 gitea/act_runner:latest Pulled 8.3s
✓ Image gitea/act_runner:latest Pulled 8.3s
✓ 748572556419 Pull complete 3.6s
✓ 9824c27679d3 Pull complete 1.8s
✓ 96e70a89510f Pull complete 2.7s
✓ 7fd1ed8ea89d Pull complete 0.7s
✓ f1bc6cc15ba1 Download complete 0.0s
✓ Container gitea Running 0.0s
✓ Container gitea_runner Created 0.3s
```

Gambar 11.4.15 Docker compose up



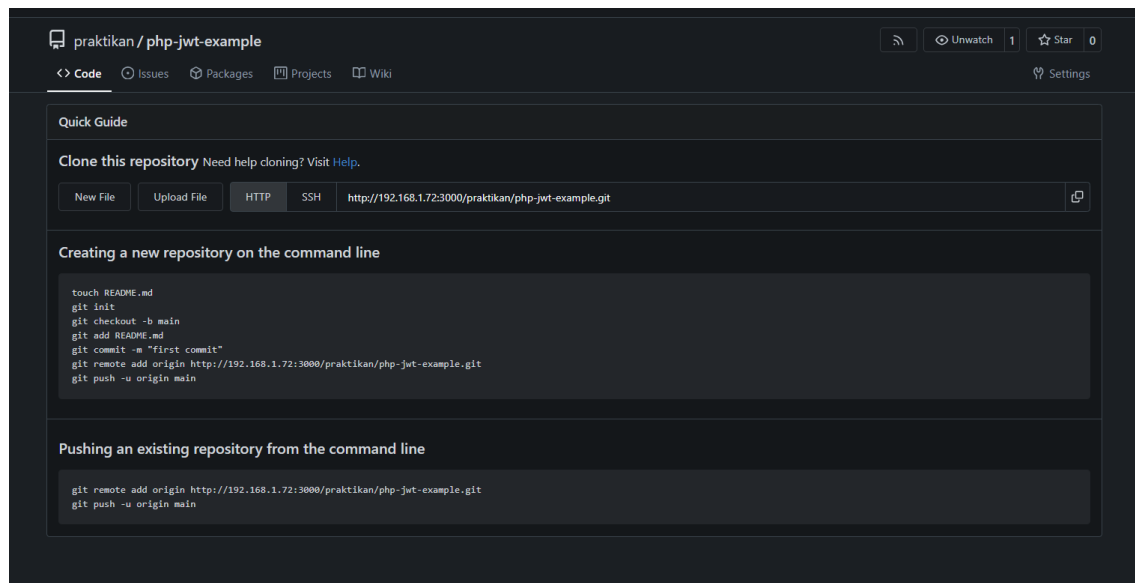
Gambar 11.4.16 Runner Gitea tersedia

Kembali lagi ke halaman awal gitea, kita akan membuat repository baru dengan menekan tombol “+” yang ada di navbar ataupun yang ada di kartu Repositories Gambar 11.4.11. Buatlah repository bernama php-jwt-example dengan default branch bernama main seperti Gambar 11.4.17. Klik create repository untuk membuat repository baru.

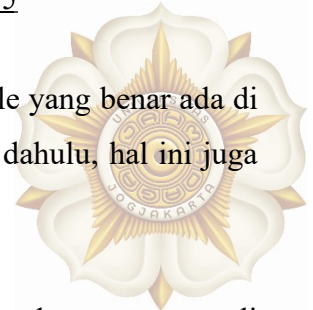


Gambar 11.4.17 Repository baru php-jwt-example

Setelah membuat repository baru, maka Anda akan diperlihatkan dengan petunjuk awal penggunaan repository seperti Gambar 11.4.18. Kita akan kembali lagi ke bagian ini setelah tahap menyiapkan aplikasi yang akan di push.



Gambar 11.4.18 Petunjuk awal konfigurasi repository



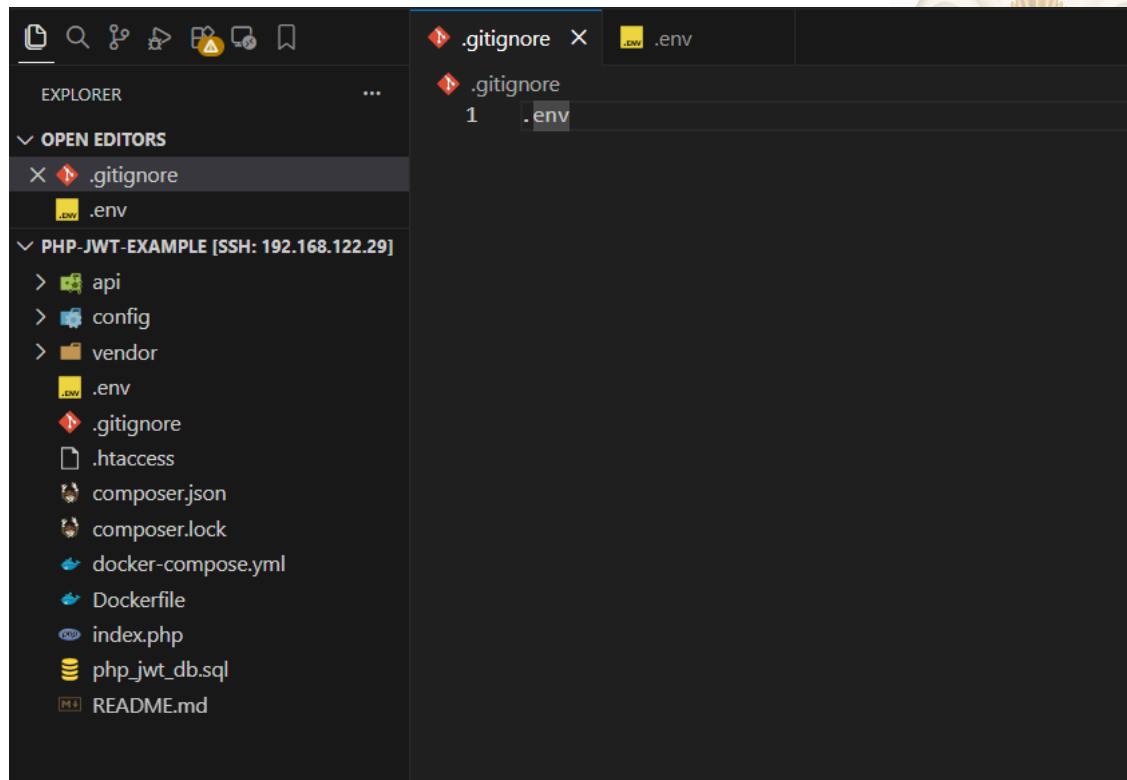
Oleh karena di modul ini menganggap bahwa kode php-jwt-example yang benar ada di VPS, maka kita akan mencoba melakukan push dari VPS terlebih dahulu, hal ini juga dilakukan guna menghubungkan kode yang ada di VPS ke gitea.

Tahap pertama (**dilakukan di VPS**) yang harus dilakukan adalah menghapus remote asli agar tidak melakukan push ke repository aslinya (Hanan Hanafi) atau bisa juga dengan menghapus folder .git agar seluruh track commit terhapus. Di modul ini akan dilakukan penghapusan folder .git sebagai upaya mempercepat praktikum. **Perhatikan baik-baik**, di dalam root project php-jwt-example, jalankan perintah berikut!

```
● praktikan@praktikan:~/psait/php-jwt-example$ rm -rf .git
○ praktikan@praktikan:~/psait/php-jwt-example$
```

Gambar 11.4.19 Menghapus folder .git

Kemudian tambahkan file .gitignore di root project php-jwt-example, isi file tersebut seperti Gambar 11.4.20. File .gitignore ini akan mengabaikan beberapa file yang kita maksud. Tujuan dari hal ini adalah agar .env kita tidak tersebar ke tempat umum (jika menggunakan github atau git cloud management service lainnya) dan juga memastikan konfigurasi .env di VPS sesuai dengan kondisi VPS.



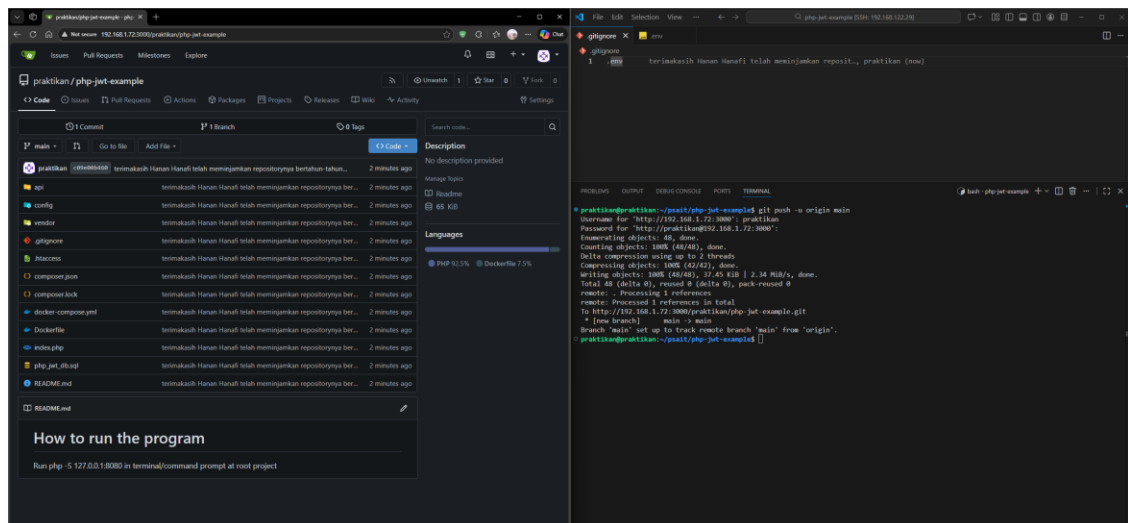
Gambar 11.4.20 Menambahkan .gitignore

Lakukan setup repository git lokal seperti Gambar 11.4.18 dan jangan lupa tambahkan keseluruhan kode (git add .) seperti Gambar 11.4.21.

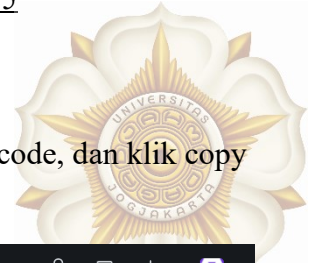
```
praktikan@praktikan:~/psait/php-jwt-example$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/praktikan/psait/php-jwt-example/.git/
praktikan@praktikan:~/psait/php-jwt-example$ git checkout -b main
Switched to a new branch 'main'
praktikan@praktikan:~/psait/php-jwt-example$ git add .
praktikan@praktikan:~/psait/php-jwt-example$ git commit -m "terimakasih Hanan Hanafi telah meminjamkan repositorynya bertahun-tahun <3"
[main (root-commit) c99e00b] terimakasih Hanan Hanafi telah meminjamkan repositorynya bertahun-tahun <3
36 files changed, 3654 insertions(+)
create mode 100644 .gitignore
create mode 100644 .htaccess
create mode 100644 Dockerfile
create mode 100644 README.md
create mode 100644 api/login/index.php
create mode 100644 api/register/index.php
create mode 100644 api/users/index.php
create mode 100644 composer.json
create mode 100644 composer.lock
create mode 100644 config/database.php
create mode 100644 docker-compose.yml
create mode 100644 index.php
create mode 100644 php_jwt_db.sql
create mode 100644 vendor/autoload.php
create mode 100644 vendor/composer/ClassLoader.php
create mode 100644 vendor/composer/InstalledVersions.php
create mode 100644 vendor/composer/LICENSE
create mode 100644 vendor/composer/autoload_classmap.php
create mode 100644 vendor/composer/autoload_namespaces.php
create mode 100644 vendor/composer/autoload_psr4.php
create mode 100644 vendor/composer/autoload_real.php
create mode 100644 vendor/composer/autoload_static.php
create mode 100644 vendor/composer/installed.json
create mode 100644 vendor/composer/installed.php
create mode 100644 vendor/composer/platform_check.php
create mode 100644 vendor/firebase/php-jwt/CHANGELOG.md
create mode 100644 vendor/firebase/php-jwt/LICENSE
create mode 100644 vendor/firebase/php-jwt/README.md
create mode 100644 vendor/firebase/php-jwt/composer.json
create mode 100644 vendor/firebase/php-jwt/src/BeforeValidException.php
create mode 100644 vendor/firebase/php-jwt/src/CachedKeySet.php
create mode 100644 vendor/firebase/php-jwt/src/ExpiredException.php
create mode 100644 vendor/firebase/php-jwt/src/JWK.php
create mode 100644 vendor/firebase/php-jwt/src/JWT.php
create mode 100644 vendor/firebase/php-jwt/src/Key.php
create mode 100644 vendor/firebase/php-jwt/src/SignatureInvalidException.php
praktikan@praktikan:~/psait/php-jwt-example$ git remote add origin http://192.168.1.72:3000/praktikan/php-jwt-example.git
praktikan@praktikan:~/psait/php-jwt-example$
```

Gambar 11.4.21 Staging kode ke gitea

Kemudian tahap terakhir yaitu git push. Mungkin Anda akan diminta untuk memasukkan username dan password, gunakan username dan akun yang telah dibuat di Gitea. Jika berhasil, maka kode akan masuk ke dalam gitea seperti Gambar 11.4.22.

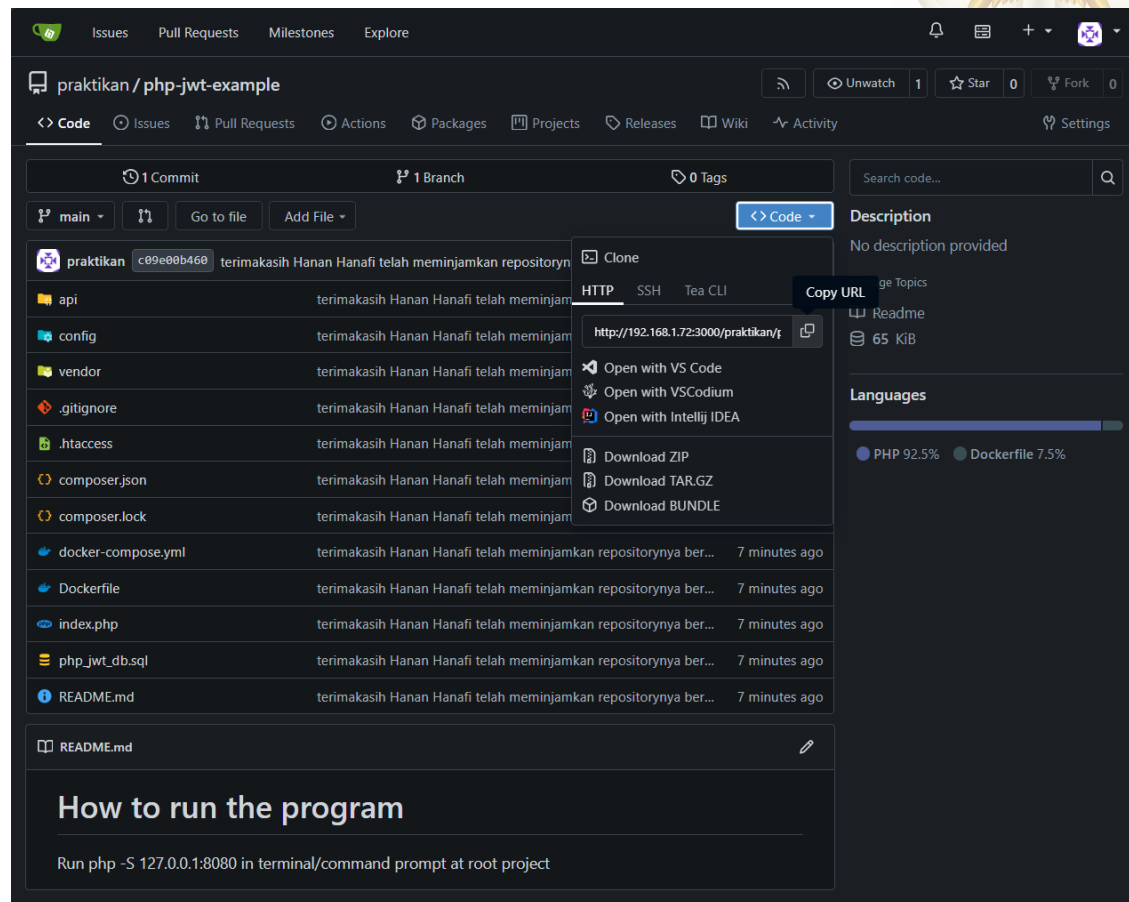


Gambar 11.4.22 Git push



4. Clone repository

Untuk melakukan cloning repository, Anda dapat menekan tombol code, dan klik copy URL seperti Gambar 11.4.23.



Gambar 11.4.23 Copy URL Repository

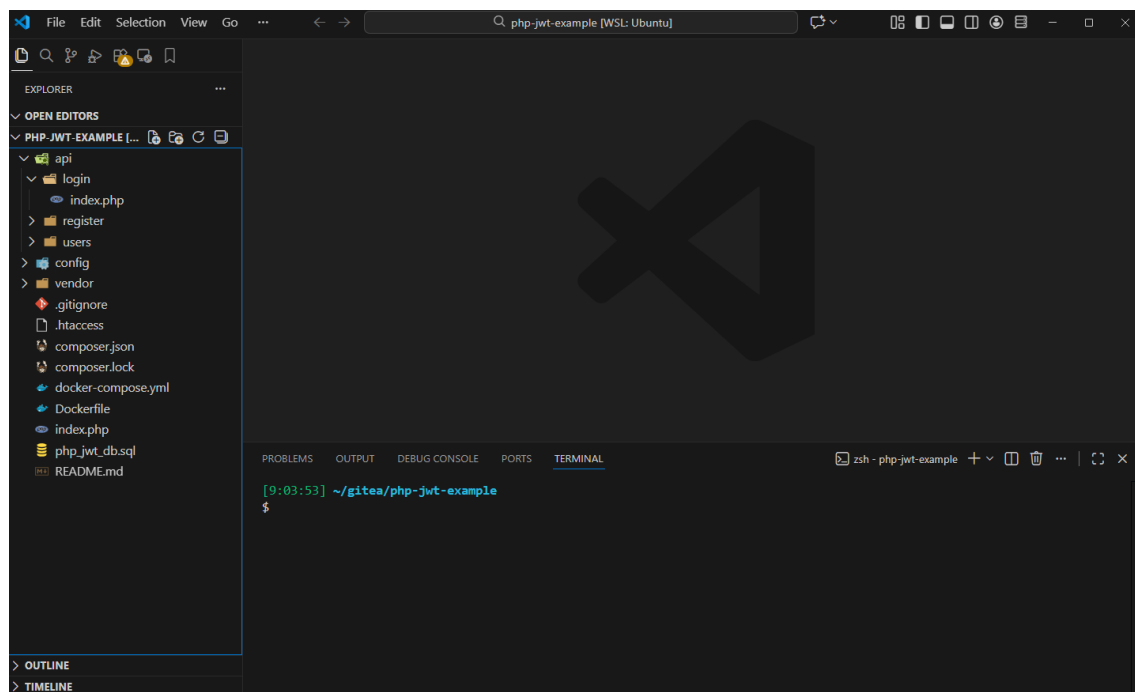
Buka terminal **di Laptop atau komputer** yang terkoneksi dengan jaringan internet yang sama dengan VPS dan jalankan kode seperti berikut Gambar 11.4.24. Jika berhasil maka folder php-jwt-example akan muncul.



```
[9:01:53] ~/gitea
$ git clone http://192.168.1.72:3000/praktikan/php-jwt-example.git
Cloning into 'php-jwt-example'...
remote: Enumerating objects: 48, done.
remote: Counting objects: 100% (48/48), done.
remote: Compressing objects: 100% (42/42), done.
remote: Total 48 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (48/48), 37.45 KiB | 7.49 MiB/s, done.
[9:02:04] ~/gitea
$ ls
php-jwt-example
[9:02:09] ~/gitea
$
```

Gambar 11.4.24 Git clone

Buka kode repository tersebut di Visual Studio Code seperti Gambar 11.4.25. Jika Anda ingin menjalankan aplikasi di Laptop atau komputer, Anda harus menambahkan .env karena di pengaturan yang kita lakukan, file .env tidak akan disimpan di repository. Namun, kita tidak akan menjalankan di lokal dan menganggap aplikasi telah berhasil jalan dan selesai dilakukan perubahan.

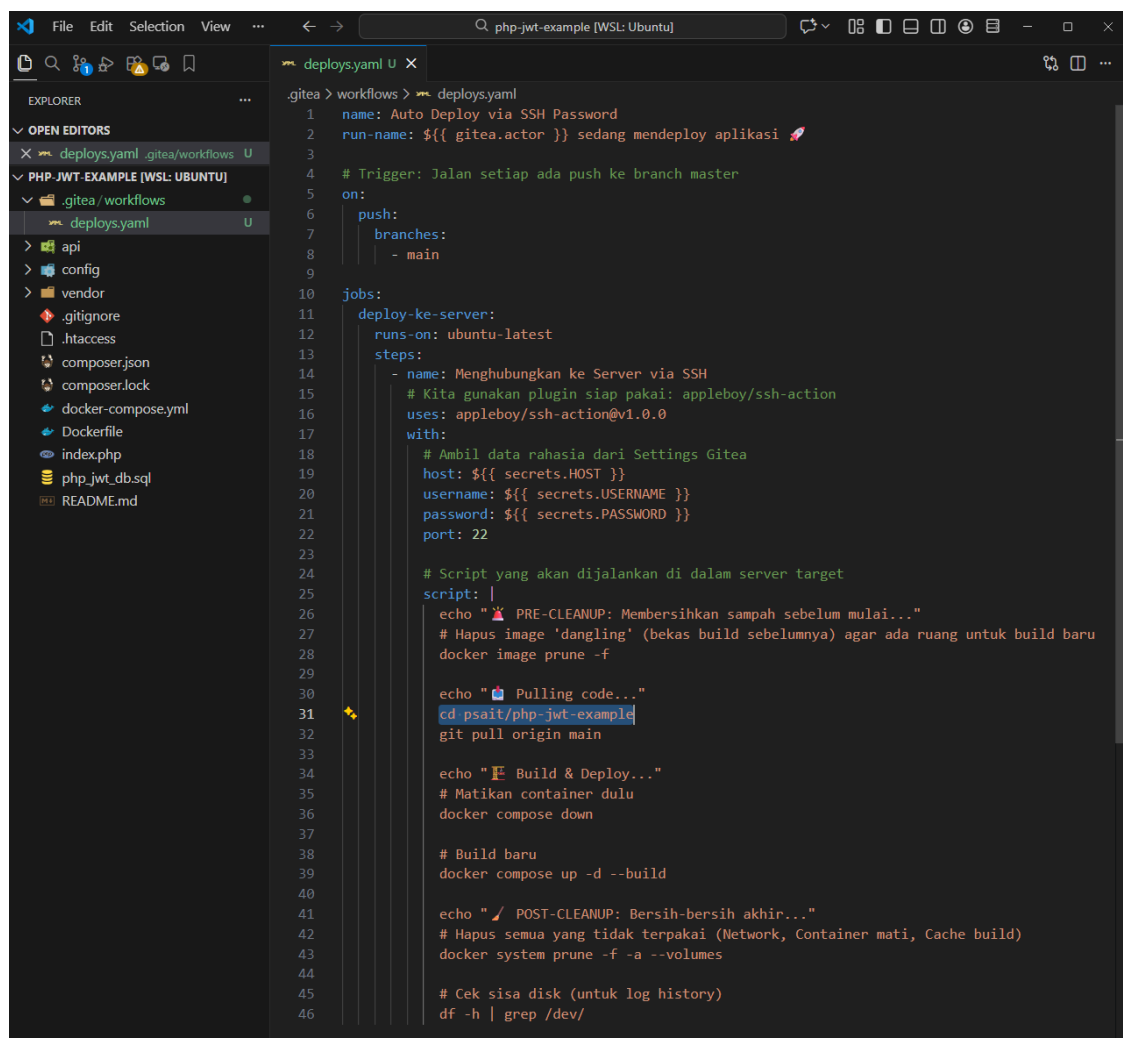


Gambar 11.4.25 Membuka kode di VS code

5. Implementasi Gitea runner untuk CICD

Kita masih melanjutkan di VS Code yang ada di laptop atau komputer yang berada di jaringan internet yang sama dengan VPS, kita akan menambahkan script workflow

untuk CICD. Script ini serupa dengan Github Action, sehingga dapat juga digunakan di Github. Tambahkan file `deploys.yaml` di dalam folder `“.gitea/workflows/”` (jangan lupa tanda titik di depan) seperti Gambar 11.4.26. Di kode yang di highlight, terdapat `change directory` ke lokasi `php-jwt-example` kita di VPS. Anda perlu mengubahnya sesuai dengan kondisi masing-masing VPS! Pada contoh ini, lokasi `php-jwt-example` berada di home VPS, masuk ke folder `psait`, dan masuk lagi ke folder `php-jwt-example`. Jika diperhatikan dengan seksama, script yang ada di dalam workflows CICD ini mirip seperti apa yang kita lakukan secara manual. Namun, script ini akan dijalankan oleh robot sehingga kita tidak lagi mengupdate secara manual.

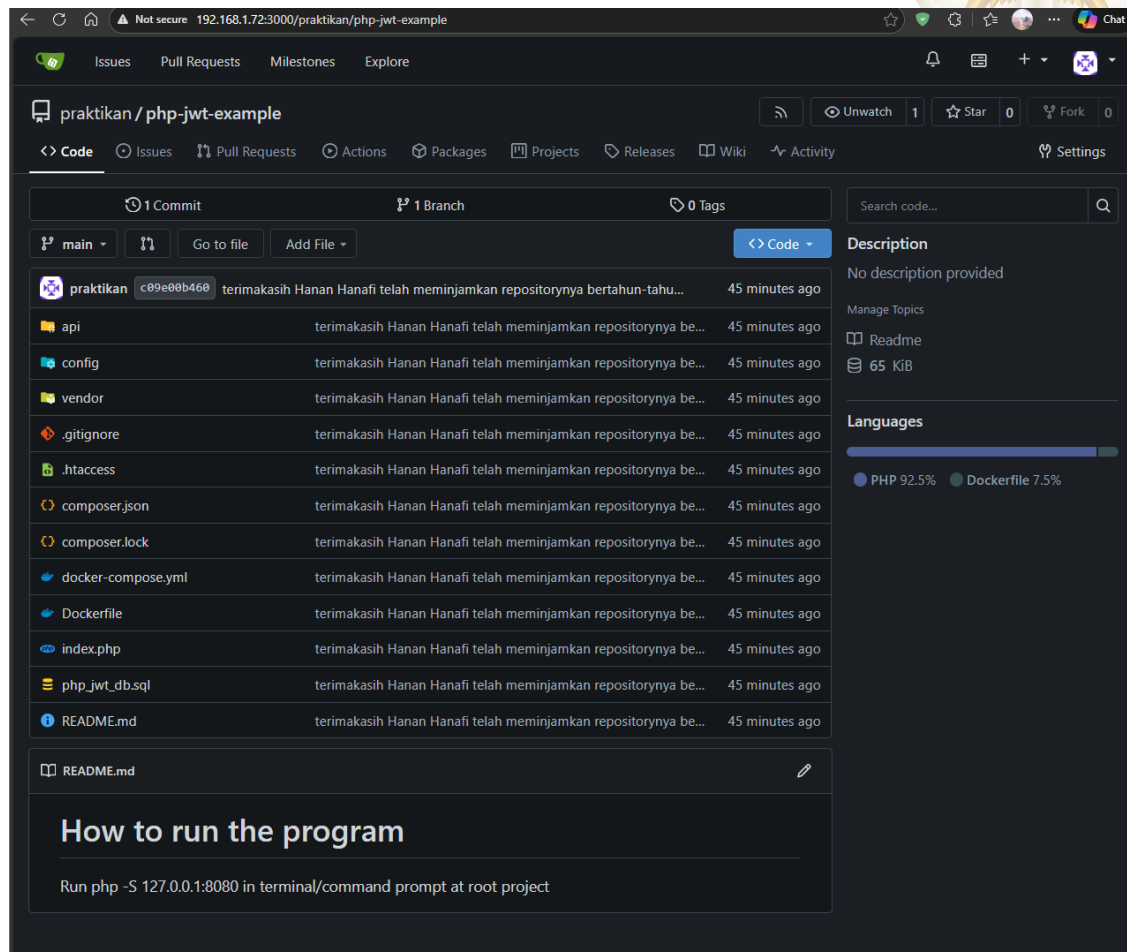


```

1 name: Auto Deploy via SSH Password
2 run-name: ${GITHUB_ACTOR} sedang mendeploy aplikasi 🚀
3
4 # Trigger: Jalan setiap ada push ke branch master
5 on:
6   push:
7     branches:
8       - main
9
10 jobs:
11   deploy-ke-server:
12     runs-on: ubuntu-latest
13     steps:
14       - name: Menghubungkan ke Server via SSH
15         # Kita gunakan plugin siap pakai: appleboy/ssh-action
16         uses: appleboy/ssh-action@v1.0.0
17         with:
18           # Ambil data rahasia dari Settings Gitea
19           host: ${GITHUB_SECRETS_HOST}
20           username: ${GITHUB_SECRETS_USERNAME}
21           password: ${GITHUB_SECRETS_PASSWORD}
22           port: 22
23
24       # Script yang akan dijalankan di dalam server target
25       script: |
26         echo "🧹 PRE-CLEANUP: Membersihkan sampah sebelum mulai..."
27         # Hapus image 'dangling' (bekas build sebelumnya) agar ada ruang untuk build baru
28         docker image prune -f
29
30         echo "📦 Pulling code..."
31         cd psait/php-jwt-example
32         git pull origin main
33
34         echo "🔧 Build & Deploy..."
35         # Matikan container dulu
36         docker compose down
37
38         # Build baru
39         docker compose up -d --build
40
41         echo "🧹 POST-CLEANUP: Bersih-bersih akhir..."
42         # Hapus semua yang tidak terpakai (Network, Container mati, Cache build)
43         docker system prune -f -a --volumes
44
45         # Cek sisa disk (untuk log history)
46         df -h | grep /dev/
  
```

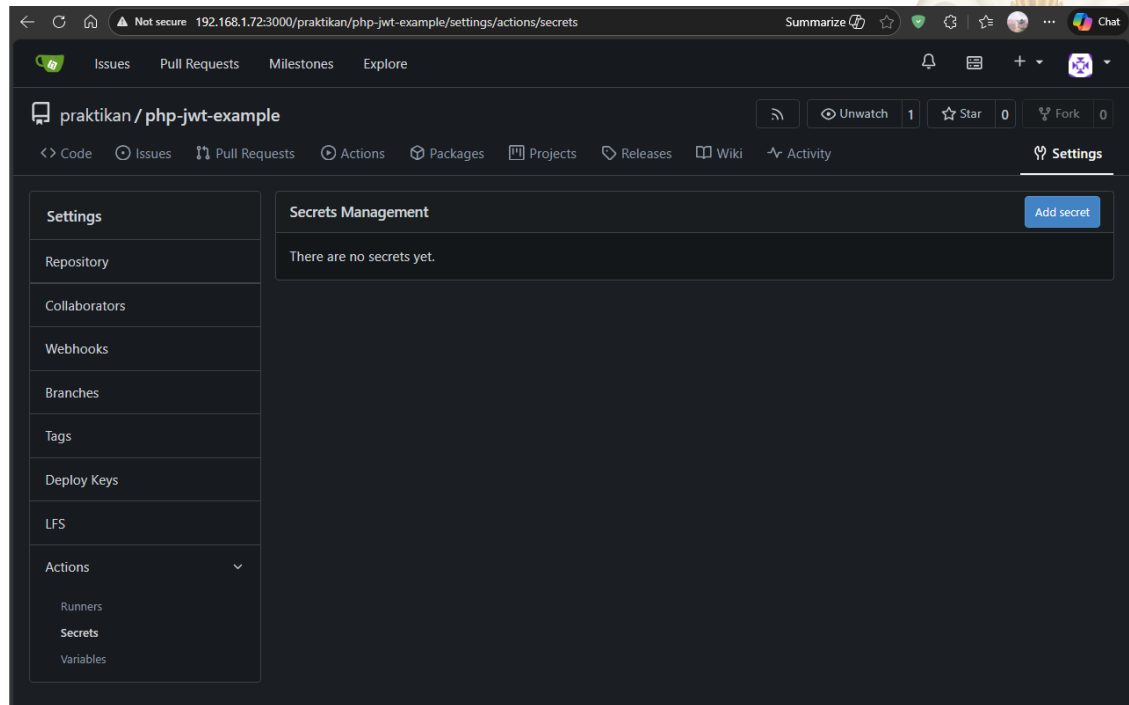
Gambar 11.4.26 Deploys YAML Workflows

Di script Gambar 11.4.26, terdapat host, username, dan juga password. Kita perlu menambahkan secrets variable tersebut di website gitea yang telah kita buat. Buka repository Anda di gitea, dan klik setting (bagian kanan baris ketiga).



Gambar 11.4.27 Tombol setting di kanan baris 3

Pilih menu Actions>Secrets maka akan tertampil seperti Gambar 11.4.28. Di halaman ini Anda perlu menambahkan konfigurasi SSH server VPS kalian! Sesuaikan data VPS yang kalian miliki.



Gambar 11.4.28 Halaman Secrets

Add secret

Secrets will be passed to certain actions and cannot be read otherwise.

Name

HOST

Value

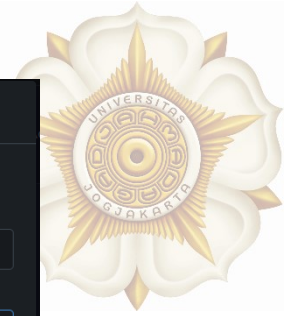
192.168.1.72

Description

IP dari VPS kalian

× Cancel ✓ Confirm

Gambar 11.4.29 Tambah Server IP VPS



Edit secret

Secrets will be passed to certain actions and cannot be read otherwise.

Name

USERNAME

Value

praktikan

Description

Username VPS kalian!

× Cancel ✓ Confirm

Gambar 11.4.30 Tambah Username VPS

Edit secret

Secrets will be passed to certain actions and cannot be read otherwise.

Name

PASSWORD

Value

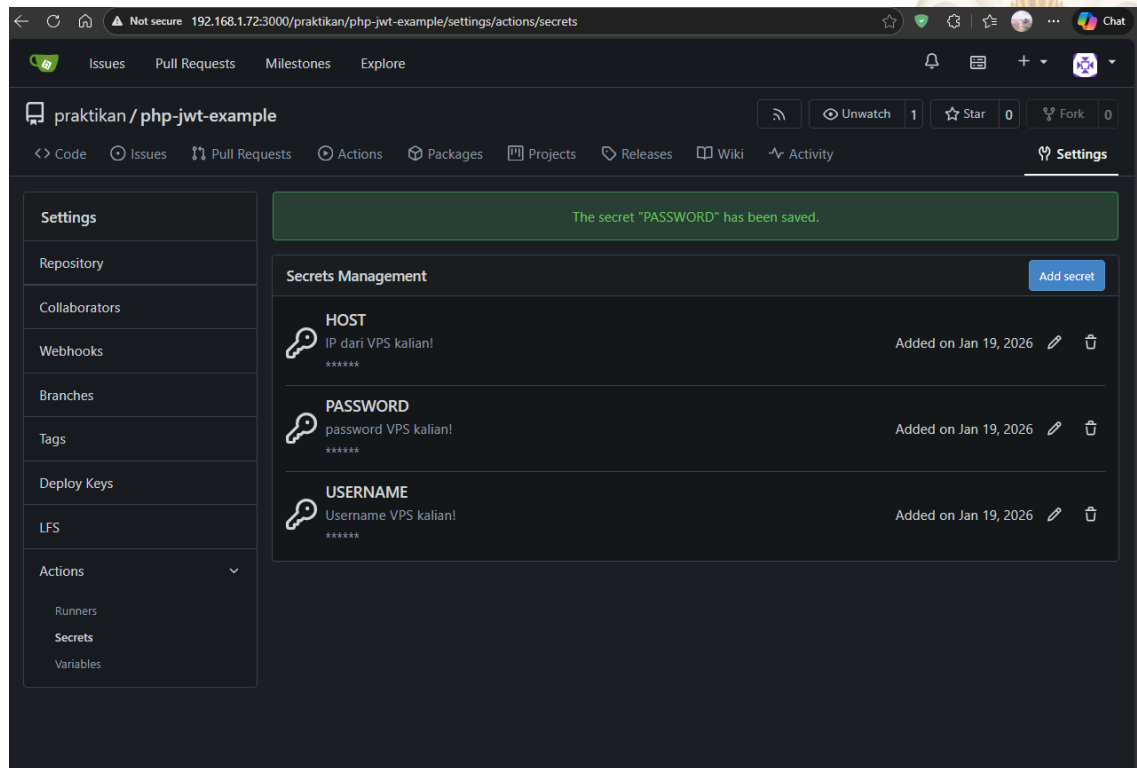
praktikan

Description

password VPS kalian!

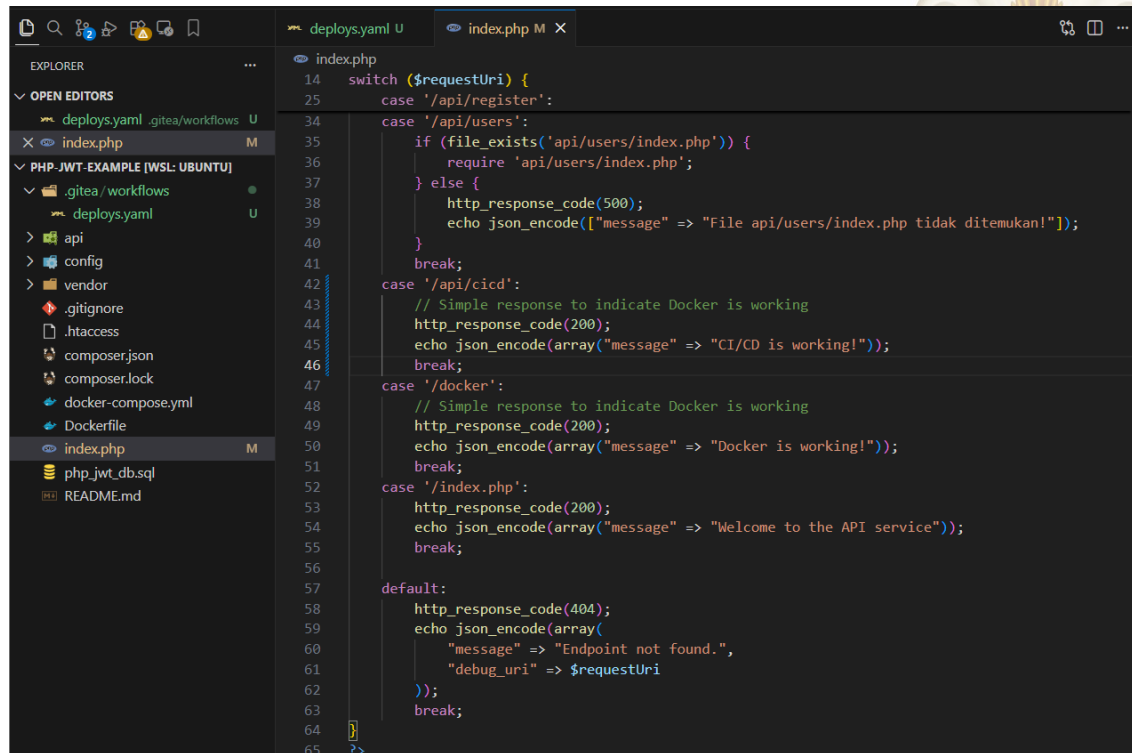
× Cancel ✓ Confirm

Gambar 11.4.31 Tambah Password VPS



Gambar 11.4.32 Tiga secrets

Untuk mengetes CICD, kita akan mencoba membuat route endpoint baru bernama cicd. Tambahkan route tersebut di index.php yang pernah kita buat sebagai routes aplikasi seperti Gambar 11.4.33.



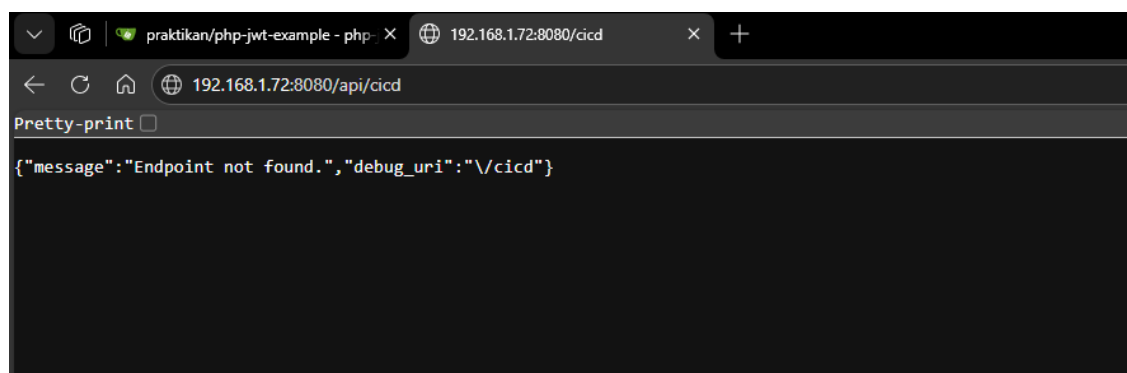
```

14  switch ($requestUri) {
25      case '/api/register':
34      case '/api/users':
35          if (file_exists('api/users/index.php')) {
36              require 'api/users/index.php';
37          } else {
38              http_response_code(500);
39              echo json_encode(["message" => "File api/users/index.php tidak ditemukan!"]);
40          }
41          break;
42      case '/api/cicd':
43          // Simple response to indicate Docker is working
44          http_response_code(200);
45          echo json_encode(array("message" => "CI/CD is working!"));
46          break;
47      case '/docker':
48          // Simple response to indicate Docker is working
49          http_response_code(200);
50          echo json_encode(array("message" => "Docker is working!"));
51          break;
52      case '/index.php':
53          http_response_code(200);
54          echo json_encode(array("message" => "Welcome to the API service"));
55          break;
56      default:
57          http_response_code(404);
58          echo json_encode(array(
59              "message" => "Endpoint not found.",
60              "debug_uri" => $requestUri
61          ));
62          break;
63      }
64  }
65  }

```

Gambar 11.4.33 Route baru cicd

Sebelum kita melakukan push, buka url cicd di IP VPS Anda saat ini, maka web akan menampilkan seperti Gambar 11.4.34. Route endpoint CICD belum ada di VPS. Kita akan mencoba melihat apakah sistem CICD berhasil melakukan update aplikasi secara otomatis yang ditandai dengan berfungsinya route endpoint cicd.



```

praktikan/php-jwt-example - php 192.168.1.72:8080/cicd
192.168.1.72:8080/api/cicd
Pretty-print
{"message":"Endpoint not found.", "debug_uri":"/cicd"}

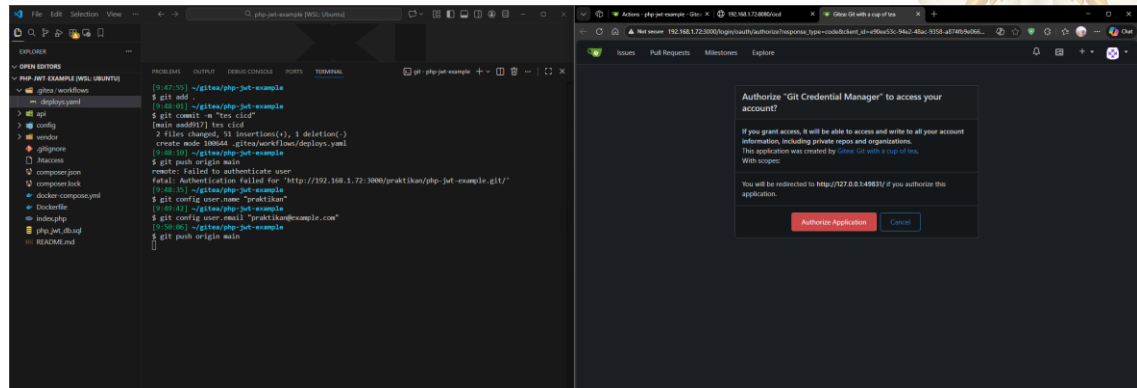
```

Gambar 11.4.34 Tampilan url CICD awal

6. Push aplikasi

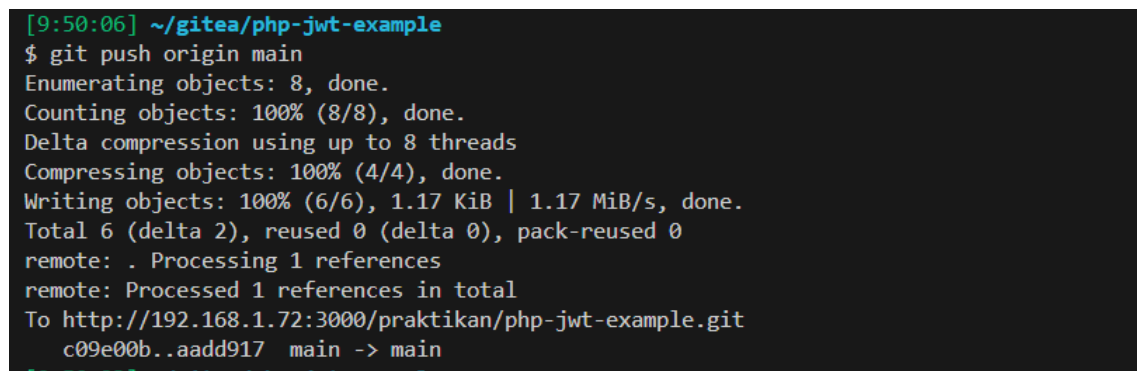
Masukkan perubahan kode ke staging dengan “git add .” (jangan lupa titik). Kemudian tambahkan commit message. Lakukan push dengan git push. Jika Anda mengalami

kredensial error seperti Gambar 11.4.35 (karena di pembuatan modul ini, akun git belum ditambahkan), tambahkan akun gitea yang telah didaftarkan. Pada halaman website gitea ketika user melakukan login, Gitea akan meminta autorisasi, Klik Authorize Application.



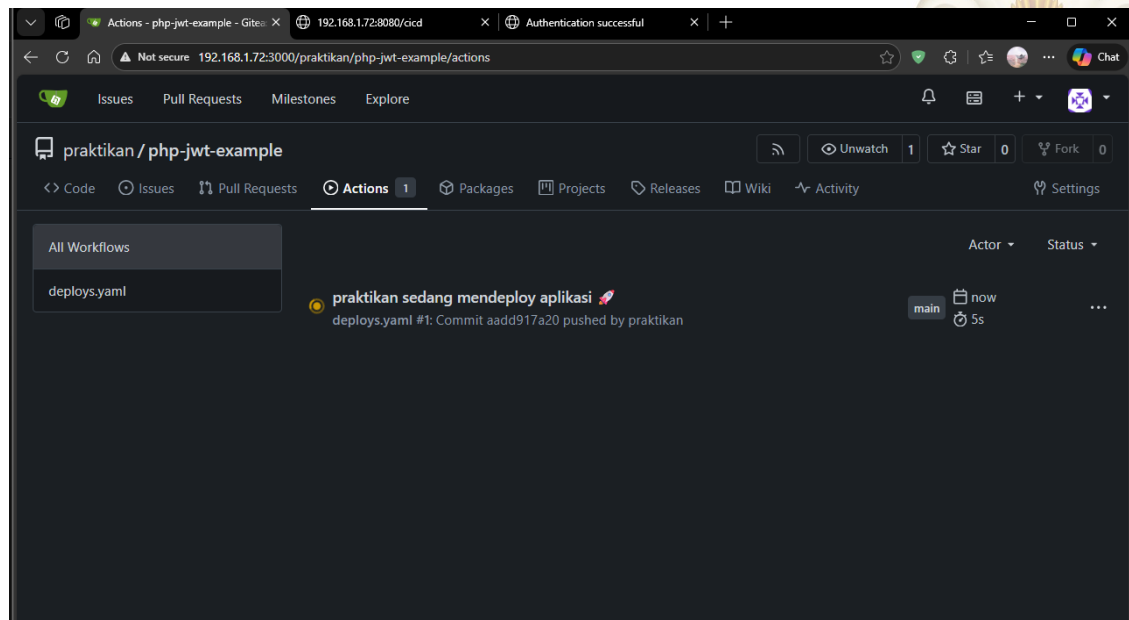
Gambar 11.4.35 Git Push dan Kredensial

Setelah klik authorize, perubahan akan ditambahkan ke dalam gitea.



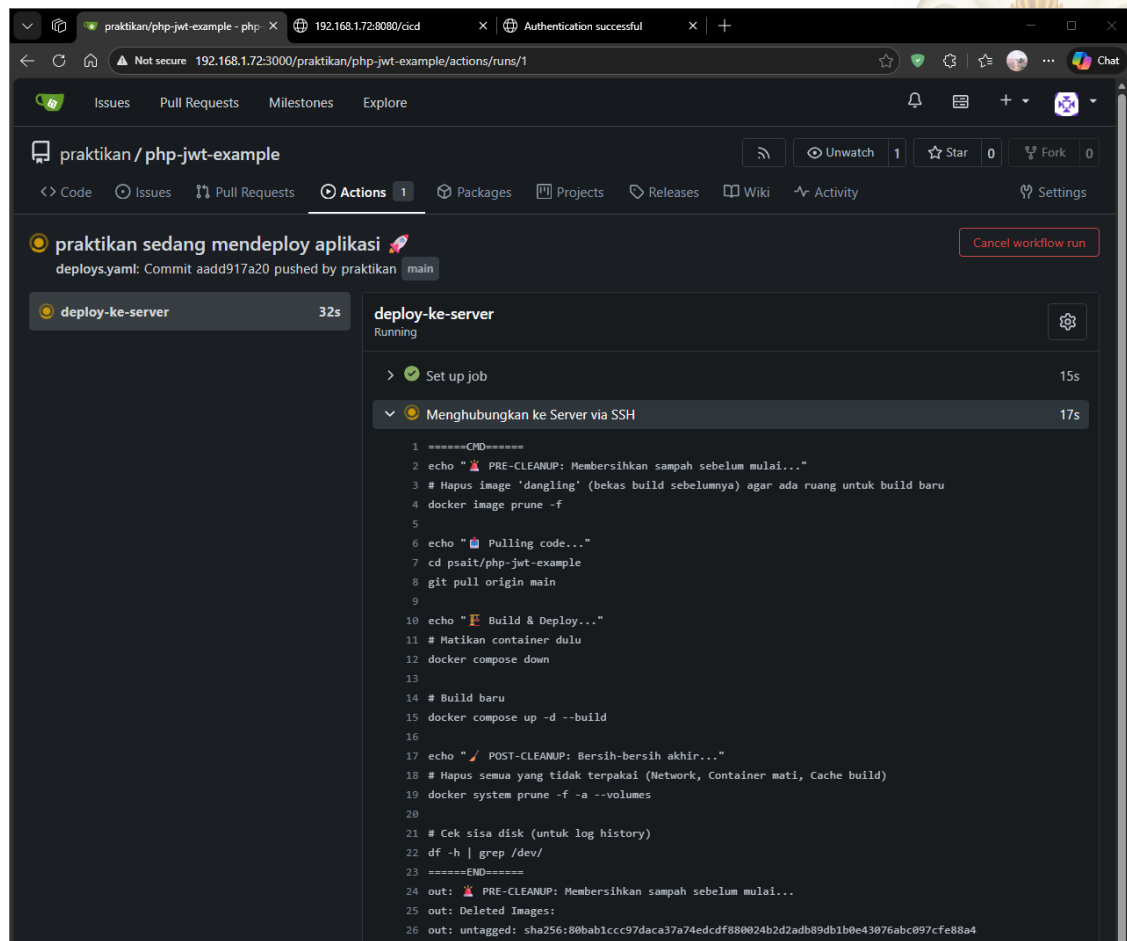
Gambar 11.4.36 Lanjutan terminal ketika push di izinkan

Jika kita membuka menu jaction di repository aplikasi di gitea, maka kita akan melihat Actions CICD kita berjalan seperti Gambar 11.4.37.



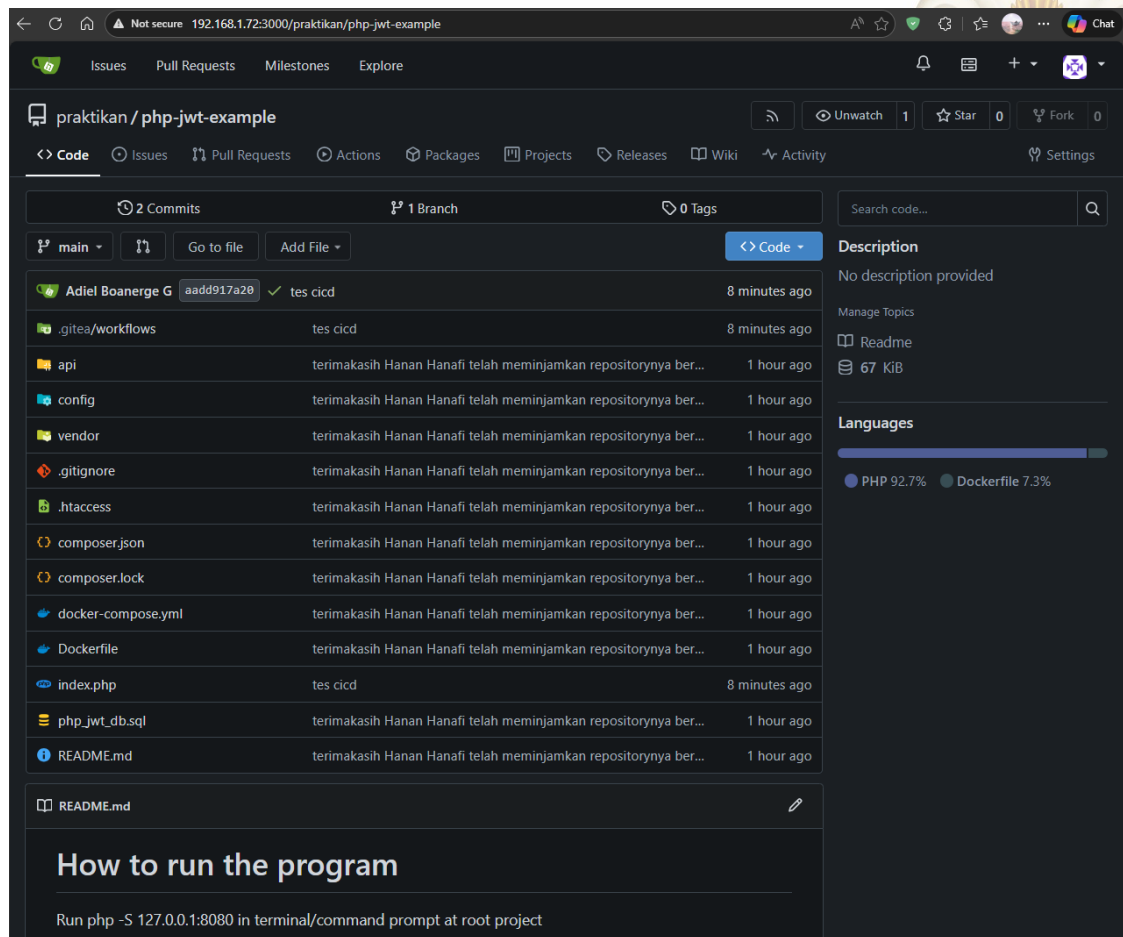
Gambar 11.4.37 Actions Berjalan di Gitea

Anda bisa melihat detail bagaimana robot menjalankan kode CICD workflows yang dibuat seperti Gambar 11.4.38.



Gambar 11.4.38 Detail Actions

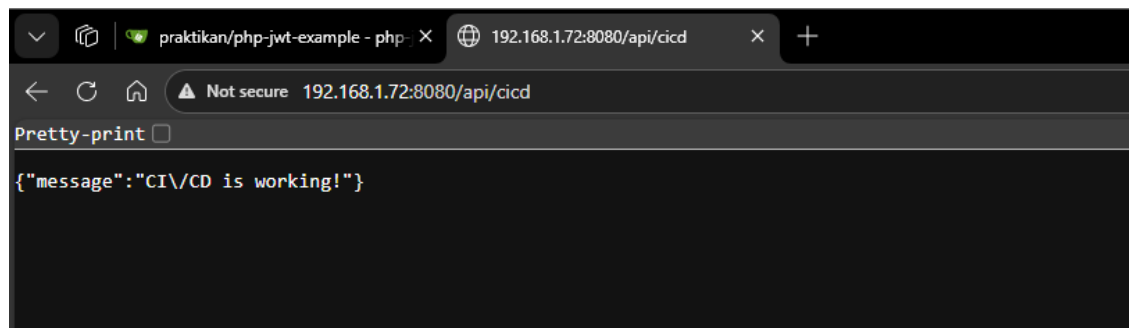
Setelah selesai Actions tersebut akan memberikan tanda centang di repository seperti Gambar 11.4.39 jika berhasil. Tanda merah jika Actions tersebut gagal, dan kuning jika masih progress.



Gambar 11.4.39 Action berhasil dijalankan

7. Cek action CICD di aplikasi

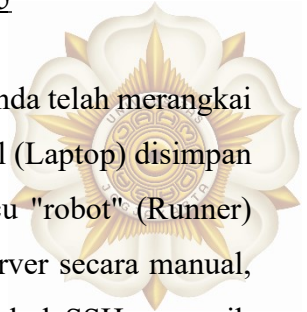
Kita buka kembali route cicd yang telah kita buat di website. Jika tertampil seperti Gambar 11.4.40, maka Action yang kita buat berhasil dilakukan.



Gambar 11.4.40 Aplikasi berhasil diperbarui

8. Penutup dan refleksi

Selamat, Anda telah berhasil menyelesaikan transformasi dari pengembangan manual menuju otomatisasi penuh dengan membangun infrastruktur Continuous Integration dan



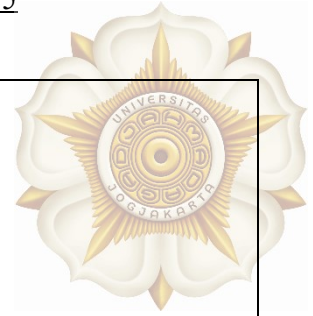
Continuous Deployment (CI/CD) mandiri. Melalui praktikum ini, Anda telah merangkai sebuah sistem terpadu di mana kode yang ditulis di lingkungan lokal (Laptop) disimpan ke dalam Version Control System (Gitea), yang kemudian memicu "robot" (Runner) untuk bekerja secara otomatis. Tanpa perlu menyentuh terminal server secara manual, Runner tersebut berhasil masuk ke server produksi melalui protokol SSH, menarik pembaruan kode terbaru, dan membangun ulang container Docker dalam hitungan detik.

Kompetensi yang Anda kuasai hari ini merupakan simulasi langsung dari standar operasional DevOps Engineer di industri teknologi modern. Di dunia profesional, metode deploy manual via FTP atau copy-paste file sudah lama ditinggalkan karena rentan terhadap human error dan sangat lambat. Perusahaan rintisan (startup) hingga korporasi besar mengandalkan otomatisasi seperti ini untuk mempercepat time-to-market fitur baru. Meskipun dalam praktikum ini kita menggunakan Gitea karena keterbatasan jaringan, logika skrip YAML dan konsep Runner yang Anda pelajari 99% identik dengan GitHub Actions atau GitLab CI. Artinya, bekal teknis ini dapat langsung Anda terapkan saat terjun ke lingkungan kerja profesional yang menggunakan platform global tersebut.

Ke depannya, pipeline ini dapat dikembangkan lebih jauh dengan menambahkan tahap pengujian otomatis (Automated Testing) sebelum deployment. Dengan demikian, sistem dapat mencegah kode yang bermasalah (bug) masuk ke server produksi, menjaga stabilitas aplikasi, dan membiarkan Anda fokus pada hal yang paling penting: menciptakan kode yang berkualitas.

11.5. TUGAS & ANALISIS

Modul praktikum langkah percobaan yang telah kalian lakukan hanya membahas CD saja (belum ada testing dan gatekeeper ketika error agar tidak di deploy), tugas kali ini adalah mencoba menambahkan testing sehingga menjadi keseluruhan CICD. Buatlah folder baru bernama test dan tambahkan file bernama test_matematika_sederhana.php, isikan file tersebut dengan kode ini. Anggap saja kode ini adalah unit test dari aplikasi php-jwt-example.



```
<?php
// Logika sederhana
$a = 5;
$b = 10;
$hasil = $a + $b;

echo "Menjalankan Test: $a + $b ... \n";

// Logika Validasi
if ($hasil == 15) {
    echo "[SUKSES] Perhitungan benar. Silakan lanjut deploy.\n";
    exit(0); // Kode 0 artinya Aman/Sukses
} else {
    echo "[GAGAL] Perhitungan salah! Jangan deploy ke server!\n";
    exit(1); // Kode 1 (atau selain 0) artinya Error/Stop
}
?>
```

1. Tugas Anda adalah menjalankan file test tersebut ke dalam Actions workflows yang telah kita buat. Modifikasi deploys.yaml agar runner yang ada di gitea mampu membatalkan aksi deploy jika logika sederhana tersebut salah! Screenshots juga detail workflow Actions yang ada di Gitea ketika sudah selesai berjalan!
2. Ubah nilai dari a menjadi 55 dan screenshot apa yang terjadi di Actions Gitea!

11.6. REFERENSI

- [1]. Docker, “Docker Documentation,” Docker Documentation, Oct. 31, 2019. <https://docs.docker.com/>
- [2]. “Gitea Official Website,” about.gitea.com. <https://about.gitea.com/>



PERTEMUAN XII

IMPLEMENTASI DEPLOYMENT DAN LOAD BALANCING PADA VPS

12.1. TUJUAN PEMBELAJARAN

- A. Mahasiswa mampu setup aplikasi Laravel
- B. Mahasiswa mampu melakukan proses deployment aplikasi Laravel dari repositori Git ke server produksi
- C. Mahasiswa mampu mengonfigurasi Apache Virtual Host untuk menjalankan aplikasi web pada port spesifik (8001 dan 8002).
- D. Mahasiswa mampu memahami konsep Horizontal Scaling dengan menduplikasi instance aplikasi untuk menangani beban trafik.
- E. Mahasiswa mampu mengimplementasikan teknik Load Balancing menggunakan modul Reverse Proxy pada Apache untuk mendistribusikan trafik ke beberapa instance aplikasi.

12.2. DASAR TEORI

Di praktikum-praktikum sebelumnya kita telah belajar cara melakukan setup aplikasi dengan berbagai service seperti front-end, backend, database, dan external api. Aplikasi tersebut tentu dapat berjalan dengan lancar. Namun, apa yang terjadi jika tiba-tiba ada lonjakan pengguna yang mengakses aplikasi tersebut?

Lonjakan pengguna secara tiba-tiba cukup berbahaya, hal ini hampir serupa dengan serangan Distributed Denial of Service (DDOS). Sistem aplikasi dapat lumpuh seketika jika hal tersebut terjadi. Kelumpuhan sistem ini tentu dapat menyebabkan kerugian bagi perusahaan pemilik sistem aplikasi tersebut. Sebagai contoh, jika perusahaan belanja online tidak mempersiapkan sistemnya untuk hari belanja nasional, aplikasi belanja tersebut akan lumpuh ketika pengguna tiba-tiba menyerbu aplikasi tersebut. Perusahaan tentu akan kehilangan banyak kesempatan keuntungan dari kegagalan sistem dan pelanggan yang pergi. Oleh karena itu, kita sebagai Software Engineer tentu harus memikirkan mitigasi dari hal tersebut.

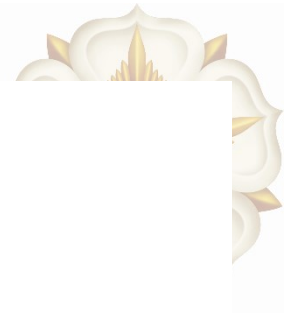


Salah satu cara mengatasi hal tersebut adalah dengan melakukan scaling aplikasi. Scaling dapat dibagi menjadi dua, yaitu scaling secara horizontal dan scaling secara vertical. Berikut penjelasan singkat dari perbedaan keduanya.

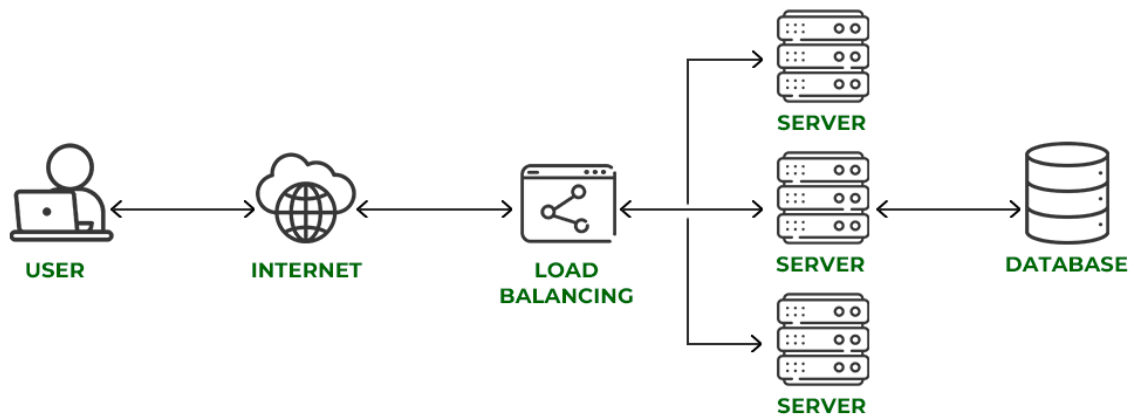
Horizontal Scaling vs Vertical Scaling

- Vertical Scaling: Menambah kekuatan satu server (misal: upgrade RAM 4GB ke 16GB). Ada batas fisiknya.
- Horizontal Scaling: Menambah jumlah server atau instance aplikasi (misal: dari 1 aplikasi menjadi 2 aplikasi yang berjalan paralel).

Di modul praktikum ini, kita akan mencoba menerapkan load balancing yang merupakan salah satu cara dari Horizontal Scaling. Sewajarnya, load balancing dilakukan di banyak server sehingga satu server akan menjalankan satu instance aplikasi. Tujuan dari hal ini adalah menyebar beban trafik ke beberapa server. Selain itu, jika salah satu server lumpuh, aplikasi kita tetap dapat berjalan. Secara konsep diagram akan terlihat seperti Gambar 12.2.1. Server yang akan digunakan untuk menjalankan Load Balancing akan dijadikan sebagai Gateway dari instances aplikasi kita. Kemudian, aplikasi kita yang merupakan satu, dikloning ke berbagai server. Perlu diperhatikan bahwa aplikasi yang dikloning merupakan aplikasi bagian stateless (tidak menyimpan data) sehingga database dan storage tidak boleh dikloning untuk tetap memegang pedoman “a single source of truth”. Namun, kita tetap bisa membuat semacam Slave DB atau cache di setiap server untuk mempercepat pembacaan data (proses Write hanya boleh ke Master DB, Slave digunakan untuk read).



How Load Balancing Works



Gambar 12.2.1 Load Balancing (Sumber: [geeksforgeeks.org](https://www.geeksforgeeks.org))

Oleh karena keterbatasan sumber daya di laboratorium praktikum ini. Praktikum load balancing hanya dilakukan di satu server. Hal ini bukanlah load balancing yang sejati karena jika server mati, maka aplikasi kita akan mati. Namun, dalam praktikum ini, penggunaan Load Balancing berguna untuk meningkatkan kapasitas pekerja di server. Kita mengetahui bahwa PHP memiliki keterbatasan dan juga secara native tidak support concurrency. Hal tersebut dapat diatasi dengan banyak hal seperti menggunakan server PHP-FPM, FrankenPHP, maupun load balancing. Perlu diketahui bahwa load balancing di satu server akan dibatasi oleh limitasi spesifikasi hardware dari server itu sendiri.

Load Balancing yang akan dilakukan akan menggunakan web server Apache2. Di dalam Apache2 tersebut terdapat Apache Load Balancer yang bertindak sebagai polisi lalu lintas. Load Balancer ini akan menerima semua permintaan masuk di pintu utama (Port 80) dan mendistribusikannya ke server-server di belakangnya (contohnya jika diatur ke Port 8001, 8002, dst). Apache menggunakan modul `mod_proxy` dan `mod_proxy_balancer` untuk melakukan ini. Algoritma standar yang digunakan biasanya adalah Round Robin, di mana permintaan dibagi secara bergilir (Request 1 -> App A, Request 2 -> App B, Request 3 -> App A, dst).

12.3. ALAT DAN BAHAN



12.3.1. Perangkat Keras

- Laptop/PC (Host).
- Virtual Private Server.

12.3.2. Perangkat Lunak

- Terminal / SSH Client (PuTTY, VS Code Terminal).
- Git (untuk cloning repository).
- Apache2 Web Server.
- PHP 8.2 (berserta ekstensi: xml, curl, gd, mysql, zip, dll).
- Composer.
- Node.js & NPM (via NVM).
- MySQL Server.

12.4. LANGKAH PERCOBAAN

Pada bab ini, kita akan melakukan simulasi penerapan arsitektur aplikasi monolith menggunakan framework Laravel. Laravel adalah framework yang cukup kompleks, kita perlu memastikan lingkungan server (environment) sudah benar-benar siap sebelum masuk ke tahap koding. Setelah aplikasi berjalan, kita akan mencoba skenario Horizontal Scaling sederhana dengan membagi beban trafik ke dua aplikasi kloning menggunakan teknik Load Balancing.

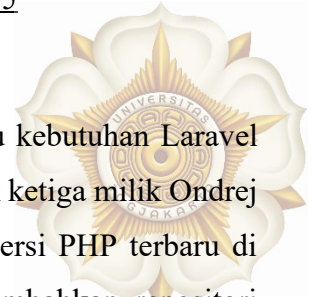
1. Menyiapkan Environment VPS

Memastikan Ekstensi PHP telah terinstal

Laravel versi terbaru membutuhkan PHP versi modern agar fitur-fiturnya berjalan optimal. Untuk modul ini, kita menargetkan penggunaan PHP 8.2. Langkah pertama, silakan cek apakah PHP sudah terinstal di sistem Anda dengan perintah `php -v`.

```
praktikan@praktikan:~$ php -v
PHP 8.3.6 (cli) (built: Jan 7 2026 08:40:32) (NTS)
Copyright (c) The PHP Group
Zend Engine v4.3.6, Copyright (c) Zend Technologies
    with Zend OPcache v8.3.6, Copyright (c), by Zend Technologies
praktikan@praktikan:~$
```

Gambar 12.4.1 Tampilan terminal cek versi PHP



Jika PHP belum ada atau versinya masih jadul (di bawah 8.2 atau kebutuhan Laravel yang Anda gunakan), kita perlu mengambilnya dari repositori pihak ketiga milik Ondrej Sury. Repositori ini adalah standar industri untuk mendapatkan versi PHP terbaru di Ubuntu. Jalankan perintah berikut secara berurutan untuk menambahkan repositori tersebut:

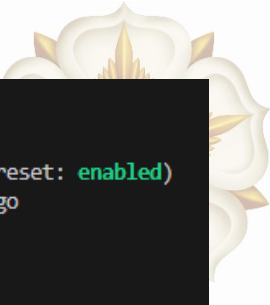
```
sudo apt install software-properties-common
sudo add-apt-repository ppa:ondrej/php
sudo apt update
```

Setelah repositori siap, kita akan menginstal PHP 8.2 beserta beberapa ekstensinya. Karena laravel membuauthkan beberapa ekstensi seperti php-xml untuk manipulasi data, php-curl untuk akses API, php-gd untuk olah gambar, hingga php-mysql untuk koneksi database. Salin dan jalankan perintah berikut:

```
sudo apt install php8.2-common php8.2-pgsql php8.2-xml php8.2-xmldrpc php8.2-
curl php8.2-gd php8.2-imagick php8.2-cli php8.2-dev php8.2-imap php8.2-mbstring
php8.2-opcache php8.2-soap php8.2-zip php8.2-redis php8.2-intl php8.2-fpm
php8.2-bcmath php8.2-mysql -y
```

Memastikan Apache2 telah berjalan

Kita menggunakan Apache2 sebagai web server yang bertugas melayani permintaan HTTP. Pastikan layanannya sudah aktif dengan mengecek statusnya via perintah `systemctl status apache2`. Jika tulisan "Active: active (running)" muncul, berarti aman.



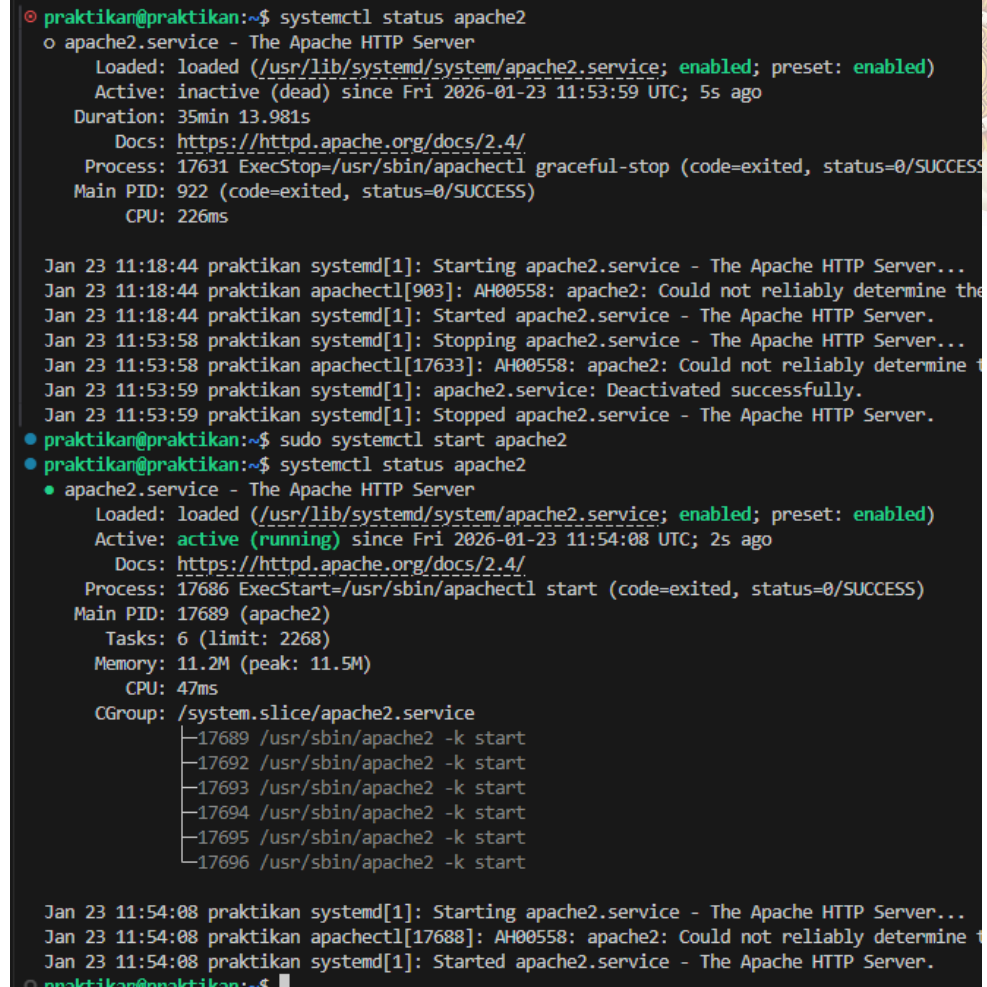
```

● praktikan@praktikan:~$ systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-01-23 11:18:44 UTC; 34min ago
     Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 922 (apache2)
     Tasks: 6 (limit: 2268)
   Memory: 9.9M (peak: 18.3M swap: 1.2M swap peak: 1.2M)
      CPU: 161ms
   CGroup: /system.slice/apache2.service
           └─922 /usr/sbin/apache2 -k start
             └─924 /usr/sbin/apache2 -k start
               └─925 /usr/sbin/apache2 -k start
                 └─926 /usr/sbin/apache2 -k start
                   └─927 /usr/sbin/apache2 -k start
                     └─928 /usr/sbin/apache2 -k start

Jan 23 11:18:44 praktikan systemd[1]: Starting apache2.service - The Apache HTTP Server...
Jan 23 11:18:44 praktikan apachectl[903]: AH00558: apache2: Could not reliably determine t
Jan 23 11:18:44 praktikan systemd[1]: Started apache2.service - The Apache HTTP Server.
○ praktikan@praktikan:~$
    
```

Gambar 12.4.2 Status apache2 running

Namun, jika statusnya mati (dead) atau belum berjalan, Anda harus menjalankannya terlebih dahulu dengan perintah `sudo systemctl start apache2`, lalu cek kembali statusnya.



```

praktikan@praktikan:~$ systemctl status apache2
○ apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: inactive (dead) since Fri 2026-01-23 11:53:59 UTC; 5s ago
   Duration: 35min 13.981s
   Docs: https://httpd.apache.org/docs/2.4/
   Process: 17631 ExecStop=/usr/sbin/apachectl graceful-stop (code=exited, status=0/SUCCESS)
   Main PID: 922 (code=exited, status=0/SUCCESS)
   CPU: 226ms

Jan 23 11:18:44 praktikan systemd[1]: Starting apache2.service - The Apache HTTP Server...
Jan 23 11:18:44 praktikan apachectl[903]: AH00558: apache2: Could not reliably determine the
Jan 23 11:18:44 praktikan systemd[1]: Started apache2.service - The Apache HTTP Server.
Jan 23 11:53:58 praktikan systemd[1]: Stopping apache2.service - The Apache HTTP Server...
Jan 23 11:53:58 praktikan apachectl[17633]: AH00558: apache2: Could not reliably determine the
Jan 23 11:53:59 praktikan systemd[1]: apache2.service: Deactivated successfully.
Jan 23 11:53:59 praktikan systemd[1]: Stopped apache2.service - The Apache HTTP Server.
● praktikan@praktikan:~$ sudo systemctl start apache2
● praktikan@praktikan:~$ systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-01-23 11:54:08 UTC; 2s ago
   Docs: https://httpd.apache.org/docs/2.4/
   Process: 17686 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Main PID: 17689 (apache2)
   Tasks: 6 (limit: 2268)
   Memory: 11.2M (peak: 11.5M)
   CPU: 47ms
   CGroup: /system.slice/apache2.service
           └─17689 /usr/sbin/apache2 -k start
             └─17692 /usr/sbin/apache2 -k start
               └─17693 /usr/sbin/apache2 -k start
                 └─17694 /usr/sbin/apache2 -k start
                   └─17695 /usr/sbin/apache2 -k start
                     └─17696 /usr/sbin/apache2 -k start

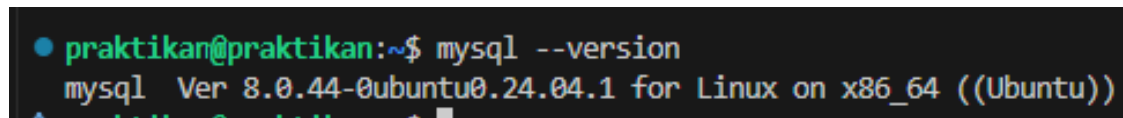
Jan 23 11:54:08 praktikan systemd[1]: Starting apache2.service - The Apache HTTP Server...
Jan 23 11:54:08 praktikan apachectl[17688]: AH00558: apache2: Could not reliably determine the
Jan 23 11:54:08 praktikan systemd[1]: Started apache2.service - The Apache HTTP Server.
praktikan@praktikan:~$

```

Gambar 12.4.3 Menjalankan apache2

Memastikan MySQL telah terinstal

Selanjutnya untuk urusan database, kita akan menggunakan MySQL yang sudah dipelajari sebelumnya. Pastikan service-nya sudah terinstal dan dikenali sistem dengan perintah `mysql --version`. Jika terminal menampilkan detail versi, berarti MySQL sudah ready dan aman digunakan.



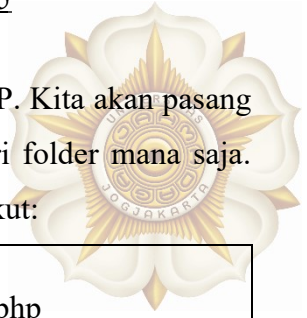
```

praktikan@praktikan:~$ mysql --version
mysql Ver 8.0.44-0ubuntu0.24.04.1 for Linux on x86_64 ((Ubuntu))
praktikan@praktikan:~$

```

Gambar 12.4.4 Cek version mysql

Instalasi Composer



Selanjutnya, kita instal Composer sebagai dependency manager PHP. Kita akan pasang secara global di `/usr/local/bin` agar perintahnya bisa dipanggil dari folder mana saja. Pastikan Anda berada di direktori home, lalu eksekusi perintah berikut:

```
cd ~
curl -sS https://getcomposer.org/installer -o /tmp/composer-setup.php
sudo php /tmp/composer-setup.php --install-dir=/usr/local/bin --filename=composer
```

Jika proses selesai, akan muncul pesan "Composer successfully installed". Ini tandanya Composer sudah siap digunakan.

```
● praktikan@praktikan:~$ cd ~
● praktikan@praktikan:~$ curl -sS https://getcomposer.org/installer -o /tmp/composer-setup.php
● praktikan@praktikan:~$ sudo php /tmp/composer-setup.php --install-dir=/usr/local/bin --filename=composer
[sudo] password for praktikan:
All settings correct for using Composer
Downloading...

Composer (version 2.9.4) successfully installed to: /usr/local/bin/composer
Use it: php /usr/local/bin/composer
```

Gambar 12.4.5 Output sukses instalasi Composer

Atau untuk mengecek dapat juga dilakukan dengan perintah Composer. Jika Composer menampilkan hasil seperti dibawah ini, maka composer telah terinstal.

```
praktikan@praktikan:~$ composer

 _____
/  _ \   / \   _ __| |__ \   _ __| |__ \
| |_) | / _ \ / _ \| '_ \| |_) | / _ \|
| |_) | / _ \ / _ \| '_ \| |_) | / _ \|
|  __/ |  __/|  __/|  __/|  __/|  __/
\_____|_____|_____|_____|_____|_____|
Composer version 2.9.4 2026-01-22 14:08:50

Usage:
  command [options] [arguments]
```

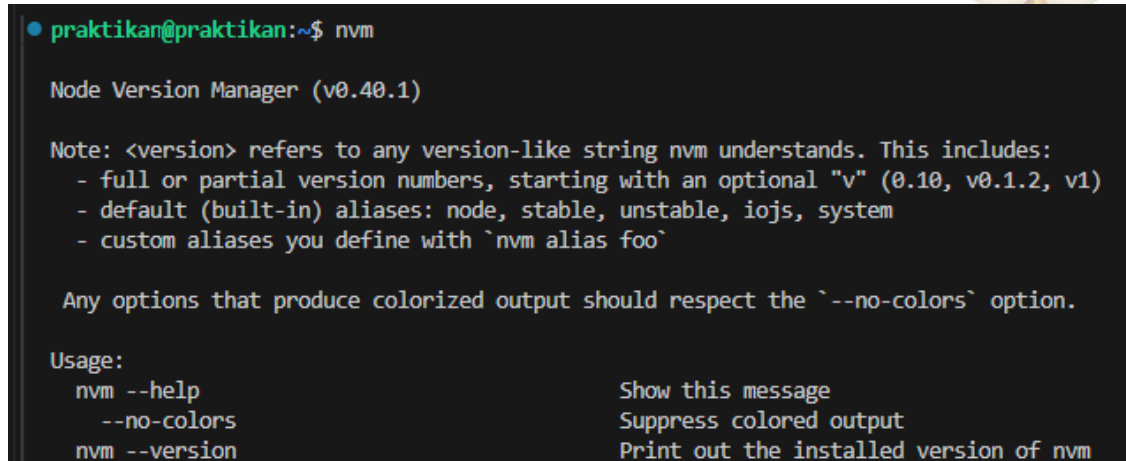
Gambar 12.4.6 Output perintah composer

Instalasi Node dan NPM (Opsional)

Jika aplikasi membutuhkan runtime JavaScript (misalnya untuk compile aset frontend), kita perlu instal Node.js. Agar lebih fleksibel mengelola versi, kita gunakan NVM (Node Version Manager). Jalankan installer-nya dengan perintah:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
```

Penting: Restart terminal Anda (tutup lalu buka lagi) agar konfigurasi path terbaca, lalu ketik nvm untuk verifikasi.

A terminal window showing the output of the 'nvm' command. The prompt is 'praktikan@praktikan:~\$ nvm'. The output includes the title 'Node Version Manager (v0.40.1)', a note about version strings, a list of aliases (node, stable, unstable, iojs, system), and usage instructions for various flags like --help, --no-colors, and --version.

```
praktikan@praktikan:~$ nvm

Node Version Manager (v0.40.1)

Note: <version> refers to any version-like string nvm understands. This includes:
- full or partial version numbers, starting with an optional "v" (0.10, v0.1.2, v1)
- default (built-in) aliases: node, stable, unstable, iojs, system
- custom aliases you define with `nvm alias foo`

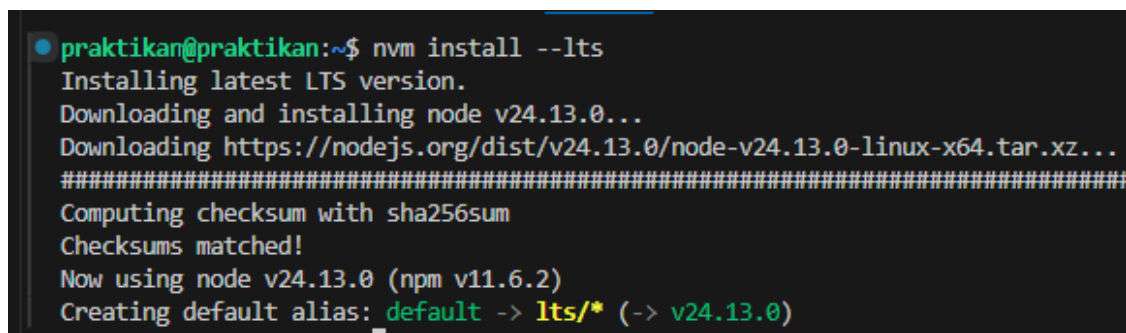
Any options that produce colored output should respect the `--no-colors` option.

Usage:
  nvm --help                Show this message
  --no-colors               Suppress colored output
  nvm --version              Print out the installed version of nvm
```

Gambar 12.4.7 Cek nvm

Selanjutnya, instal Node.js versi LTS (Long Term Support) agar lebih stabil digunakan di production. Perintah ini otomatis menyertakan NPM.

```
nvm install --lts
```

A terminal window showing the output of the 'nvm install --lts' command. The prompt is 'praktikan@praktikan:~\$ nvm install --lts'. The output shows the process of downloading and installing the latest LTS version of Node.js (v24.13.0), including checksum verification and the creation of a default alias.

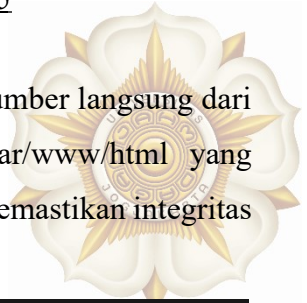
```
praktikan@praktikan:~$ nvm install --lts
Installing latest LTS version.
Downloading and installing node v24.13.0...
Downloading https://nodejs.org/dist/v24.13.0/node-v24.13.0-linux-x64.tar.xz...
#####
Computing checksum with sha256sum
Checksums matched!
Now using node v24.13.0 (npm v11.6.2)
Creating default alias: default -> lts/* (-> v24.13.0)
```

Gambar 12.4.8 Proses instalasi node versi lts

2. Menyiapkan Aplikasi Laravel

Pada tahap ini, kita akan melakukan inisiasi kode sumber (source code) dan mengonfigurasi parameter sistem agar aplikasi dapat berjalan pada VPS.

Inisiasi Source Code via GIT



Langkah standar dalam deployment modern adalah menarik kode sumber langsung dari repositori. Kita akan menempatkan aplikasi pada direktori `/var/www/html` yang merupakan DocumentRoot standar Apache. Penggunaan git clone memastikan integritas file tetap terjaga dibandingkan metode transfer manual seperti FTP.

```
● praktikan@praktikan:/var/www/html$ sudo git clone https://github.com/yusuf140298/Project280223
Cloning into 'Project280223'...
remote: Enumerating objects: 315, done.
remote: Counting objects: 100% (315/315), done.
remote: Compressing objects: 100% (205/205), done.
remote: Total 315 (delta 112), reused 291 (delta 93), pack-reused 0 (from 0)
Receiving objects: 100% (315/315), 151.47 KiB | 2.16 MiB/s, done.
Resolving deltas: 100% (112/112), done.
○ praktikan@praktikan:/var/www/html$
```

Gambar 12.4.9 Proses cloning repositori via terminal

Mengubah kepemilikan direktori

Secara default, eksekusi perintah dengan sudo akan memberikan atribut kepemilikan direktori kepada root user. Hal ini akan menimbulkan konflik izin (permission denied) saat user praktikan atau layanan web server mencoba memodifikasi file (misalnya untuk penulisan log atau cache). Oleh karena itu, diperlukan perubahan kepemilikan secara rekursif (-R) kepada user yang relevan menggunakan utilitas chown. Normalnya pemilik dari DocumentRoot adalah www-data seperti pada pertemuan sebelumnya.

```
sudo chown -R <user>:<group> <folder path>
```

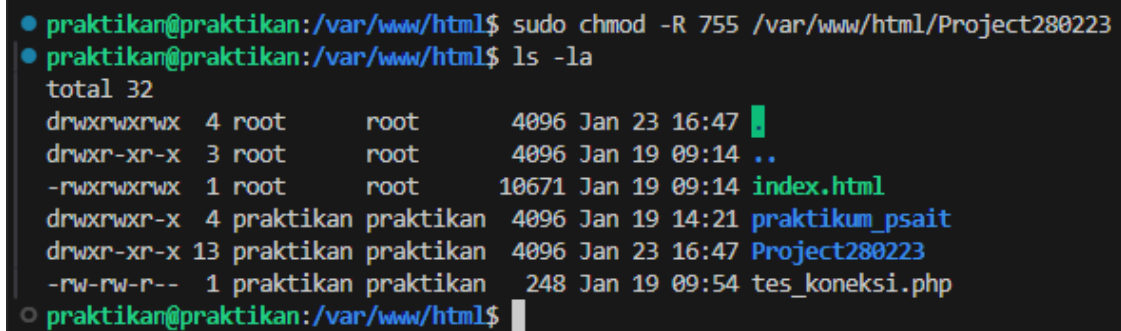
```
● praktikan@praktikan:/var/www/html$ ls
index.html praktikum_psait Project280223 tes_koneksi.php
● praktikan@praktikan:/var/www/html$ sudo chown -R praktikan:praktikan /var/www/html/Project280223
○ praktikan@praktikan:/var/www/html$
```

Gambar 12.4.10 Mengubah kepemilikan folder

Mengubah perizinan folder

Untuk menjaga keamanan sistem kita akan melakukan konfigurasi untuk memberikan akses penuh (Read, Write, Execute) kepada pemilik file, namun membatasi grup dan user lain (termasuk public requests) hanya pada akses baca dan eksekusi. Ini mencegah modifikasi file sistem yang tidak sah dari pihak eksternal.

```
sudo chmod -R 755 <folder path>
```

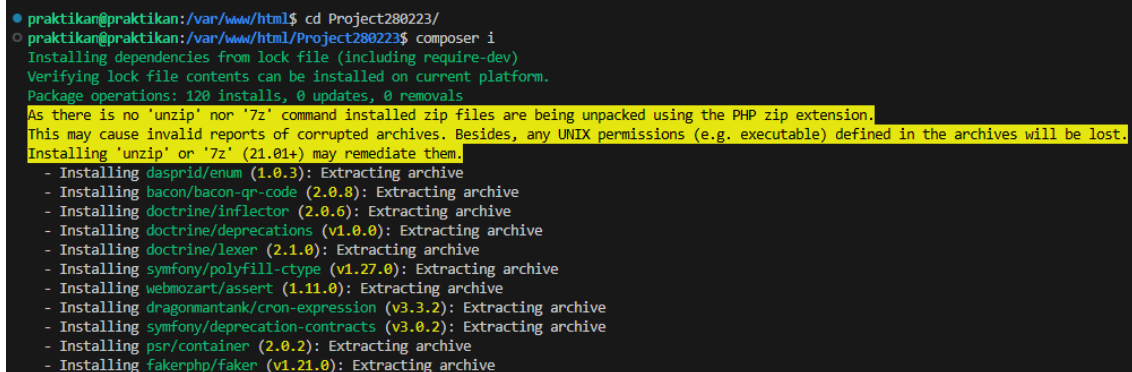


```
praktikan@praktikan:/var/www/html$ sudo chmod -R 755 /var/www/html/Project280223
praktikan@praktikan:/var/www/html$ ls -la
total 32
drwxrwxrwx  4 root      root      4096 Jan 23 16:47 
drwxr-xr-x  3 root      root      4096 Jan 19 09:14 ..
-rwxrwxrwx  1 root      root     10671 Jan 19 09:14 index.html
drwxrwxr-x  4 praktikan praktikan 4096 Jan 19 14:21 praktikum_psait
drwxr-xr-x 13 praktikan praktikan 4096 Jan 23 16:47 Project280223
-rw-rw-r--  1 praktikan praktikan 248 Jan 19 09:54 tes_koneksi.php
praktikan@praktikan:/var/www/html$
```

Gambar 12.4.11 Mengubah perizinan folder

Instal vendor Laravel

Selanjutnya kita menjalankan perintah composer i, ini berfungsi untuk membaca file composer.json dan composer.lock, kemudian mengunduh seluruh dependensi yang presisi ke dalam direktori /vendor. Tanpa proses resolusi dependensi ini, core system aplikasi tidak akan berfungsi.



```
praktikan@praktikan:/var/www/html$ cd Project280223/
praktikan@praktikan:/var/www/html/Project280223$ composer i
Installing dependencies from lock file (including require-dev)
Verifying lock file contents can be installed on current platform.
Package operations: 120 installs, 0 updates, 0 removals
As there is no 'unzip' nor '7z' command installed zip files are being unpacked using the PHP zip extension.
This may cause invalid reports of corrupted archives. Besides, any UNIX permissions (e.g. executable) defined in the archives will be lost.
Installing 'unzip' or '7z' (21.01+) may remediate them.
- Installing dasprid/enum (1.0.3): Extracting archive
- Installing bacon/bacon-qr-code (2.0.8): Extracting archive
- Installing doctrine/inflector (2.0.6): Extracting archive
- Installing doctrine/deprecations (v1.0.0): Extracting archive
- Installing doctrine/lexer (2.1.0): Extracting archive
- Installing symfony/polyfill-ctype (v1.27.0): Extracting archive
- Installing webmozart/assert (1.11.0): Extracting archive
- Installing dragonmantank/cron-expression (v3.3.2): Extracting archive
- Installing symfony/deprecation-contracts (v3.0.2): Extracting archive
- Installing psr/container (2.0.2): Extracting archive
- Installing fakerphp/faker (v1.21.0): Extracting archive
```

Gambar 12.4.12 Proses instalasi vendor Laravel

Instal nodes_module dan build jika menggunakan node

Jika aplikasi Anda menggunakan tampilan modern (seperti React atau Vue), kita perlu memprosesnya terlebih dahulu. Jalankan npm install untuk mengunduh paket JavaScript, lalu npm run build untuk menyusun (compile) kode-kode tersebut agar siap tampil di browser.



```

● praktikan@praktikan:/var/www/html/Project280223$ npm install

added 101 packages, and audited 102 packages in 5s

20 packages are looking for funding
  run `npm fund` for details

11 vulnerabilities (6 moderate, 4 high, 1 critical)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
● praktikan@praktikan:/var/www/html/Project280223$ npm run build

> build
> vite build

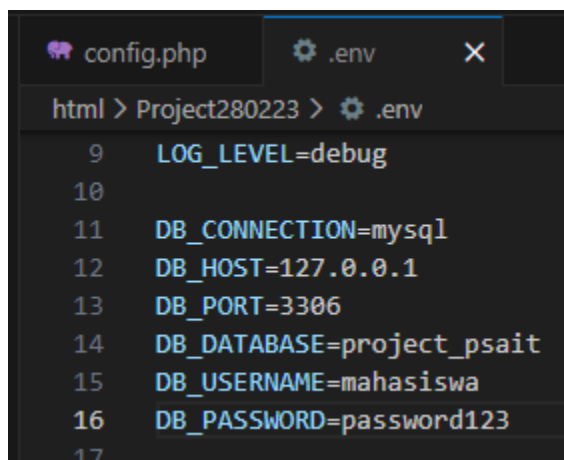
vite v4.1.4 building for production...
transforming (9) node_modules/axios/lib/axios.jsBrowserslist: caniuse-lite is outdated. Please run:
  npx update-browserslist-db@latest
  why you should do it regularly: https://github.com/browserslist/update-db#readme
✓ 51 modules transformed.
public/build/manifest.json 0.26 kB
public/build/assets/app-ad25429c.css 48.00 kB | gzip: 8.32 kB
public/build/assets/app-17ff25f3.js 154.40 kB | gzip: 56.57 kB
❖ praktikan@praktikan:/var/www/html/Project280223$

```

Gambar 12.4.13 Proses npm install dan npm run build

Buat .env file dan lakukan konfigurasi

Laravel menyimpan pengaturan penting seperti nama database dan password di file bernama .env. Karena file ini bersifat rahasia, kita harus membuatnya sendiri dengan menyalin dari contoh yang ada (.env.example). Setelah disalin, edit file tersebut dan sesuaikan bagian DB_DATABASE, DB_USERNAME, dan DB_PASSWORD dengan data MySQL Anda.

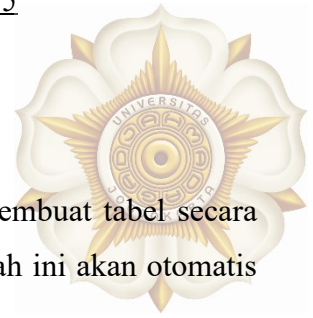


```

config.php .env
html > Project280223 > .env
9 LOG_LEVEL=debug
10
11 DB_CONNECTION=mysql
12 DB_HOST=127.0.0.1
13 DB_PORT=3306
14 DB_DATABASE=project_psait
15 DB_USERNAME=mahasiswa
16 DB_PASSWORD=password123
17

```

Gambar 12.4.14 Mengedit konfigurasi database di file .env



Migrasikan dan import database

Saat ini, database MySQL kita masih kosong. Kita tidak perlu membuat tabel secara manual. Cukup jalankan perintah `php artisan migrate:fresh`. Perintah ini akan otomatis membuatkan semua tabel yang dibutuhkan oleh aplikasi dari nol.

```
praktikan@praktikan:/var/www/html/Project280223$ php artisan migrate:fresh

Dropping all tables ..... 23ms DONE

INFO Preparing database.

Creating migration table ..... 182ms DONE

INFO Running migrations.

2014_10_12_000000_create_users_table ..... 316ms DONE
2014_10_12_100000_create_password_resets_table ..... 813ms DONE
2014_10_12_200000_add_two_factor_columns_to_users_table ..... 455ms DONE
2019_08_19_000000_create_failed_jobs_table ..... 346ms DONE
2019_12_14_000001_create_personal_access_tokens_table ..... 536ms DONE
2023_02_28_085128_create_sessions_table ..... 794ms DONE
2023_02_28_092250_create_reimbursements_table ..... 277ms DONE
2023_02_28_164529_create_payments_table ..... 286ms DONE
```

Gambar 12.4.15 Migrasi dan import database

Generate Laravel APP KEY

Selanjutnya dilakukan generate APP_KEY dengan menjalankan perintah “`php artisan key:generate`”. Perintah ini berfungsi sebagai kunci enkripsi yang umumnya digunakan oleh Laravel untuk mengamankan session, token, dan data sensitif lainnya. Setelah key berhasil dibuat, Laravel akan secara otomatis menyimpan data tersebut di dalam file `.env`.

```
praktikan@praktikan:/var/www/html/Project280223$ php artisan key:generate

INFO Application key set successfully.
```

Gambar 12.4.16 Generate app key

Jalankan aplikasi menggunakan artisan serve

Untuk mengecek apakah semua langkah di atas berhasil, mari kita coba nyalakan aplikasinya menggunakan server bawaan. Gunakan IP VPS Anda agar aplikasi bisa dibuka dari laptop/komputer.

```
php artisan serve --host=<IP_VPS> --port=8000
```

```

praktikan@praktikan:/var/www/html/Project280223$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:28:69:2b brd ff:ff:ff:ff:ff:ff
    inet 192.168.18.195/24 metric 100 brd 192.168.18.255 scope global dynamic enp0s3
        valid_lft 2726sec preferred_lft 2726sec
    inet6 fe80::a00:27ff:fe28:692b/64 scope link
        valid_lft forever preferred_lft forever
praktikan@praktikan:/var/www/html/Project280223$ php artisan serve --host=192.168.18.195 --port=8000

INFO Server running on [http://192.168.18.195:8000].

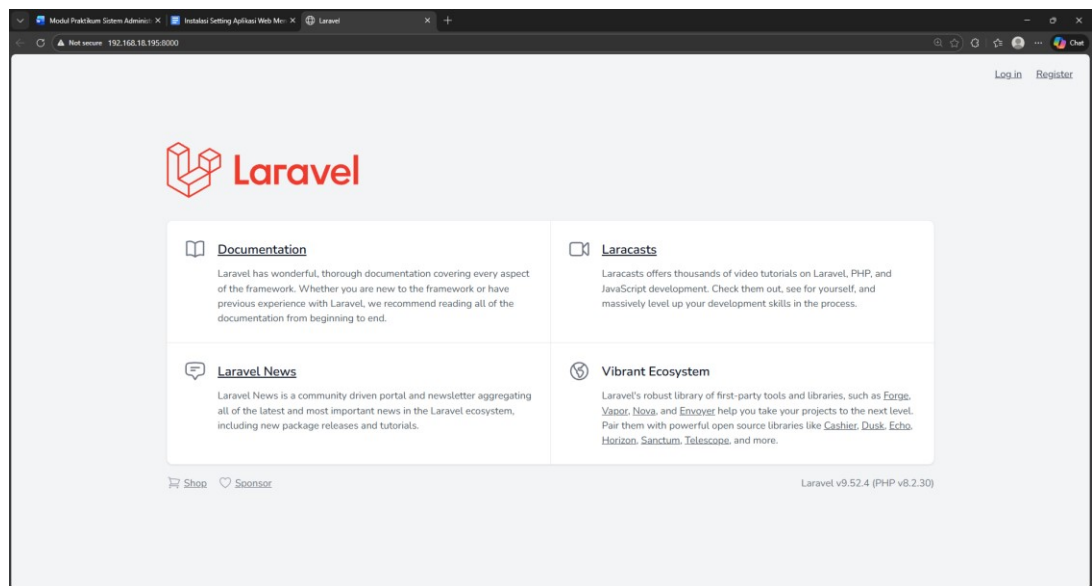
Press Ctrl+C to stop the server

```

Gambar 12.4.17 Jalankan aplikasi laravel

Buka aplikasi pada host dan port yang telah didefinisikan

Terakhir, buka browser dan ketik alamat `http://<IP_VPS>:8000`. Jika halaman utama website muncul, berarti aplikasi sudah berhasil dijalankan



Gambar 12.4.18 Buka aplikasi laravel dengan website

3. Deployment Aplikasi Utama (Instance 1)

Setelah aplikasi siap secara kode, langkah berikutnya adalah membuatnya berjalan secara mandiri. Sebelumnya kita menggunakan `php artisan serve`, namun cara itu hanya cocok untuk testing sementara. Untuk penggunaan jangka panjang (production), kita



harus mengatur agar aplikasi dijalankan sepenuhnya oleh Apache Web Server di latar belakang.

Instalasi Dependensi dan Konfigurasi Environment

Pertama, pastikan seluruh kebutuhan aplikasi terpenuhi. Masuklah ke folder proyek dan jalankan instalasi paket PHP menggunakan user www-data (user milik Apache) agar tidak terjadi masalah izin akses di kemudian hari.

```
● praktikan@praktikan:~$ cd /var/www/html/Project280223
● praktikan@praktikan:/var/www/html/Project280223$ sudo chown -R praktikan:www-data /var/www/html/Project280223
● praktikan@praktikan:/var/www/html/Project280223$ sudo -u www-data composer install
The repository at "/var/www/html/Project280223" does not have the correct ownership and git refuses to use it:

fatal: detected dubious ownership in repository at '/var/www/html/Project280223'
To add an exception for this directory, call:

git config --global --add safe.directory /var/www/html/Project280223

Installing dependencies from lock file (including require-dev)
Verifying lock file contents can be installed on current platform.
Nothing to install, update or remove
Generating optimized autoload files
```

Gambar 12.4.19 Membuat folder menjadi www-data

Jika aplikasi Anda menggunakan Node.js, jangan lupa untuk memproses aset frontend terlebih dahulu dengan npm install dan npm run build.



```

● praktikan@praktikan:/var/www/html/Project280223$ npm install

up to date, audited 102 packages in 3s

20 packages are looking for funding
  run `npm fund` for details

11 vulnerabilities (6 moderate, 4 high, 1 critical)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
● praktikan@praktikan:/var/www/html/Project280223$ npm run build

> build
> vite build

vite v4.1.4 building for production...
transforming (9) node_modules/axios/lib/axios.jsBrowserslist: caniuse-lite is outdated. Please run:
  npx update-browserslist-db@latest
  Why you should do it regularly: https://github.com/browserslist/update-db#readme
✓ 51 modules transformed.
public/build/manifest.json          0.26 kB
public/build/assets/app-ad25429c.css 48.00 kB | gzip: 8.32 kB
public/build/assets/app-17ff25f3.js 154.40 kB | gzip: 56.57 kB
❖ praktikan@praktikan:/var/www/html/Project280223$

```

Gambar 12.4.20 Menjalankan npm install dan npm run build

Konfigurasi Virtual Host (Port 8001)

Agar aplikasi ini bisa diakses secara spesifik, kita akan menaruhnya di Port 8001. Kita perlu memberi tahu Apache untuk membuka port tersebut dan mengarahkan pengunjungnya ke folder aplikasi kita.

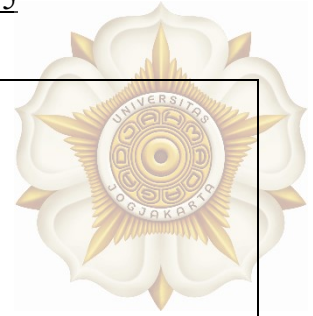
Pertama Buka daftar port Apache dan tambahkan Listen 8001 di baris paling bawah.

```
sudo nano /etc/apache2/ports.conf
```

Selanjutnya kita perlu membuat file konfigurasi "Virtual Host" baru khusus untuk aplikasi pertama ini.

```
sudo nano /etc/apache2/sites-available/Project280223.conf
```

Salin konfigurasi di bawah ini. Konfigurasi ini memberi tahu Apache untuk mengarahkan trafik Port 8001 ke folder Project280223.



```
<VirtualHost *:8001>
    DocumentRoot /var/www/html/Project280223/public
    <Directory /var/www/html/Project280223>
        AllowOverride All
        Require all granted
    </Directory>
    ErrorLog ${APACHE_LOG_DIR}/error_project1.log
    CustomLog ${APACHE_LOG_DIR}/access_project1.log combined
</VirtualHost>
```

4. Simulasi Horizontal Scaling (Instance 2)

Untuk mensimulasikan Load Balancing, kita akan menduplikasi folder Project280223 menjadi aplikasi kedua yang berjalan di Port 8002

Duplikasi Aplikasi

Selanjutnya kita akan menduplikasi aplikasi yang sudah kita miliki yaitu Project280223 menjadi Project280223_dua

```
● praktikan@praktikan:/var/www/html/Project280223$ cd /var/www/html
● praktikan@praktikan:/var/www/html$ sudo cp -r Project280223 Project280223_dua
● praktikan@praktikan:/var/www/html$ ls
index.html praktikum_psait Project280223 Project280223_dua tes_koneksi.php
○ praktikan@praktikan:/var/www/html$
```

Gambar 12.4.21 Duplikasi folder aplikasi

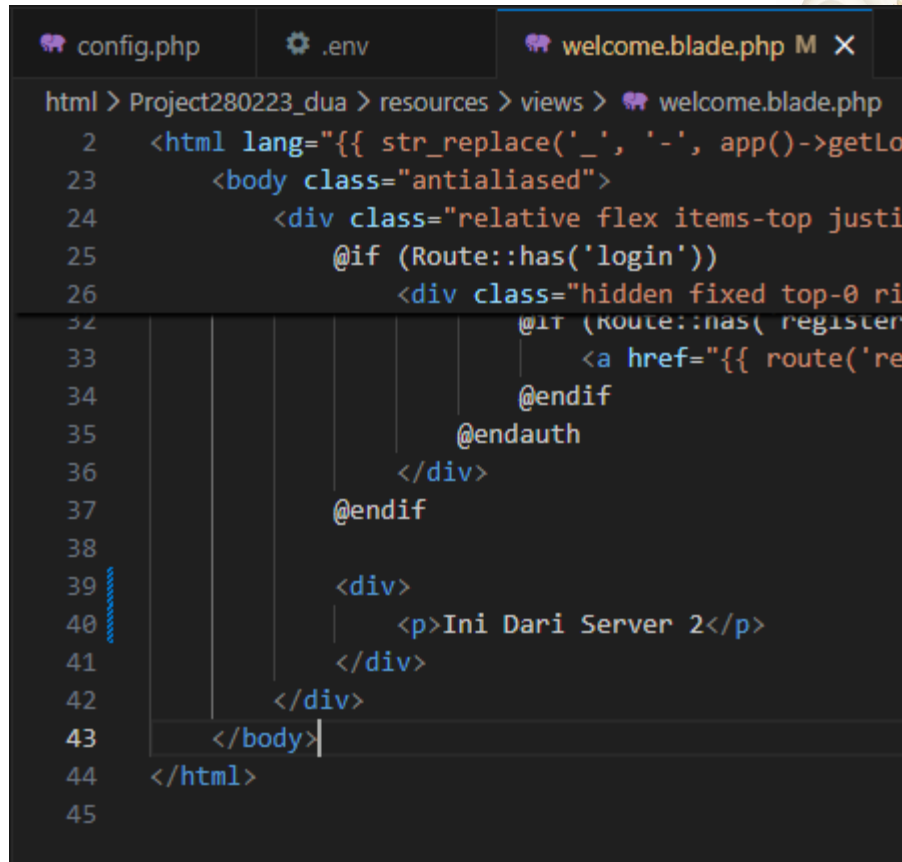
Penting: Jangan lupa ubah kembali kepemilikan folder hasil duplikat tersebut agar bisa diakses oleh Apache.

```
● praktikan@praktikan:/var/www/html$ ls
index.html praktikum_psait Project280223 Project280223_dua tes_koneksi.php
● praktikan@praktikan:/var/www/html$ sudo chown -R praktikan:www-data /var/www/html/Project280223_dua
○ praktikan@praktikan:/var/www/html$
```

Gambar 12.4.22 Mengatur kepemilikan folder duplikat

Memberi Tanda Visual

Agar kita bisa membedakan mana Server 1 dan mana Server 2 saat pengujian, edit halaman depan (welcome.blade.php) yang ada di folder Project280223_dua. Tambahkan tulisan besar seperti "INI DARI SERVER 2".



```

config.php .env welcome.blade.php M X
html > Project280223_dua > resources > views > welcome.blade.php
 2  <html lang="{{ str_replace('_', '-', app()->getLocale()) }}">
23    <body class="antialiased">
24      <div class="relative flex items-top justify" style="height: 100vh;">
25        @if (Route::has('login'))
26          <div class="hidden fixed top-0 right-0 px-6 py-4">
32            @if (Route::has('register'))
33              <a href="{{ route('register') }}" class="text-sm text-gray-600 hover:text-gray-900">
34                Register
35            @endif
36          @endif
37        @endif
38      </div>
39      <div>
40        <p>Ini Dari Server 2</p>
41      </div>
42    </div>
43  </body>
44 </html>
45

```

Gambar 12.4.23 Mengubah welcome.blade.php

Konfigurasi Apache untuk Server 2

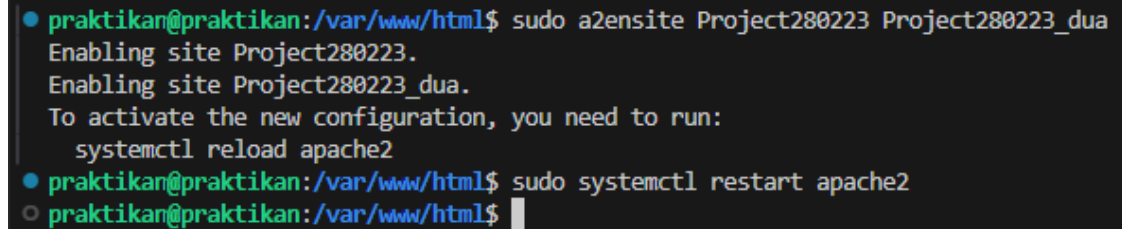
Setelah itu buka kembali ports.conf dan tambahkan Listen 8002. Setelah itu, buat file konfigurasi baru bernama Project280223_dua.conf. Isinya mirip dengan yang pertama, tapi ubah Port menjadi 8002 dan jalur folder ke Project280223_dua.

```

<VirtualHost *:8002>
    DocumentRoot /var/www/html/Project280223_dua/public
    <Directory /var/www/html/Project280223_dua>
        AllowOverride All
        Require all granted
    </Directory>
    # ... (log bisa disesuaikan)
</VirtualHost>

```

Terakhir, aktifkan kedua situs tersebut dan restart Apache.



```

● praktikan@praktikan:/var/www/html$ sudo a2ensite Project280223 Project280223_dua
Enabling site Project280223.
Enabling site Project280223_dua.
To activate the new configuration, you need to run:
    systemctl reload apache2
● praktikan@praktikan:/var/www/html$ sudo systemctl restart apache2
○ praktikan@praktikan:/var/www/html$
    
```

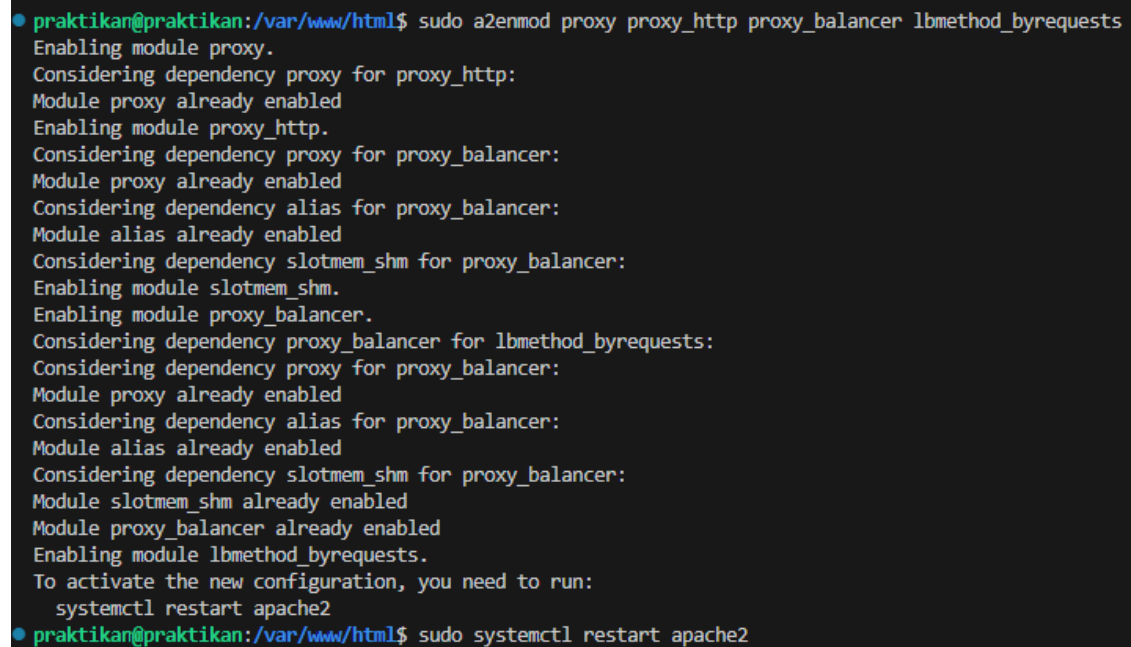
Gambar 12.4.24 Mengaktifkan kedua situs

5. Konfigurasi Load Balancing

Saat ini, Project280223 berjalan di port 8001 dan Project280223_dua di port 8002. Kita akan menjadikan Port 80 sebagai pintu gerbang utama (Load Balancer).

Mengaktifkan Modul Proxy

Selanjutnya untuk mengaktifkan modul proxy pada apache2 kita bisa menjalankan perintah sesuai yang ada di Gambar 12.4.25.



```

● praktikan@praktikan:/var/www/html$ sudo a2enmod proxy proxy_http proxy_balancer lbmethod_byrequests
Enabling module proxy.
Considering dependency proxy for proxy_http:
Module proxy already enabled
Enabling module proxy_http.
Considering dependency proxy for proxy_balancer:
Module proxy already enabled
Considering dependency alias for proxy_balancer:
Module alias already enabled
Considering dependency slotmem_shm for proxy_balancer:
Enabling module slotmem_shm.
Enabling module proxy_balancer.
Considering dependency proxy_balancer for lbmethod_byrequests:
Considering dependency proxy for proxy_balancer:
Module proxy already enabled
Considering dependency alias for proxy_balancer:
Module alias already enabled
Considering dependency slotmem_shm for proxy_balancer:
Module slotmem_shm already enabled
Module proxy_balancer already enabled
Enabling module lbmethod_byrequests.
To activate the new configuration, you need to run:
    systemctl restart apache2
● praktikan@praktikan:/var/www/html$ sudo systemctl restart apache2
    
```

Gambar 12.4.25 Mengaktifkan Modul Proxy

Mengatur Logika Pembagian Beban Port

Selanjutnya kita akan mengedit file konfigurasi default Apache (000-default.conf). Hapus isinya dan ganti dengan skrip di bawah ini.

```
sudo nano /etc/apache2/sites-available/000-default.conf
```

Skrip ini membuat grup (cluster) yang berisi aplikasi utama dan aplikasi duplikat Anda.

```
<VirtualHost *:80>
    ProxyRequests Off

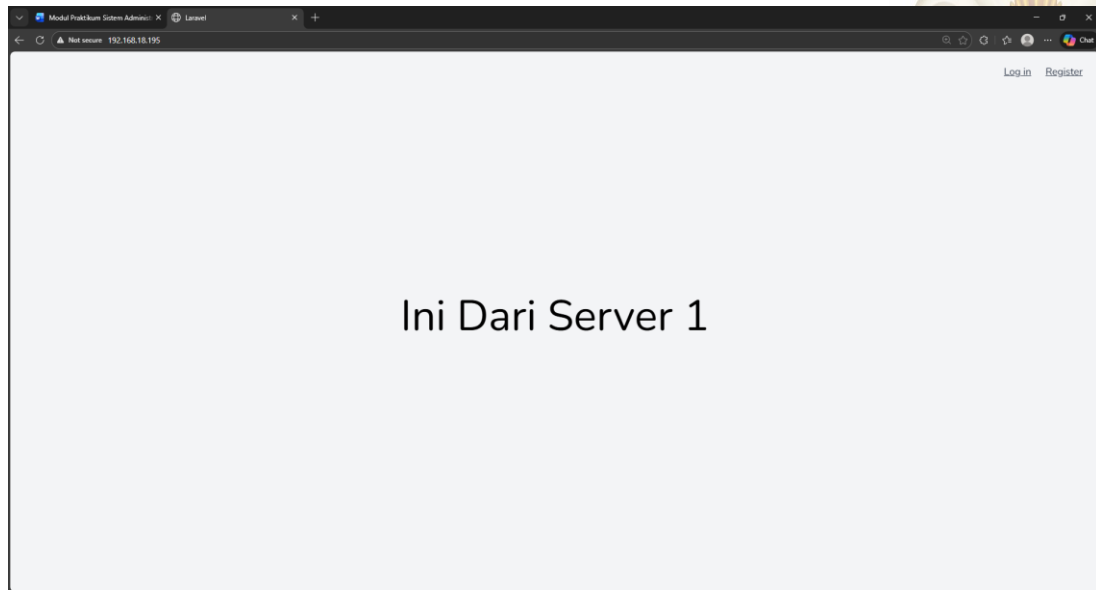
    # Membuat grup penyeimbang beban
    <Proxy balancer://mycluster>
        BalancerMember http://127.0.0.1:8001
        BalancerMember http://127.0.0.1:8002
    </Proxy>

    # Mengarahkan trafik masuk ke grup tersebut
    ProxyPass / balancer://mycluster/
    ProxyPassReverse / balancer://mycluster/
</VirtualHost>
```

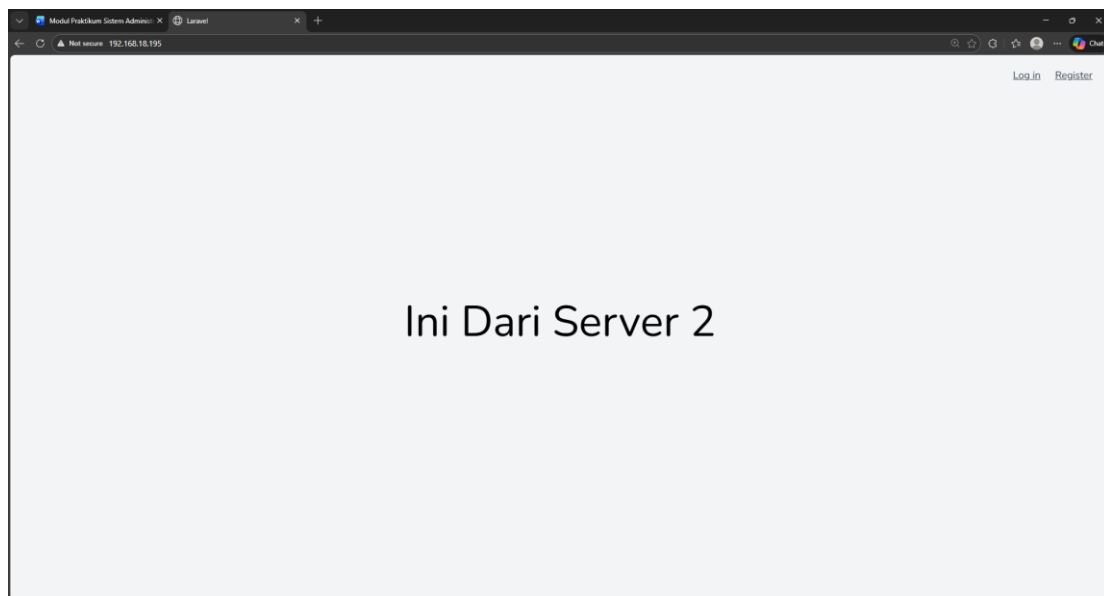
Jangan lupa restart Apache sekali lagi dengan `sudo systemctl restart apache2`.

6. Pengujian Hasil Load Balancing

Untuk memverifikasi hasil konfigurasi, silakan buka browser dan akses alamat IP VPS Anda secara langsung tanpa menambahkan nomor port di belakangnya. Pada akses awal, Anda akan disugahi halaman muka dari aplikasi utama Project280223. Cobalah lakukan refresh halaman tersebut beberapa kali secara berturut-turut. Perhatikan bahwa tampilan akan berubah-ubah secara bergantian antara halaman asli dan halaman yang memiliki penanda teks "INI DARI SERVER 2" yang berasal dari Project280223_dua. Perubahan dinamis ini membuktikan bahwa mekanisme Load Balancing pada Apache telah berfungsi dengan baik dalam membagi rata lalu lintas akses ke kedua folder proyek yang berbeda tersebut.



Gambar 12.4.26 Mengakses server 1

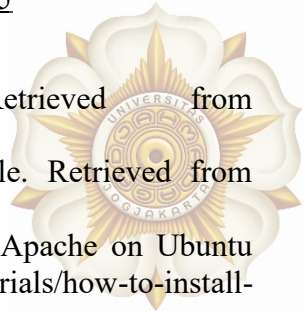


Gambar 12.4.27 Mengakses server 2

12.5. TUGAS & ANALISIS

1. Jika Anda menggunakan aplikasi yang sama seperti modul, coba gunakan aplikasi lain dan terapkan seperti percobaan, berilah tanda visual bahwa terdapat lebih dari 1 aplikasi untuk menunjukkan load balancing berjalan!

12.6. REFERENSI

- 
- [1].Laravel Documentation. (n.d.). Deployment. Retrieved from <https://laravel.com/docs/10.x/deployment>
- [2].Apache HTTP Server Project. (n.d.). mod_proxy_balancer Module. Retrieved from https://httpd.apache.org/docs/2.4/mod/mod_proxy_balancer.html
- [3].DigitalOcean. (2023). How To Install and Configure Laravel with Apache on Ubuntu 20.04. Retrieved from <https://www.digitalocean.com/community/tutorials/how-to-install-and-configure-laravel-with-apache-on-ubuntu-20-04>
- [4].GetComposer.org. (n.d.). Introduction. Retrieved from <https://getcomposer.org/doc/00-intro.md>